Less Precise, More Reliable

A Tutorial on Quantization for Efficient and Robust Road-Scene Perception

Fabio Arnez*
Université Paris-Saclay, CEA, List
F-91120, Palaiseau, France
fabio.arnez@cea.fr

Version 1.0, 9 November 2026

Abstract

A camera-based perception model in a vehicle has to stay accurate when the image degrades and has to run on small, cheap, cool hardware. Low-bit quantization answers the second demand and is widely assumed to cost the first. These notes teach, from first principles and for undergraduate and master's students, everything needed to read two recent papers that question that assumption: a large-scale study of quantized vision-language models (Bouguerra et al., 2026a) and a matched-budget study of a road-scene detector and recognizer under blur (Bouguerra et al., 2026b). Part I builds the toolkit. It covers number formats and what a bit costs in hardware, the uniform quantizer and its rounding and clipping errors, rounding error as noise, where quantization enters a vision transformer, post-training quantization and quantization-aware training, the covariance spectrum of a representation, what added noise does to that spectrum, blur as a low-pass filter, CLIP and DINOv2, object detection, reliability beyond accuracy, and the statistics needed to compare models honestly. Part II reads both papers with that toolkit, result by result, and then side by side. The reading is deliberately calibrated. Quantization-aware training can make a recognition model more robust to blur it never saw, but only inside a window of severities, and not every reliability measure improves. The spectral picture of a noise floor burying weak feature directions is a necessary part of the explanation, not a sufficient one, and what completes it is an open question. Wherever the distinction matters, a claim carries a tag saying whether it is established, suggestive, or open.

*Lecture notes for the theory half of the IEEE ICVES 2026 tutorial, Cochabamba, Bolivia, 9 November 2026. Slides, notebook and updates at https://fabioarnez.github.io/LessPrecise-MoreReliable-Tutorial-ICVES-2026-Public/. © 2026 Fabio Arnez. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0), https://creativecommons.org/licenses/by/4.0/. Correspondence welcome.

Contents

I Foundations **7**

1 Can less precise be more reliable? **9**
1.1 The deployment problem . . . 9
1.2 What these notes are . . . 9
1.3 How to read the notes . . . 11
1.4 The tutorial, section by section . . . 12

2 Numbers in hardware and what a bit costs **14**
2.1 Integers, a uniform ruler . . . 14
2.2 Floats, a ruler whose ticks spread out . . . 14
2.3 What a bit costs in memory . . . 20
2.4 An integer dot product . . . 21
2.5 Fake and true integer quantization . . . 24
2.6 Where speed comes from . . . 28

3 The uniform quantizer **32**
3.1 Divide, round, clamp, rescale . . . 32
3.2 The clip range and the two kinds of error . . . 34
3.3 Shifting the grid with a zero point . . . 37
3.4 One step for how many numbers? Granularity . . . 39
3.5 Weights and activations . . . 39

4 Rounding error as noise **41**
4.1 The high-resolution model . . . 42
4.2 Signal-to-quantization-noise ratio and the 6 dB rule . . . 43
4.3 When the model fails . . . 44
4.4 Rounding is deterministic; injected noise is redrawn . . . 46

5 Where quantization enters a network **48**
5.1 From a feature vector to a decision . . . 49
5.2 The Vision Transformer, one block at a time . . . 49
5.3 Where the quantizers sit . . . 51
5.4 Many quantizers, one residual . . . 51

6 Post-training quantization and quantization-aware training **55**
6.1 Post-training quantization . . . 55
6.2 Outliers and rotation . . . 57
6.3 From a uniform grid to a codebook . . . 63
6.4 Quantization-aware training . . . 65
6.5 Two kinds of QAT, and bit width . . . 68

7 Representations and their spectrum **70**
7.1 Feature vectors, cosines, and coordinates . . . 70
7.2 Covariance and principal directions . . . 71
7.3 Spectrum, rank index, head, and tail . . . 72
7.4 How many directions matter . . . 74
7.5 Fixed basis versus recomputed basis . . . 74

8 What noise does to a spectrum **77**
8.1 One strong and one weak feature . . . 77
8.2 Independent noise on a feature vector, the flat floor . . . 77
8.3 The per-rank measurement . . . 79
8.4 White and colored residuals . . . 80
8.5 When the error is correlated with the signal . . . 81
8.6 What a model trained through noise learns . . . 83
8.7 Which spectrum? . . . 83
8.8 What the picture explains, and what it leaves open . . . 83

9 Images, blur, and frequency **85**
9.1 An image is a two-dimensional signal . . . 85
9.2 Blur is a convolution, and a convolution is a filter . . . 86
9.3 A blur σ is measured in pixels of the image it is applied to . . . 88
9.4 Noise, brightness, and contrast on the frequency axis . . . 89
9.5 Pixel frequency is not feature rank . . . 90

10 Vision foundation models, CLIP and DINOv2 **91**
10.1 Reusing a pretrained encoder . . . 92
10.2 CLIP, a shared space for images and text . . . 92
10.3 Zero-shot classification with prompts . . . 93
10.4 The CLIP family . . . 94
10.5 DINOv2 and a recognizer built on it . . . 95

11 Object detection in one section **97**
11.1 Boxes, intersection over union, and matching . . . 97
11.2 Precision, recall, and average precision . . . 98
11.3 In what sense mAP@[.5:.95] is dominated by localization . . . 100
11.4 Set prediction with DETR and RT-DETR . . . 100
11.5 Recognition on ground-truth crops . . . 101

12 Reliability beyond accuracy **103**
12.1 Plain accuracy, per-class recall, and balanced accuracy . . . 104
12.2 Differences, ratios, and retention . . . 105
12.3 Probability calibration . . . 105
12.4 Out-of-distribution detection . . . 107
12.5 Kinds of shift . . . 108
12.6 Spurious correlations and group robustness . . . 109
12.7 Severity windows, and why averages hide them . . . 110

13 Statistics for comparing models **111**
13.1 Describing a set of runs . . . 112
13.2 Paired differences and the bootstrap . . . 113
13.3 p-values, effect sizes, and many tests . . . 114
13.4 No significant difference is not no difference . . . 116
13.5 Aggregation and selection . . . 116
13.6 Designing a comparison that can be read causally . . . 117

II Reading the Two Papers **121**

14 The question, and why the answer is surprising **123**
14.1 How Part II grades what the papers say . . . 123
14.2 The assumed trade-off . . . 123
14.3 What reliability means in each paper . . . 125
14.4 Two papers, two scales, one claim . . . 125
14.5 A reading guide to Part II . . . 126

15 The ICML paper, design and metrics **127**
15.1 The grid . . . 128
15.2 Five dimensions and their metrics . . . 129
15.3 Signs and units . . . 130
15.4 Which runs are counted . . . 131
15.5 What the paper leaves unstated . . . 131

16 The ICML paper, results axis by axis **132**
16.1 All four quadrants of Figure 1 . . . 133
16.2 Accuracy, with redundancy and fidelity . . . 134
16.3 Calibration and a one-parameter repair . . . 134
16.4 OOD detection follows accuracy . . . 135
16.5 Corruptions versus natural distribution shift . . . 135
16.6 Spurious correlations, where quantization hurts . . . 136
16.7 Failure rates by method . . . 136
16.8 The deployed engine . . . 137

17 The ICML paper, the spectral-filtering account **139**
17.1 Two stages on two bases . . . 139
17.2 Eqs. (2) to (12) as measurements . . . 139
17.3 Stage 1, passive filtering, read with both meters . . . 141
17.4 Stage 2, active concentration . . . 142
17.5 The truncation and dropout controls . . . 143
17.6 What is derived, what is measured, what is asserted . . . 144

18 The ICVES paper, a matched-budget road-scene testbed **146**
18.1 DAWN and the two testbeds . . . 146
18.2 The variants and the matched protocol . . . 147
18.3 Conditions, budgets, and the evaluation grid . . . 149
18.4 What the error bars and intervals cover . . . 149
18.5 Deployment, and the scope of the testbed . . . 150

19 The ICVES paper, a neutral detector and a more robust recognizer **151**
19.1 The detector, a bound and one rare class . . . 152
19.2 The recognizer, a gain in balanced accuracy . . . 153
19.3 The severity window . . . 154
19.4 Deployment, what it costs and what it can vouch for . . . 155

20 The ICVES paper, the shared subspace and its controls **157**
20.1 Two residuals on one basis . . . 157
20.2 The profile cosine and its shuffled-rank null . . . 158
20.3 Necessary, not sufficient . . . 160
20.4 Three simpler explanations and how firmly each is excluded . . . 160
20.5 The matched-noise control and the untrained variant . . . 161
20.6 What remains open . . . 162

20.7 The paper's own limits . . . 163

21 Reading the two papers together **164**
21.1 One object, two papers . . . 164
21.2 Where the two accounts part . . . 165
21.3 What is established, suggestive, and open . . . 166
21.4 Where the effect holds, and what to do about it . . . 166
21.5 Open problems for a student project . . . 168
21.6 The sentence to leave with . . . 169

A Notation **170**

B Glossary of terms the two papers use differently **171**

C Key equations **173**

Part I

Foundations

1 Can less precise be more reliable?

What you will be able to do after this section

Explain why a camera-based perception model in a vehicle must be both reliable under image degradation and cheap to run, and why low-bit quantization is expected to trade one for the other. Say in numbers what 8 bits save and what rounding costs a strong and a weak feature. Read the box types and the running example of these notes. Find, for each topic of the tutorial and each part of its notebook, the sections that support it.

1.1 The deployment problem

An automated vehicle depends on its cameras more than on any other sensor, and the camera is also the sensor most exposed to the world. Blur, soiling, condensation, rain, and fog act on the lens before any network sees a pixel (Ceccarelli & Secci, 2023), and the perception model must stay reliable when they do. The same model must also run on embedded hardware inside the vehicle, under tight limits on cost, power, and heat. The two demands pull in opposite directions.

Low-bit quantization is the standard answer to the second demand. It stores weights and computes activations with a few bits per number instead of the 32 bits of the usual floating-point format (FP32). A common choice, written W8A8, uses 8-bit integers (INT8) for the weights (W) and for the activations (A), meaning the inputs of every quantized layer. It cuts weight memory fourfold relative to FP32 and runs on the integer units of modern accelerators (Jacob et al., 2018); Section 2 does the arithmetic of bits and bytes. A network is quantized in one of two ways.

- *Post-training quantization* (PTQ) takes a trained network and sets the step of each quantizer from a small *calibration set* of unlabeled inputs, with no training of the network on its task.
- *Quantization-aware training* (QAT) trains the network, or part of it, with the quantizers in place, so that the network adapts to them.

Section 6 develops both.

Quantization is widely assumed to cost the first demand. Rounding every number to one of 256 levels adds error, and a model that already carries error seems a poor candidate to stay accurate when its input degrades too. Robustness itself is usually measured by applying synthetic corruptions such as blur and noise at graded severities (Hendrycks & Dietterich, 2019; Michaelis et al., 2019). The question of this tutorial is whether the assumed trade-off always holds. Quantization is more than a compression step, because it changes which features a model can rely on, and it is at least conceivable that this change leaves a model more robust to a degraded image rather than less. Part I builds the tools to pose that question precisely, and Part II reads the evidence.

1.2 What these notes are

These notes are the companion to a half-day tutorial at the 2026 IEEE International Conference on Vehicular Electronics and Safety (ICVES), three hours split into 1.5 h of theory and 1.5 h of hands-on work in a Google Colab notebook. The level is intermediate. The notes assume first-year linear algebra and probability, basic Python, and a working idea of how a neural network is trained, and nothing about quantization, spectra of representations, vision-language models, detection metrics, or reliability metrics. Each section stands on its own, so you can read the ones a session needs.

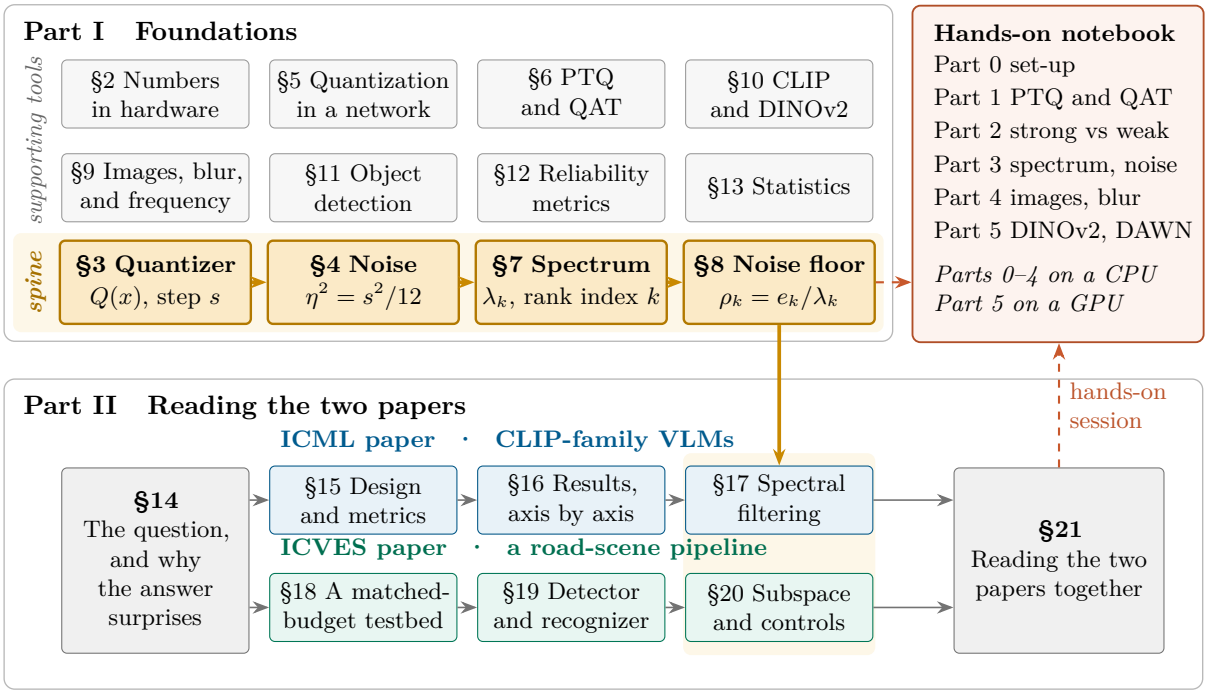


Figure 1: How the notes fit together. Sections 2 to 13 of Part I are the toolkit, and four of them form the spine (amber): the quantizer, the noise model of its rounding error, the spectrum of a set of features, and what noise does to that spectrum. The spine feeds the two mechanism readings of Part II (amber column). Part II reads the two papers in parallel lanes, each from design to results to mechanism, between a common question (§14) and a joint reading (§21). The hands-on notebook (orange) runs the spine on toy problems in Parts 1 to 4 and a real encoder in Part 5, after the theory.

Part I (Section 1 to Section 13) builds the fundamentals with synthetic examples. Part II reads the two papers at the center of the tutorial with those tools in hand. Bouguerra et al. (2026a), the *ICML paper* from here on, after the International Conference on Machine Learning (ICML) that publishes it, evaluate quantized vision-language models (VLMs) of the CLIP family (contrastive language-image pre-training; Radford et al., 2021) at large scale and well beyond accuracy. Bouguerra et al. (2026b), the *ICVES paper*, a paper in the same conference program as this tutorial, study a road-scene pipeline, an object detector followed by a recognition encoder, under camera blur. Part I does not quote their results. Every number, result, and judgment of evidence from the two papers lives in Part II.

Figure 1 shows how the pieces connect. Four sections form the *spine*, the argument on which the mechanism readings of Part II rest.

1. Section 3 defines the uniform quantizer Q with step s, whose rounding error is at most $s/2$ inside its clip range.

2. Section 4 models that error as noise of variance $\eta^2 = s^2/12$ (Bennett, 1948; Widrow & Kollár, 2008) and says when the model fails.

3. Section 7 describes a cloud of feature vectors by its *spectrum*, the variances λ_k along its principal directions, ordered by rank index k.

4. Section 8 joins the three through the per-rank ratio ρ_k of residual energy to signal variance.

In the idealized model rounding adds the same variance to every direction, a flat floor that

buries the weakest directions first. Measured residuals need not be flat, and Section 8 reads the ratio as graded, never as a hard cutoff.

Two reading paths. Part I reads straight through, each section ending where the next begins. If you want the central argument first, take the spine, go on to Part II, and return to the supporting sections when Part II points back to them. If deployment is your interest, add Sections 2 and 6, since a speed-up means little without its baseline.

1.3 How to read the notes

Each box color has one meaning.

- *Structure.* Slate boxes open a section with its goals and close it with a checkpoint answered in footnotes.
- *The notes' own results.* Blue boxes hold key equations reused later. Green, violet, and blue-gray boxes hold the notes' numbered definitions, propositions with a proof or proof sketch, and lemmas.
- *Results from papers.* Gray boxes, used in Part II only, restate a result from a paper or a cited work, with its locator in the title and no number of their own.
- *Examples.* Olive boxes hold worked examples with numbers. Teal boxes hold the running example and analogies, which never carry weight.
- *Cautions.* Amber boxes untangle a term used in different senses, and amber-red boxes state open questions and limitations.
- *Practice.* Cyan boxes hold exercises, each followed by a solution sketch, orange boxes point to the notebook, and steel boxes hold pseudocode.

A *check yourself* prompt asks one question inline and answers it in a footnote. Part II adds claim tags that grade each paper result as established, suggestive, or open (Section 14).

Running example: the bus in the rain

A camera behind the windshield sees a bus through rain and a fogged lens. A detector proposes a box, and the crop inside it, resized to 224×224 pixels, goes to a recognition encoder that must answer car, truck, bus, or person, the four classes of the notebook's Part 5. The encoder runs on an embedded board with 8-bit weights and activations, and the example keeps asking one question, whether the 8-bit encoder still says bus when the lens fogs. It returns as a quantized activation vector (Section 3), a feature vector (Section 5), a spectrum (Section 8), a blurred crop (Section 9), and a confusion between bus and truck (Section 12). In Part I its numbers are synthetic and computed on the page.

Worked example, what eight bits save and what they cost

Give the bus encoder 20 million weights, a round number for a small vision encoder. In FP32 each weight takes 4 bytes, so the weights fill 80 megabytes (MB). In INT8 each takes 1 byte, so they fill 20 MB, four times less.

Now the cost. Put an 8-bit quantizer with step $s = 1/32 = 0.03125$ on one of the encoder's activations. Its 256 integer levels, -128 to 127, cover the clip range $[-128s, 127s] = [-4, 3.96875]$, and inside that range rounding moves no value by more than $s/2 = 1/64 \approx 0.0156$. Section 4 models this rounding error as noise of variance

$$\eta^2 = \frac{s^2}{12} = \frac{(1/32)^2}{12} \approx 8.1 \times 10^{-5}.$$

Table 1: How the three-hour tutorial maps onto the notes. The first block lists the topics of the theory session in order, the second the parts of the hands-on notebook, each with the sections that support it.

Theory session, 1.5 h	Sections
The deployment problem and the assumed trade-off	§1, §14
How quantization works, from number formats to rounding error in a network	§2 to §5
Post-training quantization and quantization-aware training	§6
Representations, their spectrum, and what noise does to it	§7, §8
Images, blur, and frequency	§9
The models, CLIP and DINOv2 encoders and object detectors	§10, §11
Reliability beyond top-1 accuracy, and how to compare two models fairly	§12, §13
A large-scale evaluation of quantized VLMs, the ICML paper	§15 to §17
A road-scene detector and recognition encoder under blur, the ICVES paper	§18 to §20
Where each paper's claims hold and where they stop	§21
Hands-on session, 1.5 h, the notebook in Google Colab	
Part 0. Set-up, the shared colors, six short talk clips	§1
Part 1. A 1-D network through FP32, naive 3-bit, PTQ, and QAT	§3, §6
Part 2. One rounding step on a strong and a weak sinusoid	§4, §8
Part 3. A power-law spectrum against rounding noise, linear classifiers	§7, §8
Part 4. Images with a coarse and a fine cue, quantized and blurred	§8, §9
Part 5. DINOv2 on DAWN crops, W8A8 PTQ and QAT, a blur sweep	§5, §10, §12, §19

Compare it with two feature directions of the crop's representation. If the error adds the same variance to every direction, the idealized flat floor, the ratio of error to signal along direction k is $\rho_k = \eta^2/\lambda_k$.

- A strong direction of variance $\lambda_1 = 1$ has $\rho_1 = 8.1 \times 10^{-5}$, a signal-to-quantization-noise ratio (SQNR) of $\mathrm{SQNR}_1 = -10 \log_{10} \rho_1 \approx 40.9$ dB.
- A weak direction far down the spectrum, of variance $\lambda_{300} = 10^{-4}$, has $\rho_{300} = 0.81$, an SQNR of about 0.9 dB.

The same rounding that is negligible for the strong direction is almost as large as the signal of the weak one. Real residuals are not flat, and Section 8 replaces this picture with a graded one. Whether any of it bears on robustness is the question Part II puts to the papers.

1.4 The tutorial, section by section

Table 1 maps the tutorial onto the notes. The theory session runs from the fundamentals to the two papers and their limits, and the hands-on session runs the six parts of the notebook.

The hands-on notebook, part by part

The second half of the tutorial runs the notebook of Table 1 in Google Colab. Parts 0 to 4 are small and seeded and run on a central processing unit (CPU). Part 5 needs a graphics processing unit (GPU) and downloads the DINOv2 weights (Oquab et al., 2024) and the DAWN driving images (Kenk & Hassaballah, 2020); without a GPU, read the published blur sweep in Section 19 in place of Part 5's output. Part 0 downloads the six talk clips, `media.zip` (16 MB), from a Google Drive share link, so nothing needs to be mounted. Two

cautions apply.

- The toy parts explain robustness by the size of the rounding noise alone, the idealized picture of the worked example above. Section 20 tests that picture on the real encoder, so read your answer to the notebook's Exercise 3.2 against it.
- Part 5 runs one seed, and the notebook itself warns that any single point moves by about 2 percentage points between runs.

Exercises

Exercise I.1: Four bits instead of eight

Redo the worked example at 4 bits. (a) How many megabytes do the 20 million weights fill in 16-bit floating point (FP16) and in 4-bit integers (INT4), and by what factor is each smaller than FP32? (b) A 4-bit quantizer has integer levels -8 to 7. Which step s puts the lower end of its clip range at -4, what is the whole clip range, and what is η^2? (c) Find ρ and the signal-to-quantization-noise ratio in dB for directions of variance 1 and 10^{-2}. (d) At which variance λ does the ratio reach 1, at 8 bits and at 4 bits, and by what factor does each bit removed move that point?

Solution sketch. (a) FP16 takes 2 bytes per weight, 40 MB, half of FP32; INT4 takes half a byte, 10 MB, eight times less. (b) $s = 4/8 = 0.5$, the clip range is $[-4, 3.5]$, and $\eta^2 = 0.5^2/12 \approx 0.0208$, 256 times the 8-bit value because the step is 16 times larger. (c) For $\lambda = 1$, $\rho \approx 0.021$ and SQNR ≈ 16.8 dB, down from 40.9 dB. For $\lambda = 10^{-2}$, $\rho \approx 2.1$ and SQNR ≈ -3.2 dB, so the error now exceeds the signal. (d) The ratio is 1 where $\lambda = \eta^2$, at 8.1×10^{-5} for 8 bits and 0.0208 for 4 bits. Each bit removed doubles s and so raises that point fourfold, the 6.02 dB per bit of Section 4.

Exercise I.2: Designing a fair comparison

You want to know whether an 8-bit QAT encoder is more robust to blur than its full-precision twin. (a) List four things the two models must share so that a difference in accuracy on blurred crops can be charged to quantization alone. (b) A colleague argues that any gain could come from training with noise of the right size, not from quantization as such. Which third model would test that argument, and which result would support the colleague? (c) Why does one training run per model not suffice?

Solution sketch. (a) The same starting checkpoint; the same training steps, seeds, and data order; the same evaluation crops, blurred identically pixel for pixel; and the same metric. Anything else that differs is a second possible cause. (b) A full-precision model trained identically, with random noise of the quantizer's size injected where the quantizers sit. If it gains as much as the QAT model, noise size alone suffices to explain the gain; if it does not, the quantizer contributes something beyond the size of its error. (c) Two runs of the same recipe already differ by chance, so one pair cannot separate a real gain from seed noise; several seeds and an interval are needed (Section 13). Section 18 shows how one of the two papers builds such a comparison.

Check yourself before moving on

1. Which two demands on an in-vehicle perception model pull in opposite directions, and why is low-bit quantization expected to hurt the first?[a]
2. At 8 bits with step $s = 1/32$, is a direction of variance 10^{-3} above or below the flat floor, by how many dB, and why is the answer only as good as the model behind it?[b]
3. Name the four sections of the spine and the quantity that joins them.[c]

[a] Reliability when the camera image degrades, and efficient execution on embedded hardware under cost,

power, and heat limits. Quantization serves the second, and the error it adds to every number seems likely to compound with a degraded input.

[b]Above. $\rho = 8.1 \times 10^{-5}/10^{-3} \approx 0.081$, so SQNR ≈ 10.9 dB. The flat floor assumes the error adds the same variance to every direction, which measured residuals need not do (Section 8).

[c]The quantizer, the rounding-noise model, the spectrum, and what noise does to a spectrum (Sections 3, 4, 7 and 8), joined by the per-rank ratio $\rho_k = e_k/\lambda_k$ of residual energy to signal variance.

2 Numbers in hardware and what a bit costs

What you will be able to do after this section

Decode a two's-complement integer; encode and decode FP32, FP16, and BF16 values by hand, read off their range and precision, and tell normal from subnormal values; contrast the spacing of INT8 with that of the three float formats; compute weight memory (Equation (7)) and read W8A8; follow one INT8 dot product from its steps and codes through INT32 accumulation and requantization to the next layer; tell fake from true integer quantization, explain why the two agree in exact arithmetic and where they can differ, and say what each can measure; and use the roofline model and Amdahl's law (Equation (12)) to explain why INT8 deployments usually gain less than 2× over FP16.

Later sections study quantization as rounding onto a grid. This section is about the machine underneath. The bits of a value decide how many distinct values exist, how many bytes a tensor occupies and moves, and which arithmetic units can process it. The first is the cost; the other two are the reason anyone pays it.

2.1 Integers, a uniform ruler

A pattern of b bits $(\beta_{b-1}, \ldots, \beta_0)$, each $\beta_i \in \{0, 1\}$, takes 2^b values. Signed integers use *two's complement*, in which the top bit carries a negative weight,

$$v = -\beta_{b-1}\, 2^{b-1} + \sum_{i=0}^{b-2} \beta_i\, 2^i, \qquad v \in \{-2^{b-1}, \ldots, 2^{b-1} - 1\}. \tag{1}$$

For $b = 8$ this is the range $[-128, 127]$ of INT8, the 8-bit signed integer. The pattern 01111111 is 127, 10000011 is $-128 + 2 + 1 = -125$, and 11111111 is -1. The range is lopsided because zero uses up one of the patterns with a leading 0, which is why Section 3 writes the levels as $q_{\min} = -2^{b-1}$ and $q_{\max} = 2^{b-1} - 1$. Two wider formats appear later in this section, INT16 with the range $[-32768, 32767]$ and INT32 with $[-2^{31}, 2^{31} - 1]$, about $\pm 2.1 \times 10^9$.

Neighboring integers are exactly 1 apart everywhere. An integer code is not yet a real number, so a quantized tensor carries one real-valued *step* s with it, and the code q stands for the value sq. With $s = 0.02$, for example, the code 69 stands for 1.38, and the INT8 codes $-128, \ldots, 127$ stand for $-2.56, -2.54, \ldots, 2.54$. These values form a *uniform grid*, with the same gap s everywhere. Section 2.4 shows where the step comes from, and Section 3 what happens to values that fall between the grid points or beyond its ends.

2.2 Floats, a ruler whose ticks spread out

An integer code carries no scale of its own; the step s that gives it a meaning is stored once for a whole tensor. A floating-point number, or *float*, spends some of its bits on a scale of its own, so that every value carries its own power of two. A float has three fields, stored in this order: one *sign bit* S, an *exponent field* of n_e bits that holds an unsigned integer code c, and a

Table 2: The three float formats in numbers. Each row follows from Equation (2), from the subnormal value of Equation (6), or from the gap of Equation (3), with the formula in the second column, and the values are rounded to three significant figures. FP32 and BF16 share one exponent range and differ in precision; FP16 has three more mantissa bits than BF16 and a far smaller range.

Quantity	Formula	FP32	FP16	BF16
Bits $S + n_e + m$		$1+8+23$	$1+5+10$	$1+8+7$
Bias κ	$2^{n_e-1}-1$	127	15	127
Normal exponents j	$1-\kappa, \ldots, \kappa$	$-126, \ldots, 127$	$-14, \ldots, 15$	$-126, \ldots, 127$
Precision p in bits	$m+1$	24	11	8
Largest finite value	$2^{\kappa}(2-2^{-m})$	3.40×10^{38}	65504	3.39×10^{38}
Smallest normal value	$2^{1-\kappa}$	1.18×10^{-38}	6.10×10^{-5}	1.18×10^{-38}
Smallest subnormal value	$2^{1-\kappa-m}$	1.40×10^{-45}	5.96×10^{-8}	9.18×10^{-41}
Gap above 1	2^{-m}	1.19×10^{-7}	9.77×10^{-4}	7.81×10^{-3}
Unit roundoff u	2^{-m-1}	5.96×10^{-8}	4.88×10^{-4}	3.91×10^{-3}
Decimal digits	$p \log_{10} 2$	7.2	3.3	2.4
Every integer exact up to	2^{m+1}	16,777,216	2048	256

mantissa field (also called the fraction) of m bits that holds an unsigned integer F. A *normal* float has the value

$$x = (-1)^S \, 2^{c-\kappa} \left(1 + \frac{F}{2^m}\right), \qquad \kappa = 2^{n_e-1} - 1, \quad 1 \le c \le 2^{n_e} - 2 \tag{2}$$

(Goldberg, 1991; IEEE, 2019). Each term has its own role.

- The factor $(-1)^S$ is $+1$ for $S = 0$ and -1 for $S = 1$. Because the sign is a bit of its own, the format is symmetric about zero, unlike two's complement, and it has two zeros, $+0$ and -0.
- The exponent code c is an unsigned integer, and the *bias* κ turns it into the signed *exponent* $j = c - \kappa$. The allowed codes $1, \ldots, 2^{n_e} - 2$ give the exponents $1 - \kappa, \ldots, \kappa$, centered near zero, so a format reaches about as far below 1 as above it.
- The mantissa integer F runs from 0 to $2^m - 1$. The *significand* $1 + F/2^m$ then lies in $[1, 2)$. In binary it is the number $1.f_1 f_2 \ldots f_m$, whose digits after the point are the bits of F, each bit f_i adding 2^{-i}. The significand holds the digits of x, and 2^j moves its binary point.
- The leading 1 of the significand is not stored. In binary the only nonzero digit is 1, so every nonzero number written as $1.f_1 f_2 \cdots \times 2^j$ starts with it, and storing it would waste a bit. This *hidden bit* gives a normal float $p = m + 1$ significant bits, its *precision*.
- The range of c leaves out two codes. The code $c = 0$ holds zero and the subnormal values, and the code $c = 2^{n_e} - 1$ holds $\pm\infty$ and not-a-number (NaN), as Definition 2.1 below sets out.

FP32 (single precision) splits its 32 bits $1+8+23$ and FP16 (half precision) its 16 bits $1+5+10$, both as defined by IEEE 754 (IEEE, 2019). BF16 (bfloat16), which IEEE 754 does not define, splits $1 + 8 + 7$. It keeps the exponent field of FP32, and so its range, and gives up precision, so a BF16 pattern has the layout of the upper 16 bits of an FP32 pattern (Figure 2a). Table 2 collects what Equation (2) implies for the three formats, and the rest of this subsection derives each of its rows.

Reading a float by hand. Equation (2) is also a recipe for decoding a bit pattern. Patterns are usually written in hexadecimal, one hex digit for every four bits, so first expand each digit into its four bits. Decoding then takes four steps.

1. Split the bits into the fields S, c, and F, of 1, n_e, and m bits.
2. Convert c to decimal and subtract the bias, $j = c - \kappa$.
3. Read the mantissa bits as the significand $1.f_1 f_2 \dots f_m$ in binary.
4. Multiply, $x = (-1)^S \, 2^j \cdot 1.f_1 f_2 \dots f_m$.

Worked example, decoding one pattern in each format

Format	Hex	Fields S, c, F	$c \to j$	Significand	Value x
FP32	`C0D00000`	`1 10000001 101`$0 \cdots 0$	$129 \to 2$	$1.101_2 = 1.625$	$-4 \cdot 1.625 = -6.5$
FP16	`4A40`	`0 10010 1001000000`	$18 \to 3$	$1.1001_2 = 1.5625$	$8 \cdot 1.5625 = 12.5$
BF16	`3F40`	`0 01111110 1000000`	$126 \to -1$	$1.1_2 = 1.5$	$0.5 \cdot 1.5 = 0.75$

In the first row, `C0D00000` expands to `1100 0000 1101 0000` followed by sixteen zeros. This regroups as the sign bit 1, the exponent field `10000001`, which is 129, and the mantissa `101` followed by twenty zeros, so the significand is $1 + \frac{1}{2} + \frac{1}{8} = 1.625$. The FP16 significand is $1 + \frac{1}{2} + \frac{1}{16}$ in the same way. Two more FP16 patterns are worth remembering. `3C00` has $c = 15 = \kappa$ and $F = 0$, so it is 1, and `7BFF` has the largest normal code $c = 30$ and $F = 2^{10} - 1$, so it is $2^{15}(2 - 2^{-10}) = 65504$, the largest finite FP16 value. Because BF16 is the upper half of FP32, the FP32 pattern `3F400000` is 0.75 as well.

Writing a float by hand. Encoding runs Equation (2) backward. A real number x rarely has an exact float, so the format stores the float nearest to it, written $\mathrm{fl}(x)$, and breaks an exact tie toward an even F (round half to even, the default of IEEE 754). Seven steps find it.

1. Set $S = 0$ for $x \geq 0$ and $S = 1$ for $x < 0$, and continue with $|x|$.
2. Find the *octave* of $|x|$, the interval $[2^j, 2^{j+1})$ between neighboring powers of two that contains it, so that $j = \lfloor \log_2 |x| \rfloor$. Bracketing $|x|$ between two powers of two is enough, and $2^{10} = 1024 \approx 10^3$ helps for large and small numbers.
3. Divide by the lower power, $r = |x|/2^j$, which lies in $[1, 2)$.
4. Round the fraction to m bits, $F = \mathrm{round}\big((r - 1)\, 2^m\big)$, sending an exact tie to the even integer.
5. If the rounding gave $F = 2^m$, the significand has reached 2, so set $F = 0$ and raise j by one.
6. Store $c = j + \kappa$. If c exceeds $2^{n_e} - 2$, the value overflows to $\pm\infty$, and if c falls below 1, it is subnormal (Definition 2.1).
7. Check that $\mathrm{fl}(x) = (-1)^S \, 2^j (1 + F/2^m)$ lies within half a gap of x, where the gap is 2^{j-m} (Equation (3)).

Step 4 also works without a calculator. Doubling a fraction moves its binary point one place to the right, so double $r - 1$, write down the integer part as the next binary digit, and continue with the fractional part. Keep m digits and look at the first digit dropped. If it is 0, keep the digits as they are. If it is 1 and any later digit is 1, add 1 to the kept digits. If it is 1 and every later digit is 0, the value is a tie, and you add 1 only when the last kept digit is 1, which makes it even.

Worked example, 0.1 in FP32, FP16, and BF16

Since $2^{-4} = 0.0625 \le 0.1 < 0.125 = 2^{-3}$, the octave is $j = -4$ in all three formats, and $r = 0.1/0.0625 = 1.6$. Doubling the fraction 0.6 gives 1.2, 0.4, 0.8, 1.6, and then 1.2 again, so its binary digits are `1001` repeated without end, $0.6 = 0.1001\,1001\,1001\ldots_2$. Each format keeps its own number of digits.

Format	m	$0.6 \cdot 2^m$	F	c	Hex	fl(0.1)	Error
FP32	23	5033164.8	5033165	123	`3DCCCCCD`	0.1000000015	$+1.5 \times 10^{-9}$
FP16	10	614.4	614	11	`2E66`	0.0999755859	-2.4×10^{-5}
BF16	7	76.8	77	123	`3DCD`	0.1000976563	$+9.8 \times 10^{-5}$

In FP16 the eleventh digit of 0.6 is 0, so the ten kept digits `1001100110` stay as they are and $F = 614$. In BF16 the eighth digit is 1 and later digits are not all 0, so `1001100` becomes `1001101` and $F = 77$. The FP16 pattern is therefore `0 01011 1001100110` and the BF16 pattern `0 01111011 1001101`. Each error lies within half a gap, $2^{-4-m}/2$, which is 3.7×10^{-9}, 3.1×10^{-5}, and 2.4×10^{-4}. No binary format stores 0.1 exactly, because its binary digits never end.

Three cases need care. *A tie.* 50000 lies in the FP16 octave $j = 15$, since $32768 \le 50000 < 65536$, and $(r-1)\,2^{10} = 538.5$ is exactly halfway, so the even $F = 538$ wins and $\mathrm{fl}(50000) = 2^{15}(1 + 538/1024) = 49984$. *A carry.* For 1.9999 in FP16, $(r-1)\,2^{10} = 1023.9$ rounds to $1024 = 2^m$, so $F = 0$, $j = 1$, and $\mathrm{fl}(1.9999) = 2$. *Integers.* The gap equals 1 in the octave $j = m$, so every integer up to 2^{m+1} is exact, and above it the gap is 2 or more. FP16 stores every integer up to 2048, BF16 only up to 256, and FP32 up to $2^{24} = 16{,}777{,}216$. In FP16, 2049 is a tie between 2048 and 2050 and goes to 2048, so $2048 + 1$ returns 2048, and a count kept in BF16 stops growing at 256.

The gap between neighbors. Read Equation (2) one octave at a time. For a fixed exponent $j = c - \kappa$, the 2^m values of F give the floats $2^j + F\,2^{j-m}$, a uniform grid with step 2^{j-m} that starts at 2^j and stops one step short of 2^{j+1}, where the next octave starts with twice the step. Every octave $[2^j, 2^{j+1})$ receives the same 2^m values however wide it is, so the gap between a normal float x and its upper neighbor is

$$\mathrm{gap}(x) = 2^{\lfloor \log_2 |x| \rfloor - m}, \tag{3}$$

which doubles each time $|x|$ crosses a power of two (Figure 2b). The gap is the value of the last mantissa bit, which is why numerical analysts call it one *ulp* (unit in the last place). At a power of two it is lopsided. In FP16 the upper neighbor of 1 is $1 + 2^{-10} = 1.0009765625$, but the lower neighbor is $1 - 2^{-11} = 0.99951171875$, only half a gap away, because it lies in the octave below. Counting values works the same way. FP16 has 1024 values in $[1, 2)$ and as many in $[1024, 2048)$. Of the 31744 bit patterns that FP16 has for non-negative finite values, 15360 lie below 1.

Inside one octave a float is therefore an integer grid in disguise, and rounding to the nearest float is the uniform quantizer of Section 3 with a step chosen by the octave of x,

$$\mathrm{fl}(x) = 2^{j-m}\,\mathrm{round}\!\left(\frac{x}{2^{j-m}}\right), \qquad j = \lfloor \log_2 |x| \rfloor. \tag{4}$$

For positive x the integer $\mathrm{round}(x/2^{j-m})$ is $2^m + F$, or 2^{m+1} after a carry. This is often the fastest way to encode by hand. For 50000 in FP16 the step is $2^{15-10} = 32$, and $50000/32 = 1562.5$ rounds to the even 1562, so $\mathrm{fl}(50000) = 1562 \cdot 32 = 49984$ at once. For 1.9999 the step is 2^{-10}, and $1.9999 \cdot 1024 = 2047.9$ rounds to 2048, which gives 2 and handles the carry by itself. Seen this way, a float is a quantizer whose step follows the magnitude of its input, and an integer tensor with step s is a float whose exponent has been frozen and shared.

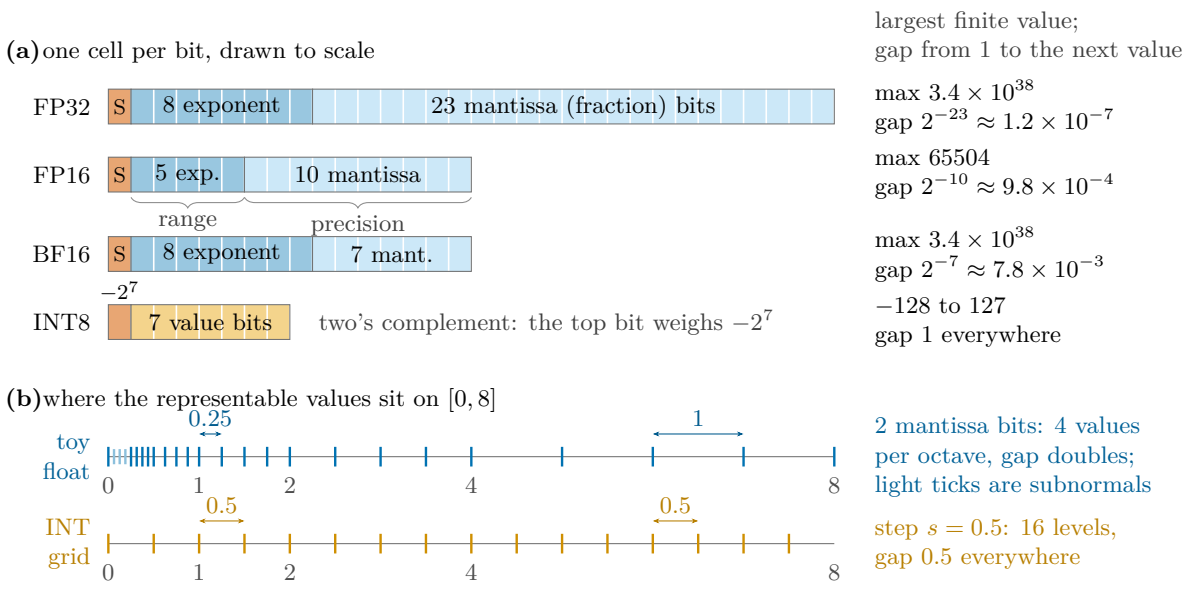


Figure 2: Where the bits go. (a) The four formats to scale, one cell per bit. The exponent sets the range and the mantissa the relative precision; BF16 keeps the range of FP32 and gives up precision, FP16 does the opposite, and INT8 has no exponent. (b) A toy float with $n_e = 3$ exponent bits and $m = 2$ mantissa bits ($\kappa = 3$), $x = 2^j(1 + F/4)$, puts four values in every octave, so its ticks crowd toward zero and spread out as x grows (Equation (3)). Below its smallest normal value 0.25, three subnormal values (Equation (6), light ticks) continue the step 0.0625 down to zero. An integer grid with step $s = 0.5$ has the same gap everywhere.

Relative precision. Relative to the value, the gap hardly changes. Dividing Equation (3) by $|x|$, with $2^j \le |x| < 2^{j+1}$, gives

$$2^{-m-1} < \frac{\text{gap}(x)}{|x|} \le 2^{-m}, \tag{5}$$

with the upper end at the start of each octave, $x = 2^j$, and the lower end approached at its top. Goldberg (1991) calls this factor of 2 the *wobble*. Rounding to the nearest float errs by at most half a gap, so every rounding obeys

$$\text{fl}(x) = x\,(1 + \delta), \qquad |\delta| \le u = 2^{-m-1},$$

where u is the *unit roundoff*. One rounding thus changes a value by at most 6.0×10^{-8} of itself in FP32, 4.9×10^{-4} in FP16, and 3.9×10^{-3} in BF16. In decimal terms, p significant bits are worth about $p \log_{10} 2 \approx 0.3\,p$ significant digits, 7.2 for FP32, 3.3 for FP16, and only 2.4 for BF16. The value 87.3 lies in the octave $[64, 128)$, where the BF16 gap is $2^{6-7} = 0.5$, so BF16 stores it as 87.5. FP16, whose gap there is $2^{6-10} = 0.0625$, stores 87.3125.

Resolving an ambiguity about "machine epsilon"

The name *machine epsilon* is given to two numbers that differ by a factor of 2. Goldberg (1991) defines it as $(\beta/2)\beta^{-p}$ for base β and precision p, the bound on the relative error of one rounding, which for a binary format is 2^{-m-1}, the unit roundoff u. Programming languages and libraries, among them C and NumPy, report the gap above 1, 2^{-m}, instead. The C constant `FLT_EPSILON` is 2^{-23} for FP32, and NumPy's `numpy.finfo(numpy.float16).eps` is 2^{-10}. These notes avoid the bare name and write "the gap above 1" for 2^{-m} and "the unit roundoff" for $u = 2^{-m-1}$.

The integer grid is the mirror image, a constant absolute gap s and a relative precision that degrades toward zero. For $x = sq$ the relative gap is $s/|x| = 1/|q|$, smallest at the clipping edge ($1/127 \approx 7.9 \times 10^{-3}$ for INT8) and growing to 10% at $|q| = 10$ and to 100% at $|q| = 1$. The best relative gap of INT8 is thus about the worst of BF16, $2^{-7} \approx 7.8 \times 10^{-3}$. INT8 keeps its relative gap at or below 2^{-6} only in its top octave, $|q| \geq 64$, whereas a float keeps it within a factor of 2 in every octave of its range.

▷ **Check yourself.** What is the FP16 gap near $x = 1000$, and near $x = 0.001$?[1]

Normal, subnormal, and special values. Equation (2) stops at the smallest normal value $2^{1-\kappa}$. The two codes it leaves out give the format its remaining values.

Definition 2.1: Normal, subnormal, and special floats

The exponent code c sorts every bit pattern of a float into one of four classes.

- *Normal*, $1 \leq c \leq 2^{n_e} - 2$, with the value of Equation (2).
- *Subnormal*, $c = 0$ and $F \neq 0$, with the value

$$x = (-1)^S \, 2^{1-\kappa} \, \frac{F}{2^m}, \tag{6}$$

  which has no hidden 1 and keeps the exponent $1 - \kappa$ of the smallest normal octave.
- *Zero*, $c = 0$ and $F = 0$, which is $+0$ or -0; the two compare equal.
- *Infinity or NaN*, $c = 2^{n_e} - 1$. With $F = 0$ the pattern is $\pm\infty$, the result of an overflow; with $F \neq 0$ it is not-a-number, the result of an invalid operation such as $0/0$ or $\infty - \infty$.

IEEE 754 defines the four classes (IEEE, 2019). Older texts, Goldberg (1991) among them, call subnormal values *denormalized*.

Subnormals repair a hole. Without them, the next value below $2^{1-\kappa}$ would be 0, a jump 2^m times wider than the gap just above it, 6.1×10^{-5} against 6.0×10^{-8} in FP16. Equation (6) fills $[0, 2^{1-\kappa})$ with the $2^m - 1$ values $F\,2^{1-\kappa-m}$, spaced exactly like the smallest normal octave, so the grid runs down to zero without a jump. This is why a subnormal keeps the exponent $1 - \kappa$ of the smallest normal octave rather than $-\kappa$, one lower (Goldberg, 1991). With $-\kappa$ its values would be spaced half as wide and would stop just below $2^{-\kappa}$, which would leave a new hole below $2^{1-\kappa}$. Equation (4) covers subnormals as well if j is not allowed below $1 - \kappa$.

The repair has a useful consequence. A result too small to be normal now loses precision one step $2^{1-\kappa-m}$ at a time instead of jumping to zero, which Goldberg (1991) calls *gradual underflow*, and it guarantees that $x - y = 0$ only when $x = y$. In FP16 the two smallest normal values, $2^{-14} + 2^{-24}$ and 2^{-14}, differ by 2^{-24}, the smallest subnormal, which is stored exactly. A format without subnormals would return 0 for this difference of two unequal numbers. The price is precision. Subnormals form a uniform grid with the constant step $2^{1-\kappa-m}$, an integer grid like that of Equation (1), so their relative precision falls toward zero as an integer grid's does. Each leading zero bit of F costs one significant bit, and the smallest subnormals keep only one or two.

The ends of the range. The extreme codes give the remaining rows of Table 2. The largest finite value takes the largest normal code $c = 2^{n_e} - 2$, so $j = 2^{n_e} - 2 - \kappa = \kappa$, and the largest

[1] $1000 \in [2^9, 2^{10})$ gives $2^{9-10} = 0.5$, and $0.001 \in [2^{-10}, 2^{-9})$ gives $2^{-20} \approx 9.5 \times 10^{-7}$. The relative gaps, 5×10^{-4} and 9.5×10^{-4}, differ by less than 2.

mantissa $F = 2^m - 1$, so the significand is $2 - 2^{-m}$,

$$x_{\max} = 2^{\kappa}\left(2 - 2^{-m}\right).$$

The smallest normal value is $2^{1-\kappa}$ ($c = 1$, $F = 0$), and the smallest subnormal value is $2^{1-\kappa-m}$ ($c = 0$, $F = 1$). For FP16 all three can be done by hand, $x_{\max} = 2^{15}(2 - 2^{-10}) = 2^{16} - 2^5 = 65504$, then $2^{-14} = 2^{-10} \cdot 2^{-4} \approx 0.000977 \cdot 0.0625 \approx 6.1 \times 10^{-5}$, and $2^{-24} \approx 6.0 \times 10^{-8}$. For FP32 the powers are too large to multiply out, so convert them with $\log_{10} 2 \approx 0.30103$. Then $2^{128} = 10^{38.53} \approx 3.4 \times 10^{38}$, which is $x_{\max}$ to three figures since $2 - 2^{-23}$ is almost 2, and $2^{-126} = 10^{-37.93} = 10^{0.07} \cdot 10^{-38} \approx 1.18 \times 10^{-38}$. BF16 has the same exponent range, and its $x_{\max} = 2^{128}(1 - 2^{-8}) \approx 3.39 \times 10^{38}$ differs from that of FP32 only in the third figure.

Rounding to nearest also decides what happens at the two ends. A value overflows to ∞ only from half a gap above $x_{\max}$, so in FP16, where the top gap is $2^{15-10} = 32$, 65519 rounds to 65504 but 65520 rounds to ∞. At the bottom, every value up to half the smallest subnormal, $2^{-25} \approx 3.0 \times 10^{-8}$ in FP16, rounds to zero.

Worked example, normal and subnormal values in FP16

With $\kappa = 15$ and $m = 10$, an FP16 subnormal is $F \cdot 2^{-14}/2^{10} = F \cdot 2^{-24}$, an integer times $2^{-24} \approx 5.96 \times 10^{-8}$, so decoding it takes one multiplication.

Hex	Fields S, c, F	Class	Value	Note
`0001`	`0 00000 0000000001`	subnormal, $F = 1$	$2^{-24} \approx 5.96 \times 10^{-8}$	smallest positive
`0200`	`0 00000 1000000000`	subnormal, $F = 512$	$2^{-15} \approx 3.05 \times 10^{-5}$	
`03FF`	`0 00000 1111111111`	subnormal, $F = 1023$	$\approx 6.0976 \times 10^{-5}$	largest subnormal
`0400`	`0 00001 0000000000`	normal, $c = 1$, $F = 0$	$2^{-14} \approx 6.1035 \times 10^{-5}$	smallest normal

From `03FF` to `0400` the value grows by 2^{-24}, the same step as everywhere below, so the grid passes from subnormal to normal without a jump. To encode $x = 10^{-5}$, note that it lies below 2^{-14}, so $c = 0$ and $F = \text{round}(10^{-5}/2^{-24}) = \text{round}(167.77) = 168$, which is `0010101000` in ten bits. The pattern is `0 00000 0010101000` (`00A8`), the stored value is $168 \cdot 2^{-24} \approx 1.0014 \times 10^{-5}$, and the relative error is 1.4×10^{-3}, about three times the unit roundoff of a normal FP16 value. The two leading zeros of F leave 8 significant bits, as many as BF16 has, and BF16 indeed stores 10^{-5} as the same $168 \cdot 2^{-24}$. Lower down, 10^{-7} gives $F = \text{round}(1.68) = 2$ and the value 1.19×10^{-7}, a relative error of 19%. FP32 and BF16 reach their subnormals only below 1.18×10^{-38}, so both store 10^{-5} and 10^{-7} as normal values, with relative errors below their unit roundoff.

2.3 What a bit costs in memory

Weight memory. If the parameters of a model fall into groups ℓ, and group ℓ holds P_ℓ values of b_ℓ bits each, the weights occupy

$$M_{\text{w}} = \frac{1}{8}\sum_{\ell} P_\ell\, b_\ell \text{ bytes}, \qquad \text{which is} \quad M_{\text{w}} = \frac{P\,b}{8} \quad \text{when all } P \text{ parameters use } b \text{ bits.} \tag{7}$$

The steps add one number per tensor, or one per output channel. For a layer with n_{in} inputs, per-channel steps are a fraction $1/n_{\text{in}}$ of its weights, small enough to ignore here.

Worked example, a detector in bytes

A synthetic object detector has $P = 20.0$M parameters. With 1 MB $= 10^6$ bytes, Equation (7) gives 80.0 MB in FP32, 40.0 MB in FP16, and 20.0 MB in INT8, the ideal ratios

4× and 2×.

The weights are not the only bytes that move. The activations do not count in M_{w}, but every layer reads its input and writes its output for every image, and 8-bit activations halve that traffic relative to FP16 (Section 2.6 returns to it).

A tag WxAy gives the bit widths of the weights (W) and of the activations (A), meaning the inputs of each quantized layer. W8A8 quantizes both to 8 bits, the case that Section 2.4 follows, and W4A8 lowers only the weights. A weight-only format such as W4A16 saves memory but not arithmetic. An integer unit multiplies two integers, so with FP16 activations the 4-bit weights must be converted back to FP16 before each product, and the multiplication runs in floating point. Which layers carry the tag, and with which granularity of steps, is set out in Sections 3 and 5.

2.4 An integer dot product

This subsection follows one output of one quantized layer through an integer kernel, from the float inputs to the 8-bit code that the next layer reads (Figure 3). A *kernel* is one compiled routine that a processor, such as a GPU or a CPU, runs as one unit, usually for one operation such as a matrix product. A network runs as a sequence of kernels. An *integer kernel* reads integer codes, here INT8, and computes their products and sums in integers. Section 2.5 then compares this integer path with the way quantization is usually simulated. Every linear and convolution layer reduces to dot products. A linear layer with weight matrix W computes each output as $y_i = \sum_j W_{ij} x_j$, and a convolution computes the same kind of sum over each patch of its input. It is therefore enough to follow one dot product,

$$y = \sum_{j=1}^{K} w_j x_j,$$

where $w = (w_1, \ldots, w_K)$ holds the trained weights of one output and $x = (x_1, \ldots, x_K)$ is the input vector, for example the $K = 384$ features of one token in a small vision transformer. The five steps below are the five stages of Figure 3, from left to right.

Step 1, the operands become codes. To quantize a tensor v with step s, divide it by the step, round to the nearest integer, and clamp the result to the INT8 range,

$$q = \mathrm{clamp}\big(\mathrm{round}(v/s),\, -128,\, 127\big), \qquad v \approx s\, q.$$

The clamp replaces a rounded value below -128 by -128 and one above 127 by 127, and it leaves every value in between unchanged, so the code always fits in 8 bits. The integer q is the *code* that is stored and computed with, and sq is the real value that it stands for. Section 3 studies this map in full, including the error it makes. Here it is enough that every quantized number is an integer code times a step.

Where the steps come from. A step is fixed before deployment, once for each tensor, so that the grid covers the values the tensor takes. The simplest rule, *max-abs*, maps the largest magnitude onto the largest code, $s = \max |v|/127$, so that no value is clipped. Weights and inputs differ in when their values are known.

- The weights are fixed once training ends. Their step s_w comes from the weights themselves, often with one step for each output, so the codes $q_{w,j}$ are computed once, offline, and stored together with s_w. The float weights are not needed at run time.

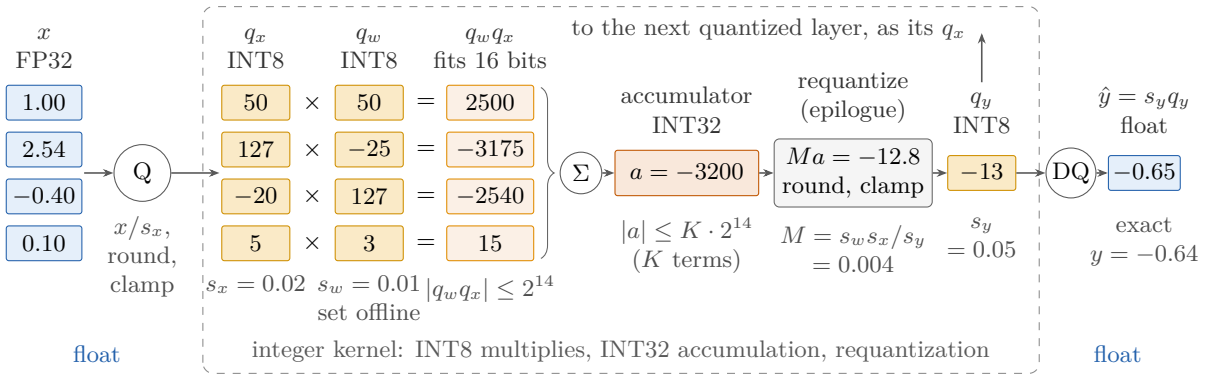


Figure 3: One INT8 dot product, with the numbers of the worked example. The weight codes q_w were computed offline with the max-abs step s_w, and a quantize node (Q) turns the float inputs into the codes q_x at run time. Each product of two codes fits 16 bits, their sum needs a 32-bit accumulator, and the epilogue rescales it by $M = s_w s_x / s_y$, rounds, and clamps it to INT8. The code q_y goes on to the next quantized layer as its input code, or, where a float operation follows, through a dequantize node (DQ) back to a float. Inside the dashed box the arithmetic is integer (M is applied as a 64-bit integer multiply and a shift), and after the inputs the only rounding is in the epilogue.

- The inputs change with every image. Their step s_x is fixed in advance, either from the activations that a small *calibration set* of images produces (post-training quantization) or by training (quantization-aware training), as Section 6 explains. At run time a *quantize* (Q) node turns each incoming float x_j into the code $q_{x,j}$ with this fixed step.

In the worked example below the largest weight magnitude is 1.27, and the largest input magnitude seen on the calibration set is 2.54, so $s_w = 1.27/127 = 0.01$ and $s_x = 2.54/127 = 0.02$.

Step 2, multiply and accumulate in integers. Substitute $w_j \approx s_w q_{w,j}$ and $x_j \approx s_x q_{x,j}$ into the dot product,

$$y \;\approx\; \sum_{j=1}^{K} (s_w q_{w,j})(s_x q_{x,j}) \;=\; s_w s_x \sum_{j=1}^{K} q_{w,j}\, q_{x,j} \;=\; s_w s_x\, a. \tag{8}$$

The steps leave the sum because each is the same for all K terms, s_w for the weights of this output and s_x for the whole input vector. A step that changed from term to term would stay inside the sum, which is why weight steps may differ between outputs but not within one (Section 3 returns to this). What remains is the *accumulator* $a = \sum_j q_{w,j} q_{x,j}$, a sum of products of integers that integer units compute exactly. It needs more bits than its terms. A product of two INT8 codes lies between $-128 \cdot 127 = -16256$ and $(-128)^2 = 2^{14}$, so it fits in 16 bits, and K products can add up to $K \cdot 2^{14}$. For $K = 384$, the token width of DINOv2 ViT-S/14 (Oquab et al., 2024), a small Vision Transformer (ViT, Section 5), that is 6.3×10^6, far beyond INT16 and far inside INT32, which stays safe for every K up to 131071. Integer hardware therefore multiplies 8-bit codes and accumulates in 32 bits (Jacob et al., 2018; Nagel et al., 2021).

Step 3, what the accumulator stands for. The accumulator is itself a code. Its step is $s_w s_x$, so the integer a stands for the real number $s_w s_x\, a$, the dot product of the quantized operands. A layer's bias b is added at this point, in the same units, as the INT32 code $\mathrm{round}(b/(s_w s_x))$, so that the sum still stands for $s_w s_x$ times an integer (Jacob et al., 2018, Sec. 2.4). With $s_w s_x = 0.0002$, for example, a bias of 0.3 becomes the code 1500.

Step 4, requantize for the next layer. The next quantized layer reads INT8 codes on its own input grid. Its step s_y is set in the same way, from the range that this layer's outputs take on the calibration set. *Requantization* converts the accumulator to that grid. Divide the real value $s_w s_x a$ by s_y, round, and clamp,

$$q_y = \mathrm{clamp}(\mathrm{round}(M\,a),\, -128,\, 127), \qquad M = \frac{s_w s_x}{s_y}. \tag{9}$$

This is the quantizer of Step 1 with step s_y, applied to $s_w s_x a$. It does in one integer operation what two float operations would do, recovering the real value and quantizing it again, and only one real number enters it, the multiplier M, which is known before deployment.

An integer-only processor has no floating-point unit, so it cannot multiply by M directly. It replaces M by a fraction with an integer numerator and a power-of-two denominator,

$$M \approx \frac{M_0}{2^n}, \qquad M_0 = \mathrm{round}(2^n M), \qquad \text{so that} \qquad M\,a \approx \frac{M_0\,a}{2^n}.$$

Both M_0 and a are integers, so the numerator takes one ordinary integer multiplication. The division by 2^n is a right shift by n bits. The shift drops the n lowest bits of $M_0\,a$, just as dropping the last two digits of a decimal number divides it by 100. The sign is $\approx$ because most real numbers have no exact form $M_0/2^n$, among them the $M = 0.004$ of the worked example below. Like M, the integers M_0 and n are fixed before deployment. The last item of the worked example runs the multiplication and the shift by hand.

Real kernels choose n so that M_0 lies between 2^{30} and 2^{31}, and they make the shift round to the nearest integer, as Equation (9) asks (Jacob et al., 2018, Sec. 2.2). With M_0 that large, the fraction $M_0/2^n$ matches M to better than one part in 2^{30}, about one in a billion, which is at least 30 bits of relative accuracy. A plain right shift rounds down, and adding 2^{n-1} before the shift is one way to turn this into rounding to the nearest integer. Because M_0 alone takes 31 bits, the product $M_0\,a$ is computed in 64 bits, twice the width of the accumulator. All of this happens in the kernel's *epilogue*, its last stage, which processes the accumulator before it writes the result out.

An activation function between the two layers fits into this step when it is a clamp. The rectified linear unit, $\mathrm{ReLU}(u) = \max(u, 0)$, commutes with a positive step, $\mathrm{ReLU}(s_y q) = s_y\,\mathrm{ReLU}(q)$, so it becomes a clamp of q_y at 0 and stays in integers, which is how Jacob et al. (2018) fuse ReLU and ReLU6, $\min(\max(u, 0), 6)$, into the epilogue. Smooth functions such as the Gaussian error linear unit (GELU), the softmax, and LayerNorm (layer normalization, Section 5) need real arithmetic. Their input is dequantized, the function runs in floating point, and its output is quantized again for the next integer layer.

Step 5, on to the next layer. The codes q_y are the input codes of the next quantized layer, whose input step is s_y. A chain of quantized layers therefore hands INT8 codes from one integer kernel to the next without leaving the integers. A *dequantize* (DQ) node, which multiplies codes by their step, $\hat{y} = s_y q_y$, is needed only where a floating-point operation follows or where the network ends.

Worked example, one INT8 dot product by hand

Take one output of a layer with $K = 4$ inputs, the trained weights $w = (0.50, -0.25, 1.27, 0.03)$, and the input $x = (1.00, 2.54, -0.40, 0.10)$, and follow Figure 3 from left to right.

- *Steps.* Max-abs gives $s_w = 1.27/127 = 0.01$. On the calibration set the inputs reach 2.54 in magnitude and the outputs 6.35, so $s_x = 2.54/127 = 0.02$, and the next layer's input step is $s_y = 6.35/127 = 0.05$.

- *Codes.* Offline, $q_w = \text{round}(w/0.01) = (50, -25, 127, 3)$. At run time the Q node computes $q_x = \text{round}(x/0.02) = (50, 127, -20, 5)$. No value is clipped, and every value happens to lie on its grid, so quantizing the operands costs nothing here.
- *Accumulate.* The products $50 \cdot 50 = 2500$, $(-25) \cdot 127 = -3175$, $127 \cdot (-20) = -2540$, and $3 \cdot 5 = 15$ all exceed INT8 and fit INT16. Their sum is $a = -3200$, which stands for $s_w s_x a = 0.0002 \cdot (-3200) = -0.64$, the float result exactly. The bias is 0 here.
- *Requantize.* $M = 0.01 \cdot 0.02/0.05 = 0.004$ and $Ma = -12.8$, which rounds to $q_y = -13$. This code goes on to the next layer. It stands for $\hat{y} = 0.05 \cdot (-13) = -0.65$, an error of 0.01, below $s_y/2 = 0.025$.
- *In integers only.* A short multiplier that you can handle by hand takes $n = 15$ and $M_0 = \text{round}(0.004 \cdot 2^{15}) = \text{round}(131.07) = 131$. The epilogue computes $-3200 \cdot 131 = -419200$ and shifts it right by 15 bits, which divides it by 32768 and gives -12.79, rounded to the same -13.

After the inputs, the only rounding happens at the end, in the epilogue.

2.5 Fake and true integer quantization

The integer path of Section 2.4 is how a quantized network runs once it is deployed. It is not how a quantized network is trained, and it is rarely how quantization is studied. Training and most studies run an ordinary floating-point network that *simulates* the integer path, for three reasons.

- Training needs gradients. Rounding has a zero derivative almost everywhere, so training replaces it by a surrogate in the backward pass (Section 6), a float computation that integer inference kernels do not provide. The weights also stay in floating point during training, so that small gradient steps can move them, and only their rounded copies enter the forward pass (Jacob et al., 2018, Sec. 3).
- Integer kernels exist only for the bit widths and step granularities that a given chip and its software support. A simulation can try any bit width, for example 6-bit or 3-bit weights, and any granularity.
- A simulation runs on the hardware used for training, and the same code can test many quantization options, layer by layer. It is much easier to build than a run on quantized hardware or with quantized kernels (Nagel et al., 2021, Sec. 2.3).

Definition 2.2: Fake and true integer quantization

A *quantize* (Q) node divides by a step, rounds, clamps, and emits codes; a *dequantize* (DQ) node multiplies codes by their step and emits floats. *Fake* (simulated) quantization applies DQ directly after Q, with the same step and nothing in between,

$$x \;\mapsto\; s\,\text{clamp}\big(\text{round}(x/s),\, q_{\min},\, q_{\max}\big),$$

and keeps the result as a float. A fake-quantized tensor is therefore a float tensor whose values lie on the grid of its step, at most 2^b distinct values, while its storage and arithmetic stay in FP32 or FP16. *True* integer quantization stores the codes themselves and runs integer kernels with INT32 accumulation and requantization (Equations (8) and (9)) (Gholami et al., 2022; Jacob et al., 2018; Krishnamoorthi, 2018).

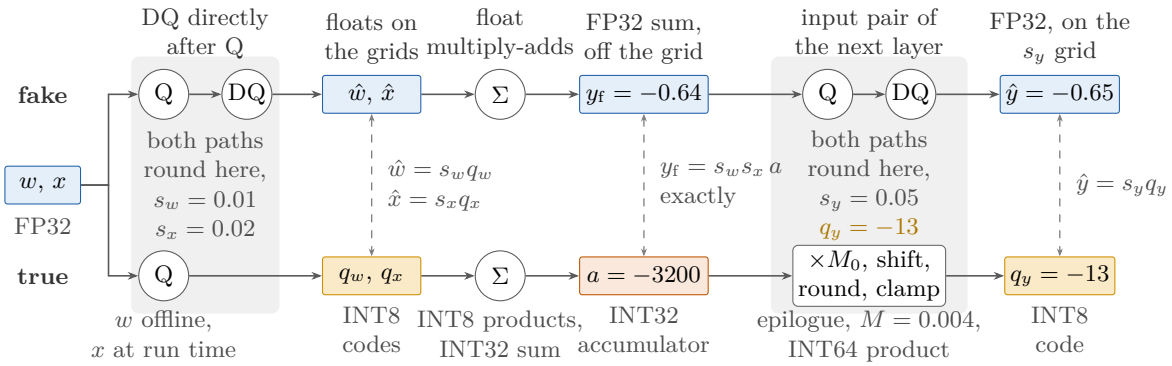


Figure 4: The layer of Figure 3 on the two paths of Definition 2.2, with the numbers of the worked example. Blue boxes hold floats, and orange boxes hold integers. Dashed arrows are not operations. Each arrow links a float to the integer that stands for it. Both paths round only in the gray bands, with the same steps, and $y_f = s_w s_x a$ exactly, so both reach the same code, $q_y = -13$. On integer-only hardware, the epilogue applies $M \approx M_0/2^n$ as a 64-bit product $M_0\, a$ and a right shift by n bits (Step 4 of Section 2.4). The true path needs no DQ node, since $M = s_w s_x / s_y$ applies the steps of the first DQ node and the next quantized layer reads q_y directly. The fake path computes the same numbers with float tensors, so it saves no memory and no time.

Other texts call fake quantization *simulated quantization* (Jacob et al., 2018, Sec. 3) or *quantization simulation* (Nagel et al., 2021, Sec. 2.3). They call the true path *integer-arithmetic-only* (Jacob et al., 2018) or *on-device* (Nagel et al., 2021, Fig. 4) quantized inference, or, informally, real quantization. These notes write fake and true.

Where the fake nodes sit. A fake-quantized layer is the float layer with a Q-DQ pair at each place where the integer layer rounds (Jacob et al., 2018, Sec. 3). Each node of the fake layer then stands for one operation of the integer layer, as Figure 4 shows with the numbers of the worked example.

- The pair on the weights, with step s_w, turns each w_j into the float $\hat{w}_j = s_w q_{w,j}$. It stands for the codes $q_{w,j}$, which the integer path computes once, offline.
- The pair on the input, with step s_x, turns each x_j into $\hat{x}_j = s_x q_{x,j}$. It stands for the Q node that runs on every input.
- The float sum of products, $y_f = \sum_j \hat{w}_j \hat{x}_j$, stands for the accumulator. Like the real value $s_w s_x a$ of the accumulator, it lies on no 8-bit grid.
- The input pair of the next quantized layer, with step s_y, rounds y_f onto the next grid. It stands for requantization (Equation (9)).

Only the pairs round, and everything between them is ordinary float arithmetic. The fake layer therefore carries the rounding of the integer layer but none of its integer storage or arithmetic. A pair belongs exactly where the integer path rounds, no more and no less, and Section 5 marks these places in a whole vision transformer.

Table 3 sets the two paths side by side for one layer. They start from the same steps and the same codes, and they differ in what they keep and in what they cost.

Why the two paths agree. Factor the steps out of the fake path's float sum,

$$y_f = \sum_j \hat{w}_j \hat{x}_j = \sum_j (s_w q_{w,j})(s_x q_{x,j}) = s_w s_x \sum_j q_{w,j} q_{x,j} = s_w s_x\, a.$$

Table 3: One quantized layer on the two paths of Definition 2.2, with the clamps omitted. Both paths use the same steps and codes; the last row gives the result for the worked example.

	Fake (simulated) quantization	True integer quantization
Weights	floats on the grid, $\hat{w}_j = s_w q_{w,j}$	INT8 codes $q_{w,j}$ and one step s_w
Inputs	floats on the grid, $\hat{x}_j = s_x q_{x,j}$	INT8 codes $q_{x,j}$
Products, sum	float multiply-adds, $y_{\mathrm{f}} = \sum_j \hat{w}_j \hat{x}_j$, each addition rounded	integer multiply-adds, $a = \sum_j q_{w,j} q_{x,j}$, exact in INT32
Bias	a float b	the INT32 code $\mathrm{round}\big(b/(s_w s_x)\big)$
Output	the float $\hat{y} = s_y\,\mathrm{round}(y_{\mathrm{f}}/s_y)$, from the input pair of the next layer	the INT8 code $q_y = \mathrm{round}(Ma)$, with $M \approx M_0/2^n$
Runs on	any GPU or CPU, at any bit width, with gradients	integer units, at the bit widths they support
Bytes per weight	4 in FP32, 2 in FP16	1, plus the steps
Speed	no gain, since every pair adds work to the float layer	from integer units and fewer bytes (Section 2.6)
Measures	what the network predicts	what the network predicts and what quantization saves
Worked example	$y_{\mathrm{f}} = -0.64$, so $\hat{y} = -0.65$	$a = -3200$, so $q_y = -13$, which stands for -0.65

This is Equation (8) again, now as an exact identity. The fake sum equals the real value that the true accumulator stands for, and dividing it by s_y gives $y_{\mathrm{f}}/s_y = (s_w s_x/s_y)\,a = Ma$. The two paths therefore round the same number Ma and produce the same code, -13 in the worked example. In exact arithmetic, and with its pairs where the integer path rounds, fake quantization computes what an integer deployment computes.

Why they can still differ. Two kinds of difference separate the paths in practice. The first lives in the last bits of each number. The second changes the function itself.

Last-bit differences. Computer arithmetic is not exact, and three approximations separate the two paths.

1. The float sum rounds after every addition, to 24 significant bits in FP32 and to 11 in FP16 (Table 2), whereas INT32 adds integers exactly.

2. The true path rounds two numbers that the fake path keeps, the bias to a code with step $s_w s_x$ and the multiplier M to $M_0/2^n$.

3. Operations that stay unquantized, such as the softmax and LayerNorm, often run in FP16 on the device but in FP32 in a simulation.

These differences are tiny next to one step s_y, so they change a code only when Ma lies very close to a rounding boundary, halfway between two integers. There a tiny difference moves q_y by one code. Suppose that the accumulator of the worked example had been $a = -3126$. The fake path rounds $Ma = -12.504$ to -13, while the epilogue with the short multiplier $131/2^{15}$ computes $-3126 \cdot 131/32768 = -12.497$ and rounds it to -12, one step $s_y = 0.05$ away. Real kernels shrink these zones a great deal. A 31-bit multiplier changes Ma by less than 10^{-8} here, and FP32 rounding in the fake path by about 10^{-6}. A single output rarely falls that close to a boundary, but a network computes millions of outputs, so a few codes can still flip, and each flipped code is a different input for the next layer.

Placement differences. The identity above assumes that both paths round at the same places. A deployment that rounds at other places computes another function, and the difference is then not confined to the last bits. Three cases show how this happens.

- A layer is quantized on one path only. A deployment may keep a layer in FP16 because no integer kernel exists for it, or round a tensor that the simulation keeps in floating point, such as the sum of a residual connection, which an integer network may quantize (Jacob et al., 2018, Sec. 3). One path then rounds once more than the other.
- Operations are fused differently. A deployment that fuses an addition with the ReLU after it rounds only once, after both, so a simulation must not put a pair between them (Krishnamoorthi, 2018, Sec. 3.2.1). A pair there would round a tensor that the deployment never rounds, on a grid set for a different range.
- A granularity has no kernel. Step 2 of Section 2.4 showed that a step may change between outputs but not along the sum. A simulation with one step for each input feature therefore has no direct integer counterpart, and its deployment must round onto other grids.

Jacob et al. (2018, Sec. 2.4) state the rule that prevents such differences. The operations that the inference code fuses into one integer kernel must match the placement of the fake quantizers in the training graph. Nagel et al. (2021, Sec. 2.3.3) note that the mismatch between a simulation and the hardware sometimes comes from layers that are not properly quantized. Unlike a last-bit difference, a placement difference does not shrink with a longer multiplier or a wider float format.

What each path can measure. Fake quantization reproduces the numbers of an integer deployment, not its savings. A fake-quantized network keeps every tensor in FP32 or FP16, so it needs as many bytes as the float network. It also performs all the float arithmetic of the float network plus the rounding of every pair, so it runs no faster. The two paths therefore support different measurements.

- The fake path measures what the quantized network predicts, for example its accuracy, its probability calibration, and its robustness to corrupted images. These depend only on the values that the network computes, which the fake path reproduces up to the differences above.
- Only a deployment measures what quantization saves in memory, latency, and energy, since only a deployment stores codes and runs the kernels that read them (Section 2.6).

A run on true integer kernels also checks the first item. Evaluate the integer network, its fake-quantized simulation, and the full-precision network on the same inputs. The accuracy of the integer network minus that of its simulation is the *deploy-versus-simulation drift*, and it links the effect of quantization measured in simulation to the effect in deployment,

$$\underbrace{A_{\text{dep}} - A_f}_{\text{deployed effect}} = \underbrace{A_{\text{sim}} - A_f}_{\text{simulated effect}} + \underbrace{A_{\text{dep}} - A_{\text{sim}}}_{\text{drift}}, \tag{10}$$

where A_{dep}, A_{sim}, and A_f are the accuracies of the deployed integer network, of its simulation, and of the full-precision network. The equation is an identity, since A_{sim} cancels on the right, and it holds for any other metric as well. A simulated effect therefore carries over to the deployment when the drift is small compared with it.

Worked example, drift against effect

The numbers are synthetic. A quantized classifier scores 2.0 percentage points of accuracy above its full-precision version in simulation, and its integer deployment scores 0.3 points below the simulation, a drift of -0.3 points. Equation (10) gives a deployed effect of $2.0 - 0.3 = +1.7$ points, so the gain survives deployment. A simulated gain of 0.2 points with the same drift gives $0.2 - 0.3 = -0.1$ points, a small loss, so that gain does not carry

over. Both differences are measured with noise, from the training seeds and the finite test set, and Section 13 shows how to attach an interval to each. Neither says anything about speed.

▷ **Check yourself.** A fake-quantized network runs 10% slower than its float version on the same GPU. Does this show that INT8 is slow on that GPU?[2]

Resolving an ambiguity about "quantized model"

A text that reports results for "the quantized model" may mean either path. Its size and latency must come from a deployment, which stores codes, while its accuracy may come from a fake-quantized simulation. Before you compare two such numbers, or two such texts, find out which path each number comes from and, for a simulated accuracy, whether a deploy-versus-simulation drift (Equation (10)) was measured.

2.6 Where speed comes from

A layer's time is set by three things, the arithmetic it performs, the bytes it moves between memory and the processor, and fixed overheads such as kernel launches and format conversions. INT8 can help with the first two and hurt the third. Most arithmetic sits in *GEMMs* (general matrix multiplies), the kernels behind linear and convolution layers.

The roofline. The roofline model (Williams et al., 2009) bounds the time of one kernel by its two resources. A kernel that performs N_{op} operations and moves N_{B} bytes, on a chip with peak arithmetic rate P and memory bandwidth B, needs at least

$$t = \max\left(\frac{N_{\text{op}}}{P},\ \frac{N_{\text{B}}}{B}\right), \tag{11}$$

because arithmetic and memory traffic can overlap, but neither can run faster than its peak. The ratio $I = N_{\text{op}}/N_{\text{B}}$, the *arithmetic intensity*, decides which term wins. The two terms are equal at the *ridge point* $I = P/B$. A kernel with I below it is *memory-bound*, and one with I above it is *compute-bound*.

Worked example, a linear layer on the roofline

Take a linear layer with $n = 384$ inputs and outputs, and a synthetic chip with an FP16 rate of $P = 10^{14}$ operations per second, twice that in INT8, and $B = 10^{12}$ bytes per second, so that P/B is 100 operations per byte for FP16 and 200 for INT8. Applied to T tokens, the layer performs $2Tn^2$ operations (one multiply and one add per weight and token). It moves its n^2 weights and the $2Tn$ values of its input and output, $(n^2 + 2Tn)\,b/8$ bytes at b bits.

- *One token.* The layer performs 294912 operations and moves 296448 bytes in FP16, so $I \approx 1$, and INT8 halves the bytes, so $I \approx 2$. Both lie far below their ridge points, so both are memory-bound. The times are 0.30 and 0.15 μs, and INT8 gains 2× because it moves half the bytes.
- *256 tokens.* The layer performs 7.55×10^7 operations and moves 688128 bytes in FP16, so $I \approx 110$, above 100, and INT8 gives $I \approx 219$, above 200. Both are compute-bound. The times are 0.75 and 0.38 μs, and INT8 gains 2× because it computes twice as fast.

[2] No. The fake network performs all the float work of the float network plus the rounding of every pair, so it runs no faster than the float network, and its time says nothing about integer kernels (Section 2.6).

The gain is 2× in both regimes, for two different reasons. On a chip whose INT8 rate only equaled its FP16 rate, the second case would gain nothing, since its time is set by arithmetic.

So the gain of one GEMM does not grow with its size. A memory-bound GEMM gains the byte ratio, 2× against FP16, and a compute-bound one gains the ratio of the chip's INT8 to FP16 peak rates, which its data sheet states. The rest of the network and every fixed overhead gain little or nothing. This is why end-to-end ratios usually fall below the GEMM ratio, and why they tend to rise as GEMMs take a larger share of the time, in wider models and larger batches. Amdahl's law below makes this precise.

Fusion and overheads. Overheads are where INT8 can lose. An FP16 graph *fuses* a matrix product, its bias, and its activation into one kernel, so the intermediate results never travel to memory and back. Every Q or DQ node can break such a chain into separate kernels, each with its own launch and its own memory traffic. An INT8 path can then end up slower than FP16, unless the rescaling and the activation are folded into the epilogue of the integer kernel, as in Step 4 of Section 2.4 (Figure 3).

Amdahl's law. If a fraction f of a baseline cost (time, or bytes) is reduced by a factor r and the rest is untouched, the overall gain is

$$G = \frac{1}{(1-f) + f/r} \;\le\; \frac{1}{1-f} \tag{12}$$

(Amdahl, 1967). *Proof.* With the baseline cost normalized to 1, the untouched part still costs $1-f$ and the improved part f/r; G is the reciprocal of their sum, and the bound is the limit $r \to \infty$. □

Worked example, why 1.85× and not 2×

Suppose only the GEMM layers of the synthetic detector become INT8, holding 18.4M of its 20.0M parameters, a fraction $f = 0.92$ of its 40.0 MB in FP16, and the rest stays FP16. It then takes $18.4 + 1.6 \cdot 2 = 21.6$ MB, and $40.0/21.6 = 1.85$. Equation (12) gives the same number, $G = 1/(0.08 + 0.92/2) = 1.85$, and reaches 2× only if every byte becomes INT8. Read backward, a measured size ratio gives $f = 2(1 - 1/G)$. For time, f and r are both unknown (Exercise I.6).

Baselines. A speed-up means nothing without its baseline. Suppose that an optimized inference engine runs a model in FP16 four times faster than a plain framework runs it in FP32, and that the same engine runs it in INT8 1.5 times faster than in FP16. Against the plain FP32 framework the INT8 engine is then $4 \cdot 1.5 = 6$ times faster, although precision accounts for only 1.5× of it. Only the same software stack in both precisions isolates the gain of precision, and every speed-up in these notes states its baseline.

Running example: the bus in the rain

The camera quantizes first. The bus crop of Section 1 arrives as 224×224 pixels in three 8-bit channels, 150,528 bytes of integers in $[0, 255]$, and reaches the encoder as numbers. Follow one synthetic activation, $x = 1.3712$, into a layer whose INT8 step is $s = 0.02$, with the encoding recipe of Section 2.2 and Step 1 of Section 2.4. FP16 stores $c = 15$ and $F = \text{round}(0.3712 \cdot 2^{10}) = 380$, the value 1.37109375, off by -1.1×10^{-4}. BF16 stores $c = 127$ and $F = \text{round}(0.3712 \cdot 2^{7}) = 48$, the value 1.375, off by $+3.8 \times 10^{-3}$. INT8 stores

$q = \text{round}(68.56) = 69$, the value 1.38, off by $+8.8 \times 10^{-3}$, inside $s/2$ but about 80 times the FP16 error. INT8 buys fewer bytes and integer arithmetic, not precision. Whether the precision it gives up matters for telling the bus from a truck is what the rest of Part I takes apart.

In the hands-on session

Every quantizer in the notebook is fake (Definition 2.2). In Part 5, `ActQuantizer` and `quantize_weight` round onto $[-128, 127]$ and `QLinear` calls a float matrix product. Count with `torch.unique` the distinct values of a quantized activation (at most 256), then those of a `QLinear` output, which, like the accumulator of Figure 3, lies on no 8-bit grid until the next quantizer rounds it. Then call `quantize_weight` on a weight matrix and check `element_size()` of the result, the bytes per value. It is still 4, since a fake-quantized weight is an FP32 tensor, which is why the notebook can measure accuracy and robustness but not the memory or speed of an INT8 network.

Exercises

Exercise I.3: Two's complement and the accumulator

(a) Write the INT4 range and decode 0111, 1000, and 1010 with Equation (1). (b) What is the largest magnitude of a product of two INT4 codes, and how many such products can an INT16 accumulator sum with no risk of overflow? (c) Why is FP16 a poor accumulator for INT8 products, although its largest value 65504 exceeds the product bound 2^{14}?

Solution sketch. (a) $[-8, 7]$; 7, -8, and $-8 + 2 = -6$. (b) $(-8)(-8) = 64$. The positive side binds, $32767/64 = 511.98$, so 511 products are always safe. (c) FP16 holds every integer only up to $2^{11} = 2048$ (Table 2). Above that its gap is 2 or more, so a running sum in the thousands is rounded at almost every step, and beyond 65504 it overflows.

Exercise I.4: Floats by hand

(a) Encode $x = -6.3$ in FP16 and in BF16. Give S, c, F, the stored value, and its error, and check the error against half a gap. (b) Decode the FP16 pattern `0 00011 0100000000` and the BF16 pattern `1 10000101 0101111`. (c) Which of 10^{-6} and 10^{-4} is subnormal in FP16? Encode that one and give its relative error.

Solution sketch. (a) $4 \le 6.3 < 8$ gives $j = 2$, $r = 1.575$, and $S = 1$. In FP16, $0.575 \cdot 2^{10} = 588.8$ gives $F = 589$ and $c = 17$, so $\text{fl}(x) = -6.30078125$, an error of -7.8×10^{-4} within half a gap, $2^{2-10}/2 \approx 2.0 \times 10^{-3}$. In BF16, $0.575 \cdot 2^{7} = 73.6$ gives $F = 74$ and $c = 129$, so $\text{fl}(x) = -6.3125$, an error of -1.25×10^{-2} within $2^{2-7}/2 \approx 1.6 \times 10^{-2}$. (b) FP16, $c = 3$ and $j = -12$, with significand $1.01_2 = 1.25$, so $x = 1.25 \cdot 2^{-12} \approx 3.05 \times 10^{-4}$. BF16, $S = 1$, $c = 133$, $j = 6$, and $F = 47$, so $x = -64\,(1 + 47/128) = -87.5$. (c) 10^{-6} lies below $2^{-14} \approx 6.1 \times 10^{-5}$, so it is subnormal, with $F = \text{round}(10^{-6}/2^{-24}) = \text{round}(16.78) = 17$ and the value $17 \cdot 2^{-24} \approx 1.013 \times 10^{-6}$, a relative error of 1.3%. 10^{-4} is normal.

Exercise I.5: Requantization by hand

A layer's largest weight magnitude is 0.635. On the calibration set its inputs reach 5.08 in magnitude and its outputs 12.7. (a) Find s_w, s_x, and s_y with the max-abs rule, and the multiplier M of Equation (9). (b) Write M as $M_0/2^{16}$ with an integer M_0. (c) For the accumulators $a = 2500$ and $a = -8751$, find q_y on the fake path (exact M) and on the true path ($M_0/2^{16}$), and explain any difference. (d) Which INT32 code stores a bias of 0.3?

Solution sketch. (a) $s_w = 0.635/127 = 0.005$, $s_x = 5.08/127 = 0.04$, $s_y = 12.7/127 = 0.1$, so $M = 0.005 \cdot 0.04/0.1 = 0.002$. (b) $0.002 \cdot 65536 = 131.07$, so $M_0 = 131$. (c) For $a = 2500$, $Ma = 5$ and $2500 \cdot 131/65536 = 4.997$ both give 5. For $a = -8751$, $Ma = -17.502$ gives -18 but $-8751 \cdot 131/65536 = -17.492$ gives -17. Ma lies near a half-integer, where the small error of the short multiplier moves the code by one. (d) $\text{round}\big(0.3/(0.005 \cdot 0.04)\big) = 1500$.

Exercise I.6: Reading a latency ratio with Amdahl's law

A synthetic detector takes 6.0 ms per image in FP16 and 4.0 ms with INT8 GEMMs, in the same stack. (a) If INT8 GEMMs are twice as fast ($r = 2$), what fraction f of the FP16 time do GEMMs take? (b) What is the smallest f for any r? (c) Why can the two times alone not settle f? (d) A plain framework runs the same detector in FP32 in 24 ms. What speed-up would a report against that baseline print for INT8, and how much of it is due to precision?

Solution sketch. (a) $G = 6.0/4.0 = 1.5$, and Equation (12) solved for f gives $f = (1 - 1/G)/(1 - 1/r)$, here $(1/3)/(1/2) \approx 0.67$. (b) As $r \to \infty$, $f \to 1 - 1/G \approx 0.33$; for $r = 4$, $f \approx 0.44$. (c) One ratio gives one equation in two unknowns; separating them takes a per-layer profile. (d) $24/4.0 = 6\times$, of which precision accounts for $1.5\times$, the ratio within one stack; the other factor of 4 is the engine's own gain over the plain framework.

Check yourself before moving on

1. Why does the gap between neighboring floats double at every power of two, while an integer grid's gap is constant?[a]
2. What changes in Equation (2) for a subnormal value, and why does its exponent stay at $1 - \kappa$?[b]
3. A 60M-parameter model keeps 5% of its parameters in FP16 and the rest in INT8. How many megabytes do its weights take, and how much smaller is that than the all-FP16 model?[c]
4. Why must an INT8 dot product accumulate in 32 bits, and what does requantization do?[d]
5. Why do fake and true integer quantization give the same code in exact arithmetic, and what can make them differ in practice?[e]

[a] Each octave gets the same 2^m mantissa values however wide it is (Equation (3)); the integer grid has one step s for its whole range.

[b] The hidden 1 becomes a 0, so $x = (-1)^S 2^{1-\kappa} F/2^m$ (Equation (6)). With the exponent of the smallest normal octave, the subnormals keep that octave's step $2^{1-\kappa-m}$ and fill $[0, 2^{1-\kappa})$ without a hole.

[c] $60\,(0.95 + 0.05 \cdot 2) = 63.0$ MB against 120 MB, a ratio of 1.90, which is Equation (12) with $f = 0.95$ and $r = 2$.

[d] Two products of up to 2^{14} can already overflow INT16. Requantization rescales the accumulator by M, rounds, and clamps it onto the next INT8 grid (Equation (9)).

[e] Both round the same number. The fake float sum $\sum_j (s_w q_{w,j})(s_x q_{x,j})$ equals $s_w s_x a$, and dividing it by s_y gives Ma. In the last bits they can differ because the float sum rounds at every addition, because the true path rounds the bias and M, or because unquantized operations run in different float formats, and a code flips only when Ma lies very near a half-integer. A deployment that rounds at other places than the simulation computes another function.

Where this is used. Sections 16 and 19 read measured deployments with Equations (7) and (12), their baselines, and the deploy-versus-simulation drift of Equation (10).

3 The uniform quantizer

What you will be able to do after this section

Quantize a number and a short vector by hand with Equation (13), writing down the stored integer code and the value it stands for. Tell the bounded rounding error from the unbounded clipping error on a staircase plot, and explain why at a fixed bit width the step and the clip range are one knob. Write the affine form with a zero point. Explain per-tensor, per-channel, and per-token steps, and why an integer matrix product wants per-channel steps on weights and per-token or per-token steps on activations.

Section 2 built the target of quantization, a grid of integers such as the 256 INT8 codes from -128 to 127, and showed why an integer kernel wants its operands there (Section 2.4). This section builds the map that takes a real number onto that grid and back. It is one short formula with one real parameter, the step s, and all of the "less precise" in the title of these notes lives inside it. Every later statement about rounding noise (Sections 4 and 8) is a statement about what this map does. The presentation follows the standard treatments (Gholami et al., 2022; Krishnamoorthi, 2018; Nagel et al., 2021).

3.1 Divide, round, clamp, rescale

Definition 3.1: Symmetric uniform quantizer

Fix a bit width b, the integer range $q_{\min} = -2^{b-1}$ and $q_{\max} = 2^{b-1} - 1$, and a step $s > 0$. The *symmetric uniform quantizer* maps a real number x to

$$Q(x) \;=\; s \cdot \operatorname{clamp}\Big(\operatorname{round}(x/s),\; q_{\min},\; q_{\max}\Big), \tag{13}$$

where round rounds to the nearest integer, and clamp limits its first argument u to the interval $[q_{\min}, q_{\max}]$ given by the other two,

$$\operatorname{clamp}(u, q_{\min}, q_{\max}) \;=\; \min\big(\max(u, q_{\min}),\, q_{\max}\big) \;=\; \begin{cases} q_{\min}, & u < q_{\min}, \\ u, & q_{\min} \le u \le q_{\max}, \\ q_{\max}, & u > q_{\max}. \end{cases}$$

In Equation (13) the clamped argument is the rounded value $u = \operatorname{round}(x/s)$. The resulting integer $q = \operatorname{clamp}(\operatorname{round}(x/s), q_{\min}, q_{\max})$ is the *code* that is stored and multiplied by an integer kernel, and $Q(x) = sq$ is the real value it stands for (the dequantized value). The quantization error is $e(x) = Q(x) - x$.

Read Equation (13) from the inside out, as four operations.

1. Divide by s, which measures x in steps.
2. Round, which snaps that measurement to the nearest integer $u = \operatorname{round}(x/s)$.[3]
3. Clamp, which keeps u inside the range of the b-bit codes. The inner $\max(u, q_{\min})$ raises a value below $q_{\min}$ up to $q_{\min}$ and leaves every other value as it is. The outer $\min(\cdot\,, q_{\max})$ then lowers a value above $q_{\max}$ down to $q_{\max}$. A value that already lies in $[q_{\min}, q_{\max}]$ passes both operations unchanged, so the clamp acts only on values that the codes cannot hold, and it *saturates* them at the nearer end of the range.

[3] Exact ties such as 2.5 need a rule. NumPy and PyTorch round half to even, so $\operatorname{round}(2.5) = 2$ and $\operatorname{round}(3.5) = 4$. Ties have probability zero for continuous inputs, and the examples here avoid them.

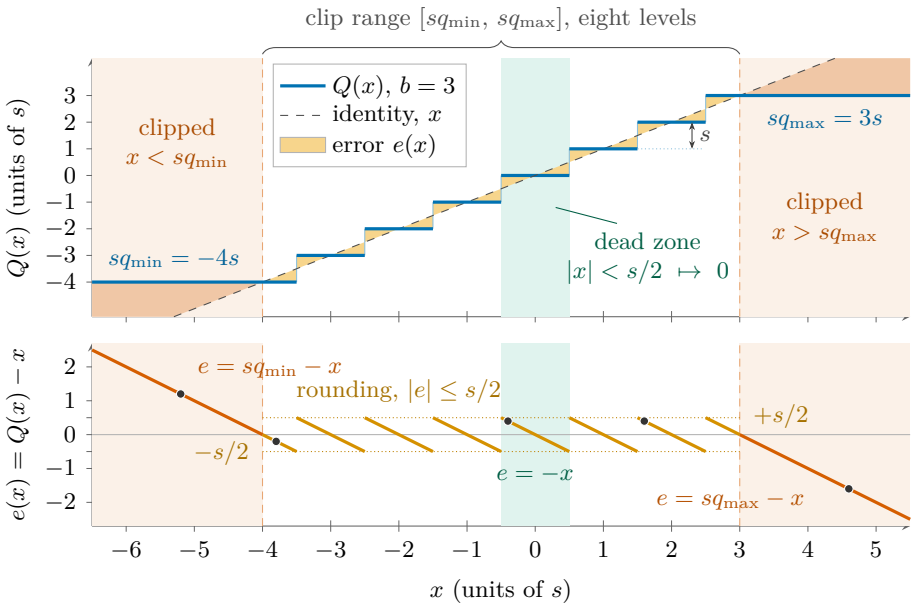


Figure 5: The 3-bit symmetric mid-tread quantizer of Equation (13), in units of the step. Top, the staircase $Q(x)$ and the identity line, with the error $e(x) = Q(x) - x$ as the shaded gap between them, yellow inside the clip range $[-4s, 3s]$ and red outside it. Bottom, the signed height of that gap, a sawtooth within $\pm s/2$ inside the clip range and a line that grows without bound outside it. Green band, the dead zone $|x| < s/2$, where $Q(x) = 0$ and $e = -x$. Dots, the five inputs of the worked example. The bottom panel is the whole story of the section, a bounded error set by the step and an unbounded one set by the range.

4. Multiply by s, which returns to real units.

Operations 1 to 3 form the Q node of Definition 2.2, and operation 4 is the DQ node, so $Q(x)$ is what a fake quantizer computes. From here on, a node labeled Q in a figure stands for this whole pair.

A 3-bit instance with $s = 0.5$ has $q_{\min} = -4$, $q_{\max} = 3$, and the levels $-2.0, -1.5, \ldots, 1.5$. The table follows five inputs through the four operations, and its clamp column computes $\min(\max(u, -4), 3)$.

x	x/s	$u = \text{round}(x/s)$	$q = \text{clamp}(u, -4, 3)$	$Q(x) = sq$	$e(x)$
0.80	1.60	2	2	1.00	+0.20
−0.20	−0.40	0	0	0.00	+0.20
−1.90	−3.80	−4	−4	−2.00	−0.10
2.30	4.60	5	3	1.50	−0.80
−2.60	−5.20	−5	−4	−2.00	+0.60

In the first three rows u already lies in $[-4, 3]$, so the clamp changes nothing, and the error is at most $s/2 = 0.25$. In the second row the code is 0, so the output is 0 and the error, +0.20, is the input with its sign flipped. In the last two rows the clamp acts. For $x = 2.30$, the inner $\max(5, -4) = 5$ leaves the value as it is, and the outer $\min(5, 3) = 3$ lowers it to $q_{\max}$. For $x = -2.60$, the inner $\max(-5, -4) = -4$ raises the value to $q_{\min}$, and the outer $\min(-4, 3) = -4$ leaves it there. Both errors exceed $s/2$, and the rest of the section is organized around that difference.

Figure 5 draws this quantizer for every input, with both axes in units of the step, so that the picture holds for any s. The graph of Q is a staircase. Each flat *tread* holds the inputs that the quantizer sends to one level, the tread of an inner level k running from $(k - \frac{1}{2})s$ to

$(k + \frac{1}{2})s$, and each vertical *riser* sits halfway between two levels. The error $e(x) = Q(x) - x$ is the vertical gap between the staircase and the dashed identity line, shaded in the top panel, and the bottom panel plots its signed height. On the tread of level k the output stays at ks while x grows, so $e = ks - x$ falls with slope -1, from $+s/2$ at the left end of the tread to $-s/2$ at its right end. At the riser the output rises by one step, and the error jumps back to $+s/2$. Each tread therefore adds one tooth to a sawtooth.

Past the highest level, $sq_{\max} = 3s$, no riser follows. The output stays at $3s$, and the error $3s - x$ keeps falling without bound. Below the lowest level, $sq_{\min} = -4s$, the same happens with the sign reversed. These two levels are the ends of the clip range $[-4s, 3s]$. The outermost treads reach half a step into it, so the first and the last tooth are halves, and the error does not jump where clipping begins. The dots in the bottom panel are the five rows of the table, placed at x/s (the second column) and at the height e/s, twice the tabulated error. The first row lies on a full tooth, the second on the tooth of level 0, the third on the half tooth at $-4s$, and the last two on the clipping lines.

▷ **Check yourself.** With the same quantizer, what are $Q(1.24)$ and $Q(1.26)$?[4]

Three properties of Equation (13) matter later.

- *Uniform grid.* The 2^b levels $sq_{\min}, \ldots, sq_{\max}$ are equally spaced, one step apart. Unlike the floating-point spacing of Section 2.2, the spacing does not grow with magnitude, so the same absolute error bound $s/2$ holds for small and large inputs alike.
- *Symmetric.* There is no offset, so $Q(0) = 0$ and $Q(-x) = -Q(x)$ as long as neither side is clipped. The level set is still one level short at the top, because two's complement has one more negative code than positive ones, $-128s$ to $127s$ at $b = 8$.
- *Mid-tread.* Zero sits in the middle of a tread, so zero is a level. A *mid-rise* quantizer instead puts a riser at zero and has the levels $\pm s/2, \pm 3s/2, \ldots$, with no level at zero.

The tread of level 0, the inputs with $|x| < s/2$, is called the *dead zone* (the green band in Figure 5). Every input on it is sent to exactly 0, so its error is $e = -x$, the input with its sign flipped, as in the second row of the table. The size of this error is not special, since the tooth of the dead zone is as large as any other. What is special is that the output is 0 whatever the input, so nothing of the input survives, not even its sign. On any other tread inside the clip range the output has the sign of the input. A weight in the dead zone adds nothing to a dot product, as if it had been pruned. The first property explains why small inputs suffer most. The bound $s/2$ is the same for every input inside the clip range, so the relative error $|e|/|x| \le s/(2|x|)$ grows as the input gets smaller, and on the dead zone it is 100% for every input. Section 4.3 returns to the dead zone. An error equal to $-x$ is not noise added to the signal; it removes the signal.

Learned step size quantization (LSQ; Esser et al., 2020) uses the map of Equation (13), up to the order of rounding and clamping (Exercise I.7), and makes s a trainable parameter (Section 6); the hands-on notebook codes it line for line.

3.2 The clip range and the two kinds of error

The two regimes of Figure 5 hold at every bit width, and the next box states them in general, with a proof.

[4] $1.24/0.5 = 2.48$ rounds to 2, so $Q(1.24) = 1.0$ with $e = -0.24$; $1.26/0.5 = 2.52$ rounds to 3, so $Q(1.26) = 1.5$ with $e = +0.24$. An input change of 0.02 moves the output by a full step. The quantizer is piecewise constant, so its derivative is zero almost everywhere, and training through it needs the straight-through estimator of Equation (27).

Clip range and the two error regimes. For the quantizer of Equation (13),

$$\text{clip range} \;=\; \big[\, sq_{\min},\; sq_{\max} \,\big] \;=\; \big[\, -2^{b-1}s,\; (2^{b-1}-1)s \,\big], \tag{14}$$

and the error obeys

$$e(x) = \begin{cases} s\,\mathrm{round}(x/s) - x, & \text{with } |e(x)| \le s/2, \quad sq_{\min} \le x \le sq_{\max} \quad \text{(rounding)}, \\ sq_{\max} - x \;< 0, & x > sq_{\max} \quad \text{(clipping)}, \\ sq_{\min} - x \;> 0, & x < sq_{\min} \quad \text{(clipping)}. \end{cases}$$

Proof. Write $u = x/s$. If $q_{\min} \le u \le q_{\max}$, then $\mathrm{round}(u)$ also lies in $[q_{\min}, q_{\max}]$, because both endpoints are integers and rounding cannot carry a number past an integer. The clamp therefore does nothing, and $|e| = s\,|\mathrm{round}(u) - u| \le s/2$. If $u > q_{\max}$, then $\mathrm{round}(u) \ge q_{\max}$, the clamp returns $q_{\max}$, and $Q(x) = sq_{\max}$ whatever x is. The case $u < q_{\min}$ is the mirror image. □

Two conventions. These notes write the error as quantized minus original, $e = Q(x) - x$, and some papers write it the other way round. An energy or a variance does not depend on the sign, but the cross term $\mathbb{E}[x\,e]$ of Section 4 does, so here e always means $Q(x) - x$. And quantization theory calls the two regimes the *granular* and the *overload* region (Gray & Neuhoff, 1998), where the neural-network literature says rounding and clipping error (Nagel et al., 2021). These notes use the second pair of names.

The asymmetry between the two regimes is the point. Rounding error never exceeds $s/2$, while clipping error is the distance past the edge, so one large input can cost many steps (28 in the running example below). The obvious fix, a wider range, is not free, for a reason plain from Equation (14).

The step and the range are one knob. At a fixed bit width the clip range is s times two fixed integers. Halving s halves the rounding bound and quarters its variance, and it also halves the clip range. No choice of s lowers both errors at once. Choosing s is called *range calibration* (not to be confused with the probability calibration of Section 12), and every rule for it in Section 6, from max-abs to a learned step, picks a point on this trade-off (Figure 6).

For example, INT8 with $s = 0.02$ has the clip range $[-2.56, 2.54]$ and the rounding bound 0.01. Halving the step to 0.01 halves the bound to 0.005, but the range shrinks to $[-1.28, 1.27]$. An input of 2.0, which the coarser grid stored exactly, is now clipped to 1.27, an error of -0.73, which is 146 times the new rounding bound.

Worked example on the optimum step

Take $x \sim \mathcal{N}(0, 1)$ and $b = 4$, the setting of Figure 6, so the levels run from $-8s$ to $7s$. The mean squared error averages the squared error over the inputs,

$$\mathbb{E}\big[e^2\big] \;=\; \int_{-\infty}^{\infty} e(x)^2\, \varphi(x)\, dx,$$

with φ the standard normal density. In the terms of Figure 5, redrawn for $b = 4$, it squares the bottom panel and weights each input by its density. Splitting the integral at the edges of the clip range gives the two colored curves of Figure 6, the *rounding part* over $[-8s, 7s]$ and the *clipping part* over the two tails outside it, and their sum is the total. Each step gives one point on each curve. An input spread evenly over a cell of width s has a rounding error

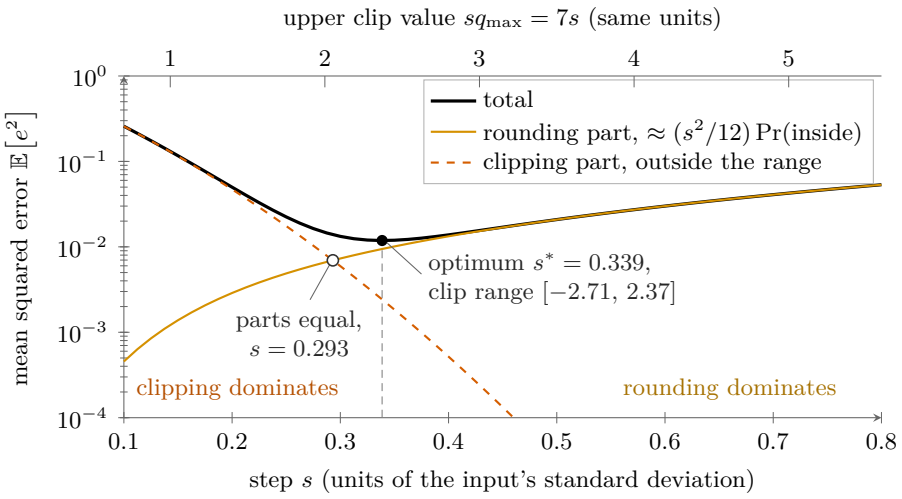


Figure 6: Mean squared error of the 4-bit quantizer ($q_{\min} = -8$, $q_{\max} = 7$) against its step, for a standard Gaussian input, on a logarithmic axis. Each curve averages $e(x)^2$ over the input (Figure 5 draws e for $b = 3$), the rounding part inside the clip range and the clipping part outside it. The top axis gives the upper edge $7s$ of the clip range, the same knob in other units. Small steps clip and large steps round coarsely. The total is lowest at $s^* = 0.339$, not where the two parts are equal (open circle), since there the clipping part still falls faster than the rounding part rises. Computed from the stated model (rounding part by quadrature, clipping part in closed form).

of mean square $\frac{1}{s}\int_{-s/2}^{s/2} t^2\,dt = s^2/12$ (Section 4 gives the assumptions), so the rounding part is about $(s^2/12)\Pr(-8s \le x \le 7s)$. An input past an edge is sent to the edge, so an edge at $a > 0$ adds the clipping part

$$\int_a^\infty (x-a)^2\,\varphi(x)\,dx \;=\; (1+a^2)(1-\Phi(a)) - a\,\varphi(a),$$

with Φ the standard normal distribution function. The identity follows from $\varphi'(x) = -x\varphi(x)$, which gives $\int_a^\infty x\varphi(x)\,dx = \varphi(a)$ and, by parts, $\int_a^\infty x^2\varphi(x)\,dx = a\varphi(a) + 1 - \Phi(a)$; expand $(x-a)^2$ and add the three terms. The upper edge is $a = 7s$, and, because the Gaussian is symmetric, the lower edge $-8s$ contributes the same expression at $a = 8s$. Numerical minimization of the sum gives $s^* = 0.339$, a clip range of $[-2.71, 2.37]$, and a total of 0.01186 (rounding 0.00944, clipping 0.00242). The optimum is not where the two parts are equal (the open circle, $s = 0.293$, where each is 0.00695) but where their slopes cancel, where a slightly larger step adds as much rounding error as it removes clipping error. At the crossing the clipping part still falls about three times as fast as the rounding part rises (slopes -0.155 and $+0.051$), so a larger step still lowers the total; at s^* the slopes are ± 0.058. The logarithmic axis hides this, because on it a curve looks steeper the smaller its value. The optimum clips the 1.2% of inputs above 2.37 or below -2.71 on purpose.

The same minimization at other bit widths gives the table below.

b	s^*	clip range	inputs clipped	$\mathbb{E}[e^2]$	rounding share of $\mathbb{E}[e^2]$
2	1.048	$[-2.10, 1.05]$	16.5%	0.149	52%
3	0.602	$[-2.41, 1.81]$	4.4%	0.0406	71%
4	0.339	$[-2.71, 2.37]$	1.2%	0.0119	80%
6	0.104	$[-3.33, 3.23]$	0.10%	0.00104	87%
8	0.0308	$[-3.94, 3.91]$	0.009%	0.0000877	90%

Read the table from top to bottom. With more bits the optimal clip range widens, fewer inputs are clipped, and rounding takes a growing share of the error. Because the range widens, each extra bit divides the error by only 3.4 to 3.7, not by the 4 of a fixed range, where one more bit halves the step and quarters $s^2/12$. Section 4 restates this in decibels. The max-abs rule of Section 2.4 never clips the data it is set on, so it picks a larger step than s^*; Section 6 measures what that costs.

Open question on what matched noise leaves out

LSQ trains s on the training loss (a task or distillation loss) (Esser et al., 2020), not on the quantizer's own rounding-plus-clipping error, so nothing pins a learned step to the optimum above, and every move of it trades rounding against clipping. A natural test of whether only the size of the rounding error matters is to train with additive uniform noise of the same variance in place of the quantizer. Equation (14) shows what such noise copies and what it leaves out. It copies only the size of the rounding error. It is redrawn at every pass and blind to the signal, whereas rounding is a fixed function of the input (the dead zone above), and it has no counterpart of the clipping regime. A learned step moves rounding and clipping at once, so changing the step cannot separate them. Which of these differences matters for a network under corrupted inputs is open. Section 4 lists them, and Section 20 returns to them.

Running example: the bus in the rain

At the input of one quantized linear layer of the encoder, one token of the bus crop carries 384 activations. Six of them are (synthetic numbers) $x = (2.54, -0.731, 0.0093, -1.2468, 0.445, -0.0162)$. Suppose one step is shared by the whole tensor and was set by max-abs range calibration on clean crops, whose largest activation here was 2.54, so $s = 2.54/127 = 0.02$ and the clip range is $[-2.56, 2.54]$.

x	x/s	code q	$Q(x)$	$e(x)$
2.5400	127.000	127	2.54	0.0000
−0.7310	−36.550	−37	−0.74	−0.0090
0.0093	0.465	0	0.00	−0.0093
−1.2468	−62.340	−62	−1.24	+0.0068
0.4450	22.250	22	0.44	−0.0050
−0.0162	−0.810	−1	−0.02	−0.0038

The INT8 kernel multiplies the codes $(127, -37, 0, -62, 22, -1)$ and applies 0.02 once for the tensor. Every error is within $s/2 = 0.01$; the third entry sits in the dead zone, so its error is minus its value. The mean squared error of the six is 4.2×10^{-5}, the order of $s^2/12 = 3.3 \times 10^{-5}$ (Section 4 explains why).

Now suppose a rain streak drives the first entry to 3.10. Then $3.10/0.02 = 155$ is clamped to 127, the value is flattened to 2.54, and the error is −0.56, which is 28 steps and 56 times the rounding bound $s/2 = 0.01$. The range fixed on clean crops is the range the deployed encoder keeps, and whether such flattening helps or hurts recognition under corruption is one form of the open question above.

3.3 Shifting the grid with a zero point

A symmetric grid wastes codes on a tensor that is mostly of one sign. The output of a Gaussian error linear unit (GELU), which feeds the second linear layer of the multilayer perceptron (MLP) in every Vision Transformer (ViT) block (Dosovitskiy et al., 2021), never goes below

about -0.17. A symmetric grid scaled to its largest value keeps almost all of its negative codes for values that never occur, so nearly half of the grid sits unused. The fix is an integer offset, the *zero point* z, which slides the codes so that they cover the range the tensor actually takes.

Affine (asymmetric) quantizer. With an integer zero point z,

$$Q_{s,z}(x) \;=\; s\Big(\mathrm{clamp}\big(\mathrm{round}(x/s) + z,\; q_{\min},\; q_{\max}\big) - z\Big), \tag{15}$$

with clip range $[s(q_{\min} - z),\; s(q_{\max} - z)]$; $z = 0$ gives back Equation (13). To cover a range $[x_{\min}, x_{\max}]$ that contains 0, take $s = (x_{\max} - x_{\min})/(2^b - 1)$ and $z = q_{\min} - \mathrm{round}(x_{\min}/s)$. The real value 0 maps to the code z and back to exactly 0.

The recipe in the box follows from counting. The 2^b codes span $2^b - 1$ steps, which must cover the width $x_{\max} - x_{\min}$, and z shifts the codes so that $x_{\min}$ lands on $q_{\min}$. Exact zero matters because convolutions pad their inputs with zeros, and a zero that came back as a small nonzero value would add a spurious signal along every border, so Jacob et al. (2018, Sec. 2.1) make it a requirement.

The price is arithmetic. Let the weight codes q^W_{ij} have the zero point z_w and the input codes q^x_j the zero point z_x. The integer sum of output i then expands into four terms (Jacob et al., 2018, Sec. 2.3),

$$\sum_{j=1}^{n}(q^W_{ij} - z_w)(q^x_j - z_x) = \sum_{j=1}^{n} q^W_{ij} q^x_j \;-\; z_x \sum_{j=1}^{n} q^W_{ij} \;-\; z_w \sum_{j=1}^{n} q^x_j \;+\; n\, z_w z_x.$$

The first term is the product a symmetric kernel computes. The second depends only on the weights, so it is computed once, offline, and folded into the bias, and the fourth is a constant. Only the third depends on the input, so it costs extra work for every input, and it vanishes when $z_w = 0$. This is why a common compromise is symmetric weights with affine activations (Nagel et al., 2021).

Worked example on a GELU output at INT8, symmetric against affine

Let the input of the second MLP layer range over $[-0.17, 5.93]$ (the maximum is synthetic).

- *Symmetric.* The step is $5.93/127 = 0.04669$. The data reach down only to the code $\mathrm{round}(-0.17/0.04669) = -4$, so they use the 132 codes from -4 to 127 out of 256.
- *Affine.* The step is $(5.93 + 0.17)/255 = 6.10/255 = 0.02392$, and the zero point is $z = -128 - \mathrm{round}(-0.17/0.02392) = -128 - \mathrm{round}(-7.107) = -121$. All 256 codes now cover the data. By the box the clip range is $[s(q_{\min} - z),\; s(q_{\max} - z)] = [-7s,\; 248s] = [-0.167,\; 5.93]$, and the smallest value, -0.17, rounds onto its lower end.

The affine step is 1.95 times finer, so its rounding variance $s^2/12$ is 3.8 times smaller. The input $x = 0$ gets the code $\mathrm{round}(0) + z = -121$ and returns exactly 0. The input $x = 1.0$ gets the code $\mathrm{round}(41.80) - 121 = -79$ and returns $s\,(-79 + 121) = 42s = 1.0047$, where the symmetric grid returns $21 \cdot 0.04669 = 0.9806$.

Code often writes Equation (15) as fake quantization (Definition 2.2) in three steps, with the step called the scale (Jacob et al., 2018). It quantizes, $x_{\mathrm{q}} = \mathrm{round}(x/\mathrm{scale} + \mathrm{zero_point})$, clamps x_{q} to $[q_{\min}, q_{\max}]$, and dequantizes, $(x_{\mathrm{q}} - \mathrm{zero_point}) \times \mathrm{scale}$. With scale $= s$ and zero_point $= z$ this is Equation (15), because $\mathrm{round}(u + z) = \mathrm{round}(u) + z$ for an integer z away from exact ties (Exercise I.7).

3.4 One step for how many numbers? Granularity

The step of a quantized tensor is shared by a group of its numbers, and the size of that group is the *granularity*. Three granularities are common.

- *Per-tensor*, one step for a whole weight matrix or activation tensor.
- *Per-channel*, one step for each output channel of a weight matrix $W \in \mathbb{R}^{m \times n}$, that is, for each row, so that the weights of every output get their own step.
- *Per-token*, one step for each row of an activation matrix, which in a transformer means one step for each token (Xiao, Lin, et al., 2023).

Which groups are allowed is decided by the integer dot product of Equation (8) (Figure 3). A linear layer applied to a sequence of tokens computes, for token t and output i, $y_{ti} = \sum_j W_{ij} x_{tj}$, summed over the input index j. Give row i of the weights its own step, $W_{ij} \approx s_i^W q_{ij}^W$, and token t its own step, $x_{tj} \approx s_t^x q_{tj}^x$. Then

$$y_{ti} = \sum_{j=1}^{n} W_{ij} x_{tj} \;\approx\; s_i^W \, s_t^x \, \sum_{j=1}^{n} q_{ij}^W \, q_{tj}^x .$$

Both steps are constant along the summed index j, so they leave the sum as the factor $s_i^W s_t^x$, and what remains is pure integer arithmetic accumulated in 32 bits. The rule fits in one line. *A step may depend on every index except the summed one.* The per-channel weight step depends on i, the per-token step on t, and a per-tensor step on neither, so all three are allowed. A step that varies along j, such as one for each input channel of the activations, would stay inside the sum, $\sum_j s_j^x \, q_{ij}^W \, q_{tj}^x$, and force a rescaling of every product before accumulation, which integer kernels are not built to do (Nagel et al., 2021; Xiao, Lin, et al., 2023). Hence the usual pattern of per-channel weights and per-tensor or per-token activations.

Worked example on a dead output channel

Quantize $W = \begin{pmatrix} 0.84 & -0.35 & 0.12 \\ 0.03 & -0.05 & 0.02 \end{pmatrix}$ to $b = 4$ ($q_{\max} = 7$) with max-abs steps. **Per-tensor,** $s = 0.84/7 = 0.12$. Row 1 gets codes $(7, -3, 1)$. Row 2 gives 0.25, −0.42, 0.17, which all round to 0, because all three weights lie in the dead zone $|w| < s/2 = 0.06$. The second output channel is therefore zero for every input, as if it had been pruned. **Per-channel,** $s_2 = 0.05/7 = 0.00714$, and row 2 gets codes $(4, -7, 3)$. For $x = (0.5, -1, 2)$ the exact output is (1.010, 0.105), per-tensor gives (1.020, 0), and per-channel (1.020, 0.107). At $b = 8$ per-tensor the row-2 codes are $(5, -8, 3)$ and $y_2 = 0.109$, still 3.9% off. More bits shrink the problem without removing it.

Output channels of a real layer can differ in range by a large factor, and per-channel weight steps are the standard remedy (Krishnamoorthi, 2018; Nagel et al., 2021). Activations raise the same problem along their channels, which form the summed index j and so cannot take their own steps. When a few activation channels carry values far larger than the rest, they set the per-tensor step, and every other channel is rounded coarsely (Xiao, Lin, et al., 2023). Section 6.2 measures this cost and presents the remedies.

3.5 Weights and activations

Weights are fixed once training ends, so their range is known exactly and a step can be computed from the tensor itself. Activations change with every input, so their range is set in one of three ways.

- *Static.* The range is estimated on a small range-calibration set and then frozen. This costs nothing at run time, but the range does not follow the input. A static range set on clean images is the range the deployed model keeps when the rain arrives, as in the running example above.
- *Learned.* The step is trained together with the network, as LSQ does (Section 6), and is then frozen like a static one.
- *Dynamic.* The range is recomputed for every input at run time. It follows the input, at the price of finding the range of each tensor before the integer kernel can start.

The tag W8A8 of Section 2 counts the bits of the weights and of the input activations of each quantized layer, and nothing else. A complete description of a quantization scheme states six items, for weights and activations separately. The table fills them in for the hands-on notebook. Section 5 places its quantized layers in a ViT block (Figure 9), and Section 6 explains how its ranges are set.

Item	Weights	Activations (layer inputs)
bit width	8	8
form	symmetric	symmetric
granularity	one step per output channel	one step per tensor
range setting	max-abs of each row	moving average of batch maxima; in QAT, learned by LSQ
data	none, read off the weights	256 clean training crops (8 batches of 32); in QAT, the labeled training crops
layers	the 48 linear layers of the blocks	the inputs of the same 48 layers

In the hands-on session

Part 1 of the notebook codes Equation (13) as `step * np.clip(np.round(y/step), QMIN, QMAX)` with `QMIN, QMAX = -4, 3` and plots $Q(x)$ and $Q(x) - x$ for $s = 2$ and $s = 1$, the two panels of Figure 5. Predict the output range for $s = 2$ from Equation (14) before running it; the legend's "range ±8" rounds $[-8, 6]$. The Part 1 talk clips draw a mid-rise grid, while the code is mid-tread; trust the code. When the quantization-aware training (QAT) cell learns the step, watch the printed s and the saturated peaks (Section 6).

Exercises

Exercise I.7: Round then clamp, or clamp then round?

LSQ writes its quantizer as $s \cdot \mathrm{round}(\mathrm{clamp}(x/s, q_{\min}, q_{\max}))$ (Esser et al., 2020), while Equation (13) clamps after rounding. (a) Show that the two agree for every x when $q_{\min}$ and $q_{\max}$ are integers. (b) Find an input where they disagree if the upper bound were 2.5. (c) The three-step code form of the affine quantizer matches Equation (15) because $\mathrm{round}(u + z) = \mathrm{round}(u) + z$ for an integer z. Check this identity at $u = 0.5$ and $z = 1$ under round half to even.

Solution sketch. (a) Inside $[q_{\min}, q_{\max}]$ neither clamp acts, since rounding keeps a number of that interval inside it when the endpoints are integers. Above, clamping first gives $\mathrm{round}(q_{\max}) = q_{\max}$, and rounding first gives an integer $\geq q_{\max}$ that the clamp sends to $q_{\max}$; below is the mirror image. (b) With $u = 2.6$, clamp then round gives $\mathrm{round}(2.5) = 2$, while round then clamp gives $\min(3, 2.5) = 2.5$, not even an integer code. Integer bounds keep the output on the grid. (c) $\mathrm{round}(1.5) = 2$, but $\mathrm{round}(0.5) + 1 = 0 + 1 = 1$, so the identity fails exactly at ties, which have probability zero for continuous inputs.

Exercise I.8: The trade-off in closed form

Let x be uniform on $[0,1]$, quantized with L unsigned levels $0, s, \ldots, (L-1)s$, so the clip range is $[0,c]$ with $c = (L-1)s \le 1$. (a) Show that the rounding part of the mean squared error is exactly $c\,s^2/12$ and the clipping part is $(1-c)^3/3$. (b) Minimize the total over c. (c) Evaluate for $L = 16$ and compare with $c = 1$.

Solution sketch. (a) Inside $[0,c]$ there are $L-2$ full treads of width s, each contributing $s^3/12$, and two half treads at the ends, each contributing $\int_0^{s/2} t^2\,dt = s^3/24$, a total of $(L-1)s^3/12 = c\,s^2/12$. The clipping part is $\int_c^1 (x-c)^2 dx = (1-c)^3/3$. (b) With $s = c/(L-1)$ the total is $c^3/(12(L-1)^2) + (1-c)^3/3$, and a zero derivative gives $c/(2(L-1)) = 1-c$, so $c^* = 2(L-1)/(2L-1)$. (c) $c^* = 30/31 = 0.968$, and the error falls from 3.70×10^{-4} at $c = 1$ to 3.47×10^{-4}, a 6.3% gain. Even a bounded input with no outliers is best served by clipping its top 3%.

Check yourself before moving on

(1) For $b = 8$ and $s = 0.05$, give the clip range and $Q(0.024)$, $Q(-0.026)$, $Q(7.0)$. (2) Why can no change of s at a fixed bit width reduce both the rounding and the clipping error? (3) In an integer matrix product, why may weight steps vary by output channel while activation steps are shared across input channels? (4) If training shrinks the running example's step from 0.02 to 0.016 (synthetic), what are the new clip range and the clipping error of the rain value 3.10, and what does that say about a learned step?[5]

Where this is used. Sections 15 and 18 read what each paper states about its quantizer against the description of a scheme given above, and Section 20 returns to clipping by a learned step.

4 Rounding error as noise

What you will be able to do after this section

- Derive the rounding-noise variance $\eta^2 = s^2/12$ by integration and state the three assumptions that turn a deterministic rounding error into "noise".
- Compute a signal-to-quantization-noise ratio in decibels, derive the 6.02 dB-per-bit rule, and say correctly what removing one bit, or going from 8-bit to 4-bit weights, does to the step.
- Recognize the four ways the model fails (few levels, the dead zone, clipping, signal-dependent error), and use $\mathbb{E}[Q(x)^2] = \mathbb{E}[x^2] + 2\mathbb{E}[x\,e] + \mathbb{E}[e^2]$ to tell an attenuated signal from a signal with noise added.
- Explain why rounding and injected random noise of the same variance are different perturbations, and name the three properties that separate them.

Section 3 built the quantizer and showed that, inside the clip range, its error never exceeds half a step. That is a worst-case statement about one number. Analyses of quantized networks need more, a statement about the typical size and the structure of the error across many

[5] (1) $[-6.40, 6.35]$; $0.024/0.05 = 0.48$ rounds to 0 (dead zone), so $Q = 0$; -0.52 rounds to -1, so $Q = -0.05$; 140 is clamped to 127, so $Q = 6.35$, a clipping error of -0.65 or 13 steps. (2) The clip range is s times two fixed integers (Equation (14)), so a smaller step tightens the rounding bound and shrinks the range by the same factor. (3) A step that is constant along the summed index j factors out of $\sum_j W_{ij}x_j$ and leaves an integer sum; a per-row weight step is constant along j, a per-input-channel activation step is not. (4) $[-2.048, 2.032]$; $3.10/0.016 = 193.75$ is clamped to 127, so $Q = 2.032$ and $e = -1.068$, about 67 steps, and even the clean maximum 2.54 is now clipped ($e = -0.508$). Through Equation (14) a learned step also sets a learned clip range.

numbers, and they usually make it by calling quantization "noise". This section writes down the model behind that word, turns it into decibels, and shows where it fails.

4.1 The high-resolution model

Take a real number x drawn from a distribution with a smooth density p, round it with the mid-tread quantizer of Equation (13), and assume for the moment that x stays inside the clip range of Equation (14), so only rounding acts. The error $e = Q(x) - x$ is then a fixed function of x, the sawtooth of Figure 5. In the cell of level m, $x \in [(m - \frac{1}{2})s, (m + \frac{1}{2})s)$, it equals $e = ms - x$ and runs linearly from $+s/2$ down to $-s/2$.

Nothing about that function is random; the randomness comes from x. If p is nearly constant across each cell, then, given that x landed in cell m, its position inside the cell is nearly uniform, and so is e. Because this holds in every cell, it holds overall. The condition, a step small compared with the scale on which p changes, gives the model its name. It is the classical model of Bennett (1948); Gray and Neuhoff (1998) survey its history, and Widrow and Kollár (2008) give exact conditions under which it holds.

Definition 4.1: High-resolution (additive-noise) model of rounding

Write $Q(x) = x + e$. The model treats the rounding error e as a random variable such that (i) e is uniform on $[-s/2, s/2]$; (ii) e is uncorrelated with the signal, $\mathbb{E}[x\, e] = 0$; and (iii) errors on different samples, or on different coordinates of a vector, are uncorrelated with each other. Rounding then acts as the addition of a white noise of variance η^2 (white meaning uncorrelated across samples and coordinates, so its power is the same along every direction), uncorrelated with the signal. It is a model, not a law. It is accurate when the step is fine and nothing is clipped, and it fails in the four situations of Section 4.3.

Rounding-noise variance. Under property (i) the density of e is $1/s$ on $[-s/2, s/2]$, so

$$\mathbb{E}[e] = \frac{1}{s}\int_{-s/2}^{s/2} e\, de = 0, \qquad \eta^2 \triangleq \operatorname{Var}(e) = \frac{1}{s}\int_{-s/2}^{s/2} e^2\, de = \frac{1}{s}\left[\frac{e^3}{3}\right]_{-s/2}^{s/2} = \frac{1}{s}\cdot\frac{2}{3}\cdot\frac{s^3}{8} = \frac{s^2}{12}. \tag{16}$$

The root-mean-square (RMS) error is $\eta = s/\sqrt{12} \approx 0.289\, s$, about 58% of the worst case $s/2$. The result is exact under (i); whether (i) holds is the subject of the rest of the section.

▷ **Check yourself.** An INT8 activation quantizer has step $s = 0.02$. What are the worst-case and the RMS rounding errors inside its range?[6]

Whether (i) holds can be read off the density of e, which is built cell by cell (Figure 7). An input in the cell of level m has the error $e = ms - x$, so that cell maps the slice of p over it onto $[-s/2, s/2]$, with its level at $e = 0$. Stacking the slices of all levels gives

$$p_e(e) = \sum_{m=q_{\min}}^{q_{\max}} p(ms - e), \qquad |e| < s/2.$$

If p is nearly flat across every cell, every slice is nearly flat, and so is the stack, which is property (i) and panel (a). Inputs more than half a step past an edge of the clip range land outside $[-s/2, s/2]$, in tails the model does not have. Panel (b) stacks the slices for a coarse 2-bit grid.

[6] The worst case is $s/2 = 0.01$. The RMS error is $s/\sqrt{12} = 0.02/3.464 = 0.00577$, and the variance is $\eta^2 = 0.0004/12 = 3.33 \times 10^{-5}$.

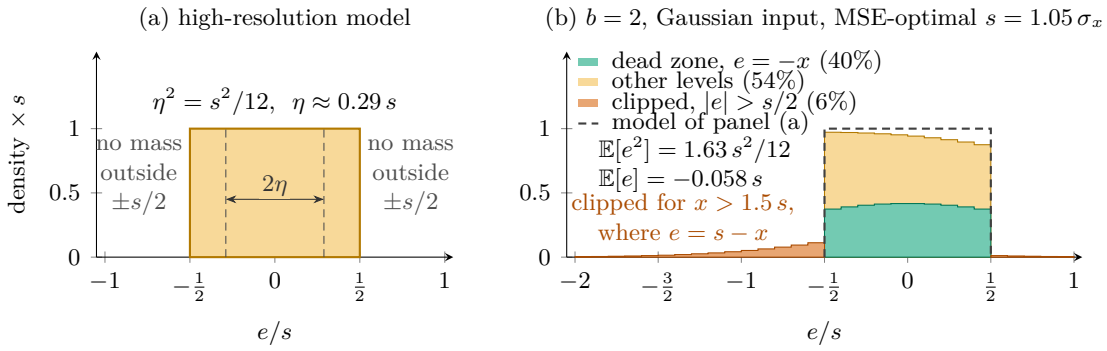


Figure 7: The rounding-error density, model and reality. (a) The high-resolution model of Definition 4.1; the dashed lines mark the RMS error $\pm\eta$. (b) The exact density of $e = Q(x) - x$ for $x \sim \mathcal{N}(0, 1)$, a 2-bit quantizer, and the step $s = 1.048$ that minimizes the mean squared error (MSE), stacked level by level as in the text and computed by numerical integration, not sampled. Inside $\pm s/2$ the density is close to the model's, although the 40% of inputs in the dead zone have $e = -x$ exactly. A density cannot show this correlation with x, which is why Section 4.3 needs $\mathbb{E}[x\,e]$. The 6.2% of inputs more than half a step beyond the clip range $[-2s, s]$ fall in tails the model forbids and carry 43% of the error power, so the total is 1.63 times the model's. The mean is negative because the grid has one level fewer at the top, so more inputs clip there.

4.2 Signal-to-quantization-noise ratio and the 6 dB rule

What matters is the size of the error relative to the signal it rides on, stated in decibels (dB), where 10 dB is a factor of ten in power, 3 dB about a factor of two, and an amplitude ratio c is worth $20\log_{10} c$ dB. A ratio of 40 dB, for example, is a factor of 10^4 in power and of 100 in amplitude, so an error at 40 dB below the signal has an RMS value one hundredth of the signal's.

Signal-to-quantization-noise ratio (SQNR). With signal power $P_S = \mathbb{E}[x^2]$ and noise power $P_N = \mathbb{E}[e^2]$,

$$\text{SQNR} \triangleq 10\log_{10}\frac{P_S}{P_N}\ \text{dB}, \qquad \text{and under Definition 4.1} \qquad \text{SQNR} = 10\log_{10}\frac{12\,P_S}{s^2}\ \text{dB}. \tag{17}$$

The 6.02 dB-per-bit rule. The grid of Equation (14) spans $s(q_{\max} - q_{\min}) = s(2^b - 1)$. Hold that width fixed at R, so $s = R/(2^b - 1)$, and substitute into Equation (17),

$$\text{SQNR}(b) = 10\log_{10}\frac{12\,P_S}{R^2} + 20\log_{10}(2^b - 1) \approx 10\log_{10}\frac{12\,P_S}{R^2} + 6.02\,b\ \text{dB}, \tag{18}$$

because $20\log_{10} 2 = 6.02$. Each bit halves the step, and each halving buys 6.02 dB, the classical 6 dB-per-bit rule (Gray & Neuhoff, 1998).

The classic instance is a sine $x(t) = A\sin\omega t$ that just fills the range, so $P_S = A^2/2$, $R \approx 2A$, the first term of Equation (18) is $10\log_{10}(12 \cdot \frac{1}{2}A^2/4A^2) = 10\log_{10} 1.5 = 1.76$ dB, and SQNR $\approx 6.02\,b + 1.76$ dB, which is 49.9 dB at 8 bits and 25.8 dB at 4 bits.

Bits and steps, stated correctly. Removing *one* bit at a fixed range multiplies the step by $(2^b - 1)/(2^{b-1} - 1)$, which is $255/127 = 2.008$ from 8 to 7 bits; η^2 quadruples and the SQNR falls by 6.05 dB. Halving the bit width is not doubling the step. From 8 to 4 bits at the same range

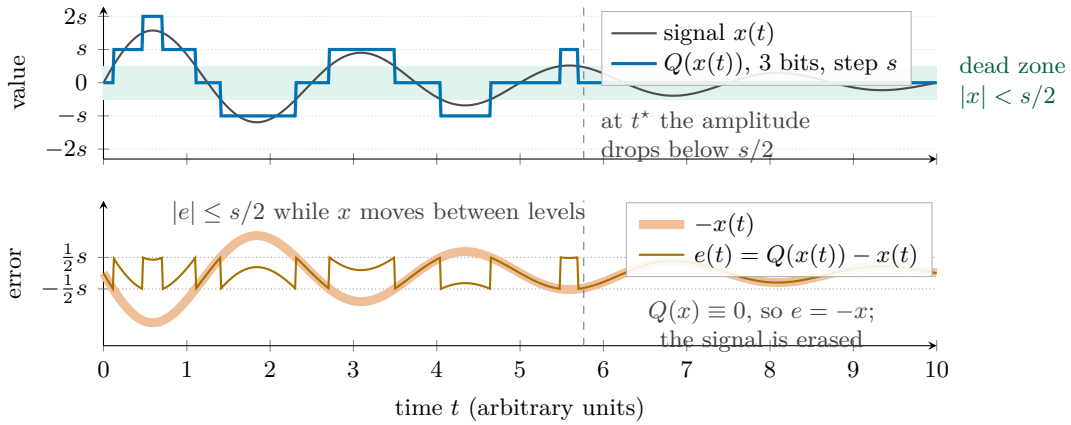


Figure 8: Erasure in the dead zone. Top, the signal $x(t) = 1.8\,e^{-t/4.5}\sin(2\pi t/2.5)$ in units of the step and its 3-bit rounding $Q(x(t))$ (levels $-4s, \dots, 3s$, of which only $-s, \dots, 2s$ are reached, so nothing is clipped); the green band is the dead zone $|x| < s/2$. Bottom, the error $e(t) = Q(x(t)) - x(t)$ drawn over $-x(t)$. The error stays within $\pm s/2$ throughout, so its size does not change at $t^\star = 4.5 \ln 3.6 = 5.76$, where the amplitude falls below $s/2$. What changes is its relation to the signal. Before $t^\star$ the error coincides with $-x$ only near the zero crossings. After $t^\star$ the output is zero and the error is exactly $-x$. Nothing was added; the signal was removed.

the step is multiplied by $255/15 = 17$, η^2 by 289, and the SQNR falls by $20 \log_{10} 17 = 24.6$ dB, close to the rule's $4 \times 6.02 = 24.1$ dB. The rule assumes a fixed range and a valid model (an MSE-optimal range gains less per bit, Section 3). Section 8 applies Equation (17) separately along each principal direction of a feature vector, Equation (38).

4.3 When the model fails

Properties (i) to (iii) are strong, and a quantized network violates them in four ways.

Few levels. When the step is comparable to the spread of the signal, the density is not flat across a cell. For a Gaussian input this bites later than you might expect. Take $x \sim \mathcal{N}(0, \sigma_x^2)$ on an unbounded grid, so nothing is clipped, and compute the error moments exactly, cell by cell.

step s	$\le \sigma_x$	$2\sigma_x$	$3\sigma_x$	$4\sigma_x$
$\mathbb{E}[e^2]\,/(s^2/12)$	1.0000	0.991	0.864	0.648
correlation of x and e	0.000	−0.025	−0.28	−0.61

Up to a step of one standard deviation the model is exact to four decimals. From about $s = 3\sigma_x$ on, the error power falls below $s^2/12$ and the error turns anti-correlated with the signal. A smooth input thus fills the cells evenly even with only a few cells per standard deviation, so a quantizer with 16 or 256 levels over a sensible range usually fails for the other three reasons. At a large step most inputs fall into one cell, which is the next failure.

The dead zone. A mid-tread quantizer sends every $|x| < s/2$ to 0, and there $e = -x$ exactly. The error is the negative of the signal, perfectly anti-correlated with it, and its power is the signal's own power rather than $s^2/12$. A signal whose whole range lies in the dead zone is not drowned in noise; it is erased. Figure 8 shows the transition on a decaying signal.

Clipping. Outside the clip range the error is $e = sq_{\max} - x$ above it and $e = sq_{\min} - x$ below it. It is unbounded, linear in x, and again anti-correlated with x, since a large positive input produces a large negative error. Even at the MSE-optimal step, clipping carries a large share of the error power (Figure 7(b); Figure 6 shows the trade-off as a function of the step).

Signal-dependent error. Because $e(x)$ is a deterministic function, structure in the signal passes into the error. A periodic signal rounded with few levels gives a periodic error, and coordinates that carry similar values get similar errors, which breaks property (iii). In the extreme case, three coordinates that all equal 0.31, rounded with $s = 0.2$, all become 0.4 and all get the error $+0.09$. Their errors are identical, not uncorrelated. Section 4.4 takes this up.

Where the power goes. One identity separates the two ways a quantizer can change a signal.

Error decomposition. Because $Q(x) = x + e$,

$$\mathbb{E}\left[Q(x)^2\right] = \mathbb{E}\left[x^2\right] + 2\,\mathbb{E}[x\,e] + \mathbb{E}\left[e^2\right]. \tag{19}$$

Under Definition 4.1 the middle term vanishes and rounding adds power, $\mathbb{E}\left[Q(x)^2\right] = \mathbb{E}\left[x^2\right] + \eta^2$. A negative $\mathbb{E}[x\,e]$ means the error partly cancels the signal, which is then *attenuated*. In the dead zone $\mathbb{E}[x\,e] = -\mathbb{E}\left[x^2\right]$ and $\mathbb{E}\left[e^2\right] = \mathbb{E}\left[x^2\right]$, so $\mathbb{E}\left[Q(x)^2\right] = 0$.

The middle term is often summarized by the gain $g = \mathbb{E}[x\,Q(x)]\,/\mathbb{E}\left[x^2\right] = 1 + \mathbb{E}[x\,e]\,/\mathbb{E}\left[x^2\right]$, the slope of the least-squares line through the origin that fits $Q(x)$ against x. A gain below 1 means that the output is, on average, a shrunk copy of the input. The model predicts $g = 1$, the dead zone gives $g = 0$, and clipping gives $g < 1$. Section 8 applies the same decomposition along each feature direction (Equation (39), Proposition 8.2). One case deserves a proof here, because it shows that attenuation is a consequence of good settings, not of bad ones.

Proposition 4.1: A step chosen for minimum error attenuates

Let x have a density and a finite second moment, fix b, and let $s^* > 0$ minimize $\mathbb{E}\left[(Q(x) - x)^2\right]$ over s. Then at $s = s^*$

$$\mathbb{E}[Q(x)\,e] = 0, \quad \text{hence} \quad \mathbb{E}[x\,e] = -\mathbb{E}\left[e^2\right] < 0 \quad \text{and} \quad \mathbb{E}\left[Q(x)^2\right] = \mathbb{E}\left[x^2\right] - \mathbb{E}\left[e^2\right].$$

Proof sketch. Write $Q(x) = sq$ with the code $q = \mathrm{clamp}(\mathrm{round}(x/s), q_{\min}, q_{\max})$. As s varies, q changes only where x/s crosses a half-integer, and there the two neighboring levels are equally far from x, so $(Q(x) - x)^2$ is continuous in s; between such points its derivative is $2(sq - x)q = 2e\,Q(x)/s$. Differentiating under the expectation, $\frac{d}{ds}\mathbb{E}\left[e^2\right] = \frac{2}{s}\mathbb{E}[e\,Q(x)]$, which vanishes at the minimum. Then $\mathbb{E}[x\,e] = \mathbb{E}[(Q(x) - e)\,e] = -\mathbb{E}\left[e^2\right]$, and Equation (19) gives the last identity. The proposition and its proof are the notes' own. □

A range chosen for minimum error therefore never adds power to a signal; it removes exactly the error power, the opposite of the additive model's prediction. The reason is geometric. With the codes q held fixed, the step that minimizes $\mathbb{E}\left[(sq - x)^2\right]$ is the least-squares coefficient of x on q, and the condition $\mathbb{E}[Q(x)\,e] = 0$ is its normal equation, which makes the error orthogonal to the fitted value $Q(x)$. The input $x = Q(x) - e$ then splits into two orthogonal parts, and Pythagoras gives $\mathbb{E}\left[x^2\right] = \mathbb{E}\left[Q(x)^2\right] + \mathbb{E}\left[e^2\right]$. The input's power is shared between the output and the error, so the output must carry less power than the input. Learned steps are not pinned to this optimum (Section 3).

Worked example on the minimum-error steps for a Gaussian input

Section 3 found the steps s^* that minimize the MSE for $x \sim \mathcal{N}(0,1)$ at several bit widths. Hold three of them against the model and against Equation (19); every entry is computed exactly by integrating over the cells.

b	s^*	$\mathbb{E}[e^2] \,/\, (s^{*2}/12)$	$\mathbb{E}[x\,e]$	g
2	1.048	1.63	−0.1494	0.851
4	0.339	1.24	−0.0119	0.988
8	0.0308	1.11	−0.000088	1.000

The model underestimates the error power at every bit width, by 63% at 2 bits and still by 11% at 8, because a minimum-error range clips and clipping adds to rounding. $\mathbb{E}[x\,e] = -\mathbb{E}[e^2]$ in every row, as Proposition 4.1 says (at 2 bits, $1.63 \times 1.048^2/12 = 0.1494$), so $g < 1$ throughout. In decibels the model is off by at most $10\log_{10} 1.63 = 2.1$ dB, but what it misses is clipping error, which is large, one-sided, and anti-correlated with the signal.

Running example: the bus in the rain

Section 3 rounded six activations of one token of the bus crop with the per-tensor INT8 step $s = 0.02$ (clip range $[-2.56, 2.54]$) and found a mean squared error of 4.2×10^{-5}, of the order of $s^2/12 = 3.3 \times 10^{-5}$. Follow two of those coordinates across many crops, A, which was −1.2468 on the bus crop, and B, which was 0.0093 and sat in the dead zone. Suppose (synthetic numbers) that A varies from crop to crop as a zero-mean Gaussian with standard deviation $\sigma_A = 0.8$, and B with $\sigma_B = 0.008$. The table sets the model against the exact computation, at INT8 and at 4 bits over the same range width, where the step is $17 \times 0.02 = 0.34$. The model's error power is $s^2/12$ for both coordinates, and SQNR $= 10\log_{10}(\sigma^2/\text{error power})$.

		error power		SQNR (dB)		outputs
coordinate	step	model	exact	model	exact	at 0
A, INT8	0.02	3.33×10^{-5}	1.60×10^{-4}	42.8	36.0	1%
B, INT8	0.02	3.33×10^{-5}	3.16×10^{-5}	2.8	3.1	79%
A, 4 bits	0.34	9.63×10^{-3}	9.79×10^{-3}	18.2	18.2	17%
B, 4 bits	0.34	9.63×10^{-3}	6.40×10^{-5}	−21.8	0.0	100%

At INT8 the exact computation disagrees with the model for both coordinates, in opposite ways. A's error power is 4.8 times the model's, because the 0.14% of crops on which A leaves the clip range contribute almost four times as much error power as the rounding of all the rest. B's error power is within 5% of the model's, yet B is exactly zero on 79% of the crops, and its error is correlated with B at −0.12. The model gets the size of B's error right and its shape wrong. At 4 bits A behaves as the model says, because the coarse rounding dwarfs the rare clips. B is zero on every crop, so its error power is its own power, $\sigma_B^2 = 6.4 \times 10^{-5}$. The model says B drowns under noise 22 dB stronger than itself; in fact nothing is added and B is removed. One erased coordinate is not yet an erased direction, since a direction of a feature vector sums the errors of many coordinates. Section 8 turns A and B into the head and the tail of a feature spectrum.

4.4 Rounding is deterministic; injected noise is redrawn

The error $e(x) = Q(x) - x$ is a function. Feed the same crop through the same quantizer twice and you get the same error twice. The randomness in Definition 4.1 is over the population of inputs, across which the errors only look like uniform draws. An injected perturbation,

such as dropout or Gaussian noise added during training, is redrawn at every forward pass, independently of the input. Two perturbations with the same variance $s^2/12$ can therefore differ in three ways the variance does not see.

- *Determinism.* Rounding error is fixed for a given input and step; injected noise is fresh on every pass.
- *Signal dependence.* Rounding error depends on x and is anti-correlated with it in the dead zone and under clipping.
- *Clipping.* A quantizer saturates large inputs, which no additive perturbation does.

A training control that injects uniform noise of variance $s^2/12$ at a quantizer's insertion points is Definition 4.1 turned into an experiment. It keeps the magnitude of property (i) and drops all three, so it tests whether the magnitude of the error alone is enough to do what the quantizer does.

Dithering. Signal processing makes rounding behave like the model on purpose. Add a random d, uniform on $[-s/2, s/2]$ and independent of x, before rounding. Inside the range the output is then unbiased, $\mathbb{E}[Q(x+d) \mid x] = x$ (Exercise I.10), so a signal smaller than half a step, which plain rounding erases, survives on average; subtracting the same d afterward makes the total error exactly uniform and independent of x (Gray & Neuhoff, 1998; Widrow & Kollár, 2008). Stochastic rounding, which rounds up with probability equal to the fractional part of x/s, is the same device. Dither buys independence with randomness and extra error power. Rounding error and noise become the same object only when randomness is added deliberately.

Resolving an ambiguity about "noise"

These notes keep four terms apart: *rounding error* $e(x)$, a deterministic function of the input; the *rounding-noise variance* η^2 of Definition 4.1, a property of a population of inputs; *injected noise*, redrawn at every forward pass; and *corruption noise*, added to image pixels (Section 9). The single word *noise* is commonly used for all four, sometimes in one paragraph, and a sentence true of one can be false of another. When you meet the word, work out from the context which of the four is meant; these notes always name it.

In the hands-on session

Notebook Part 1 builds the quantizer of Equation (13) at 3 bits and plots $Q(x)$ and its error for $s = 2$ and $s = 1$; its "about $s/\sqrt{12}$" RMS error is Equation (16). Part 2 rounds a strong sine (amplitude 3) and a weak one (amplitude 0.15) with the same step, growing to $s = 1$. At $s = 1$ the weak sine lies inside the dead zone, so what you watch is erasure, as in Figure 8, not drowning. Use the sliders to find where the weak signal vanishes, and compare ratios of amplitude to step, since the notebook sets the step differently in Parts 1 and 2.

Exercise I.9: Truncation instead of rounding

Some integer pipelines truncate, $Q_T(x) = s\lfloor x/s \rfloor$. (a) Under the high-resolution assumptions, find the distribution, mean, variance, and mean square of $e_T = Q_T(x) - x$. (b) By how many decibels is the SQNR worse than for rounding when the noise power is the mean square, and when it is the variance?

Solution sketch. (a) In each cell e_T runs from 0 down to $-s$, so it is uniform on $(-s, 0]$, with mean $-s/2$, variance $s^2/12$, and mean square $s^2/12 + s^2/4 = s^2/3$. (b) As a mean square the noise power is four times larger, 6.02 dB worse, as if a bit had been lost; as a variance there is no difference. A bias

is invisible to a variance and counts in full in a mean square, so always check whether a reported noise power subtracts the mean (Section 8.3).

Exercise I.10: Dither makes the mean right

Let d be uniform on $[-s/2, s/2]$ and independent of x, and ignore clipping. (a) For $x = 0.3\,s$, which plain rounding sends to 0, find the distribution and the mean of $Q(x+d)$. (b) Write $x = (m+r)s$ with an integer m and $r \in [0,1)$ and show that $\mathbb{E}[Q(x+d) \mid x] = x$. (c) Show that $\mathbb{E}\left[(Q(x+d)-x)^2 \mid x\right] = r(1-r)s^2$ and average it over r uniform on $[0,1)$.

Solution sketch. (a) $Q(x+d) = s$ exactly when $d \geq 0.2\,s$, with probability 0.3, and 0 otherwise; the mean is $0.3\,s = x$. (b) The output is $(m+1)s$ with probability r and ms with probability $1-r$, so the mean is $(m+r)s = x$. (c) The error is $(1-r)s$ with probability r and $-rs$ with probability $1-r$, so its mean square is $r(1-r)^2s^2 + (1-r)r^2s^2 = r(1-r)s^2$, and $\int_0^1 r(1-r)\,dr = \frac{1}{6}$. Dither doubles the average error power, from $s^2/12$ to $s^2/6$, and makes it depend on x, in exchange for an unbiased output.

Check yourself before moving on

1. At a fixed range, what does removing one bit do to s, η^2, and the SQNR, and what does going from 8 to 4 bits do?[a]
2. Name the four failures of the high-resolution model and the sign of $\mathbb{E}[x\,e]$ in the dead zone and under clipping.[b]
3. Why can a quantizer whose step minimizes the MSE never add power to its input?[c]
4. Rounding with step s and uniform noise redrawn on every pass both have variance $s^2/12$. What three properties separate them?[d]

[a] Removing one bit about doubles the step (2.008 from 8 to 7 bits), quadruples η^2, and lowers the SQNR by about 6 dB. Going from 8 to 4 bits multiplies the step by 17 and η^2 by 289, and lowers the SQNR by 24.6 dB.

[b] Few levels, the dead zone, clipping, and signal-dependent error. In the dead zone $\mathbb{E}[x\,e] = -\mathbb{E}\left[x^2\right] < 0$; under clipping it is negative too, since large inputs produce errors of the opposite sign.

[c] At the optimal step $\mathbb{E}[e\,Q(x)] = 0$ (Proposition 4.1), so $\mathbb{E}[x\,e] = -\mathbb{E}\left[e^2\right]$, and Equation (19) gives $\mathbb{E}\left[Q(x)^2\right] = \mathbb{E}\left[x^2\right] - \mathbb{E}\left[e^2\right]$.

[d] Determinism, signal dependence (the dead zone and the anti-correlation of e with x), and clipping.

Where this is used. Section 17 reads a measured per-rank SQNR with Equation (17), and Section 20 reads a matched-noise control against the three properties of Section 4.4.

5 Where quantization enters a network

What you will be able to do after this section

Compute softmax probabilities, the cross-entropy, and its gradient by hand; draw one Vision Transformer block and name its four linear layers, the source of the running example's 48 quantized layers; mark where each fake quantizer sits; and explain, with a proposition and a two-layer example, why the end-to-end residual is not one rounding error of variance $s^2/12$.

Sections 3 and 4 studied one quantizer acting on one number. A quantized encoder contains dozens of them, and what matters downstream is the difference between the outputs of a full-precision and a quantized network. This section builds the running example's recognizer from the output backward, in four steps: the classifier head that turns a feature vector into a decision, the Vision Transformer (ViT) that produces the feature vector, the places where the quantizers sit, and the residual they leave together in the last feature vector. Where the encoder's weights come from is the subject of Section 10.

5.1 From a feature vector to a decision

A *linear layer* maps $\boldsymbol{x} \in \mathbb{R}^d$ to $\boldsymbol{y} = W\boldsymbol{x} + \boldsymbol{b}$, with weights $W \in \mathbb{R}^{m \times d}$ and a bias $\boldsymbol{b} \in \mathbb{R}^m$. Nearly all the arithmetic of the networks in these notes has this form, which is why quantization targets it. The last linear layer of a classifier is its *classifier head*. For K classes its outputs $\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^K$ are the *logits*, one unnormalized score per class. In the running example $K = 4$ (car, truck, bus, person) and $d = 384$, so the head holds $4 \times 384 + 4 = 1540$ numbers.

Softmax and cross-entropy. For logits $\boldsymbol{z} \in \mathbb{R}^K$ and true class y, the softmax probabilities, the cross-entropy loss, and its gradient with respect to the logits are

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad L(\boldsymbol{z}, y) = -\log p_y = -z_y + \log \sum_{j=1}^{K} e^{z_j}, \qquad \frac{\partial L}{\partial z_k} = p_k - \mathbf{1}[k = y]. \quad (20)$$

Proof of the gradient. The derivative of $\log \sum_j e^{z_j}$ with respect to z_k is $e^{z_k} / \sum_j e^{z_j} = p_k$, and the term $-z_y$ contributes -1 exactly when $k = y$. □

The softmax makes the scores positive, normalizes them to sum to one, and keeps their order, so the predicted class $\arg\max_k z_k$ is the most probable one and its probability is the model's *confidence* (Section 12). The gradient $\boldsymbol{p} - \boldsymbol{u}_y$, with $\boldsymbol{u}_y$ the one-hot vector of the true class, raises the true logit by $1 - p_y$ and lowers every wrong logit in proportion to its probability. Training lowers the average loss over mini-batches by such gradient steps, usually with AdamW (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), and a *frozen* parameter receives no update. The notebook trains its heads this way (Part 5).

Worked example with the bus crop's four logits

Let the head return the synthetic logits $\boldsymbol{z} = (0.6, 2.2, 3.0, -1.1)$ for (car, truck, bus, person) on a crop whose true class is bus.

Probabilities and loss. The exponentials are 1.822, 9.025, 20.086, and 0.333, with sum 31.266, so $\boldsymbol{p} = (0.058, 0.289, 0.642, 0.011)$. The crop is called bus with confidence 0.642, truck is the runner-up at 0.289, and $L = -\ln 0.642 = \ln 31.266 - 3.0 = 0.443$.

Gradient. $\boldsymbol{p} - \boldsymbol{u}_{\text{bus}} = (0.058, 0.289, -0.358, 0.011)$. A step raises the bus logit and pushes hardest on truck, the class the model confuses with bus.

Margin. The decision rests on the *margin* $m = z_y - \max_{k \neq y} z_k$, here $m = z_{\text{bus}} - z_{\text{truck}} = 0.8$. A change $\boldsymbol{\delta}$ of the features moves this margin by $(\boldsymbol{w}_{\text{bus}} - \boldsymbol{w}_{\text{truck}})^\top \boldsymbol{\delta}$, where $\boldsymbol{w}_k^\top$ is row k of W. The running example below puts a quantization residual in place of $\boldsymbol{\delta}$.

▷ **Check yourself.** Add 5 to every logit. What happens to $\boldsymbol{p}$ and to L?[7]

5.2 The Vision Transformer, one block at a time

The Vision Transformer (ViT) (Dosovitskiy et al., 2021) processes an image as a sequence of patches with the Transformer of Vaswani et al. (2017). The running example's encoder is the DINOv2 model ViT-S/14 (Oquab et al., 2024), where S (small) means width 384 and 14 is the patch side in pixels. The shapes below are those of the implementation the notebook loads, and Figure 9 draws the recognizer, the whole pipeline in its top row and one block below it.

Patches become tokens. A 224×224 color crop is cut into a 16×16 grid of 14×14 patches, 256 patches of $14 \cdot 14 \cdot 3 = 588$ numbers each. The *patch embedding* maps every patch to

[7] Nothing. $e^{z_k+5} = e^5 e^{z_k}$, and e^5 cancels between numerator and denominator. Only differences of logits, the margins, matter.

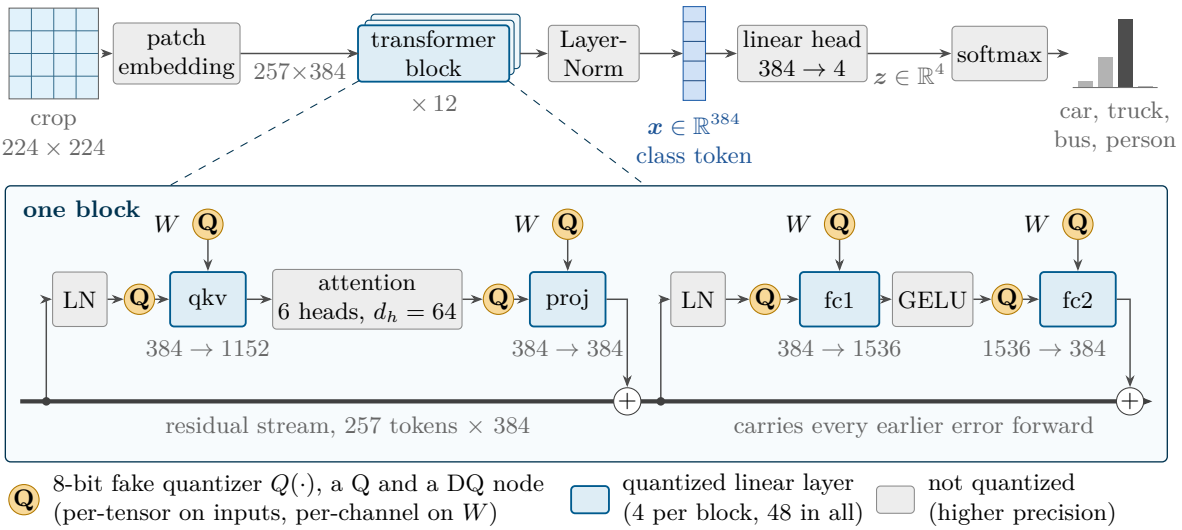


Figure 9: Where quantization enters the running example's recognizer, with the probabilities $\boldsymbol{p}$ of the first worked example at the top right. Each blue linear layer rounds its input (one per-tensor step) and its weights (one step per output channel) at the yellow fake quantizers; gray parts are not quantized. Read it as a count, 4 quantized layers per block and 48 in all, each leaving its error in the residual stream that carries it to $\boldsymbol{x}$. Shapes from the notebook's model.

$\mathbb{R}^{384}$ with one shared linear map. A learned *class token* is prepended, and a learned position embedding is added to each of the 257 tokens so that the network knows where each patch came from. The resulting 257×384 array H, which every later layer reads and updates in place, is the *residual stream*, the thick line along the bottom of the block in Figure 9.

One block. Each of the 12 blocks, the lower panel of Figure 9, applies two sublayers, each wrapped in a residual connection (He et al., 2016),

$$H \leftarrow H + \mathrm{Attn}(\mathrm{LN}(H)), \qquad H \leftarrow H + \mathrm{MLP}(\mathrm{LN}(H)). \tag{21}$$

LayerNorm (LN) standardizes each token over its 384 coordinates and applies a learned scale and shift (Ba et al., 2016). DINOv2 also multiplies each sublayer's output by a learned per-channel vector (LayerScale) before adding it to the stream, and the notebook does not quantize it. The attention sublayer computes a query, a key, and a value for every token with one linear layer, *qkv* ($384 \to 1152$), split into 6 heads of dimension $d_h = 64$. Stacked over the 257 tokens, one head's queries, keys, and values form the matrices $Q_{\mathrm{att}}, K_{\mathrm{att}}, V_{\mathrm{att}} \in \mathbb{R}^{257\times 64}$; the subscript keeps them apart from the quantizer $Q(\cdot)$ and the class count K. The head outputs $\mathrm{softmax}(Q_{\mathrm{att}}K_{\mathrm{att}}^{\top}/\sqrt{d_h})V_{\mathrm{att}}$, the softmax taken row by row, so every token becomes a weighted average of the values, with weights set by how well its query matches each key (Vaswani et al., 2017). A second linear layer, *proj* ($384 \to 384$), maps the six heads back into the stream. The multilayer perceptron (MLP) acts on each token with *fc1* ($384 \to 1536$), the GELU nonlinearity, and *fc2* ($1536 \to 384$).

Counting the linear layers. Each block has exactly four linear layers with trained weights, holding 442,368 (qkv), 147,456 (proj), and 589,824 (fc1, fc2 each) weights, 21,233,664 in all 12 blocks. The two attention products multiply one activation by another and have no trained weights, so they are not linear layers. Twelve blocks of four give the 48 layers the running example's encoder quantizes.

The vector the head reads. After the last block a final LayerNorm is applied and one 384-d vector per image is read out, here the class token, the default of the model the notebook loads. Averaging the patch tokens is the common alternative, so a paper that says "pooled features" may mean either; check which one before comparing results. This vector $\boldsymbol{x} \in \mathbb{R}^{384}$ enters the classifier head, so it is the *penultimate* representation, the one Sections 7 and 8 analyze.

5.3 Where the quantizers sit

Inside each of the 48 layers quantization enters at two places, the yellow fake quantizers of Figure 9. With the quantizer of Definition 3.1, a quantized linear layer computes

$$\tilde{\boldsymbol{y}} = Q_w(W)\, Q_a(\boldsymbol{x}) + \boldsymbol{b}, \tag{22}$$

where Q_a rounds the layer's input with one step s_a for the whole tensor (per-tensor) and Q_w rounds W with one step per row, that is per output channel (per-channel), the granularities of Section 3. The output $\tilde{\boldsymbol{y}}$ is not rounded here. If it feeds another linear layer, that layer's input quantizer rounds it, so every tensor that enters one of the 48 weight-by-activation products is an 8-bit integer times a step, which is what an INT8 kernel needs (Figure 3). The encoder then carries 48 activation steps, one for each layer input, and 41,472 weight steps, one for each output channel, since each block has $1152 + 384 + 1536 + 384 = 3456$ output channels and $12 \times 3456 = 41{,}472$. In training the quantizers are usually simulated, returning floats that lie on the grid (fake quantization, Definition 2.2), and true integer kernels take over at deployment. No quantizer sits on the output of these 48 layers, and each output feeds a floating-point operation (attention, GELU, or the residual sum). A deployment that matches the simulation therefore skips requantization. Each kernel multiplies its INT32 accumulator by $s_w s_a$ and returns a float, while the DQ branch of Figure 3 first rounds the output to an INT8 code.

Why these parts? The 48 weight matrices hold 21,233,664 of the encoder's 21,628,800 parameters (98.2%), and at 257 tokens the four linear layers perform 454.8 million of a block's 505.5 million multiply-accumulates (about 90%, counted from the shapes). LayerNorm and softmax need square roots, divisions, and exponentials, which an integer matrix kernel does not speed up. LayerNorm's *output* is still rounded, since it enters qkv or fc1 through an input quantizer.

In the hands-on session

Notebook Part 5.1, "The quantizers, now inside a transformer", builds Figure 9 in code. `quantize_backbone` replaces every `nn.Linear` whose name contains `blocks` with a `QLinear` and prints the count, 48. A `QLinear` computes `F.linear(aq(x), quantize_weight(W), bias)`, which is Equation (22) with a per-tensor `ActQuantizer`, a per-channel `quantize_weight`, and a floating-point bias. Print the wrapped names and check that they are each block's qkv, proj, fc1, and fc2. Part 5 needs a GPU.

5.4 Many quantizers, one residual

Run N crops through the full-precision encoder and through its W8A8 twin, and stack the features $\boldsymbol{x}$ as the rows of X and $X_Q \in \mathbb{R}^{N \times 384}$. The *end-to-end residual* is $E = X_Q - X$, quantized minus original as in Section 3. It is not rounding noise of variance $s^2/12$ (Equation (16)). It collects the errors of 96 quantizers (48 on activations, 48 on weight matrices), each transformed by every layer after it, and part of it is even a fixed linear function of the input.

Proposition 5.1: Rounding errors pushed through a chain of linear layers

Consider n linear layers without nonlinearities, each with its own quantizers. The full-precision chain computes $\boldsymbol{h}_\ell = W_\ell \boldsymbol{h}_{\ell-1}$ and the quantized chain $\tilde{\boldsymbol{h}}_\ell = Q_w(W_\ell)\, Q_a(\tilde{\boldsymbol{h}}_{\ell-1})$,

both from $\boldsymbol{h}_0 = \tilde{\boldsymbol{h}}_0 = \boldsymbol{x}$. Write $\boldsymbol{e}_\ell = Q_a(\tilde{\boldsymbol{h}}_{\ell-1}) - \tilde{\boldsymbol{h}}_{\ell-1}$ for the error of the ℓ-th input quantizer and $\Delta_\ell = Q_w(W_\ell) - W_\ell$ for that of the ℓ-th weight matrix. Then exactly

$$\tilde{\boldsymbol{h}}_n - \boldsymbol{h}_n = \sum_{\ell=1}^{n} B_\ell \Big(W_\ell\, \boldsymbol{e}_\ell + \Delta_\ell\, Q_a(\tilde{\boldsymbol{h}}_{\ell-1}) \Big), \qquad B_\ell = W_n W_{n-1} \cdots W_{\ell+1}, \quad B_n = I. \tag{23}$$

The map B_ℓ carries an error from the output of layer ℓ to the end of the chain. If moreover $\Delta_\ell = 0$ and the $\boldsymbol{e}_\ell$ are zero-mean, mutually uncorrelated, with covariance $\eta_\ell^2 I$, then with $A_\ell = B_\ell W_\ell = W_n \cdots W_\ell$, the map from the input of layer ℓ to the end,

$$\operatorname{Cov}(\tilde{\boldsymbol{h}}_n - \boldsymbol{h}_n) = \sum_{\ell=1}^{n} \eta_\ell^2\, A_\ell A_\ell^\top. \tag{24}$$

Proof. Write $\boldsymbol{d}_\ell = \tilde{\boldsymbol{h}}_\ell - \boldsymbol{h}_\ell$. Substituting $Q_w(W_\ell) = W_\ell + \Delta_\ell$ and $Q_a(\tilde{\boldsymbol{h}}_{\ell-1}) = \tilde{\boldsymbol{h}}_{\ell-1} + \boldsymbol{e}_\ell$ into $\tilde{\boldsymbol{h}}_\ell$ gives the linear recursion $\boldsymbol{d}_\ell = W_\ell \boldsymbol{d}_{\ell-1} + W_\ell \boldsymbol{e}_\ell + \Delta_\ell\, Q_a(\tilde{\boldsymbol{h}}_{\ell-1})$ with $\boldsymbol{d}_0 = \boldsymbol{0}$, and unrolling it gives Equation (23). With $\Delta_\ell = 0$ the residual is $\sum_\ell A_\ell \boldsymbol{e}_\ell$, $\operatorname{Cov}(A\boldsymbol{e}) = A \operatorname{Cov}(\boldsymbol{e}) A^\top$, and the cross terms vanish because the errors are uncorrelated. □

Equation (23) is exact algebra. The covariance formula adds to the model of Section 4 an assumption that model does not supply, since each $\boldsymbol{e}_\ell$ is computed from activations that already carry the earlier errors. Three consequences follow, each visible in Equation (24).

- *The residual accumulates.* The variances of the n contributions add, so more quantized layers leave a larger residual.
- *The residual is colored*, meaning that its covariance is not a multiple of I. Equation (24) is proportional to the identity when every A_ℓ is a scaled orthogonal matrix (Exercise I.11), and for generic trained weights it is not, so even if each quantizer's own error were white the residual at the end would not be.
- *Early errors are shaped like the signal.* The error of the input quantizer passes through the same map A_1 as the signal, so for a white input it spreads over the output directions exactly as the signal does. The errors of later quantizers skip the early layers and are not shaped by them.

The flat floor of Section 8 (Proposition 8.1), white noise that lifts every eigenvalue of the feature covariance by the same amount, models one quantizer's error, not E.

Weight rounding is not noise. The weight term of Equation (23) is different in kind. Rounding the weights replaces the map W_ℓ by the slightly different map $W_\ell + \Delta_\ell$, and Δ_ℓ is fixed once the weights are rounded. The error $\Delta_\ell\, Q_a(\tilde{\boldsymbol{h}}_{\ell-1})$ is therefore a fixed linear function of the layer's input, the same every time the same input arrives, and correlated with the signal across a data set. For a single weight it is a pure gain, a fixed rescaling of the output (Exercise I.12). In general it is part gain and part leakage from other directions. Section 8 writes this split along each direction $\boldsymbol{v}_k$ of the features as the signal's component scaled by a gain g_k, plus a part uncorrelated with it (Equation (39)).

Real blocks are not linear chains. For small errors, replace each nonlinear map f (LayerNorm, GELU, softmax) by its Jacobian J_f, the matrix of its partial derivatives at the full-precision input, and Equation (23) holds to first order with Jacobians in place of the W_ℓ. A residual connection $H \leftarrow H + f(H)$ maps a small error $\boldsymbol{d}$ in its input to $\boldsymbol{d} + J_f \boldsymbol{d} = (I + J_f)\, \boldsymbol{d}$, because $f(H + \boldsymbol{d}) \approx f(H) + J_f \boldsymbol{d}$. The identity part carries every earlier error forward unchanged, so in a deep residual network the errors of early layers are not washed out by later ones. The

linearization fails where errors are not small, in particular at a clipped value, whose error is not bounded by $s/2$ (Equation (14)).

Worked example with two quantizers and one colored residual

Take $\boldsymbol{x} \in \mathbb{R}^2$ with $\mathrm{Cov}(\boldsymbol{x}) = I$ and the layers $W_1 = \left(\begin{smallmatrix} 2 & 1 \\ 0 & 0.5 \end{smallmatrix}\right)$ and $W_2 = \left(\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right)$, weights unquantized. The input quantizer has step $s_1 = 0.1$, and the hidden one the larger step $s_2 = 0.25$ because its input has the larger range (the first hidden coordinate has standard deviation $\sqrt{5} \approx 2.24$). The model gives $\eta_1^2 = 0.1^2/12 = 8.33 \times 10^{-4}$ and $\eta_2^2 = 0.25^2/12 = 5.21 \times 10^{-3}$.

Downstream maps. The input quantizer's error passes through both layers, $A_1 = W_2 W_1 = \left(\begin{smallmatrix} 2 & 1 \\ 2 & 1.5 \end{smallmatrix}\right)$, and the hidden quantizer's error only through the second, $A_2 = W_2$. So $A_1 A_1^\top = \left(\begin{smallmatrix} 5 & 5.5 \\ 5.5 & 6.25 \end{smallmatrix}\right)$ and $A_2 A_2^\top = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 2 \end{smallmatrix}\right)$.

Residual. Equation (24) gives $\mathrm{Cov}(E) = \eta_1^2 A_1 A_1^\top + \eta_2^2 A_2 A_2^\top = \left(\begin{smallmatrix} 0.00938 & 0.00979 \\ 0.00979 & 0.01563 \end{smallmatrix}\right)$, with eigenvalues 0.0228 and 0.0022. It is far from white, although each quantizer's error was white by assumption.

Along the signal. The output signal has covariance $A_1 A_1^\top$, with eigenvalues $\lambda_1 = 11.16$ and $\lambda_2 = 0.0896$ along $\boldsymbol{v}_1 = (0.666, 0.746)$ and $\boldsymbol{v}_2 = (-0.746, 0.666)$. The residual energy $\boldsymbol{v}_k^\top \mathrm{Cov}(E)\, \boldsymbol{v}_k$ is 0.0226 and 0.0024. It falls by a factor of 9.3 from the strong direction to the weak one, while the signal falls by a factor of 125.

By source. The input quantizer contributes $\eta_1^2 \lambda_k$ to direction k, exactly proportional to the signal because the input is white, the third consequence above. The hidden quantizer contributes 0.0133 and 0.0023, not shaped by W_1. The residual-to-signal ratios ρ_k are 0.0020 and 0.027, so the weak direction is 13 times worse off relative to its own signal.

Check. Rounding 400,000 standard Gaussian inputs gives energies 0.0221 and 0.0023, within 5% of the model.

The toy ends with a shape you meet again in Section 8, a residual that decays with the rank index more slowly than the signal, so that ρ_k rises toward the tail. A quantizer placed after signal-shaping layers suffices for it (the input quantizer alone gives $\rho_k = \eta_1^2$ at both ranks). Whether this explains a real encoder's residual is an empirical question.

What else is in the residual. If quantization-aware training freezes the backbone and trains only the steps and the head, as the notebook does, E is a pure quantization residual of fixed weights, learned clipping included. If it also retrains the weights, E carries their drift too, a different object under the same symbol (Section 6.5).

Running example: the bus in the rain

The bus crop's $224 \cdot 224 \cdot 3 = 150{,}528$ pixel values become 256 patches of 14×14 pixels and a stream of 257×384 numbers. The 48 quantized layers round $12 \times 257 \times (3 \cdot 384 + 1536) = 8{,}289{,}792$ activations per crop, and all of them feed the class token $\boldsymbol{x} \in \mathbb{R}^{384}$, whose quantized twin is $\boldsymbol{x} + \boldsymbol{\delta}$, one row of E.

Keep the head of the first worked example, with bus-truck margin 0.8, and suppose (synthetic numbers) $\|\boldsymbol{w}_{\text{bus}} - \boldsymbol{w}_{\text{truck}}\| = 1.5$ and a residual of root-mean-square 0.03 per coordinate, so $\|\boldsymbol{\delta}\| = 0.03\sqrt{384} = 0.59$. The bias cancels, and the margin moves by $(\boldsymbol{w}_{\text{bus}} - \boldsymbol{w}_{\text{truck}})^\top \boldsymbol{\delta}$. If $\boldsymbol{\delta}$ pointed along $\boldsymbol{w}_{\text{truck}} - \boldsymbol{w}_{\text{bus}}$, the margin would fall by $1.5 \times 0.59 = 0.88$, the Cauchy-Schwarz bound, more than 0.8, and the bus would become a truck. If $\boldsymbol{\delta}$ had covariance $0.03^2 I$, the move would have standard deviation $1.5 \times 0.03 = 0.045$, eighteen times smaller than the margin. The same residual norm is harmless or decisive depending on which directions it occupies, and Sections 7 and 8 give you the tools to measure them.

Exercise I.11: When does the residual stay white?

(a) Show that if every A_ℓ in Equation (24) is c_ℓ times an orthogonal matrix, then $\mathrm{Cov}(E) = (\sum_\ell c_\ell^2 \eta_\ell^2) I$. (b) In the second worked example, replace W_2 by the rotation $R = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & -1 \\ 1 & 1 \end{smallmatrix}\right)$. Which contribution stays white, which does not, and why?

Solution sketch. (a) $A_\ell A_\ell^\top = c_\ell^2 I$ for each ℓ, and a sum of multiples of I is one. (b) The hidden quantizer contributes $\eta_2^2 R R^\top = \eta_2^2 I$, white. The input quantizer contributes $\eta_1^2 R W_1 W_1^\top R^\top$, whose eigenvalues are η_1^2 times those of $W_1 W_1^\top = \left(\begin{smallmatrix} 5 & 0.5 \\ 0.5 & 0.25 \end{smallmatrix}\right)$, 5.05 and 0.198. One non-orthogonal layer after a quantizer colors its error.

Exercise I.12: Weight rounding is a gain, not noise

A one-weight layer computes $y = wx$ with $w = 0.37$, rounded with step $s_w = 0.1$. (a) Compute $Q(w)$, the weight error Δ, and the output error $Q(w)x - wx$ for $x = 1, 2, -3$. (b) Show that over any data set this error has correlation ± 1 with y, and compute the ratio of signal power to error power. (c) If instead the weight were perturbed by independent noise of variance $s_w^2/12$ redrawn for every input, why could a retrained head undo the first perturbation but not the second?

Solution sketch. (a) $Q(w) = 0.4$ and $\Delta = 0.03$, with errors 0.03, 0.06, and -0.09, each $(\Delta/w)\, y$. (b) The error is a fixed multiple of y, so its correlation with y is $\mathrm{sign}(\Delta/w) = +1$ and the power ratio is $(w/\Delta)^2 = (0.37/0.03)^2 = 152$, or 21.8 dB, for every input distribution. (c) A fixed gain is undone by the fixed factor $w/Q(w) = 0.925$ downstream, while fresh noise is uncorrelated with y and changes with every input, so no fixed map removes it. Swapping a quantizer for matched random noise thus changes more than the error's size (Section 4.4).

Check yourself before moving on

1. How many linear layers does the 12-block ViT-S/14 have, which are they, and which matrix products in a block are not among them?[a]
2. In a quantized linear layer, which tensors are rounded, with which granularity, and which are not?[b]
3. Give two reasons why $X_Q - X$ at the penultimate features is not rounding noise of variance $s^2/12$.[c]

[a] 48, the qkv, proj, fc1, and fc2 of each block. The attention products $Q_{\text{att}} K_{\text{att}}^\top$ and (attention weights)$\times V_{\text{att}}$ multiply two activations and have no trained weights.

[b] The input, with one step for the whole tensor, and the weight matrix, with one step per output channel (Equation (22)). The bias and the output are not rounded; the output is rounded only if it enters the next linear layer's input quantizer.

[c] Any two of these. It sums many transformed errors (Proposition 5.1), so it accumulates and is colored. Weight rounding adds a fixed, signal-dependent term. Clipped values err beyond $s/2$. Retrained weights add drift.

Where this is used. Sections 15 and 18 place each paper's quantizers on Figure 9, and Sections 17 and 20 measure the residual E in real encoders.

6 Post-training quantization and quantization-aware training

What you will be able to do after this section

Say what post-training quantization (PTQ) and quantization-aware training (QAT) each change in a trained network. Set a step with the max-abs, percentile, and mean-squared-error rules, and predict which rule clips. Tell an outlier channel from a rare value and from an outlier token, say why outlier channels cost activations more than weights, and predict the bit width at which one erases the other channels. Rotate an outlier vector with a Hadamard matrix, compute how far the step shrinks, and say what a rotation cannot repair. Derive the two Lloyd–Max conditions, compare a codebook with a uniform grid in decibels, and say what TurboQuant and QuIP add to a rotation and what a codebook costs in hardware. Compute the gradients of the straight-through estimator and of learned step size quantization (LSQ) by hand, and say which samples push the step down and which push it up. Tell a QAT that retrains the weights from one that trains only the steps and a head, and say what each leaves in the feature residual.

Sections 3 to 5 built the quantizer $Q(x) = s\,\mathrm{clamp}(\mathrm{round}(x/s), q_{\min}, q_{\max})$, its rounding error, and its place in a transformer block, and left one number open for every tensor, the step s. Given a network trained in full precision, how are its thousands of steps chosen, and may the network itself change to accommodate them?

The two answers share a trunk and then fork (Figure 10). Post-training quantization keeps the trained weights and chooses the steps from a few example inputs. Quantization-aware training starts the same way and then keeps training with the quantizers in the forward pass, so that the steps, and some or all of the other parameters, adapt to the rounding. Either can end in an integer model that runs on integer kernels (Section 2), although a study may instead evaluate the simulated model in floating point.

6.1 Post-training quantization

Calibration data. A weight matrix is fixed once training ends, so its steps can be read off the weights, for example one per output channel from each row's largest magnitude, as the notebook does. An activation depends on the input, so its range must be estimated by running an unlabeled *calibration set* through the network (Section 3). The set is small (the notebook uses 256 clean crops) and needs no labels. In these notes PTQ therefore means *no gradient training of the network on a task*, not *no data*.

Range setters. For a symmetric quantizer, choosing s is choosing the clip value $c = s\,q_{\max}$, the largest magnitude that survives unclipped (Equation (14)). Three rules are common.

- *Max-abs* sets c to the largest magnitude seen in calibration, $s = \max_i |x_i|/q_{\max}$, the rule of Section 2.4. It never clips the calibration data, so one unusual value, a rare outlier in the sense of Section 6.2, can make the grid coarse for all the others.
- *Percentile* sets c to a high quantile of $|x|$, say the 99.9th, and sacrifices a few rare values to a finer grid for all the others (Gholami et al., 2022, arXiv version, Sec. III-C).
- *Mean squared error (MSE)* picks the s that minimizes $\frac{1}{N}\sum_i (Q(x_i) - x_i)^2$ on the calibration data, the minimum of the curve in Figure 6. It trades rounding against clipping directly (Nagel et al., 2021, Sec. 3.1).

Over several batches the statistic may be smoothed. The notebook keeps a moving average of the batch maxima, $a \leftarrow 0.9\,a + 0.1 \max|x|$, a soft percentile in effect, since one spiky batch moves a only a tenth of the way.

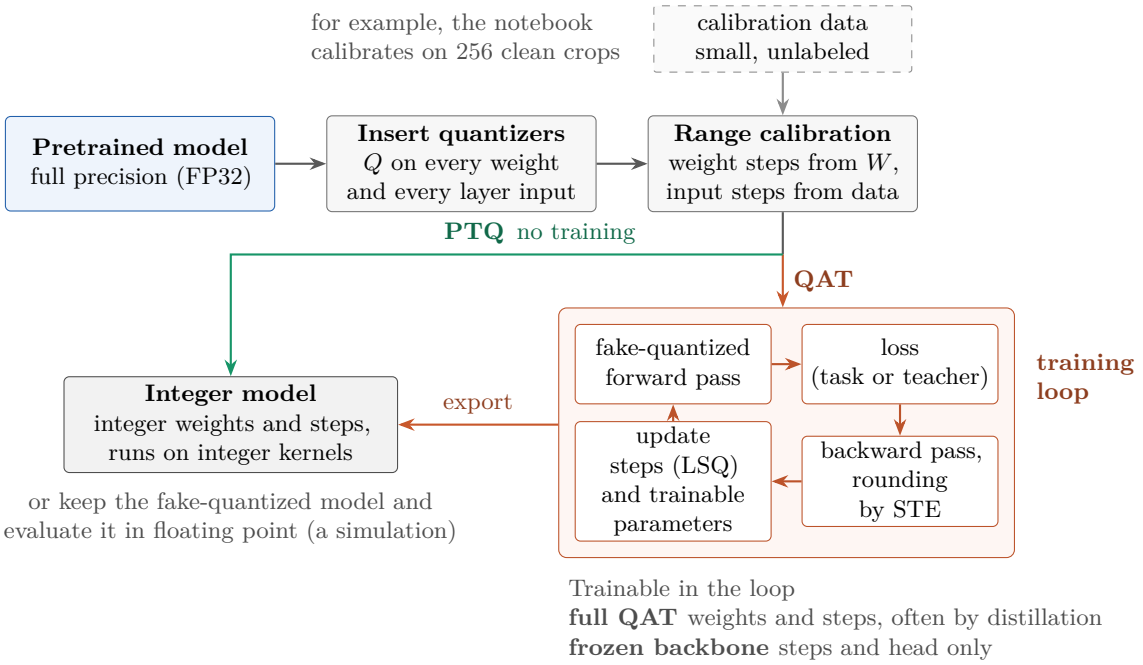


Figure 10: PTQ and QAT share a trunk. Both insert quantizers into a pretrained network and set every step by range calibration on a small unlabeled set. PTQ stops there and exports. QAT adds a training loop whose forward pass is fake-quantized (rounded and dequantized in floating point, Section 6.4), whose backward pass bypasses rounding with the straight-through estimator (Equation (27)), and whose steps are learned with LSQ (Equation (28)). What else trains in the loop differs between recipes, and the difference changes what the feature residual contains (Section 6.5).

Max-abs, min-max, and MinMax

Two rules set the range from the extreme values seen in calibration, and both are often called "MinMax". An affine quantizer (Section 3) takes the range $[\min_i x_i, \max_i x_i]$ (Jacob et al., 2018; Nagel et al., 2021). A symmetric one takes $[-\max_i |x_i|, \max_i |x_i|]$, the max-abs rule. These notes write *min-max* for the first and *max-abs* for the second. When a paper names a method MinMax, check which of the two it uses.

Worked example of three range setters on a Gaussian activation

Let an activation be $x \sim \mathcal{N}(0, 1)$, so the SQNR of Equation (17) is $10 \log_{10}(1/\text{MSE})$, and calibrate on $N = 256$ samples, as the notebook does. The largest of 256 standard Gaussian magnitudes is 3.04 on average (simulation, 20 000 repetitions), which max-abs takes as c. The 99.9th percentile of $|x|$ is 3.29, above that maximum. The two numbers measure different things. The percentile belongs to the whole distribution, while 3.04 is the largest of only 256 samples, and 256 samples cannot estimate a 99.9th percentile, which needs more than a thousand values. Here the percentile rule is therefore wider than max-abs, the reverse of the usual case, in which one outlier makes the sample maximum far larger than any high percentile. The MSE rule, evaluated on the Gaussian by numerical integration as in Section 3, picks $c^\star$.

	$b = 4$, $q_{\max} = 7$			$b = 8$, $q_{\max} = 127$		
Rule	c	s	SQNR	c	s	SQNR
Max-abs	3.04	0.435	18.0 dB	3.04	0.0240	34.2 dB
99.9th percentile	3.29	0.470	17.3 dB	3.29	0.0259	37.4 dB
MSE rule	2.37	0.339	19.3 dB	3.91	0.0308	40.6 dB

At 4 bits rounding is expensive, so the best clip value sits inside the data, at 2.37 standard deviations. At 8 bits rounding is cheap ($s^2/12 = 7.9 \times 10^{-5}$ at the optimum), so the best clip value moves out to 3.91 and max-abs is badly placed. On new inputs, the 0.23% of values beyond 3.04 contribute 3.4×10^{-4} of its total MSE of 3.77×10^{-4}. A short sample makes max-abs too narrow, and one outlier makes it far too wide.

Algorithm for max-abs post-training quantization

Input: trained model with L quantized layers (weights W_ℓ, inputs $\boldsymbol{x}_\ell$), calibration batches $B_1, \dots, B_T$, bit width b, smoothing factor $\beta \in [0, 1)$

Output: one step per output channel of every W_ℓ, one step per layer input

$q_{\max} \leftarrow 2^{b-1} - 1$, $q_{\min} \leftarrow -2^{b-1}$

$s_{w,\ell j} \leftarrow \max_i |W_\ell[j, i]| / q_{\max}$ for every layer ℓ and row j ▷ no data needed

for $t = 1, \dots, T$ **do**

  run batch B_t through the full-precision model; for every ℓ, $m_\ell \leftarrow \max |\boldsymbol{x}_\ell|$

  $a_\ell \leftarrow m_\ell$ if $t = 1$, else $a_\ell \leftarrow \max(a_\ell, m_\ell)$ ▷ or $\beta a_\ell + (1 - \beta) m_\ell$

end for

$s_{x,\ell} \leftarrow a_\ell / q_{\max}$ for every ℓ, and freeze all steps

export the integers $\mathrm{clamp}(\mathrm{round}(W_\ell / s_{w,\ell}), q_{\min}, q_{\max})$ with their steps

Beyond range setting. The three rules fix only where the range sits. They leave alone the direction in which each value rounds, the shape of the tensor, and the grid itself, and most other PTQ methods change one of these (Table 4). AdaRound, for example, chooses up or down for every weight by minimizing each layer's output error on calibration data (Nagel et al., 2020). PTQ therefore does not mean that nothing is optimized; what it excludes is end-to-end training on a task loss.

Running example: the bus in the rain

The encoder that classifies the bus crop, a frozen DINOv2 ViT-S/14 (Section 5), has 48 block linear layers quantized to W8A8, so calibration sets 48 activation steps and one weight step per output channel. Follow the input to the first MLP layer of the last block through the notebook's 8 batches of 32 clean crops. Suppose (synthetic numbers) that the batch maxima are 7.9, 8.4, 12.6, 8.1, 7.7, 9.0, 8.3, 8.0, the third batch holding one crop with an unusually large activation. Max-abs gives $s = 12.6/127 = 0.0992$. The moving average with $\beta = 0.9$ ends at 8.34, so $s = 8.34/127 = 0.0657$. The rounding variances differ by $(12.6/8.34)^2 = 2.28$, which is 3.6 dB of SQNR for every activation that is not clipped. The price is that the largest activations of three batches now saturate. QAT lets the task loss decide whether that price is worth paying (Proposition 6.1).

6.2 Outliers and rotation

To a quantizer, an *outlier* is a value much larger than the typical values that share its step, so its size is measured against those values. For a vector $\boldsymbol{x} \in \mathbb{R}^d$ quantized with one step, the

Table 4: Post-training quantization methods grouped by the problem they address, with one coarse line each. The last row does not quantize weights or activations; it compresses vectors that are stored and later used in inner products.

Problem	Idea	Methods
Where the range sits	set s from calibration statistics (maximum, percentile, MSE), or from a distribution fitted to the tensor	max-abs and min-max (Jacob et al., 2018; Nagel et al., 2021); ACIQ, from a Laplace fit (Banner et al., 2019)
Which way to round	choose up or down for each weight to minimize the output error of a layer or a block	AdaRound (Nagel et al., 2020); BRECQ, block by block (Li et al., 2021); QDrop, which randomly leaves activations unquantized while it optimizes (Wei, Gong, et al., 2022); GPTQ, from second-order information, weights only (Frantar et al., 2023)
A few very large activation channels	keep them in 16 bits	LLM.int8() (Dettmers et al., 2022)
The same	move their scale into the neighboring weights	SmoothQuant (Xiao, Lin, et al., 2023), Outlier Suppression (Wei, Zhang, et al., 2022)
The same	rotate, so that each outlier spreads over all channels (Section 6.2)	QuaRot, Hadamard-based (Ashkboos et al., 2024); SpinQuant, learned rotations (Liu et al., 2025)
Weights at four bits and below	protect the most salient weights, or rotate the weights before rounding	AWQ, salient channels found from the activations (Lin et al., 2024); QuIP, toward incoherent weights (Chee et al., 2023); QuIP#, with a lattice codebook (Tseng et al., 2024)
Activation statistics peculiar to vision transformers	quantizers designed around the post-LayerNorm, post-softmax, and post-GELU statistics	PTQ4ViT, twin uniform quantization (Yuan et al., 2022); RepQ-ViT (Li et al., 2023); APQ-ViT (Ding et al., 2022); IGQ-ViT (Moon et al., 2024)
Vision–language models	PTQ designed for large vision–language models	Q-VLM (Wang, Wang, et al., 2024)
Stored vectors used in inner products, such as a key–value cache	rotate at random, then use one precomputed codebook for every coordinate (Section 6.3)	TurboQuant (Zandieh et al., 2026)

peak-to-rms ratio

$$\frac{\max_i |x_i|}{\mathrm{rms}(\boldsymbol{x})}, \qquad \mathrm{rms}(\boldsymbol{x}) = \frac{\|\boldsymbol{x}\|}{\sqrt{d}},$$

compares the value that sets a max-abs step with the size of an average coordinate. For $d = 384$ coordinates drawn from one Gaussian it is about 3.2 (simulation), close to the 3.04 that max-abs takes from 256 Gaussian calibration samples in Section 6.1. Outliers come in three kinds, which differ in where they sit and in what can remove them (Figure 11a).

- A *rare value* is large in one input and absent from the next. A range setter can clip it at little cost (Section 6.1).
- An *outlier channel* is large at most tokens of most inputs. It is systematic and carries signal, so clipping it would damage every token. Transformer activations often contain a few such channels (Wei, Zhang, et al., 2022; Xiao, Lin, et al., 2023), and vision transformers vary strongly across channels after LayerNorm (Li et al., 2023). For language models, Dettmers et al. (2022, arXiv version, Sec. 4.1) count a hidden dimension as an outlier when its magnitude reaches at least 6 in at least 25% of the layers and at least 6% of the sequence positions.
- An *outlier token* is large in every channel of one token, like the high-norm tokens Darcet et al. (2024) report in vision transformers.

Three words, "outlier", "channel", and "token"

In quantization, as in this section, an outlier is a coordinate of an activation tensor, a value, a channel, or a token, that is large compared with the other values sharing its step. It is a matter of magnitude, measured by the peak-to-rms ratio, and an outlier channel appears for every input, ordinary images included. In out-of-distribution (OOD) detection and anomaly detection (Section 12.4), an outlier is an input, an image or a region of it, that lies outside the training distribution. It is a matter of position relative to the training data, scored on the model's outputs or in its embedding space, and it is rare by definition.

The two senses meet in one place. An unusual input can push activations past a range calibrated on clean data, and a static range then clips them, as the rain streak does in the running example of Section 3. That is a rare value in the first sense, caused by a shifted input. The senses also differ where they look alike. The high-norm tokens of vision transformers are outlier tokens in the first sense, yet they appear "primarily in low-informative background areas" of ordinary images (Darcet et al., 2024, abstract). An anomalous patch, such as debris on the road, is the reverse, informative and unusual, and its embedding need not have a large norm.

A channel, in this section, is one coordinate of the token vectors, the same feature dimension at every token, 384 of them in the running example's encoder and 1536 at the input of fc2 (Section 5.2). It is not a color channel of the image. The patch embedding mixes the three color channels and the 196 pixels of each 14×14 patch into 384 learned features, so no channel after it stands for a color, and only these learned channels reach the activation quantizers.

A token, in a vision transformer, is the vector of one image patch, one row of the activation matrix, plus one class token that stands for no patch, 257 tokens in all for a 224×224 crop. At the input of the first block token t depends only on patch t. From then on attention mixes all tokens, so a token keeps the position of its patch but carries information from the whole image. This is why an outlier token, such as a high-norm token at a background position, is not an anomalous patch. In a language model a token is a piece of text, a word or part of a word, and the definitions above apply unchanged.

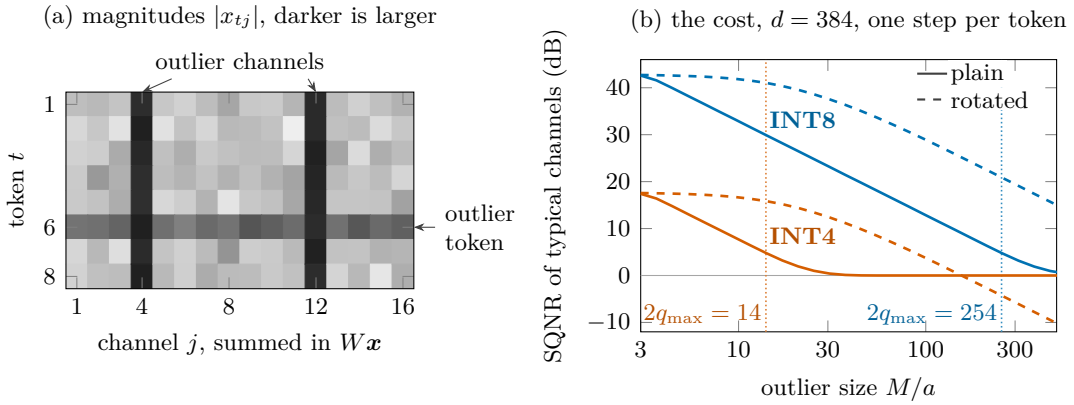


Figure 11: What an outlier channel is, and what it costs. (a) Magnitudes of a synthetic activation matrix, 8 tokens by 16 channels, on a logarithmic gray scale. Typical entries are about 1, token 6 is an outlier token (about 6), and channels 4 and 12 are outlier channels (18 to 30 at every token). (b) SQNR of the typical channels of a 384-channel token whose one outlier channel is M/a times their typical size a, under max-abs with one step per token. Plain max-abs loses 20 dB for every tenfold growth of the outlier and erases the typical channels once $M/a > 2q_{\max}$ (dotted lines). For large outliers a random rotation (dashed) keeps the SQNR that plain max-abs has at an outlier about six times smaller, $\sqrt{384}/3.2$. It lowers the cost but cannot remove it, because the outlier's energy stays in the token. Below 0 dB the error exceeds the error of zeroing the typical channels. Synthetic model and seeds in the figure file, 800 tokens per point.

Why outlier channels hurt activations most. The granularity rule of Section 3.4 decides where an outlier can be isolated. A step may depend on every index except the summed one. A large row of a weight matrix is one output channel, so it takes its own step and costs the other rows nothing. A large activation channel is the summed index j of $\sum_j W_{ij}x_{tj}$, so it shares one step with all the channels of its token, or of the whole tensor, and it sets that step for all of them.

What an outlier channel costs. Let one channel reach M while the other channels have typical values of size a. Max-abs gives $s = M/q_{\max}$, so a typical value spans

$$\frac{a}{s} = \frac{q_{\max}}{M/a} \tag{25}$$

steps, and it falls into the dead zone $|x| < s/2$ of Section 4.3 once $M/a > 2q_{\max}$. Table 5 evaluates the rule. An outlier fifty times the typical size leaves the other channels 2.5 steps at INT8, a coarse but usable grid, and erases them at INT4, whose threshold is 14. Each doubling of M/a costs the typical channels one bit, the 6 dB of Equation (18), which is the straight part of the plain curves in Figure 11b.

Four ways out. The first is to clip. A range setter can clip the outlier channel (Section 6.1), but the channel recurs at every token, so the clipping error recurs too. The second keeps the outlier channels in 16 bits. LLM.int8() computes them in a separate 16-bit matrix product, while more than 99.9% of the values are multiplied in 8 bits (Dettmers et al., 2022), at the price of a second, irregular product. The third moves part of the range into the weights. SmoothQuant divides each activation channel j by a factor $m_j > 0$ and multiplies the matching weight column by the same factor,

$$W\boldsymbol{x} = \big(W\,\mathrm{diag}(\boldsymbol{m})\big)\big(\mathrm{diag}(\boldsymbol{m})^{-1}\boldsymbol{x}\big),$$

Table 5: What one outlier channel of size M costs the channels of typical size a under max-abs. A typical value spans $a/s = q_{\max}/(M/a)$ steps (Equation (25)). Entries below 0.5, in vermilion, lie in the dead zone and round to zero.

b	$q_{\max}$	dead zone once M/a exceeds	a/s at $M/a = 10$	a/s at $M/a = 50$
8	127	254	12.7	2.54
6	31	62	3.10	0.62
4	7	14	0.70	0.14
3	3	6	0.30	0.06

which is exact in real arithmetic and shifts part of the difficulty from the activations, which are hard to quantize, to the weights, which are easier (Xiao, Lin, et al., 2023). The fourth rotates, so that each outlier spreads over all channels. Table 4 lists methods of each kind.

A rotation uses an orthogonal matrix R ($R^\top R = I$) and changes nothing in exact arithmetic. A linear layer computes $W\boldsymbol{x} = (WR^\top)(R\boldsymbol{x})$, so it may quantize the rotated activation $R\boldsymbol{x}$ against the rotated weights $WR^\top$. The rotation keeps every length, $\|R\boldsymbol{x}\|^2 = \boldsymbol{x}^\top R^\top R\,\boldsymbol{x} = \|\boldsymbol{x}\|^2$, but redistributes it over the coordinates, and the quantizer only sees coordinates. A *Hadamard matrix* has entries ± 1 and mutually orthogonal rows. Divided by $\sqrt{d}$ it is orthogonal, and it mixes every coordinate into every other with equal weight, which is what spreads an outlier. QuaRot applies rotations built from Hadamard matrices to the hidden states of language models and folds them into neighboring weights (Ashkboos et al., 2024).

Worked example of rotating $(10, 0, 0, 0)$ and a realistic neighbor

The 4×4 Hadamard matrix, scaled to be orthogonal, is

$$H = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}, \qquad H^\top H = I.$$

The extreme case. For $\boldsymbol{x} = (10, 0, 0, 0)$, $H\boldsymbol{x} = (5, 5, 5, 5)$, and the norm is 10 both times. The peak-to-rms ratio falls from 2 to 1, since $\mathrm{rms}(\boldsymbol{x}) = 10/\sqrt{4} = 5$ both times. This ratio is what matters, because max-abs sets the step from the peak, while the rms measures how large the coordinates are on average. A lower peak-to-rms ratio therefore gives the same norm a finer step relative to the size of its coordinates. With INT4 ($q_{\max} = 7$) and max-abs the step halves from $10/7 = 1.43$ to $5/7 = 0.71$, and the rounding variance of every channel falls by 4, the 6.02 dB of one extra bit (Equation (18)).

A realistic neighbor. Take $\boldsymbol{x} = (10,\ 0.8,\ -0.6,\ 0.4)$. Plain INT4 max-abs uses $s = 10/7 = 1.4286$ and gives the integers $(7, 1, 0, 0)$, so two small channels fall into the dead zone $|x| < 0.71$ and the sum of squared errors is 0.915. Rotating first gives $H\boldsymbol{x} = (5.3,\ 4.1,\ 5.5,\ 5.1)$, $s = 5.5/7 = 0.7857$, and the integers $(7, 5, 7, 6)$. Rotating the dequantized values back with $H^\top$ gives $\hat{\boldsymbol{x}} = (9.82,\ 1.18,\ -0.39,\ 0.39)$ and a sum of squared errors of 0.218, a factor 4.2 below plain PTQ, spread over all four channels instead of erasing the small ones (Figure 12). The layer itself never rotates back. It computes $(WH^\top)\,Q(H\boldsymbol{x}) = W\hat{\boldsymbol{x}}$ with $\hat{\boldsymbol{x}} = H^\top Q(H\boldsymbol{x})$, so the error it sees is $W(\hat{\boldsymbol{x}} - \boldsymbol{x})$, the error measured here in the original channels.

Why a rotation works in general. The Hadamard example spreads a single spike perfectly evenly. For a general vector a random rotation does almost as well. If R is drawn uniformly among all rotations, $R\boldsymbol{x}$ points in a uniformly random direction whatever $\boldsymbol{x}$ was, and for large d each of its coordinates is close to a Gaussian with variance $\|\boldsymbol{x}\|^2/d$. The peak-to-rms ratio of

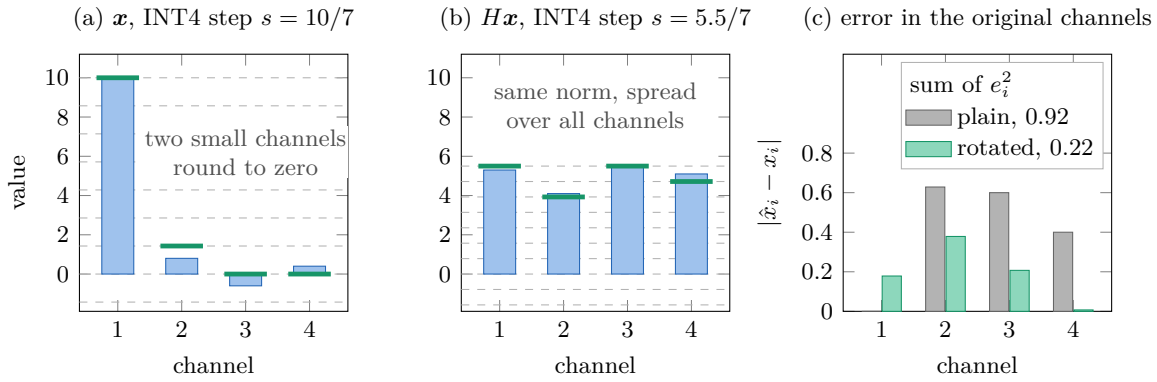


Figure 12: Rotation shrinks the step by spreading an outlier. (a) The vector $\boldsymbol{x} = (10, 0.8, -0.6, 0.4)$ with its INT4 max-abs grid, $s = 10/7$ (dashed levels); the quantized values (green ticks) erase two small channels. (b) After the Hadamard rotation the same norm is spread over all four channels, the grid is 1.8 times finer, and no channel falls into the dead zone. (c) The error $e_i = \hat{x}_i - x_i$ per original channel after rotating back; the bars show $|e_i|$, and the legend gives the sum of squared errors, 0.92 plain and 0.22 rotated. Numbers from the worked example, computed exactly.

$R\boldsymbol{x}$ is then that of d Gaussian draws, about 3.2 at $d = 384$ and 3.8 at $d = 4096$ (simulation), whatever the ratio of $\boldsymbol{x}$ was. A max-abs step therefore falls from $\max_i |x_i|/q_{\max}$ to about $3.2\,\mathrm{rms}(\boldsymbol{x})/q_{\max}$, and the error, rotated back, spreads evenly over all channels, as in panel (c) of Figure 12.

Worked example of one outlier channel in a 384-channel token

Take $d = 384$, the width of the bus encoder, with 383 channels drawn from $\mathcal{N}(0, 1)$, so $a = 1$, and one channel fixed at $M = 50$ in every token. Then $\mathrm{rms}(\boldsymbol{x}) = \sqrt{(383 + 2500)/384} = 2.74$, and the peak-to-rms ratio is 18.3. A fixed random rotation brings it to 3.1 (simulation over 2000 tokens, max-abs with one step per token).

INT8. The plain step is $50/127 = 0.394$, so by Equation (25) a typical value spans 2.54 steps, and the typical channels have an SQNR of 18.9 dB. The rotated step is 0.066, 5.9 times finer, and their SQNR is 34.3 dB, a gain of 15 dB, or about 2.6 bits.

INT4. The plain step is $50/7 = 7.14$, every typical value lies in the dead zone, and the typical channels are erased, an SQNR of 0 dB. The rotated step is 1.21, the typical channels keep an SQNR of 9.1 dB, and the sum of squared errors per token falls from 382 to 47.

What a rotation cannot do. A rotation keeps every norm, so it spreads the energy of an outlier but cannot remove it. Two limits follow. First, the rotated step grows with the token's norm. When one channel carries most of the token's energy, its spread rounding error reaches every channel. In the model of the worked example the rotated INT4 curve of Figure 11b falls below 0 dB once M/a exceeds about 150, and at $M/a = 300$ the rotation gives almost four times the squared error of plain max-abs, which at least represents the outlier itself exactly. Second, a token whose norm is far larger than the others', like the high-norm tokens of Darcet et al. (2024), keeps large coordinates after any rotation and still sets a per-tensor range (Exercise I.14). Rotation repairs outlier *channels*, not outlier *tokens*.

Rotating the weights. The same argument applies to the weights. Row i of the rotated weights $WR^\top$ is $(R\boldsymbol{w}_i)^\top$, where $\boldsymbol{w}_i^\top$ is row i of W, so the rotation lowers the peak-to-rms ratio of every row, and with it every per-channel step. A rotation therefore also helps a weight matrix

whose rows contain a few large entries, which matters most at four bits and below (Table 5).

Where the rotation is applied. The factor $R^\top$ is always merged into W offline. The factor R on the activation can be merged into the weights of the layer that produced $\boldsymbol{x}$ only if every operation between the two layers commutes with R. An elementwise nonlinearity such as GELU does not, so after it $R\boldsymbol{x}$ must be computed at run time. A Hadamard matrix makes this cheap. For d a power of two its fast transform needs $d \log_2 d$ additions and subtractions, against d^2 multiply-adds for a general matrix, about 49 thousand against 17 million at $d = 4096$.

6.3 From a uniform grid to a codebook

A rotation does more than shrink the step. It also fixes, in advance, the distribution of every coordinate the quantizer will see. One coordinate of a vector drawn uniformly from the unit sphere in $\mathbb{R}^d$ has a Beta-type density proportional to $(1 - x^2)^{(d-3)/2}$, which approaches the Gaussian $\mathcal{N}(0, 1/d)$ as d grows (Zandieh et al., 2026, arXiv version, Lemma 1). After a random rotation every coordinate of $R\boldsymbol{x}/\|\boldsymbol{x}\|$ has this density, whatever $\boldsymbol{x}$ was. A quantizer for a known density can be designed once, offline, with no calibration data. The MSE rule of Section 6.1 already picks the best uniform step for it. Giving up the uniform grid gives the best quantizer of all.

The Lloyd–Max conditions. A scalar quantizer with $L = 2^b$ levels is a *codebook* of levels $c_1 < \cdots < c_L$ with thresholds $t_1 < \cdots < t_{L-1}$ between them. It maps x to c_k when $t_{k-1} < x \le t_k$, with $t_0 = -\infty$ and $t_L = +\infty$, and stores only the index k, in b bits. For an input density p its mean squared error is $\sum_k \int_{t_{k-1}}^{t_k} (x - c_k)^2 p(x)\, dx$, and at a minimum two conditions hold (Gray & Neuhoff, 1998; Lloyd, 1982; Max, 1960),

$$t_k = \frac{c_k + c_{k+1}}{2}, \qquad c_k = \mathbb{E}[x \mid t_{k-1} < x \le t_k]\,. \tag{26}$$

The first condition sends every value to its nearest level. The second puts every level at the mean of the values it stands for. Lloyd's algorithm alternates the two until nothing moves, which is k-means clustering applied to a continuous density. A uniform grid satisfies the first condition but, in general, not the second.

Worked example of a 2-bit Lloyd–Max codebook for a Gaussian

Let $x \sim \mathcal{N}(0, 1)$ and start from the uniform levels ± 0.5 and ± 1.5, with thresholds 0 and ± 1. With φ and Φ the standard normal density and distribution function, the centroid condition moves the inner level to $\mathbb{E}[x \mid 0 < x \le 1] = (\varphi(0) - \varphi(1))/(\Phi(1) - \Phi(0)) = 0.1570/0.3413 = 0.460$ and the outer level to $\mathbb{E}[x \mid x > 1] = \varphi(1)/(1 - \Phi(1)) = 0.2420/0.1587 = 1.525$. The nearest-level condition then moves the outer threshold to $(0.460 + 1.525)/2 = 0.992$. Repeating the two steps converges to the levels ± 0.453 and ± 1.510 with thresholds 0 and ± 0.982, the 2-bit codebook that Zandieh et al. (2026, arXiv version, Sec. 3.1) list in units of $1/\sqrt{d}$. Its mean squared error is 0.1175, an SQNR of 9.30 dB. The best uniform grid, with step 0.996 and levels ± 0.498 and ± 1.494, reaches 0.1189, or 9.25 dB. The codebook gains 0.05 dB.

How much a codebook adds. Table 6 gives the answer. Over the best uniform grid the codebook gains 0.05 dB at 2 bits and 0.85 dB at 4 bits, and over the symmetric grid of Equation (13) 0.7 to 1.0 dB. Most of what a rotation buys therefore comes from the rotation itself, which removes the outliers and makes the density known, and the codebook adds the last decibel. Even the Lloyd–Max codebook stays 1.6 to 3.9 dB below the bound at 1 to 4 bits, and

Table 6: SQNR in dB of three scalar quantizers for a Gaussian input, each at its best setting, computed by numerical integration. The bound is $10 \log_{10} 4^b = 6.02\, b$ dB, from the lower bound 4^{-b} on the mean squared error that Zandieh et al. (2026, arXiv version, Theorem 3) prove for any quantizer that spends b bits per coordinate of a unit vector. The symmetric grid of Equation (13) has a zero level and no 1-bit version.

b	Lloyd–Max codebook	best uniform grid, no zero level	grid of Equation (13)	bound
1	4.40	4.40	–	6.02
2	9.30	9.25	8.26	12.04
3	14.62	14.27	13.91	18.06
4	20.22	19.38	19.26	24.08

the gap approaches 4.35 dB, a factor $\sqrt{3}\pi/2 = 2.72$ in mean squared error, as b grows. Zandieh et al. (2026, arXiv version, Theorem 1) prove this factor as an upper bound for their codebooks, using for large b the high-resolution formula of Panter and Dite (1951) (Exercise I.15). A scalar quantizer cannot close the gap, because it codes each coordinate on its own. A *vector quantizer* codes several coordinates jointly, which can close part of it. QuIP# uses such a codebook, built on the eight-dimensional E_8 lattice, for groups of eight weights (Tseng et al., 2024).

TurboQuant. Zandieh et al. (2026) combine the two steps for vectors that are stored and later used in inner products, such as the keys and values that a language model caches while it generates text, or the embeddings of a vector search index. TurboQuant rotates every vector with one random orthogonal matrix, obtained from the QR decomposition of a Gaussian matrix, and quantizes every coordinate with the Lloyd–Max codebook of the Beta density. The codebook is computed once for each bit width and stored, so no calibration data are needed (Zandieh et al., 2026, arXiv version, Sec. 3.1). The mean squared error stays within the factor $\sqrt{3}\pi/2$ of the lower bound (Zandieh et al., 2026, arXiv version, Theorems 1 and 3). On key–value caches the authors report "absolute quality neutrality with 3.5 bits per channel and marginal quality degradation with 2.5 bits per channel" (abstract), measured on LongBench with two instruction-tuned language models of 7 and 8 billion parameters (arXiv version, Sec. 4.3).

Codebooks attenuate, too. The centroid condition of Equation (26) makes every level the mean of the values it replaces, so the error is uncorrelated with the quantized value and $\mathbb{E}[Q(x)^2] = \mathbb{E}[x^2] - \mathbb{E}[e^2]$, exactly as for the minimum-error step of Proposition 4.1. The quantized vector is shorter than the original. With a random rotation, the symmetry of the sphere then gives, for any fixed $\boldsymbol{y}$,

$$\mathbb{E}[\langle \boldsymbol{y}, \hat{\boldsymbol{x}} \rangle] = \Big(1 - \frac{\mathbb{E}[\|\boldsymbol{e}\|^2]}{\|\boldsymbol{x}\|^2}\Big) \langle \boldsymbol{y}, \boldsymbol{x} \rangle,$$

so every inner product is biased toward zero by the gain of Proposition 4.1. At one bit the Gaussian codebook is $\pm\sqrt{2/\pi}$ and the factor is $2/\pi = 0.64$, the multiplicative bias that Zandieh et al. (2026, arXiv version, Sec. 3.2) state. Their remedy spends $b - 1$ bits on the codebook and the last bit on the residual $\boldsymbol{r} = \boldsymbol{x} - \hat{\boldsymbol{x}}$. It stores the signs $\operatorname{sign}(S\boldsymbol{r})$ for a matrix S with independent standard normal entries, a 1-bit quantized Johnson–Lindenstrauss (QJL) map, together with the norm $\|\boldsymbol{r}\|$, and adds back the estimated inner product of $\boldsymbol{y}$ with the residual. The estimate is then unbiased (Zandieh et al., 2026, arXiv version, Definition 1 and Theorem 2).

QuIP and QuIP#. QuIP applies the rotation idea to the weights of language models (Chee et al., 2023). It calls a weight matrix $W \in \mathbb{R}^{m \times n}$ μ-incoherent when $|W_{ij}| \le \mu \|W\|_F / \sqrt{mn}$ for all entries (Chee et al., 2023, arXiv version, Definition 1). Since $\|W\|_F / \sqrt{mn}$ is the rms of

the entries, incoherence is a bound on the peak-to-rms ratio of Section 6.2. QuIP multiplies the weights, and the Hessian that weighs their rounding errors, by random orthogonal matrices built as Kronecker products of small ones, so that both become incoherent, and then rounds adaptively with a procedure it calls LDLQ. Without the incoherence step, QuIP reduces to a more efficient implementation of OPTQ, the method published earlier as GPTQ (Frantar et al., 2023). With it, the authors report the first quantization of language models to two bits per weight with viable results. QuIP# replaces the random matrices by the randomized Hadamard transform, which is faster, rounds groups of eight weights to its E_8 lattice codebook, and adds fine-tuning (Tseng et al., 2024). Both quantize weights only.

What a codebook costs in hardware. A codebook level is not a multiple of one step, so the integer dot product of Section 2.4 no longer applies. The product of two indices is not proportional to the product of the values they stand for. A codebook model therefore stores indices and turns them back into numbers by table lookup before a floating-point multiply-add. It saves memory and memory traffic, not arithmetic. A rotation followed by a uniform grid keeps the integer kernel instead. QuaRot, for instance, quantizes all weights, activations, and the key–value cache to 4 bits and performs all matrix multiplications in 4 bits (Ashkboos et al., 2024). On an integer accelerator, such as the embedded board of the running example, only the second kind runs on the integer units.

6.4 Quantization-aware training

QAT lets the network see its quantizers while it trains. Every quantized layer runs *fake quantization*, rounding values onto the integer grid and immediately dequantizing them, so the arithmetic stays in floating point but carries the rounding and clipping errors of an integer model, up to the details of Section 2. The obstacle is immediate. Rounding is a staircase whose derivative is zero almost everywhere and undefined at the jumps (Figure 13b), so exact backpropagation would deliver a zero gradient upstream of every quantizer.

The straight-through estimator (STE). In the forward pass compute $Q(x)$ exactly. In the backward pass treat round as the identity, which amounts to differentiating the surrogate $s\,\mathrm{clamp}(x/s, q_{\min}, q_{\max})$ instead of Q:

$$\frac{\partial Q}{\partial x} \approx \begin{cases} 1, & sq_{\min} < x < sq_{\max}, \\ 0, & \text{otherwise (clipped).} \end{cases} \tag{27}$$

Inside the clip range the upstream gradient passes through unchanged. Outside it is zero, because there the clamp really is flat.

▷ **Check yourself.** With $b = 3$ and $s = 1$, an upstream gradient of 0.7 arrives at $x = 2.6$ and at $x = 5.0$. What does the STE pass back to each?[8]

Bengio et al. (2013) study this estimator, and BinaryConnect used it to train networks with binarized weights (Courbariaux et al., 2015). The STE is a heuristic, the derivative of nothing the forward pass computes, justified by the fact that training with it works. In code it is one line, `(x.round() - x).detach() + x`, which returns `x.round()` in the forward pass and the gradient of `x` in the backward pass. In the forward pass the two `x` terms cancel and leave `x.round()`. In the backward pass `.detach()` hides the bracket from differentiation, so only the final `x` is differentiated, with derivative 1. The notebook's `round_ste` is this line.

[8] 0.7 to $x = 2.6$, which is inside the range $(-4, 3)$, and 0 to $x = 5.0$, which is clipped. The exact derivative would pass back 0 to both.

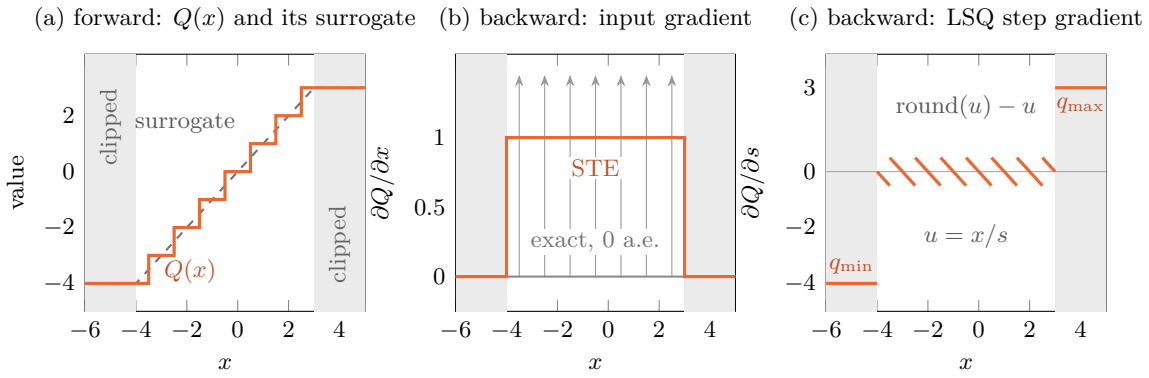


Figure 13: What the backward pass sees, for $b = 3$ and $s = 1$ ($q_{\min} = -4$, $q_{\max} = 3$). (a) The forward staircase $Q(x)$ and the surrogate $s\,\mathrm{clamp}(x/s, q_{\min}, q_{\max})$ (dashed) whose slope the STE uses; shaded regions are clipped. (b) The exact derivative of Q is zero except for impulses at the seven jumps (gray arrows); the STE replaces it by 1 inside the range and 0 outside. (c) The LSQ step gradient of Equation (28) is a sawtooth of amplitude $\frac{1}{2}$ inside the range and jumps to $q_{\min}$ or $q_{\max}$ where values are clipped, so a clipped value pulls on the step far harder than a rounded one. All curves are the stated formulas, evaluated exactly.

Learning the step. Learned Step Size Quantization (LSQ) makes s itself a trainable parameter, one per quantized tensor (Esser et al., 2020). Its gradient follows from the same substitution. Inside the range, write $u = x/s$ and $Q = s\,\mathrm{round}(u)$, and apply the product rule with the STE convention $\partial\,\mathrm{round}(u)/\partial u = 1$,

$$\frac{\partial Q}{\partial s} = \mathrm{round}(u) + s\,\frac{\partial\,\mathrm{round}(u)}{\partial u}\,\frac{\partial u}{\partial s} = \mathrm{round}(u) + s \cdot 1 \cdot \left(-\frac{x}{s^2}\right) = \mathrm{round}(u) - u.$$

Outside the range Q is the level $sq_{\max}$ or $sq_{\min}$, whose derivative with respect to s is $q_{\max}$ or $q_{\min}$.

The LSQ step gradient (Esser et al., 2020, Eq. 3), in the notes' notation with $u = x/s$:

$$\frac{\partial Q}{\partial s} = \begin{cases} \mathrm{round}(u) - u, & q_{\min} < u < q_{\max}, \\ q_{\min}, & u \le q_{\min}, \\ q_{\max}, & u \ge q_{\max}, \end{cases} \qquad \text{scaled by} \qquad g = \frac{1}{\sqrt{N\,q_{\max}}}. \tag{28}$$

N is the number of weights in the layer for a weight step and the number of features in the layer for an activation step; $q_{\max}$ plays the role of Esser et al.'s Q_P.

Inside the range the gradient is the signed distance to the nearest level in units of the step, a sawtooth between $-\frac{1}{2}$ and $+\frac{1}{2}$ (Figure 13c). It equals e/s, so inside the range panel (c) is the sawtooth of Figure 5 in units of the step. Outside it is a full $q_{\max}$ or $q_{\min}$, so a clipped value pulls on the step at least $2q_{\max}$ times harder than any rounded one (254 times at 8 bits). The scale g balances the step against the weights. Esser et al. (2020, Sec. 2.2) want each step's update, relative to the step's own size, to be about as large as the weights' updates relative to theirs. Without g it would not be, for two reasons. The step shrinks as the bit width grows, and its gradient grows with N, because one step collects a term from every element it quantizes. LSQ initializes the step at $2\langle|x|\rangle/\sqrt{q_{\max}}$ (Esser et al., 2020); the notebook starts from calibration instead, so QAT begins where PTQ stops. Under Adam and AdamW a constant factor on a gradient nearly cancels in the normalized update (Kingma & Ba, 2015; Loshchilov & Hutter, 2019), so how N is counted matters little there.

Proposition 6.1: Rounding pulls the LSQ step down, clipping pushes it up

Let $L(s) = \frac{1}{n}\sum_{i=1}^{n} e_i^2$ with $e_i = Q(x_i) - x_i$ over n samples, differentiated with the LSQ gradient Equation (28). Let $\mathcal{I}$ be the samples inside the range, $\mathcal{C}$ the clipped ones, and $q_i \in \{q_{\min}, q_{\max}\}$ the level a clipped sample saturates at. Then

$$\frac{\partial L}{\partial s} = \frac{2}{n}\Big[\underbrace{\sum_{i\in\mathcal{I}} \frac{e_i^2}{s}}_{\geq 0} + \underbrace{\sum_{i\in\mathcal{C}} q_i\, e_i}_{\leq 0}\Big],$$

so gradient descent shrinks s in proportion to the rounding error and grows it in proportion to the clipping error.

Proof. The chain rule gives $\partial L/\partial s = \frac{2}{n}\sum_i e_i\, \partial Q(x_i)/\partial s$. Inside the range $\mathrm{round}(u_i) - u_i = (Q(x_i) - x_i)/s = e_i/s$, so the term is $e_i^2/s \geq 0$. A sample clipped at the top has $q_i = q_{\max} > 0$ and $e_i = sq_{\max} - x_i \leq 0$; one clipped at the bottom has $q_i = q_{\min} < 0$ and $e_i = sq_{\min} - x_i \geq 0$. Either way $q_i e_i \leq 0$. □

The proposition concerns the LSQ estimator; the exact derivative of the piecewise-smooth L has interior terms of either sign (Exercise I.13). With a task loss the factor $2e_i/n$ becomes the upstream gradient $\partial L/\partial Q(x_i)$, which need not share the sign of e_i. The task then sets the balance, and a learned step can clip values that the calibrated range kept (hands-on box below). Through Equation (14) a learned step also sets a learned clip value $c = sq_{\max}$.

Worked example of one LSQ step by hand

Take $b = 3$ ($q_{\min} = -4$, $q_{\max} = 3$), $s = 1$, and a batch of $n = 4$ activations $x = (0.3, 1.2, 2.6, 5.0)$ under the reconstruction loss of Proposition 6.1.

Forward. The levels are $(0, 1, 3, 3)$, the last one clipped, so $e = (-0.3, -0.2, 0.4, -2.0)$ and $L = \frac{1}{4}(0.09 + 0.04 + 0.16 + 4.00) = 1.0725$.

Backward. By Equation (28), $\partial Q/\partial s = (-0.3, -0.2, 0.4, 3)$, the first three equal to e_i/s. The contributions $\frac{2}{n} e_i\, \partial Q/\partial s$ are $(+0.045, +0.020, +0.080, -3.000)$, summing to $\partial L/\partial s = -2.855$. Three rounding terms ask for a smaller step; the one clipped sample outvotes them about 20 to 1, so the step grows (Exercise I.13 takes the update).

Algorithm for one QAT step with the STE and LSQ

Input: model with fake quantizers whose steps were set by calibration, trainable set Θ (steps, and weights or a head), frozen parameters, mini-batch (X, Y), a learning rate per parameter group

1: forward pass, every quantized layer computing $Q(W)\, Q(\boldsymbol{x}) + \boldsymbol{b}$ in floating point
2: $\ell \leftarrow$ loss of the output against Y (task) or an FP32 teacher (distillation)
3: backward pass, with Equation (27) through every rounding and Equation (28), times g, for every step
4: update each parameter in Θ with AdamW at its group's learning rate
5: keep every step positive, for example $s \leftarrow \max(s, 10^{-4})$

What Θ holds differs between recipes (Section 6.5).

6.5 Two kinds of QAT, and bit width

Resolving an ambiguity about the word QAT

The word QAT names two different interventions. In *full QAT* the weights train together with the steps, often under distillation from the full-precision model, so the network can move to weights that suit the grid. In *frozen-backbone QAT* the backbone weights stay fixed and only the LSQ steps and a small head train, the setting of a linear probe (Figure 23) and of the notebook's Part 5.

The difference reaches the feature residual $X_Q - X$ of Section 5. After full QAT it mixes rounding error with the drift of the retrained weights. After frozen-backbone QAT it is a pure quantization residual of fixed weights, learned clipping included. A spectrum (Section 7) reorganized by full QAT and one left nearly unchanged by frozen-backbone QAT therefore describe different interventions and need not contradict each other.

Distillation and low-rank adapters. *Distillation* trains the quantized student to reproduce a teacher's outputs (Hinton et al., 2015); in QAT the natural teacher is the full-precision model itself, matched through its outputs or through a mean squared error between student and teacher features. *QAT-LoRA* builds on low-rank adaptation (LoRA) (Hu et al., 2022) in its quantization-aware form (Xu et al., 2024) and trains only a low-rank weight update $\Delta W = BA$ of small LoRA rank r, which is not the rank index k of Section 7.

Bit width. At a fixed clip value the step is $s = c/q_{\max}$, so going from 8 to 4 bits multiplies it by $127/7 = 18.1$ and the rounding variance by 329, a loss of 25.2 dB of SQNR. Section 4 held the full width $s(2^b - 1)$ fixed instead and found a factor of $255/15 = 17$ and 24.6 dB. The two conventions count the levels differently, and both give about 25 dB for four bits. The MSE-optimal clip value also falls from 3.91 to 2.37 standard deviations (Gaussian worked example), so a 4-bit quantizer must clip real signal as well, and more small values fall into the dead zone (Section 4.3). Weight and activation widths can be set separately. W4A8, for example, means 4-bit weights and 8-bit activations, and only the weights then pay the 4-bit price.

In the hands-on session, notebook Parts 1 and 5

Part 1 runs this section on a 1-D regression whose output is quantized to 3 bits. In a CPU run (your numbers may move slightly) the model error is 0.23 in full precision, 0.72 with a naive step 8/3, 0.56 with PTQ ($s = \max|y|/q_{\max} = 2.02$), and 0.45 with QAT, whose LSQ step shrinks to 1.48. The top level $3 \times 1.48 = 4.44$ then sits below the target's peak of 5.60 and clips 8.2% of the grid, as Proposition 6.1 allows.

Part 5 runs frozen-backbone QAT on the bus encoder, with moving-average calibration on 256 crops and then 400 steps that train only the activation steps and the head. In Parts 3 and 4 "QAT" means training a linear model on quantized inputs, with no STE or LSQ.

Exercises

Exercise I.13: The LSQ gradient by hand

Take the batch $x = (0.3, 1.2, 2.6, 5.0)$ of the LSQ worked example ($b = 3$, $s = 1$). (a) Drop the clipped sample and recompute $\partial L/\partial s$. Which way does the step move? (b) The exact derivative of the piecewise-smooth L uses $\partial Q/\partial s = n_i$, the integer level, for every sample. Compute it for the full batch and compare it with the LSQ value -2.855. (c) Take one LSQ update with learning rate 0.1 and $g = 1/\sqrt{4 \cdot 3}$, in the notebook's convention where N counts every element of the batch. What happens to the top level

and to L?

Solution sketch. (a) The three rounding terms remain, and $\partial L/\partial s = \frac{2}{3}(0.09 + 0.04 + 0.16) = +0.193$, so gradient descent shrinks the step. (b) The levels are $n = (0, 1, 3, 3)$ and the products $e_i n_i$ are $(0, -0.2, 1.2, -6.0)$, so $\partial L/\partial s = \frac{2}{4}(-5.0) = -2.5$. The sign agrees with LSQ and the size differs. The exact interior terms take either sign, while the LSQ ones e_i^2/s never do. (c) $s \leftarrow 1 - 0.1 \times 0.289 \times (-2.855) = 1.082$, the top level moves from 3 to 3.25, and L falls from 1.0725 to 0.841. Repeated updates settle at $s = 1.620$, where the two sums balance, while a grid search puts the minimum of L at 1.53, because the STE gradient is not the exact derivative of L.

Exercise I.14: What a rotation can and cannot do

In $d = 384$ one token has norm 120 and the others about $\sqrt{384} = 19.6$, with roughly standard normal coordinates. (a) Using $\max_i |y_i| \geq \mathrm{rms}(\boldsymbol{y}) = \|\boldsymbol{y}\|/\sqrt{d}$, show that after any rotation the large token keeps a coordinate of magnitude at least 6.12. (b) A typical token's largest coordinate is near $\sqrt{2 \ln d} = 3.45$ (an extreme-value heuristic; simulation gives about 3.2 at this d). Which token sets a per-tensor INT8 step after any rotation, and what fixes it?

Solution sketch. (a) An orthogonal R keeps the norm, so the rotated large token has rms $120/\sqrt{384} = 6.12$, a lower bound on its largest coordinate, since $\max_i y_i^2 \geq \frac{1}{d}\sum_i y_i^2$. (b) That exceeds the typical 3.45, so the large token sets the range after any rotation, a step of at least $6.12/127 = 0.048$ against the 0.027 the others need. A random rotation leaves it near $6.12 \times 3.45 = 21.1$, a step of 0.166. The norm bound alone forces this only for norms above $\sqrt{384} \times 3.45 \approx 68$. One step per token fixes it for any norm.

Exercise I.15: One bit, and the high-resolution limit

Let $x \sim \mathcal{N}(0, 1)$. (a) Use Equation (26) to show that the 1-bit Lloyd–Max codebook is $\pm\sqrt{2/\pi}$, and compute its mean squared error and its ratio to the bound 4^{-1}. (b) The high-resolution formula of Panter and Dite (1951) gives the smallest mean squared error of an L-level quantizer as about $\frac{1}{12L^2}\big(\int p(x)^{1/3}\,dx\big)^3$. Evaluate it for the Gaussian with $L = 2^b$, and write the result as a factor times 4^{-b} and in dB. (c) Why does the 1-bit codebook shrink the expected inner product $\mathbb{E}[\langle \boldsymbol{y}, \hat{\boldsymbol{x}} \rangle]$ by $2/\pi$ after a random rotation?

Solution sketch. (a) By symmetry the threshold is 0, and the centroid of $x > 0$ is $\mathbb{E}[x \mid x > 0] = 2\varphi(0) = \sqrt{2/\pi} = 0.798$. The error is $\mathbb{E}\big[x^2\big] - 0.798^2 = 1 - 2/\pi = 0.363$, which is 1.45 times 0.25, the ratio Zandieh et al. (2026) give for $b = 1$. (b) $p^{1/3} = (2\pi)^{-1/6} e^{-x^2/6}$ integrates to $(2\pi)^{-1/6}\sqrt{6\pi}$, whose cube is $6\sqrt{3}\,\pi$, so the formula gives $\frac{\sqrt{3}\pi}{2} 4^{-b} = 2.72 \cdot 4^{-b}$, 4.35 dB above the bound. The exact Lloyd–Max ratios are 1.45, 1.88, 2.21, and 2.43 at 1 to 4 bits, so the formula is approached only slowly. (c) The centroid condition gives $\mathbb{E}[x\,Q(x)] = \mathbb{E}\big[Q(x)^2\big] = 2/\pi$, so each rotated coordinate keeps the fraction $2/\pi$ of its projection on the input, and the symmetry of a random rotation turns this into $\mathbb{E}[\langle \boldsymbol{y}, \hat{\boldsymbol{x}} \rangle] = \frac{2}{\pi}\langle \boldsymbol{y}, \boldsymbol{x} \rangle$ (Proposition 4.1).

Check yourself before moving on

Answer these before moving on to Section 7. (1) With $b = 3$ and $s = 1$, what do the STE and the LSQ gradient return at $x = 4.2$ and at $x = 0.3$?[9] (2) Why does a Hadamard rotation repair an outlier channel but not an outlier token?[10] (3) What trains in full QAT and in frozen-backbone QAT, and which one leaves a pure quantization residual at the features?[11] (4) A token's largest channel is 20 times the typical size of the others. Under max-abs, at which of 8, 6, 4, and 3 bits does a typical value fall into the dead zone?[12] (5) Why can a Lloyd–Max codebook not run on the integer kernel of Section 2.4, and what does it save instead?[13]

Where this is used. Sections 15 and 18 read which of these recipes each paper runs and what its QAT trains (Table 8), Section 16 reads rotation and bit width across a large grid, and Section 20 returns to the learned step as a learned clip value.

7 Representations and their spectrum

What you will be able to do after this section

You will be able to compute a cosine and a covariance by hand and relate its eigenvalues to singular values; read a spectrum on semi-log and log-log axes and count its rank index from an axis that starts at 0; compute explained variance and two effective ranks, and say why one printed without its formula cannot be read; and tell a fixed basis from a recomputed one.

Section 5 ended with the bus crop turned into one vector of 384 numbers, the pooled feature that a linear classifier head reads. Thousands of such vectors form a cloud whose shape is summarized by the eigenvalues of its covariance, computed with a singular value decomposition (SVD). This section builds that tool from linear algebra, and Section 8 asks what noise does to it. Here "spectrum" always means these eigenvalues, not the Fourier spectrum of an image (Section 9).

7.1 Feature vectors, cosines, and coordinates

A feature vector is a point $\boldsymbol{x} \in \mathbb{R}^d$, with $d = 384$ for the DINOv2 ViT-S/14 encoder of the running example and the notebook (Section 10). The feature vectors of N inputs, stacked as rows, form $X \in \mathbb{R}^{N \times d}$, a 1000×384 matrix for a test set of 1000 crops. The angle between two feature vectors is measured by

$$\cos(\boldsymbol{x}, \boldsymbol{y}) \;=\; \frac{\boldsymbol{x}^\top \boldsymbol{y}}{\|\boldsymbol{x}\|\,\|\boldsymbol{y}\|} \;\in\; [-1, 1], \qquad \|\boldsymbol{x}\| = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}}. \tag{29}$$

Scaling $\boldsymbol{x}$ by $a > 0$ multiplies the numerator and $\|\boldsymbol{x}\|$ by a, so the cosine is *scale-invariant*; it reads direction and ignores length. It is *not* shift-invariant, since subtracting the same vector from both arguments changes their angle, which matters once features are centered. An *embedding cosine* compares two feature vectors, as CLIP does with an image and a prompt (Section 10); a *profile cosine* compares two per-rank profiles, vectors indexed by the rank index k (Definition 7.1).

Let the columns $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d$ of $V \in \mathbb{R}^{d \times d}$ be orthonormal, so $V^\top V = V V^\top = I$. The coordinates of $\boldsymbol{x}$ in this basis are $\boldsymbol{z} = V^\top \boldsymbol{x}$, and $\boldsymbol{x} = V\boldsymbol{z}$. Since $\boldsymbol{x}^\top \boldsymbol{y} = \boldsymbol{z}^\top V^\top V \boldsymbol{w} = \boldsymbol{z}^\top \boldsymbol{w}$ with $\boldsymbol{w} = V^\top \boldsymbol{y}$, lengths, angles and cosines are the same in every orthonormal basis, the fact that lets the outlier rotation of Section 6 be undone exactly. A basis changes only how $\|\boldsymbol{x}\|^2 = \sum_k z_k^2$

[9] At $x = 4.2$ ($u > q_{\max}$) the STE returns 0 and LSQ returns $q_{\max} = 3$. At $x = 0.3$ the STE returns 1 and LSQ returns $\mathrm{round}(0.3) - 0.3 = -0.3$.

[10] A rotation preserves every token's norm and only redistributes it over channels. A channel outlier is one large coordinate in many tokens, which the rotation spreads; an outlier token's largest coordinate can never fall below its norm divided by $\sqrt{d}$.

[11] Full QAT trains the weights and the steps, often by distillation; frozen-backbone QAT trains only the steps and a head. The frozen-backbone residual $X_Q - X$ is the pure quantization residual, since the full-QAT one also carries the drift of the retrained weights.

[12] At 4 and 3 bits, whose thresholds $2q_{\max}$ are 14 and 6 (Table 5). At 6 and 8 bits the thresholds are 62 and 254, and a typical value spans $q_{\max}/20 = 1.55$ and 6.35 steps.

[13] Its levels are not multiples of one step, so the product of two indices does not give the product of the values, and the kernel must look the levels up and multiply in floating point. It saves memory and memory traffic, not arithmetic.

is shared among the coordinates, and the rest of the section is about the basis that makes this sharing most uneven.

▷ **Check yourself.** What is $\cos(\boldsymbol{x}, -2\boldsymbol{x})$? What is the cosine between $(3,4)$ and $(4,3)$, before and after subtracting $(3,3)$ from both?[14]

7.2 Covariance and principal directions

Write $\boldsymbol{\mu} = \frac{1}{N}\sum_i \boldsymbol{x}_i$ for the mean feature vector and $X_c = X - \mathbf{1}\boldsymbol{\mu}^\top$ for the centered matrix, whose columns have mean zero. The covariance matrix is

$$C \;=\; \frac{1}{N} X_c^\top X_c \;=\; \frac{1}{N}\sum_{i=1}^{N} (\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^\top \;\in\; \mathbb{R}^{d\times d}. \tag{30}$$

C_{jj} is the variance of feature j and C_{jl} the covariance of features j and l; the unbiased factor $1/(N-1)$ would rescale every entry alike and change neither directions nor the shape of the spectrum. For a unit vector $\boldsymbol{v}$ the centered projections $\boldsymbol{v}^\top(\boldsymbol{x}_i - \boldsymbol{\mu})$ have mean zero and variance $\frac{1}{N}\sum_i \left(\boldsymbol{v}^\top(\boldsymbol{x}_i - \boldsymbol{\mu})\right)^2 = \boldsymbol{v}^\top C \boldsymbol{v} \geq 0$. So C returns the variance of the cloud along any direction, and the question is which gives the most.

Eigendecomposition and the direction of largest variance. The symmetric positive semidefinite C has orthonormal eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d$ and eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ with

$$C \;=\; V\Lambda V^\top \;=\; \sum_{k=1}^{d} \lambda_k \boldsymbol{v}_k \boldsymbol{v}_k^\top, \qquad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_d), \qquad \lambda_k = \boldsymbol{v}_k^\top C\, \boldsymbol{v}_k. \tag{31}$$

The maximum of $\boldsymbol{v}^\top C \boldsymbol{v}$ over unit vectors is λ_1, attained at $\boldsymbol{v}_1$, and $\boldsymbol{v}_k$ maximizes it among unit vectors orthogonal to $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}$.
Proof. Write a unit vector as $\boldsymbol{v} = \sum_k c_k \boldsymbol{v}_k$ with $\sum_k c_k^2 = 1$. Then $\boldsymbol{v}^\top C \boldsymbol{v} = \sum_k \lambda_k c_k^2 \leq \lambda_1 \sum_k c_k^2 = \lambda_1$, with equality at $c_1 = 1$. Orthogonality to $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}$ forces $c_1 = \cdots = c_{k-1} = 0$, and the same bound gives λ_k. □

This is principal component analysis (PCA; see Jolliffe & Cadima, 2016, for a review). The vector $\boldsymbol{v}_k$ is the k-th principal direction and $z_k = \boldsymbol{v}_k^\top(\boldsymbol{x} - \boldsymbol{\mu})$ the k-th principal coordinate. Principal coordinates are uncorrelated, since $\frac{1}{N}\sum_i z_{ik} z_{il} = \boldsymbol{v}_k^\top C \boldsymbol{v}_l = 0$ for $k \neq l$, so in them the covariance is simply Λ (Figure 14b). An eigenvector is fixed only up to its sign, and variances and squared projections, everything these notes build on the spectrum, ignore the sign.

The SVD computes the same thing. Software takes the SVD of the centered matrix, $X_c = USV^\top$, where for $N \geq d$ the matrix $U \in \mathbb{R}^{N\times d}$ has orthonormal columns, $S = \operatorname{diag}(S_{11}, \ldots, S_{dd})$ with $S_{11} \geq \cdots \geq S_{dd} \geq 0$, and $V \in \mathbb{R}^{d\times d}$ is orthogonal. Substituting into (30),

$$C = \frac{1}{N} VSU^\top US\, V^\top = V\, \frac{S^2}{N}\, V^\top \qquad \Longrightarrow \qquad \lambda_k = \frac{S_{kk}^2}{N}, \tag{32}$$

so the right singular vectors are the principal directions, and $S_{kk}/\sqrt{N} = \sqrt{\lambda_k}$ is the standard deviation along $\boldsymbol{v}_k$. The notes write S because σ is the blur width (Section 9) and Σ a noise covariance (Section 8).

[14] -1, since a negative factor flips the direction. Then $24/25 = 0.96$ before the shift, and 0 after it, because the shifted vectors are $(0,1)$ and $(1,0)$. Same points, a different origin, a different angle.

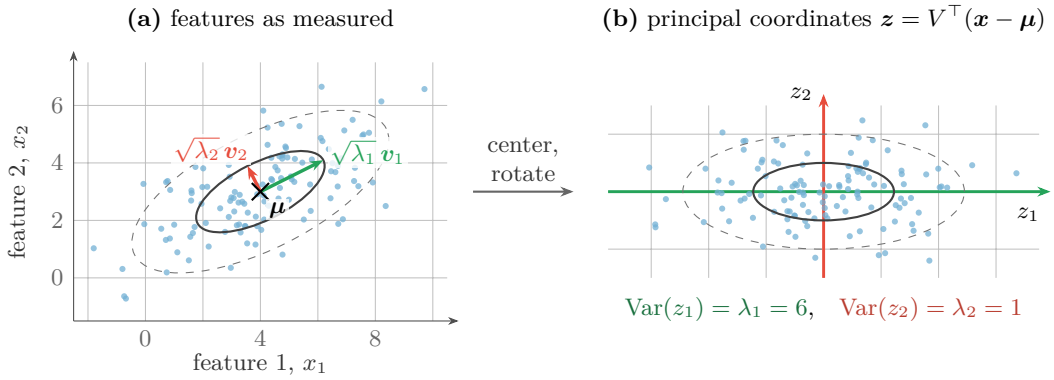


Figure 14: Principal directions of a point cloud. (a) 120 points from a Gaussian with the mean $\boldsymbol{\mu} = (4, 3)$ and covariance C of the worked example, rescaled so that their sample mean and covariance are exactly $\boldsymbol{\mu}$ and C. Ellipses are the one- and two-standard-deviation contours; the arrows $\sqrt{\lambda_1}\, \boldsymbol{v}_1$ (green, strong) and $\sqrt{\lambda_2}\, \boldsymbol{v}_2$ (red, weak) are the principal axes. (b) The same points in principal coordinates. Rotating changes no distance or angle, and centering moves the origin without changing the cloud's shape; together they align the ellipse with the axes, leaving two uncorrelated coordinates with variances 6 and 1. Grid spacing is two units in both panels.

Worked example, four crops and two features by hand

The crops give $\boldsymbol{x}_1 = (1, 1)$, $\boldsymbol{x}_2 = (3, 3)$, $\boldsymbol{x}_3 = (5, 5)$, $\boldsymbol{x}_4 = (7, 3)$. The mean is $\boldsymbol{\mu} = (4, 3)$, and the centered rows are $(-3, -2)$, $(-1, 0)$, $(1, 2)$, $(3, 0)$. Summing products and dividing by $N = 4$,

$$C = \tfrac{1}{4}\begin{pmatrix} 20 & 8 \\ 8 & 8 \end{pmatrix} = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}, \qquad \det(C - \lambda I) = \lambda^2 - 7\lambda + 6 = (\lambda - 6)(\lambda - 1),$$

so $\lambda_1 = 6$ and $\lambda_2 = 1$ (trace $7 = 6 + 1$, determinant $6 = 6 \cdot 1$). From $(C - 6I)\boldsymbol{v} = \boldsymbol{0}$, $\boldsymbol{v}_1 = (2, 1)/\sqrt{5} \approx (0.894, 0.447)$, at $26.6°$ from the first axis, and $\boldsymbol{v}_2 = (-1, 2)/\sqrt{5}$. The principal coordinates $z_1 = (-8, -2, 4, 6)/\sqrt{5}$ and $z_2 = (-1, 1, 3, -3)/\sqrt{5}$ have mean squares 6 and 1 and mean product 0. By (32) the singular values of X_c are $S_{11} = \sqrt{24} = 4.90$ and $S_{22} = 2$, and $\boldsymbol{v}_1$ carries $6/7 = 85.7\%$ of $\operatorname{tr} C = 7$.

7.3 Spectrum, rank index, head, and tail

Definition 7.1: Spectrum, rank index, spectral head and tail

Let a feature covariance C have eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ with orthonormal eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d$. The *spectrum* is the sequence $(\lambda_k)_{k=1}^{d}$ plotted against k, and the *rank index* of $\boldsymbol{v}_k$ is its position k, so $k = 1$ is the direction of largest variance. Small k is the *spectral head*, large k the *spectral tail*, and a "high-rank" direction is a tail direction, one of *low* variance.

Counting from zero. Software, and many published plots, count from 0. When an axis starts at 0, plotted index j is rank index $k = j + 1$; the notebook counts from 1. Band labels need the same care. Bands labeled "0–8", "8–16", "16–32" share their endpoints, so they read as half-open intervals $[0, 8)$, $[8, 16)$, $[16, 32)$ of plotted indices, and "0–8" is the first eight directions, $k = 1, \ldots, 8$, not nine. When a source does not state its convention, say which reading you take. Head and tail have no sharp boundary either; Figure 15 uses $k \leq 16$ and $k > 128$ only for illustration. "Head" in this sense is never the classifier head.

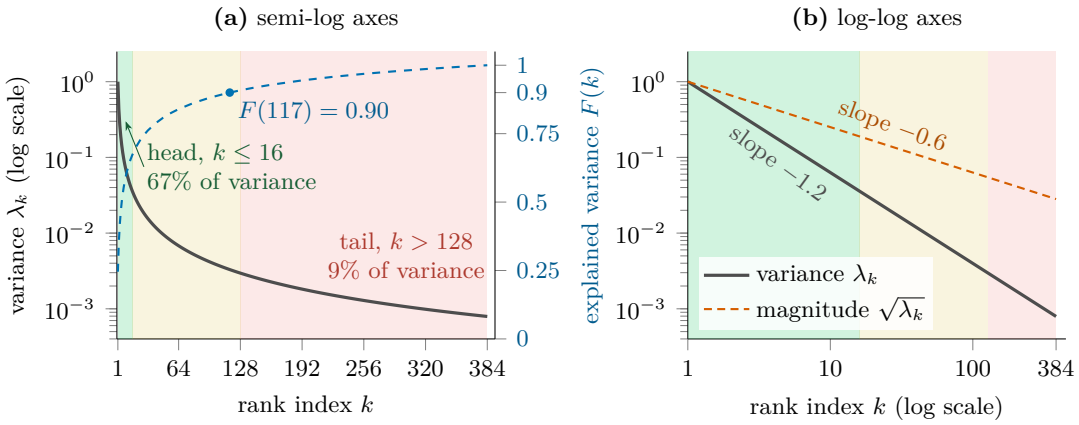


Figure 15: The synthetic model spectrum $\lambda_k = k^{-1.2}$, $k = 1, \dots, 384$. (a) On semi-log axes the power law plunges and then flattens. The dashed curve (right axis) is the explained variance $F(k)$ of (33). Half of the variance sits in the first 5 directions and 67% in the head $k \le 16$, while the tail $k > 128$, two thirds of all directions, holds 9%. (b) On log-log axes the spectrum is a line of slope -1.2 and the magnitude $\sqrt{\lambda_k}$ a line half as steep.

Reading a spectrum on logarithmic axes. Spectra span powers of ten, so their variance axis is logarithmic. The simplest model of their usual shape, steep over the first ranks and slow afterwards, is the power law $\lambda_k = \lambda_1 k^{-\alpha}$; the notes use $\alpha = 1.2$ and $d = 384$ here and in Section 8, a model chosen for convenience and not a fit to any data. Four rules let you read such plots.

- *Semi-log axes* (linear k). Equal heights are equal ratios, and one decade is a factor ten in variance, or 10 dB (Section 4). The power law plunges and then flattens (Figure 15a).
- *Log-log axes.* The power law is a straight line of slope $-\alpha$, because $\log_{10} \lambda_k = \log_{10} \lambda_1 - \alpha \log_{10} k$ (panel b). Likewise a proportionality $y = c\,x$ is a line of slope one, lifted by $\log_{10} c$.
- *Magnitude or variance.* A plot of the magnitude $\sqrt{\lambda_k}$ halves every slope and every decade count, since $\log_{10} \sqrt{\lambda_k} = \frac{1}{2} \log_{10} \lambda_k$. Check whether an axis labeled "magnitude" or "energy" shows $\sqrt{\lambda_k}$ or λ_k before you read a slope off it.
- *Normalization.* Dividing every λ_k by λ_1 or by $\operatorname{tr} C$ shifts a log plot vertically and keeps its shape, so compare shapes before heights.

▷ **Check yourself.** A magnitude spectrum falls by 1.2 decades between $k = 10$ and $k = 100$ on log-log axes. What is α, and by how many dB does the variance fall over those ranks?[15]

Running example: the bus in the rain

Small variance is not small importance. Shrink the encoder to $d = 4$ (every number synthetic) and classify by the cosine to class prototypes in centered principal coordinates, a synthetic choice. The bus crop is $\boldsymbol{z} = (1.8, 0.9, 0.3, 0.1)$, and the prototypes $\boldsymbol{m}_{\text{bus}} = (2.0, 1.0, 0.4, 0.3)$ and $\boldsymbol{m}_{\text{truck}} = (2.0, 1.0, -0.4, -0.3)$ agree on the head, both classes being large boxy vehicles, and differ only in the tail. The cosines are $4.65/(2.037 \cdot 2.291) = 0.996$ for bus and $4.35/4.668 = 0.932$ for truck, so the crop is a bus, and the head holds $(3.24 + 0.81)/4.15 = 97.6\%$ of its squared length. Now keep only $k \le 2$. The crop becomes $(1.8, 0.9, 0, 0)$, both prototypes $(2, 1, 0, 0)$, and both cosines 1.000, a tie. Dropping

[15] The magnitude slope is -1.2, so $\alpha = 2.4$, and the variance falls 2.4 decades, which is 24 dB.

2.4% of the squared length removed all the evidence that separates a bus from a truck. Section 8 asks what rounding noise does to this tail, and Section 9 what blur does and why a tail direction is not the same as fine image detail.

7.4 How many directions matter

The total variance is the mean squared distance to the center, $\frac{1}{N}\sum_i \|\boldsymbol{x}_i - \boldsymbol{\mu}\|^2 = \operatorname{tr} C = \sum_k \lambda_k$. The first k directions carry the fraction $F(k)$, and keeping only them gives the rank-k reconstruction $\hat{\boldsymbol{x}}^{(k)}$ with mean squared error

$$F(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{j=1}^{d} \lambda_j} = \frac{\sum_{j=1}^{k} S_{jj}^2}{\sum_{j=1}^{d} S_{jj}^2}, \qquad \hat{\boldsymbol{x}}^{(k)} = \boldsymbol{\mu} + \sum_{j=1}^{k} z_j \boldsymbol{v}_j, \qquad \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i^{(k)}\|^2 = (1 - F(k)) \operatorname{tr} C. \tag{33}$$

The error is Pythagoras in the principal basis. The discarded part $\sum_{j>k} z_j \boldsymbol{v}_j$ has squared length $\sum_{j>k} z_j^2$, whose mean is $\sum_{j>k} \lambda_j$. No other truncation does better. Eckart and Young (1936) showed that every $B \in \mathbb{R}^{N \times d}$ of matrix rank at most k has $\|X_c - B\|_F^2 \geq \sum_{j>k} S_{jj}^2 = N \sum_{j>k} \lambda_j$, with equality for the truncated SVD, so no k-dimensional linear summary of the features loses less variance than the top k principal directions.

Effective rank. To compress "how many directions are used" into one number, normalize the spectrum to proportions $p_k = \lambda_k / \sum_j \lambda_j$. Two standard definitions are the entropy effective rank of Roy and Vetterli (2007) and the participation ratio, a common measure (Gao et al., 2017),

$$\operatorname{erank}(C) = \exp\Big(-\sum_{k=1}^{d} p_k \ln p_k \Big), \qquad 0 \ln 0 = 0, \tag{34}$$

$$\operatorname{PR}(C) = \frac{\big(\sum_k \lambda_k\big)^2}{\sum_k \lambda_k^2} = \frac{1}{\sum_k p_k^2}. \tag{35}$$

Roy and Vetterli define (34) for any matrix through its singular values, which for C are the λ_k. Both measures lie between 1 and d, equal d for a flat spectrum and 1 for a single nonzero eigenvalue, ignore a common rescaling, and satisfy $\operatorname{PR} \leq \operatorname{erank}$ (Exercise I.16). They give different numbers for the same spectrum, so an effective rank printed without its formula cannot be compared with any other; two such numbers support a comparison only when one formula measured both.

Worked example, one spectrum and three effective ranks

For $\lambda_k = k^{-1.2}$, $d = 384$ (Figure 15), $\operatorname{tr} C = 4.071$ and $F(16) = 0.672$; $F \geq 0.9$ first at $k = 117$ and $F \geq 0.99$ at $k = 337$. Keeping $k \leq 16$ leaves a mean squared error $(1 - 0.672) \times 4.071 = 1.33$. Equation (35) gives $\operatorname{PR} = 12.0$ and (34) gives $\operatorname{erank} = 46.7$. Apply the Roy and Vetterli formula to X_c instead of C, as its definition equally allows, and $p_k \propto S_{kk} \propto \sqrt{\lambda_k}$ gives 271.0. The same 384 eigenvalues have an "effective rank" of 12.0, 46.7 or 271.0, depending on a choice a report can leave unstated.

7.5 Fixed basis versus recomputed basis

You will often compare two sets of features on the same N inputs, the full-precision FP32 features X and a set X' with mean $\boldsymbol{\mu}'$ and covariance C', for instance the features of a quantized encoder or the FP32 features of blurred inputs. There are two ways to put X' on a spectrum.

Fixed basis. Keep the FP32 directions and measure $\boldsymbol{v}_k^\top C' \boldsymbol{v}_k$. Position k keeps meaning "the FP32 direction $\boldsymbol{v}_k$" even when X' has more variance along $\boldsymbol{v}_{k+1}$; the ranks are not reordered. If X' is centered with the FP32 mean $\boldsymbol{\mu}$ rather than its own mean $\boldsymbol{\mu}'$, the measurement is $\boldsymbol{v}_k^\top C' \boldsymbol{v}_k + (\boldsymbol{v}_k^\top(\boldsymbol{\mu}' - \boldsymbol{\mu}))^2$, and a mean shift shows up as extra variance. So a fixed-basis measurement must state its centering.

Recomputed basis. Diagonalize $C' = V' \Lambda' V'^\top$ itself. Now k means the new features' own k-th direction, which need not be near $\boldsymbol{v}_k$. The k-dimensional version of the maximum property in (31), Ky Fan's maximum principle (Fan, 1949), says that no k orthonormal directions capture more variance of C' than its top k eigenvectors, so

$$\sum_{j=1}^{k} \lambda'_j \;\geq\; \sum_{j=1}^{k} \boldsymbol{v}_j^\top C' \boldsymbol{v}_j \quad \text{for every } k, \qquad \text{with equality at } k = d \text{ (both equal } \operatorname{tr} C'). \tag{36}$$

With both spectra centered at the new features' own mean $\boldsymbol{\mu}'$, a recomputed spectrum therefore always looks at least as concentrated in its head as the same features on the old basis. If the fixed-basis side is centered at the old mean $\boldsymbol{\mu}$, its mean-shift term is not covered by (36), and the inequality can fail.

Worked example, the same new features and two verdicts

Keep C from the four-crop example, with $\lambda = (6, 1)$, $\boldsymbol{v}_1 = (2, 1)/\sqrt{5}$, $\boldsymbol{v}_2 = (-1, 2)/\sqrt{5}$, and head fraction $F(1) = 6/7 = 0.857$. Let the new features have the same mean and $C' = \left(\begin{smallmatrix} 4 & 3 \\ 3 & 4 \end{smallmatrix}\right)$, total variance 8.

- *Fixed basis.* $C'\boldsymbol{v}_1 = (11, 10)/\sqrt{5}$ gives $\boldsymbol{v}_1^\top C' \boldsymbol{v}_1 = 32/5 = 6.4$, and $C'\boldsymbol{v}_2 = (2, 5)/\sqrt{5}$ gives $\boldsymbol{v}_2^\top C' \boldsymbol{v}_2 = 8/5 = 1.6$. The head fraction is $6.4/8 = 0.800$, less concentrated than FP32.
- *Recomputed basis.* C' has eigenvalues 7 and 1, with $\boldsymbol{v}'_1 = (1, 1)/\sqrt{2}$ at 18.4° from $\boldsymbol{v}_1$ (cosine $3/\sqrt{10} = 0.949$). The head fraction is $7/8 = 0.875$, more concentrated than FP32, and $7 \geq 6.4$, as (36) requires.
- *A mean shift.* Shift the new mean by $3\boldsymbol{v}_1$ and center at the old mean. The fixed-basis values become $6.4 + 3^2 = 15.4$ and 1.6, a head fraction $15.4/17 = 0.906$, now above the recomputed 0.875.

The first two verdicts are both true. Variance left the old leading direction, and the new cloud has its own leading direction, rotated by 18.4°. The third shows why a fixed-basis measurement must state its centering.

Section 8 takes the fixed basis one step further and projects the *difference* $X' - X$ on each $\boldsymbol{v}_k$, the per-rank residual energy.

Four meanings of "rank"

Matrix rank $\operatorname{rank}(A)$ counts the nonzero singular values of a matrix. Centering costs one, since the rows of X_c add up to the zero vector, so $\operatorname{rank}(X_c) \leq \min(N - 1, d)$; "full-rank noise" is noise that reaches every direction in this sense. The *rank index* k is a position in the sorted spectrum (Definition 7.1); "high-rank components" are large-k, low-variance tail directions. The *LoRA rank* r bounds the matrix rank of a weight update BA with $B \in \mathbb{R}^{m \times r}$, $A \in \mathbb{R}^{r \times n}$ (Hu et al., 2022); it concerns weights, not features. The *effective rank*, (34) or (35), is a real number between 1 and d. These notes always name which one is meant and never write "the rank" alone.

In the hands-on session

Notebook Part 3.1 builds 48 independent Gaussian features with variances $\lambda_k = k^{-1.3}$, so the rank index is the column index, counted from 1; compute $F(k)$, erank and PR before quantizing. Part 5.2 projects DINOv2 ViT-S/14 features of the DAWN test crops on the fixed basis of the clean FP32 features (Section 7.5) and keeps 383 of the 384 directions; Exercise I.17 gives a likely reason.

Exercises

Exercise I.16: A correlated pair

Let $C = \left(\begin{smallmatrix} a & b \\ b & a \end{smallmatrix}\right)$ with $0 \le b \le a$, $a > 0$. (i) Show that $\lambda_{1,2} = a \pm b$ with $\boldsymbol{v}_{1,2} = (1, \pm 1)/\sqrt{2}$. (ii) Write $F(1)$, erank and PR as functions of $r = b/a$ and evaluate them at $r = 0$, 0.5, 1. (iii) Show PR $\le$ erank for every spectrum. (Hint. The logarithm is concave.)

Solution sketch. (i) $C(1, \pm 1)^\top = (a \pm b)(1, \pm 1)^\top$. (ii) $p = \big((1+r)/2, (1-r)/2\big)$, so $F(1) = (1+r)/2$ and PR $= 2/(1+r^2)$. At $r = 0$ both effective ranks are 2; at $r = 0.5$, erank $= \exp(0.562) = 1.755$ and PR $= 1.6$; at $r = 1$ both are 1. (iii) Jensen's inequality with weights p_k gives $\sum_k p_k \ln p_k \le \ln \sum_k p_k^2$; negate and exponentiate.

Exercise I.17: Matrix rank of LayerNorm features

A LayerNorm output is $\boldsymbol{y} = \boldsymbol{\gamma} \odot (\boldsymbol{x} - m\mathbf{1})/s + \boldsymbol{\beta}$, with m and s the mean and standard deviation of the entries of $\boldsymbol{x}$ and all $\gamma_j \neq 0$. (i) Show that $\sum_j (y_j - \beta_j)/\gamma_j = 0$ for every input. (ii) Conclude that the centered matrix Y_c of N outputs satisfies $Y_c \boldsymbol{w} = \mathbf{0}$ for $w_j = 1/\gamma_j$, so $\operatorname{rank}(Y_c) \le d - 1$ and $\lambda_d = 0$. (iii) What does this suggest about notebook Part 5.2 keeping 383 directions?

Solution sketch. (i) The sum equals $\sum_j (x_j - m)/s = 0$. (ii) Every row, hence the mean row, satisfies the same equation, so every centered row is orthogonal to $\boldsymbol{w}$; a null vector means a zero singular value and, by (32), a zero eigenvalue. (iii) The notebook does not say why; the notes' guess is that its pooled feature is a final LayerNorm output, so $\lambda_{384} \approx 0$ and any ratio divided by it is meaningless. Print the smallest singular value to check.

Check yourself before moving on

1. Why does $\boldsymbol{v}^\top C \boldsymbol{v}$ give the variance along a unit vector $\boldsymbol{v}$, and which $\boldsymbol{v}$ maximizes it?[a]
2. A plot of a 768-direction spectrum counts its axis from 0 and shades a band labeled "64–768". Which rank indices does the band hold, and is their variance large or small?[b]
3. Two reports print effective ranks of 40 and 12. Which representation is more spread out?[c]
4. New features have a smaller head fraction than FP32 on the FP32 basis and a larger one on their own basis. Is that a contradiction?[d]

[a] The centered projections have mean zero and mean square $\boldsymbol{v}^\top C \boldsymbol{v}$; the maximum is λ_1, at $\boldsymbol{v}_1$.

[b] Read half-open, plotted indices 64 to 767 (index 768 does not exist), so $k = 65, \ldots, 768$, 704 tail directions of small variance.

[c] You cannot tell without the formulas and dimensions. One model spectrum gives 12.0, 46.7 or 271.0 depending on the formula.

[d] No. By (36) a recomputed basis always captures at least as much variance in its top k directions, so the two numbers answer different questions (0.800 against 0.875 in the worked example).

Where this is used. Section 17 reads a fixed-basis and a recomputed-basis analysis of real encoder features with these tools, and Section 20 reads per-rank profiles, their cosine, and a printed effective rank.

8 What noise does to a spectrum

What you will be able to do after this section

- Prove that isotropic noise lifts every eigenvalue of a feature covariance by the same amount, and explain why this flat floor is a model rather than a measurement.
- Compute the per-rank energy e_k, the ratio ρ_k and SQNR_k of any residual on a fixed full-precision basis, the one object behind every spectral figure in Part II.
- Tell a white residual from a colored one, and read SQNR_k as graded, not as a threshold.
- Show that attenuating a direction requires error anticorrelated with the signal, which the dead zone and clipping produce and independent noise cannot.
- Say what the noise-floor picture explains and what it leaves open.

Section 4 gave rounding a size, the variance $\eta^2 = s^2/12$ of Equation (16). Section 7 gave a feature vector a shape, the ordered eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d$ of its covariance. This section asks what an error of a given size does to a spectrum of a given shape. The first answer is a model; most of the section is about the measurement that tests it, the two ways real residuals depart from it, and what the picture leaves open. It starts with a single coordinate.

8.1 One strong and one weak feature

Figure 16 rounds two sinusoids with the same step $s = 1$. The strong one, amplitude $A = 3$, crosses seven levels. The weak one, $A = 0.15$, never leaves the dead zone $|x| < s/2$ of the mid-tread quantizer of Equation (13), where Section 4 showed that the error is exactly $-x$.

Worked example on burial and erasure

A sinusoid of amplitude A has power $A^2/2$, and the uniform-noise model of Section 4 gives the rounding error the power $\eta^2 = 1/12 = 0.0833$, whatever the signal.

- *Strong,* $A = 3$. The model predicts $\mathrm{SQNR} = 10\log_{10}(4.5/0.0833) = 17.3\,\mathrm{dB}$, and averaging the actual squared error (0.072) gives 18.0 dB, close.
- *Weak,* $A = 0.15$. The model predicts $10\log_{10}(0.01125/0.0833) = -8.7\,\mathrm{dB}$, a signal buried under noise 7.4 times stronger. In fact $Q(x) = 0$ at every instant, the error is $e = -x$, its power equals the signal's, and $\mathrm{SQNR} = 0\,\mathrm{dB}$ exactly.

A signal buried under fresh independent noise can be recovered by averaging many noisy copies. A signal erased by the dead zone cannot, because every copy is the same zero. Rounding is a fixed function of its input (Section 4.4), and the difference returns in Proposition 8.2 and Section 8.8.

8.2 Independent noise on a feature vector, the flat floor

Let the signal be a feature vector $\boldsymbol{x} \in \mathbb{R}^d$ with mean $\boldsymbol{\mu}$ and covariance $C = V\Lambda V^\top$ (Equations (30) and (31)), and add an error $\boldsymbol{e}$ with covariance Σ_E, uncorrelated with $\boldsymbol{x}$.

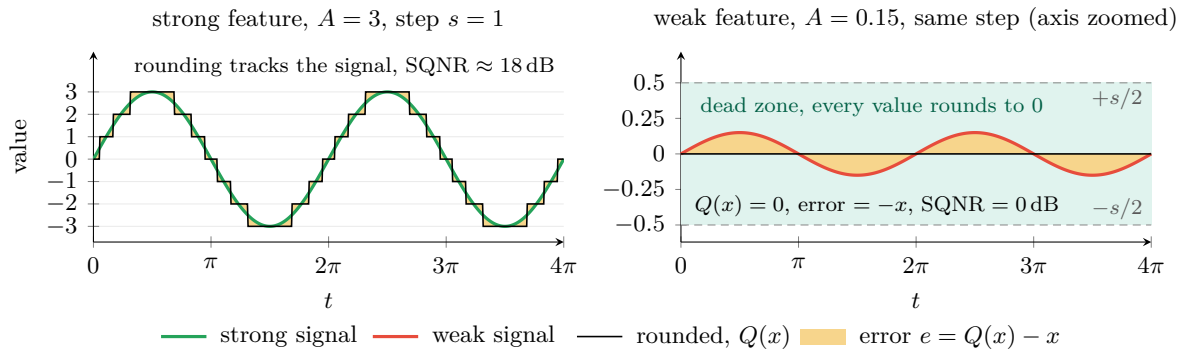


Figure 16: The same step, two fates. Left, a sinusoid of amplitude 3 rounded with $s = 1$ becomes a staircase that follows it, and the error (shaded) is a small sawtooth. Right, a sinusoid of amplitude 0.15 stays inside the dead zone (green band), so it rounds to 0 at every instant and the error is the whole signal with its sign flipped. Model, $x(t) = A \sin t$ rounded by $s\,\mathrm{round}(x/s)$ with no jitter; the notebook's Part 2 adds a small jitter to both.

Signal plus uncorrelated noise.

$$\mathrm{Cov}(\boldsymbol{x} + \boldsymbol{e}) \;=\; C + \Sigma_E, \qquad \mathrm{Var}(\boldsymbol{v}^\top(\boldsymbol{x} + \boldsymbol{e})) = \boldsymbol{v}^\top C\,\boldsymbol{v} + \boldsymbol{v}^\top \Sigma_E\,\boldsymbol{v} \;\text{ for every unit vector } \boldsymbol{v}. \tag{37}$$

The cross terms $\mathrm{Cov}(\boldsymbol{x}, \boldsymbol{e})$ and $\mathrm{Cov}(\boldsymbol{e}, \boldsymbol{x})$ of the expansion vanish by assumption. The noise power landing on direction $\boldsymbol{v}$ is $\boldsymbol{v}^\top \Sigma_E \boldsymbol{v}$.

Proposition 8.1: Isotropic noise lifts every eigenvalue by the same amount

If $\boldsymbol{e}$ is uncorrelated with $\boldsymbol{x}$ and $\Sigma_E = \eta^2 I$, then

1. $C + \eta^2 I = V(\Lambda + \eta^2 I)V^\top$, so the eigenvectors are unchanged and every eigenvalue becomes $\lambda_k + \eta^2$;
2. the noise power on every $\boldsymbol{v}_k$ is η^2, so the per-rank signal-to-noise ratio λ_k/η^2 falls with k exactly as the spectrum does;
3. the fraction of the variance along $\boldsymbol{v}_k$ that is still signal, the *signal share* $\psi_k \triangleq \lambda_k/(\lambda_k + \eta^2)$, is the squared correlation of the clean and noisy projections.

Proof. $\eta^2 I = V(\eta^2 I)V^\top$ because V is orthogonal, which gives item 1. Item 2 is Equation (37) with $\boldsymbol{v}_k^\top \boldsymbol{v}_k = 1$. For item 3, the clean projection z and the noise projection n are uncorrelated with variances λ_k and η^2, so $\mathrm{Cov}(z, z + n) = \lambda_k$ and the squared correlation is $\lambda_k^2/(\lambda_k(\lambda_k + \eta^2))$. □

The mathematics is exact; the premise is the question. Rounding every coordinate gives $\Sigma_E = \eta^2 I$ with $\eta^2 = s^2/12$ only under three conditions.

- One step serves all coordinates, so that every coordinate gets the same variance.
- The high-resolution regime of Section 4 holds, with no dead zone, no clipping, and many levels crossed (Widrow & Kollár, 2008).
- The errors of different coordinates are uncorrelated, so that Σ_E has no off-diagonal terms.

Call this the *flat-floor model*. It is a model, not a measurement, and the rest of the section shows how a measured residual can depart from it.

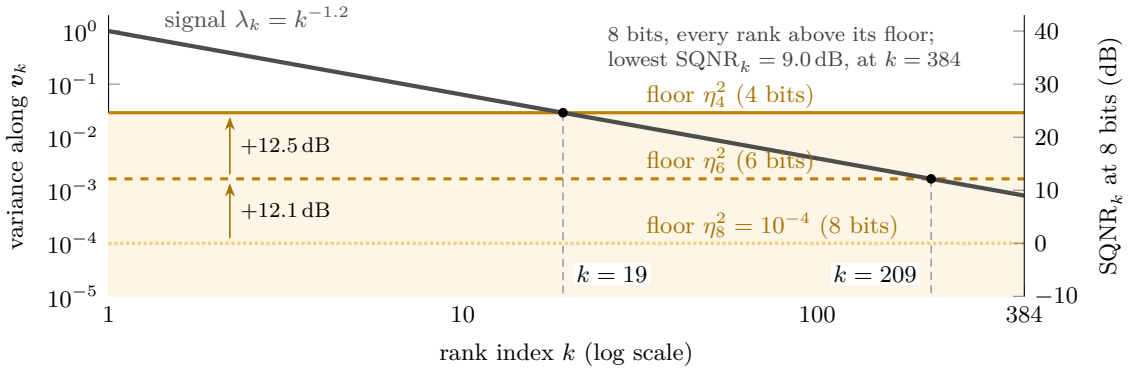


Figure 17: The flat-floor model on a power-law spectrum. The signal $\lambda_k = k^{-1.2}$ against the floors $\eta_b^2 = s_b^2/12$ for $b = 8, 6, 4$, with $s_b = 2R/(2^b - 1)$ and $R = 4.42$. Each floor sits about 6 dB per removed bit above the last, and the rank index where it meets the signal falls from beyond 384 to 209 to 19. The right axis reads the same picture as SQNR$_k$ at 8 bits. On a linear rank axis the signal line bends, but the vertical distances in dB are the same.

Worked example on a floor under a 2×2 covariance

$C = \left(\begin{smallmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{smallmatrix}\right)$ has $\lambda_1 = 4$, $\lambda_2 = 1$, $\boldsymbol{v}_1 = (1, 1)/\sqrt{2}$, $\boldsymbol{v}_2 = (1, -1)/\sqrt{2}$. With $\eta^2 = 0.25$ the noisy covariance $\left(\begin{smallmatrix} 2.75 & 1.5 \\ 1.5 & 2.75 \end{smallmatrix}\right)$ has eigenvalues 4.25 and 1.25 on the same eigenvectors. The ratios are 16 (12.0 dB) and 4 (6.0 dB), and $\psi_1 = 0.94$, $\psi_2 = 0.80$. Same noise, and the weaker direction loses more.

▷ **Check yourself.** If η^2 doubles, what happens to every per-rank ratio ρ_k and to every SQNR$_k$?[16]

Figure 17 applies the model to the spectrum shape of Figure 15, $\lambda_k = k^{-1.2}$ for $k = 1, \ldots, 384$, with floors $\eta_b^2 = s_b^2/12$ whose step comes from a fixed clip range $[-R, R]$, $s_b = 2R/(2^b - 1)$, and $R = 4.42$ so that $\eta_8^2 = 10^{-4}$. Going from 8 to 6 bits raises the floor by $20 \log_{10}(255/63) = 12.1$ dB, and from 6 to 4 bits by $20 \log_{10}(63/15) = 12.5$ dB, about 6 dB per bit as Equation (18) predicts.

The dots invite a threshold reading, with directions above the floor kept and those below it erased. Item 3 of Proposition 8.1 says otherwise. At 0 dB half the variance along $\boldsymbol{v}_k$ is still signal ($\psi_k = 0.5$), at +10 dB 0.91, at −10 dB 0.09. Information fades smoothly with rank, and the 0 dB rank is a label on a graded curve, not an erasure boundary.

8.3 The per-rank measurement

No one observes Σ_E. What you can do is run the full-precision and the quantized model on the same N inputs, subtract, and project the difference on the full-precision principal directions.

Definition 8.1: Per-rank energy and residual-to-signal ratio

Let $X \in \mathbb{R}^{N \times d}$ hold the FP32 features of N inputs as rows, with covariance $C = \frac{1}{N} X_c^\top X_c = V \Lambda V^\top$ of the centered X_c, eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d > 0$ and eigenvectors $\boldsymbol{v}_k$. For any residual $E \in \mathbb{R}^{N \times d}$ on the same inputs, such as $E = X_Q - X$ or $E = X(\text{blur}) - X$, define on this fixed FP32 basis

$$e_k(E) \triangleq \frac{1}{N} \|E\, \boldsymbol{v}_k\|^2, \qquad \rho_k(E) \triangleq \frac{e_k(E)}{\lambda_k}.$$

[16] Every ρ_k doubles, so every SQNR$_k$ falls by $10 \log_{10} 2 = 3.01$ dB, at every rank index. On a logarithmic axis the whole floor moves up by the same distance.

Per-rank SQNR.

$$\mathrm{SQNR}_k(E) \triangleq -10\log_{10}\rho_k(E) = 10\log_{10}\frac{\lambda_k}{e_k(E)} \quad \mathrm{dB}. \tag{38}$$

Positive where the residual is weaker than the signal along $\boldsymbol{v}_k$, zero where they are equal. Under the flat-floor model $e_k \to \eta^2$ as N grows, which gives the right axis of Figure 17.

Three choices hide in the definition.

- *The basis* is the FP32 one, not recomputed on the quantized features (Section 7.5).
- *The centering.* The signal is centered, but e_k is an uncentered second moment, so a mean shift of E along $\boldsymbol{v}_k$ counts as residual energy.
- *Conventions that do not matter.* Neither the sign of E nor the choice of means or sums changes ρ_k.

A per-rank plot does not always state all three, so check each before you compare two plots. With these fixed, one object reads a quantization residual $X_Q - X$ and a corruption residual $X(\mathrm{blur}) - X$ with the same eyes.

Worked example, e_k and SQNR_k by hand

Take the $N = 4$ centered rows $(2.2, 0.4)$, $(-2.2, -0.4)$, $(2.2, -0.4)$, $(-2.2, 0.4)$. The cross products cancel, so $C = \mathrm{diag}(4.84, 0.16)$ with $\boldsymbol{v}_1 = (1, 0)$, $\boldsymbol{v}_2 = (0, 1)$. Rounding with $s = 1$ sends the first coordinate to ± 2 and the second to 0, so $E = Q(X) - X$ has rows $(\mp 0.2, \mp 0.4)$ with the signs of the original entries flipped. Then $e_1 = 0.04$, $\rho_1 = 0.0083$, $\mathrm{SQNR}_1 = 20.8\,\mathrm{dB}$, and $e_2 = 0.16$, $\rho_2 = 1$, $\mathrm{SQNR}_2 = 0.0\,\mathrm{dB}$. The flat-floor model ($\eta^2 = 1/12$ on both) predicts 17.6 and 2.8 dB, wrong in both places. The tail sits at exactly 0 dB because the whole second coordinate fell into the dead zone and its residual is minus the signal. Any direction erased this way has $\rho_k = 1$, however small λ_k is, while the flat-floor ratio η^2/λ_k grows without bound.

8.4 White and colored residuals

Under the flat-floor model e_k does not depend on k, and the residual is *white* across ranks. In general the power on $\boldsymbol{v}_k$ is $\boldsymbol{v}_k^\top \Sigma_E \boldsymbol{v}_k$, which varies with k whenever Σ_E is not a multiple of I, and the residual is *colored*. Three mechanisms color a quantization residual.

- *Different steps.* Per-channel steps give $\Sigma_E = \mathrm{diag}(s_j^2/12)$, which is larger where the step is larger.
- *Accumulation.* The penultimate residual sums the errors of many quantizers, each passed through the layers that follow it, and Section 5 showed that such a sum is colored even when each quantizer's own error is white.
- *Signal dependence.* The error can depend on the signal (Section 8.5).

Which of the three dominates in a given network is open, and only a measurement can say.

A colored residual can decay with rank and still weigh most on the tail. If e_k falls, but more slowly than λ_k, then ρ_k rises toward the tail while the residual's absolute energy is largest at the spectral head; the toy network of Section 5 produced this shape. A residual of this kind sits in the high-rank tail only in this relative sense. Figure 18 contrasts it with a flat floor of the same total energy on a synthetic spectrum. The flat floor overstates the head's SQNR_k (30.1 against

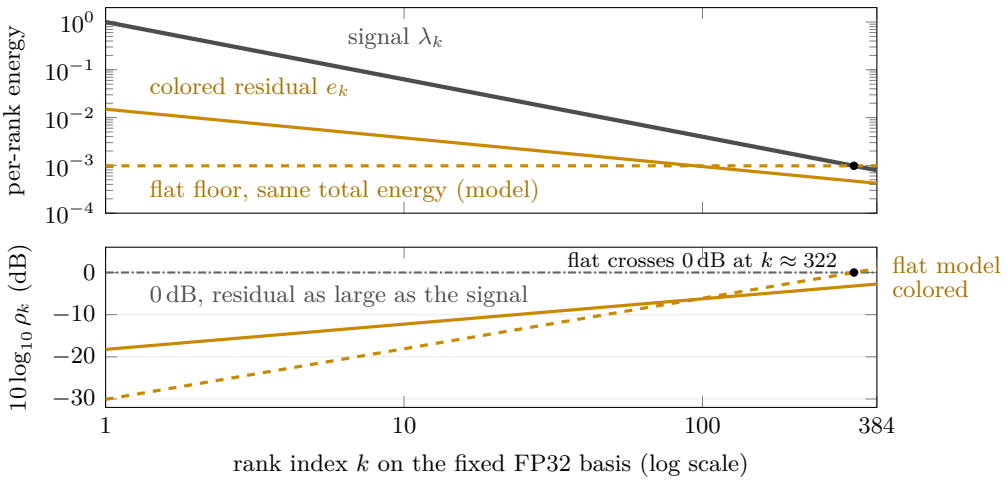


Figure 18: White against colored, at equal total residual energy (synthetic model). Top, the signal $\lambda_k = k^{-1.2}$, a colored residual $e_k = 0.015\,k^{-0.6}$ that decays more slowly than the signal, and a flat floor of the same total energy, $\eta^2 = 9.8 \times 10^{-4}$. Bottom, $10\log_{10}\rho_k = -\mathrm{SQNR}_k$. The flat floor's ratio climbs 12 dB per decade and crosses 0 dB near $k = 322$; the colored one climbs 6 dB per decade and never reaches 0 dB, so it has no erasure rank to read off.

18.2 dB at $k = 1$) and predicts a 0 dB rank. The colored residual keeps $\mathrm{SQNR}_k > 0$ at every rank index, down to $\mathrm{SQNR}_{384} = 2.7$ dB, and offers no sharp rank index at which information vanishes.

8.5 When the error is correlated with the signal

The flat-floor model also assumes the error ignores the signal. Drop that and look at one direction $\boldsymbol{v}_k$, with the clean component $z = (\boldsymbol{x} - \boldsymbol{\mu})^\top \boldsymbol{v}_k$, the projected error $\varepsilon = (\boldsymbol{x}_Q - \boldsymbol{x})^\top \boldsymbol{v}_k$, and the quantized component $z_Q = (\boldsymbol{x}_Q - \boldsymbol{\mu})^\top \boldsymbol{v}_k = z + \varepsilon$, all measured from the FP32 mean. Averaging over the N inputs, $\mathbb{E}[z^2] = \lambda_k$ and $\mathbb{E}[\varepsilon^2] = e_k$.

Gain plus noise along one direction. With $c_k \triangleq \mathbb{E}[z\varepsilon]$, the covariance of signal and error along $\boldsymbol{v}_k$,

$$\mathbb{E}\left[z_Q^2\right] = \lambda_k + 2c_k + e_k,$$
$$z_Q = g_k\, z + n_k, \qquad g_k = 1 + \frac{c_k}{\lambda_k}, \qquad \mathbb{E}[z\, n_k] = 0, \qquad \mathbb{E}\left[n_k^2\right] = e_k - \frac{c_k^2}{\lambda_k}. \tag{39}$$

The first line is Equation (19) for one projected coordinate. The second splits z_Q into a scaled copy of the signal and a part uncorrelated with it, so g_k is the least-squares slope of z_Q on z.

Proposition 8.2: Attenuation requires anticorrelated error

In the notation of Equation (39), with $\lambda_k > 0$,

1. if $c_k = 0$ then $g_k = 1$ and $\mathbb{E}\left[z_Q^2\right] = \lambda_k + e_k \geq \lambda_k$, so error uncorrelated with the signal can only add energy to a direction;
2. if $\mathbb{E}\left[z_Q^2\right] < \lambda_k$ then $c_k < -e_k/2 \leq 0$, so the error is anticorrelated with the signal;
3. $g_k < 1$ if and only if $c_k < 0$.

Proof. Expanding $\mathbb{E}\left[(z+\varepsilon)^2\right]$ gives the first line of Equation (39). With $n_k = z_Q - g_k z$, $\mathbb{E}[z n_k] = \lambda_k + c_k - g_k \lambda_k$ vanishes exactly for $g_k = 1 + c_k/\lambda_k$, and expanding $\mathbb{E}\left[n_k^2\right]$ gives $e_k - c_k^2/\lambda_k$. Item 1 is the first line at $c_k = 0$. Item 2 rearranges $\lambda_k + 2c_k + e_k < \lambda_k$. Item 3 reads off g_k. □

This proposition is the notes' own, a direct consequence of the definitions. Independent additive noise, whatever its size, never makes a direction weaker. Within one quantizer, two mechanisms do it most visibly.

- *The dead zone* sends small values to zero, so the error is $-z$ and the gain is 0.
- *Clipping* pulls large values back to the edge of the range, so the error again opposes the signal.

Other sources can add attenuation: coarse rounding, which attenuates a single coordinate even at a step chosen for minimum error (Section 4), weight rounding (Section 5), and the layers after each quantizer. Inside the range, with many levels crossed, rounding error is nearly uncorrelated with the input (Widrow & Kollár, 2008) and $g_k \approx 1$.

Figure 19 shows both on a stated model. At the head, clipping gives $g_1 = 0.91$ and $\mathbb{E}\left[z_{Q,1}^2\right]/\lambda_1 = 0.87$, an attenuation no independent noise could produce. In the tail the dead zone gives $g_2 = 0.51$, and the energy still rises to $1.47\lambda_2$, against $1 + e_2/\lambda_2 = 2.45$ times λ_2 for independent noise of the same e_2. A rise in energy is no evidence of independent error; its size, set against $1 + e_k/\lambda_k$, is. In the hand-computed example of Section 8.3, $c_1 = -0.44$ and $c_2 = -0.16$, so $\mathbb{E}\left[z_{Q,1}^2\right] = 4.84 - 0.88 + 0.04 = 4.00$ with $g_1 = 0.91$, and $\mathbb{E}\left[z_{Q,2}^2\right] = 0.16 - 0.32 + 0.16 = 0$ with $g_2 = 0$. The second direction is gone, not buried.

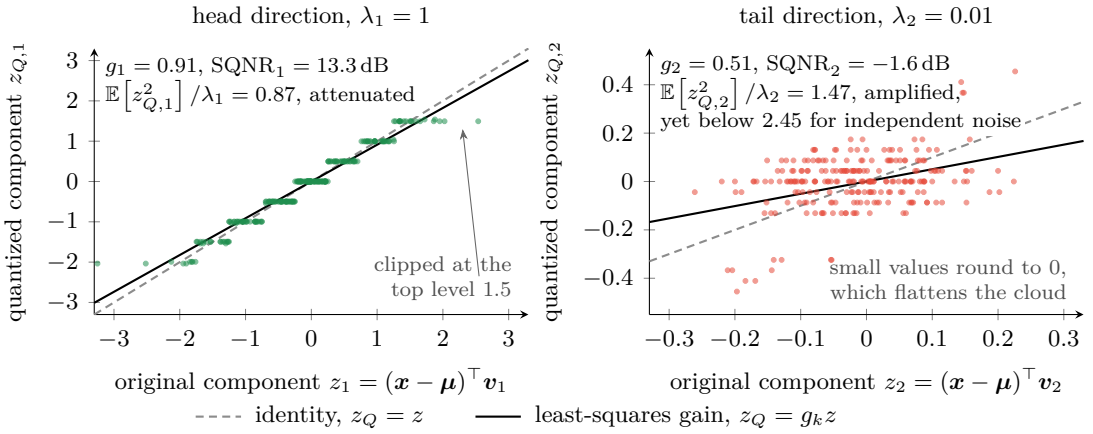


Figure 19: Gain plus noise in a two-dimensional Gaussian model. Principal variances $\lambda_1 = 1$ and $\lambda_2 = 0.01$, principal axes rotated 5° from the coordinate axes, each coordinate rounded by the 3-bit quantizer with $s = 0.5$ (levels -2 to 1.5); 240 draws shown, statistics from 2×10^6. At the head, clipping pulls both ends of the cloud toward zero and the direction loses energy although SQNR$_1$ = 13.3 dB. In the tail, the dead zone flattens the whole cloud. The error's correlation with the signal is -0.41 in both.

8.6 What a model trained through noise learns

Train a linear readout $\hat{y} = \boldsymbol{w}^\top \boldsymbol{x}$ with the squared loss on inputs carrying independent zero-mean noise $\boldsymbol{n}$ of covariance Σ_n. Expand the square and average over the noise,

$$\begin{aligned}\mathbb{E}_{\boldsymbol{n}}\left[(\boldsymbol{w}^\top(\boldsymbol{x}+\boldsymbol{n})-y)^2\right] &= (\boldsymbol{w}^\top\boldsymbol{x}-y)^2 + 2(\boldsymbol{w}^\top\boldsymbol{x}-y)\,\boldsymbol{w}^\top\mathbb{E}[\boldsymbol{n}] + \mathbb{E}\left[(\boldsymbol{w}^\top\boldsymbol{n})^2\right] \\ &= (\boldsymbol{w}^\top\boldsymbol{x}-y)^2 + \boldsymbol{w}^\top\Sigma_n\boldsymbol{w},\end{aligned}$$

because the noise has mean zero and $\mathbb{E}\left[(\boldsymbol{w}^\top\boldsymbol{n})^2\right] = \boldsymbol{w}^\top\mathbb{E}\left[\boldsymbol{n}\boldsymbol{n}^\top\right]\boldsymbol{w} = \boldsymbol{w}^\top\Sigma_n\boldsymbol{w}$. Training through the noise is therefore exactly ridge (Tikhonov) regression with penalty $\boldsymbol{w}^\top\Sigma_n\boldsymbol{w}$, of strength η^2 for isotropic noise. Bishop (1995) proved the same equivalence for general networks trained with the sum-of-squares error, to leading order in small noise. For centered features and target, the least-squares weights solve $C\boldsymbol{w} = \boldsymbol{m}$, with $\boldsymbol{m}$ the covariance of the features with y, and isotropic noise replaces C by $C + \eta^2 I$. In the eigenbasis of C, with m_k the covariance of y with the component along $\boldsymbol{v}_k$, the clean weight is therefore m_k/λ_k and the noise-trained weight $m_k/(\lambda_k+\eta^2)$, a shrinkage by the signal share $\lambda_k/(\lambda_k+\eta^2) = \psi_k$. Directions far above the floor keep their weight, those below lose most of it, and reliance moves onto strong directions. That is proved for this linear model and is a heuristic beyond it.

Two cautions. Rounding is a fixed function of its input, so the equivalence reaches quantization only through the flat-floor idealization, and LSQ also trains the step (Section 6). And whether such shrinkage makes an encoder more robust to a corruption is a separate question, which this identity cannot settle.

Running example: the bus in the rain

Give the bus crop's 384-dimensional feature vector, as a synthetic illustration, the spectrum $\lambda_k = k^{-1.2}$ of Figure 17, and suppose the direction that best separates bus from truck is a fine cue at rank 150, with $\lambda_{150} = 2.45 \times 10^{-3}$ (both choices made up). Under the flat floors of Figure 17 it sits at $\mathrm{SQNR}_{150} = 13.9\,\mathrm{dB}$ at 8 bits, 1.7 dB at 6 and $-10.7\,\mathrm{dB}$ at 4, and a readout trained through that noise keeps 0.96, 0.60 and 0.078 of its clean weight on it. At 8 bits the cue survives almost intact; at 4 bits the readout must lean on coarser directions such as overall shape. Coordinates A and B of the running example in Section 4 are the same picture for single coordinates, with B, whose spread is below the step, erased at 4 bits rather than buried. A real W8A8 residual accumulates through many quantized layers and is colored (Section 8.4), so these numbers show the direction of the effect, not its size.

8.7 Which spectrum?

Resolving an ambiguity about "spectrum"

Spectrum names three objects in work on quantization and robustness, the *image Fourier spectrum* (power against spatial frequency, on which blur is a literal low-pass filter, Section 9), the *feature covariance spectrum* (λ_k against rank index k, defined in this section), and the *singular values of a weight matrix*. Papers often say "spectral" for all three and borrow "low-pass" for the feature spectrum by analogy. These notes keep *low-pass* for images and say, for features, that quantization lowers SQNR_k in the high-rank tail.

8.8 What the picture explains, and what it leaves open

The per-rank picture, not the flat floor, is the organizing idea of these notes. The flat floor is a model that makes three things intelligible.

- Weak directions lose their signal first.

- The loss is graded rather than a cutoff.
- A linear readout trained through the noise moves its weight onto strong directions.

What you measure on a network is ρ_k on a fixed basis, and Sections 8.4 and 8.5 showed two ways it departs from the floor, color and signal dependence. When two residuals, say from quantization and from blur, both have ρ_k rising toward the tail, the picture makes their overlap intelligible. It does not turn the overlap into an explanation of what the network does downstream.

Section 4.4 listed what separates a quantizer from injected noise of the same variance, namely determinism, signal dependence, and clipping, and a training comparison adds the parameters each variant trains. A control that matches only the size of the noise therefore tests whether size alone suffices. This section adds one measurable handle, c_k and g_k of Equation (39) measured rank by rank on a residual, where $g_k < 1$ in the tail shows that part of the signal is erased, not only buried. Which of these properties matters for a given effect is an empirical question, and the variance, which all of them share, cannot answer it. The flat floor stays a model throughout, the idealization against which a measured ρ_k is read, not a description of any measured residual.

In the hands-on session

Part 3 of the notebook plots a 48-rank spectrum against a floor from 8 down to 2 bits. Its text states the flat floor of Proposition 8.1, the idealized model, but its dashed line is the measured per-rank error. It should sit at $s^2/12$ at 8 and 6 bits and, as the hand-computed example of Section 8.3 predicts, bend down onto the tail bars at 4 bits and below, where weak coordinates round to zero; verify this in the notebook's plot. Exercise 3.2 replaces the quantizer with noise of matched variance in a linear classifier. Predict the result from the ridge identity of Section 8.6 (exact for the squared loss, approximate for the notebook's logistic loss), then compare it with the quantized classifier.

Exercise I.18: Where the floor meets the signal

Take $\lambda_k = k^{-1.2}$, $\eta_b^2 = s_b^2/12$, $s_b = 2R/(2^b - 1)$ and $\eta_8^2 = 10^{-4}$, as in Figure 17. (a) Show that the 0 dB rank is $k_b^* = (\eta_b^2)^{-1/1.2}$ and evaluate it for $b = 7$ and $b = 5$. (b) Show that removing one bit multiplies k^* by about $4^{-1/1.2}$. (c) Find that factor for $\lambda_k = k^{-2}$.

Solution sketch. (a) For $b = 7$, $\eta_7^2 = 10^{-4}(255/127)^2 = 4.03 \times 10^{-4}$ and $k^* = 674 > 384$, so no rank reaches 0 dB. For $b = 5$, $\eta_5^2 = 6.77 \times 10^{-3}$ and $k^* = 64$. (b) One bit fewer about doubles s and quadruples η^2, so k^* scales by $4^{-1/1.2} = 0.31$. A bit does not halve or quarter the surviving ranks; the factor depends on the spectrum's slope. (c) 0.5.

Exercise I.19: Reading an attenuation

Along one FP32 direction a measurement gives $\mathrm{SQNR}_k = 10$ dB and $\sqrt{\mathbb{E}\left[z_Q^2\right]/\lambda_k} = 0.97$. (a) What would independent noise of the same power give? (b) Find c_k/λ_k, the correlation of error and signal, and g_k.

Solution sketch. $e_k/\lambda_k = 0.1$. (a) $\sqrt{1.1} = 1.049$, an amplification. (b) By Equation (39), $0.9409 = 1 + 2c_k/\lambda_k + 0.1$, so $c_k/\lambda_k = -0.080$; the correlation is $c_k/\sqrt{\lambda_k e_k} = -0.25$, and $g_k = 0.92$. A 3% attenuation already implies a clearly negative correlation.

Check yourself before moving on

1. Noise with covariance $\eta^2 I$, uncorrelated with the features, is added to them. What happens to the eigenvectors and the eigenvalues of the covariance?[a]

2. Under the flat-floor model, how much of the variance along $\boldsymbol{v}_k$ is still signal at the rank index where $\mathrm{SQNR}_k = 0\,\mathrm{dB}$?[b]

3. Can ρ_k rise toward the tail while e_k falls with rank?[c]

4. Why can independent noise never make $\mathbb{E}\left[z_Q^2\right]$ smaller than λ_k?[d]

[a]The eigenvectors stay, and every eigenvalue rises by the same η^2 (Proposition 8.1). On a logarithmic axis the weak directions therefore lose the most.

[b]Half, since $\psi_k = 0.5$ by item 3 of Proposition 8.1. The loss is graded, and the 0 dB rank is not an erasure rank.

[c]Yes, when e_k falls more slowly than λ_k, as in Figure 18.

[d]Then $c_k = 0$ and $\mathbb{E}\left[z_Q^2\right] = \lambda_k + e_k$ (Proposition 8.2).

Where this is used. Sections 17 and 20 read the spectral figures of the two papers with ρ_k on a fixed basis and with Proposition 8.2, and Section 20 sets the noise-floor picture against a training control that injects noise of matched size.

9 Images, blur, and frequency

What you will be able to do after this section

Read an image's radial power spectrum and its $1/f^2$ regularity. Write blur as a convolution, use the Gaussian response to compute what a σ keeps of a detail, and explain why a σ means nothing without its image. Place noise, brightness, and contrast on the frequency axis, and keep image frequency apart from the rank index of a feature spectrum.

Sections 7 and 8 built a spectrum on feature vectors. This section builds an older one on pixels, where blur is defined. The two share vocabulary (head and tail, low and high, a ratio of residual to signal) and nothing else (Section 8.7).

9.1 An image is a two-dimensional signal

Take a grayscale image $I(x, y)$ on an $M \times M$ pixel grid with intensities in $[0, 1]$; a color image is three such channels. The simplest images are gratings $\cos(2\pi(f_x x + f_y y))$, stripes whose *spatial frequency* is measured in cycles per pixel, so a grating with $f_x = 1/8$ repeats every 8 pixels. The finest grating a grid can hold alternates between neighboring pixels, so frequencies live in $[-\frac{1}{2}, \frac{1}{2}]$ and $f = 0.5$ is the *Nyquist frequency*. The discrete Fourier transform (DFT) writes an image as a weighted sum of gratings,

$$F(u, v) \;=\; \sum_{x=0}^{M-1} \sum_{y=0}^{M-1} I(x, y)\, e^{-2\pi i\,(ux+vy)/M}, \qquad (f_x, f_y) = \Big(\frac{u}{M}, \frac{v}{M}\Big), \tag{40}$$

with indices above $M/2$ read as negative frequencies. The coefficient $F(u, v)$ says how much of each grating the image contains, and with which phase. Parseval's identity says that the power of an image is the same whether you add it up over the pixels or over the frequencies (Gonzalez & Woods, 2018). So $|F|^2/M^2$ is a *power density*. Its sum over the nonzero frequency cells, each of area $1/M^2$, is the pixel variance, and its value at one frequency says how much of that variance sits there. Its average over all directions at the radial frequency $f = \sqrt{f_x^2 + f_y^2}$ is the *radial power spectrum* $P(f)$.

Natural images share the shape of that curve. Their power falls roughly as

$$P(f) \;\approx\; \frac{A}{f^2}, \tag{41}$$

with an exponent that varies from image to image around two (Field, 1987; van der Schaaf & van Hateren, 1996). Since the frequencies at radius f fill a ring of circumference $2\pi f$, the power between f and $2f$ is $\int_f^{2f} (A/f'^2)\, 2\pi f'\, df' = 2\pi A \ln 2$, the same for every octave. Two effects cancel here. The octave from f to $2f$ holds a number of frequencies proportional to f^2, since its ring is both wider and longer, while the power at each frequency falls as $1/f^2$. A 224-pixel crop spans $\log_2 112 \approx 6.8$ octaves, each carrying about a seventh of the variance, so the finest octave, where edges and lettering live, holds no more variance than the coarsest although it contains three quarters of the frequencies in the disk of radius one half. The numbers below use one stated model, a 224×224 crop following Equation (41) from $f = 1/224$ to 0.5 with pixel standard deviation 0.2. Approximating the square band by that disk, $2\pi A \ln 112 = 0.04$ gives $A = 1.35 \times 10^{-3}$.

9.2 Blur is a convolution, and a convolution is a filter

Why blur. A camera image can degrade in many ways. A catalog of RGB camera failures for automated driving lists twenty-six of them, from banding and dead pixels to condensation, dirt, flare, and rain (Ceccarelli & Secci, 2023). These notes follow blur most closely, for three reasons.

- *Several failures end in it.* An image out of focus is blurred, and the same catalog describes a broken vibration-reduction unit and uncorrected chromatic aberration as giving blurred images too (Ceccarelli & Secci, 2023). Motion adds its own blur. An object that crosses the view at speed v and distance Z moves across the image at v/Z times the focal length in pixels, and smears over that speed times the exposure time. A car crossing at 50 km/h (13.9 m/s), 20 m away, seen with a focal length of 1000 pixels, moves at about 694 pixels per second, so a night exposure of 20 ms smears it over about 14 pixels.
- *It removes what recognition needs.* Blur takes away fine detail first (Equation (45) below), the detail of distant and small objects, of text on signs, and of the cues that separate similar classes such as a truck and a bus. In the same catalog, a 10×10 box blur on driving images left no object detector with an average precision above 0.6, "even if the vehicles are visually distinguishable" (Ceccarelli & Secci, 2023).
- *It is the cleanest corruption to study.* A Gaussian blur has one parameter, σ, is deterministic, grades severity smoothly, and can be added on top of real adverse weather. It is also a literal low-pass filter, so it speaks the frequency language of Sections 7 and 8.

The other degradations are not ignored. Section 9.4 places noise, brightness, and contrast on the same frequency axis, and Part II meets them as corruptions and as controls (Section 20). Glare, night, and occlusion stay outside these notes.

Blur as a filter. A camera that is out of focus, moving, or looking through condensation replaces each pixel by a weighted average of its neighbors, with the same weights everywhere.

Blur as convolution, and the Gaussian response. A blur with kernel h, nonnegative and summing to one, maps an image I to

$$(h * I)(x, y) \;=\; \sum_m \sum_n h(m, n)\, I(x - m,\, y - n), \tag{42}$$

and in frequency the convolution becomes a product (Gonzalez & Woods, 2018),

$$\widehat{h * I}(f_x, f_y) \;=\; H(f_x, f_y)\, \widehat{I}(f_x, f_y), \tag{43}$$

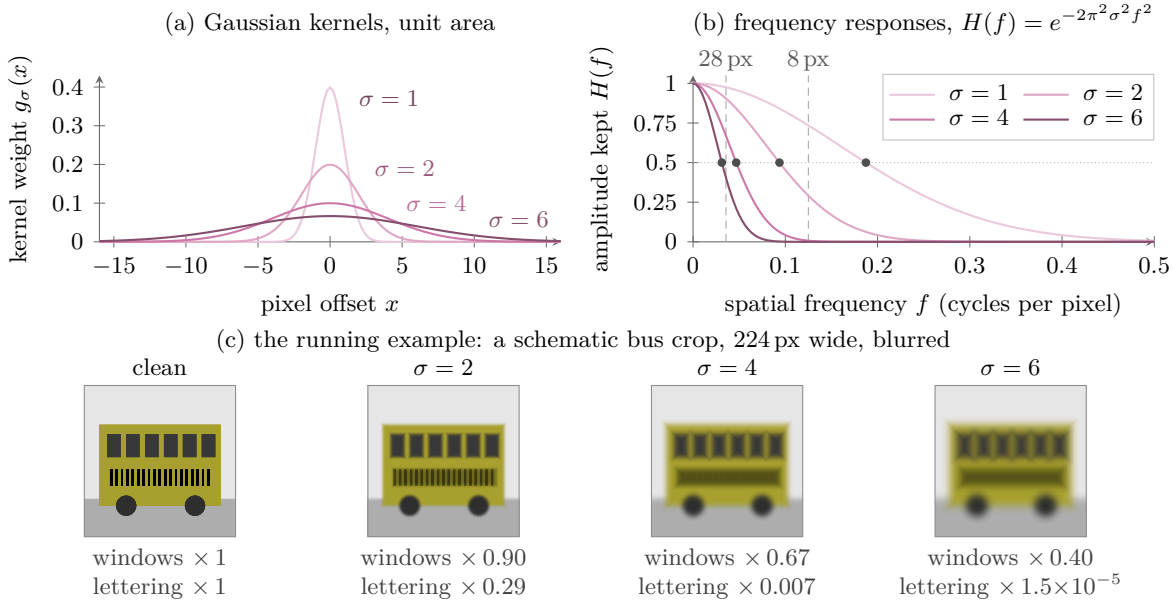


Figure 20: Gaussian blur in space, in frequency, and on the running example. (a) Kernels for $\sigma = 1, 2, 4, 6$ pixels, each of unit area. (b) Their responses, Equation (44), with dots at $f_{1/2} = 0.187/\sigma$ and dashed lines at periods of 28 and 8 pixels; a wide kernel is a narrow response, so a larger σ removes ever coarser detail. (c) A schematic 224-pixel bus crop (synthetic shapes) with edges softened over σ; under each panel, the amplitude kept by the windows (28 pixels) and the lettering (8). The lettering is gone by $\sigma = 4$; windows and outline remain at $\sigma = 6$.

where $H = \widehat{h}$ is the kernel's *frequency response*. Each grating leaves the blur as the same grating multiplied by H at its frequency, and $H(0,0) = 1$ keeps the mean brightness. For the Gaussian kernel $h_\sigma(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/(2\sigma^2)}$, with σ in pixels, and treating pixels as samples of a continuous image (accurate for $\sigma \gtrsim 1$ away from Nyquist),

$$H_\sigma(f_x, f_y) \;=\; \exp\!\left(-2\pi^2\sigma^2 f^2\right), \qquad f = \sqrt{f_x^2 + f_y^2} \text{ in cycles per pixel.} \tag{44}$$

Proof. The kernel is a product of one-dimensional Gaussians in x and y, so one dimension suffices. Completing the square gives $-\frac{x^2}{2\sigma^2} - 2\pi i f x = -\frac{(x+2\pi i\sigma^2 f)^2}{2\sigma^2} - 2\pi^2\sigma^2 f^2$, and the shifted Gaussian still integrates to one, which leaves $e^{-2\pi^2\sigma^2 f^2}$. □

Two readings carry most of what follows. The response halves at

$$f_{1/2} \;=\; \frac{\sqrt{\ln 2/2}}{\pi\,\sigma} \;\approx\; \frac{0.187}{\sigma}, \tag{45}$$

so doubling σ halves the frequency at which detail starts to go. And σ and f enter only through their product. A detail of period $\ell = 1/f$ pixels keeps $\exp\!\left(-2\pi^2(\sigma/\ell)^2\right)$ of its amplitude, so what matters is the blur width measured in periods of the detail (Figure 20).

▷ **Check yourself.** The kernel $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$ blurs a row of alternating pixels $0, 1, 0, 1, \ldots$. What remains?[17]

Defocus, motion, and "low-pass". Real blurs also come with other kernels and other responses.

[17] The constant $\frac{1}{2}$, since every pixel becomes half of itself plus a quarter of each neighbor. The kernel's response is $\cos^2(\pi f)$, which is 0 at the Nyquist frequency.

- *Defocus* spreads each point over a disk. A disk of radius r pixels has the standard response $2J_1(2\pi rf)/(2\pi rf)$, with J_1 a Bessel function. Its first zero, at $f \approx 0.61/r$, erases detail of that period outright. Beyond it the response turns negative, down to -0.13, so finer detail returns weakened and with its contrast reversed.
- *Motion* spreads each point along a segment of length L. Its response is $\sin(\pi L f_{\parallel})/(\pi L f_{\parallel})$ in the frequency component $f_{\parallel}$ along the motion, and 1 across it. It attenuates stripes that vary along the motion, erases those at $f_{\parallel} = 1/L, 2/L, \ldots$, and spares the others.

All three kernels keep coarse structure and attenuate fine structure, which is what low-pass means.

Worked example on what a blur keeps

Three details of the running example's bus crop, of period $\ell = 8$, 28, and 56 pixels, keep $e^{-2\pi^2\sigma^2/\ell^2}$ of their amplitude.

detail, period ℓ	$\sigma = 2$	$\sigma = 4$	$\sigma = 6$
lettering, 8 px	0.29	0.0072	1.5×10^{-5}
windows, 28 px	0.90	0.67	0.40
body panels, 56 px	0.98	0.90	0.80

For $\ell = 8$ and $\sigma = 4$ the exponent is $-2\pi^2 \cdot 16/64 = -4.93$, and $e^{-4.93} = 0.0072$; $\ell = 2$ with $\sigma = 1$ gives the same number, since only $\sigma/\ell = 0.5$ enters.

Running example: the bus in the rain

In Figure 20(c), by $\sigma = 6$ the lettering is gone and what tells a bus from a truck must come from the outline and proportions. A recognizer that leans on fine cues such as lettering loses them first, and one that reads the coarse shape holds out longer. The numbers are synthetic and say nothing about which cue a real encoder uses.

9.3 A blur σ is measured in pixels of the image it is applied to

Because only σ/ℓ matters, resizing changes what a given σ does. Enlarge an image by a factor m and every period in pixels grows by m, so a blur σ applied after the enlargement acts like σ/m applied before it. A σ describes a corruption only together with the resolution at which it was applied and the object's size in that image.

A pipeline with a detector and a recognizer shows why. The detector's blur acts on the whole frame, say 640 pixels wide, and the recognizer's on a square crop around each box, resized to 224 pixels.

Worked example, one σ on a frame and on a crop

Let the crop side be 1.1 times the box's longer side, as in the hands-on notebook, and let the blur come after resizing. Every object then spans about $224/1.1 \approx 204$ crop pixels, whatever its size in the frame, and $\sigma = 4$ fits $204/4 = 51$ times across it.

- A distant car 60 frame pixels long fits $\sigma = 4$ only 15 times across it on the frame, so the frame blur is the harsher one for the car.
- A near bus 400 frame pixels long fits it 100 times on the frame, so the crop blur is the harsher one for the bus.

The same σ is harsher on the crop for objects longer than about 204 frame pixels, and harsher on the frame for shorter ones. Blurred before resizing, the two are the same. Section 11

works these numbers further.

Benchmarks, severities, and augmentation. Hendrycks and Dietterich (2019) made corruption robustness a benchmark, fifteen corruption types in noise, blur, weather, and digital families at five severities, applied to test images only; Michaelis et al. (2019) carried it to object detection. A severity is a benchmark's label, not a unit, so report the kernel, its parameter, and the image size. Natural adverse weather, as in the DAWN driving photographs (Kenk & Hassaballah, 2020), is part of the image, and synthetic blur can be stacked on it. The standard remedy is augmentation, with AugMix (Hendrycks et al., 2020) as the reference method; a study of what quantization alone does leaves it out, so that no corrupted image enters training or range calibration.

9.4 Noise, brightness, and contrast on the frequency axis

White noise is flat. Add to each pixel an independent perturbation of standard deviation ν (the notes keep σ for blur). Independence makes the expected power density ν^2 at every frequency, which is what white means. Whether noise "concentrates in high frequencies" depends on the yardstick.

- *Per unit frequency* it does not, since its density is the same everywhere.
- *Per octave* it does, with three quarters of its power in the top octave, but only because that octave holds three quarters of the frequencies of the disk.
- *Relative to the image*, the sense that makes noise a high-frequency corruption, the ratio $\nu^2 f^2/A$ of noise power to image power rises by 20 dB per decade and passes one above

$$f_c = \frac{\sqrt{A}}{\nu}. \tag{46}$$

This flatness is exact, unlike the idealized flat floor of Section 8 in features. In the section's model $\sqrt{A} = 0.0367$, so $\nu = 0.08$ gives $f_c = 0.46$; even at $\nu = 0.2$, where the total signal-to-noise ratio $0.04/\nu^2$ is 0 dB and $f_c = 0.18$, the image leads by 19 dB at a period of 50 pixels, so a single signal-to-noise number describes a corruption poorly.

Brightness and contrast act where the image has its power. A contrast change $I' = \mu + c\,(I - \mu)$ scales every nonzero frequency by c. Its residual $(c-1)(I-\mu)$ has the image's own $1/f^2$ shape and, relative to the image, is the constant $(1-c)^2$. An additive brightness shift touches only $f = 0$, and a multiplicative one, $I' = b\,I$, scales all frequencies alike. Neither singles out fine detail.

Figure 21(b) puts the corruptions on one axis through the ratio $r(f)$ of residual power to clean power, the image-domain counterpart of ρ_k in Definition 8.1. A Gaussian blur residual can at most cancel what the image holds at a frequency, so Gaussian blur *erases* detail and $r \le 1$ (the disk and the segment, whose responses dip below zero, reach $r \approx 1.28$ and 1.48); additive noise has no ceiling and *buries* detail. Yin et al. (2019) sort corruptions by their residual spectra into high-frequency ones such as noise, high per octave and relative to the image, and low-frequency ones such as fog and contrast, whose residuals follow the image's own fall, and find that Gaussian noise augmentation improves robustness to the first kind while reducing it to the second.

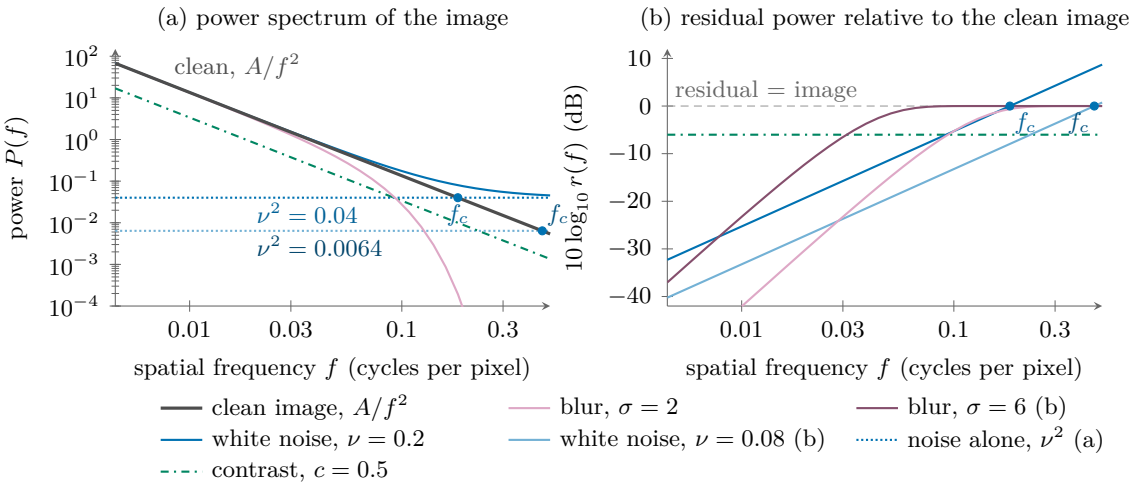


Figure 21: Where each corruption acts on the image Fourier axis, computed from the stated model $P(f) = A/f^2$, $A = 1.35 \times 10^{-3}$. (a) Blur, PH_σ^2, removes power above $f_{1/2}$; white noise adds the flat level ν^2 and overtakes the image only above $f_c = \sqrt{A}/\nu$; contrast, c^2P, lowers the whole line by 6 dB. (b) Residual power relative to the clean image, $r(f)$. Gaussian blur, $(1 - H_\sigma)^2$, approaches 0 dB and never exceeds it; noise, $\nu^2 f^2/A$, has no ceiling; contrast is the constant $(1 - c)^2$.

9.5 Pixel frequency is not feature rank

Here "feature rank" means the rank index k of a feature spectrum (Section 7), not a matrix or effective rank. It is tempting to pair fine image detail with high rank indices and coarse structure with the spectral head. Two known results point that way without settling it. Networks fit the coarse, low-frequency structure of a target function first (Rahaman et al., 2019; Xu et al., 2020), and image classifiers can rely on high-frequency components that people barely see (Wang et al., 2020). Neither result says where a band of image frequencies lands on the rank index of an encoder's feature spectrum.

Until that is measured, a blur residual found in the spectral tail is consistent with the pairing without isolating it. And if noise and contrast residuals, which sit at very different image frequencies, land in the same place, then either image frequency does not fix the rank profile in any simple way, or a cosine between profiles is too coarse to show it, since two positive profiles that both decay with rank index have a high cosine almost regardless.

A direct test adds band-limited noise of fixed energy, one octave of image frequency at a time, and computes the per-rank energy e_k of the feature residual on the fixed basis of the clean features (Definition 8.1). If the pairing holds, the profile slides toward the tail as the octave approaches Nyquist; if it does not, the two axes are distinct for that encoder.

In the hands-on session, notebook Part 4

Part 4 pits a general cue, a Gaussian blob of width 6 pixels, against a specialized cue, stripes alternating every pixel ($f = 0.5$). At $\sigma = 1$ the stripes keep $e^{-\pi^2/2} = 0.007$ by Equation (44) (about 0.014 with the notebook's sampled kernel) and the blob's peak 36/37 of its height. The stripes are high-frequency and low-variance at once, so the toy cannot say which property matters. QAT there loses less than FP32 relative to its own, lower clean accuracy but is not more accurate under blur alone, so compare absolute accuracies too (Section 12). Part 5 blurs 224-pixel crops, so its σ is in crop pixels.

Exercise I.20: A kernel wider than the image

A 32×32 thumbnail, the size of CIFAR images (Krizhevsky, 2009), is blurred with $\sigma = 8$, the kernel truncated at a half-width of 3σ. (a) How wide is the kernel? (b) Below which period does a detail keep less than half of its amplitude (Equation (45)), and what does the coarsest grating keep? (c) Which native σ matches $\sigma = 8$ applied after enlarging the thumbnail to 224 pixels?

Solution sketch. (a) $2 \cdot 24 + 1 = 49$ pixels, wider than the image. (b) $f_{1/2} = 0.187/8 = 0.023$, a period of 43 pixels, longer than that of any grating on a 32-pixel grid; the coarsest, $f = 1/32$, keeps $e^{-2\pi^2/16} = 0.29$ and finer ones less. (c) Enlarging by 7 divides σ by 7, about 1.1 native pixels, a mild blur.

Exercise I.21: Motion, contrast, and noise

(a) A horizontal motion blur averages $L = 8$ pixels along x. How much of vertical stripes of period 8 pixels survives, and of horizontal ones? (b) Lower the contrast by $c = 0.5$ before adding noise of $\nu = 0.08$. Where does the noise now overtake the image?

Solution sketch. (a) Vertical stripes have $f_{\parallel} = 1/8$ and $\sin(\pi)/\pi = 0$, so they vanish; horizontal ones have $f_{\parallel} = 0$ and pass. (b) Contrast scales A by $c^2 = 0.25$, so $f_c = \sqrt{A}/\nu$ halves to 0.23; lower contrast lets the same noise bite at coarser scales.

Check yourself before moving on

1. Why is "Gaussian blur, $\sigma = 4$" an incomplete description?[a]
2. Is white noise a high-frequency corruption?[b]
3. Does blur act on the spectral tail of a vision encoder's features?[c]

[a] H depends on σ/ℓ, and a detail's period in pixels depends on the image resolution at which the blur is applied and on the object's size in it.

[b] Relative to the image, yes, above $f_c = \sqrt{A}/\nu$. Its spectrum itself is flat.

[c] Not established. Blur removes high image frequencies; whether they map onto high rank indices is the untested pairing of Section 9.5.

Where this is used. Sections 16, 18 and 19 read blur sweeps on different image scales with the rule of Section 9.3, and Sections 17 and 20 return to pixel frequency and rank index.

10 Vision foundation models, CLIP and DINOv2

A quantized perception model is rarely trained from scratch. It starts from an encoder that others pretrained on hundreds of millions of images or image-text pairs, and the quantization recipe decides which of its parts stay frozen, which train, and which stay in higher precision. This section introduces two such encoders, CLIP and DINOv2, and the ways they are reused.

What you will be able to do after this section

- Name the three ways of reusing a pretrained encoder and say what each trains.
- Write CLIP's contrastive objective and evaluate it by hand on a batch of three.
- Classify an image zero-shot from prompts, and explain why the logit scale changes the confidence but never the predicted class.
- Say what changes when you quantize only CLIP's image encoder or both of its encoders.

- Describe a recognizer made of a frozen DINOv2 ViT-S/14 and a linear head, and count what its frozen-backbone quantization-aware training changes.

10.1 Reusing a pretrained encoder

A *foundation model* is a network pretrained on a very large and broad collection of data so that many tasks can reuse it. For images the reusable part is an *encoder*, a map from an image to a feature vector $\boldsymbol{x} \in \mathbb{R}^d$. A vision-language model (VLM) of the CLIP family has an image encoder and a text encoder trained to agree; a vision foundation model (VFM) such as DINOv2 has an image encoder trained on images alone. An encoder is reused in one of three ways, ordered by how much of the network changes.

1. **Zero-shot.** Nothing is trained; the classifier is built from class names through a text encoder (Section 10.3).

2. **Linear probe.** The encoder is frozen and only a linear classifier head is trained on labeled examples with the cross-entropy of Equation (20). It is the head of Section 5.1, written $H, \boldsymbol{c}$ here to keep W for backbone weights,

$$\boldsymbol{z} = H\boldsymbol{x} + \boldsymbol{c}, \qquad p(y \mid \boldsymbol{x}) = \operatorname{softmax}(\boldsymbol{z})_y, \qquad H \in \mathbb{R}^{K \times d},\ \boldsymbol{c} \in \mathbb{R}^K. \tag{47}$$

   It probes how much class information the frozen features hold in linearly readable form.

3. **Fine-tuning.** Encoder weights are updated too, all of them or a chosen few; low-rank adaptation (LoRA) trains a low-rank update to chosen frozen weight matrices (Hu et al., 2022).

Fine-tuning on a small, narrow data set can overwrite what the pretrained model knew and degrade it everywhere else, which is called *catastrophic forgetting*. Full fine-tuning can even end up less accurate than a linear probe under covariate shift, while it wins on the fine-tuning distribution (Kumar et al., 2022). Quantization-aware training (QAT) is training, so it belongs on this list (Section 6.5). Full QAT updates encoder weights, a form of fine-tuning with the same risk; frozen-backbone QAT is a linear probe that also trains the quantizer steps.

10.2 CLIP, a shared space for images and text

CLIP has an image encoder f_{img}, a Vision Transformer (Dosovitskiy et al., 2021) or a convolutional network, and a transformer text encoder f_{txt} (Radford et al., 2021). Each ends in a linear projection into a shared space, and outputs are normalized to unit length, $\boldsymbol{u} = f_{\text{img}}(\text{image})/\|f_{\text{img}}(\text{image})\|$ and $\boldsymbol{t} = f_{\text{txt}}(\text{text})/\|f_{\text{txt}}(\text{text})\|$, so $\boldsymbol{u}^\top \boldsymbol{t}$ is the cosine similarity of Section 7. In CLIP ViT-B/32 the shared space has 512 dimensions, while the features entering the projection have the ViT-B width of 768 (Radford et al., 2021, Table 20).

CLIP was trained on WIT (WebImageText), 400 million image-text pairs from the web (Radford et al., 2021). For a batch of B pairs, form the cosine matrix $S_{ij} = \boldsymbol{u}_i^\top \boldsymbol{t}_j$ (not the S of the singular value decomposition in Section 7). Its diagonal holds the true pairs, and training asks each image to pick its own caption among the B in the batch, and each caption its own image.

Contrastive (InfoNCE) objective of CLIP. For a batch of B image-text pairs with cosine matrix S and logit scale $\kappa > 0$,

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2B} \sum_{i=1}^{B} \left[\log \frac{e^{\kappa S_{ii}}}{\sum_{j=1}^{B} e^{\kappa S_{ij}}} + \log \frac{e^{\kappa S_{ii}}}{\sum_{j=1}^{B} e^{\kappa S_{ji}}} \right], \tag{48}$$

a B-way cross-entropy over each row (image to text) and each column (text to image). The scale is learned as $\kappa = e^{\theta}$ with θ a trainable scalar.

The name comes from contrastive predictive coding (van den Oord et al., 2018). The logit scale is an inverse temperature, $\kappa = 1/T$; CLIP starts at $T = 0.07$ ($\kappa \approx 14.3$), clips κ at 100, and draws its negatives from batches of 32,768 pairs (Radford et al., 2021, Sec. 2.5).

Worked example, InfoNCE on a batch of three

Take $B = 3$ and $\kappa = 10$, with the diagonal largest in every row and column,

$$S = \begin{pmatrix} 0.30 & 0.10 & 0.05 \\ 0.12 & 0.28 & 0.20 \\ 0.02 & 0.15 & 0.33 \end{pmatrix}, \qquad \kappa S = \begin{pmatrix} 3.0 & 1.0 & 0.5 \\ 1.2 & 2.8 & 2.0 \\ 0.2 & 1.5 & 3.3 \end{pmatrix}.$$

Row 1 gives $20.09/(20.09 + 2.72 + 1.65) = 0.821$, a loss of $-\ln 0.821 = 0.197$. Row 2, where text 3 competes with text 2, gives $16.44/(3.32 + 16.44 + 7.39) = 0.606$ and 0.502; row 3 gives 0.191. The image-to-text mean is 0.296, the columns give 0.285, and $\mathcal{L}_{\text{CLIP}} = 0.291$, against 0.163 at CLIP's starting $\kappa \approx 14.3$ and $\ln 3 = 1.099$ for a uniform guess. The loss rewards the gap between each diagonal entry and the rest of its row and column, and κ sets how strongly.

▷ **Check yourself.** What is $\mathcal{L}_{\text{CLIP}}$ when all B^2 cosines are equal?[18]

10.3 Zero-shot classification with prompts

After training, text can stand in for labeled examples. For each class y write a prompt, "a photo of a bus." in CLIP's default template (Radford et al., 2021, Sec. 3.1.4), and embed it once to get $\boldsymbol{t}_y$.

Zero-shot classification. With image embedding $\boldsymbol{u}$, prompt embeddings $\boldsymbol{t}_y$, and logit scale κ,

$$p(y \mid \text{image}) = \frac{\exp\left(\kappa\, \boldsymbol{u}^{\top} \boldsymbol{t}_y\right)}{\sum_{y'} \exp\left(\kappa\, \boldsymbol{u}^{\top} \boldsymbol{t}_{y'}\right)}, \qquad \hat{y} = \arg\max_{y}\ \boldsymbol{u}^{\top} \boldsymbol{t}_y. \tag{49}$$

It is the linear head of Equation (47) with rows $\kappa \boldsymbol{t}_y^{\top}$ and zero bias, built from text instead of trained.

Two consequences matter for quantization. First, for a fixed class list the prompt embeddings are computed once and cached, which leaves a deployment two *quantization scopes*. In the *visual-only* scope the text encoder stays in FP32, at no inference cost because it never runs, and quantization error reaches each cosine through $\boldsymbol{u}$ alone. In the *joint visual-text* scope, needed when prompts are generated at run time, both encoders are quantized and error enters through $\boldsymbol{u}$ and every $\boldsymbol{t}_y$.

Second, κ multiplies every logit by the same positive number, so it changes how peaked the probabilities are but never which class wins, as a temperature does (Equation (62)). Quantization shifts the cosines, and the pretrained κ may then no longer give well-calibrated confidences (probability calibration, Section 12). Re-fitting κ on a few labeled examples after quantization, a step called *logit tuning*, is temperature scaling (Guo et al., 2017) under another name. It can repair confidence but never accuracy.

[18] Every softmax is uniform, so $\mathcal{L}_{\text{CLIP}} = \ln B$ for any κ.

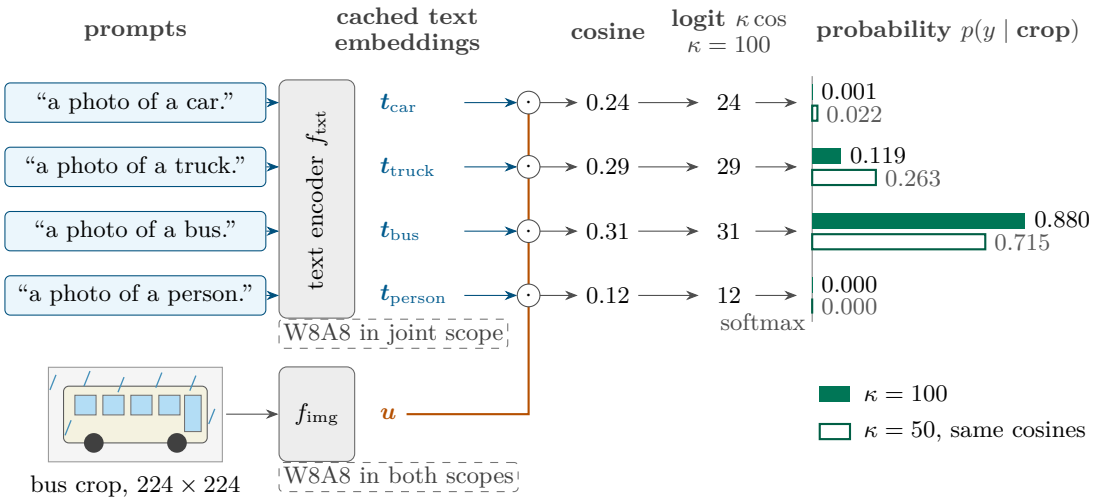


Figure 22: Zero-shot classification of the bus crop (Equation (49)). The prompts are embedded once and cached; $\boldsymbol{u}$ meets each prompt embedding in a dot product, the cosines are scaled by κ, and a softmax gives probabilities. Halving κ (outlined bars) lowers the confidence in "bus" from 0.880 to 0.715 but cannot change the winner. Dashed tags mark where each quantization scope puts W8A8. The cosines drawn are those of full-precision encoders, and the worked example below shows a quantized f_{img} moving them. Synthetic numbers, the notes' own.

Worked example, the logit scale and a small quantization shift

Let the bus crop have cosines $0.24, 0.29, 0.31, 0.12$ with the car, truck, bus, and person prompts (synthetic). At $\kappa = 100$ the logits are $24, 29, 31, 12$, and $p(\text{bus}) = 1/(1 + e^{-2} + e^{-7} + e^{-19}) = 0.880$, $p(\text{truck}) = 0.119$. At $\kappa = 50$, $p(\text{bus}) = 0.715$. A cosine gap of 0.02 becomes a logit gap of 2 only because κ is large. Now quantize the image encoder and let the cosines move to $0.245, 0.300, 0.305, 0.120$. The bus-truck gap shrinks from 2 to 0.5 and $p(\text{bus}) = 1/(1 + e^{-0.5} + e^{-6} + e^{-18.5}) = 0.622$. The class survives and the confidence falls by 0.26. Logit tuning can restore a confidence level, but no κ can undo a shift that puts the truck cosine above the bus cosine.

10.4 The CLIP family

CLIP started a family of models that keep the two-encoder design and change the loss, the architecture, or the data (Table 7), which range from curated collections to lightly filtered alt-text scraped from the web.

Table 7: Members of the CLIP family met in these notes, and what each changes.

Model	What differs from the original CLIP
CLIP	the reference, a ViT or convolutional image encoder trained on WIT (Radford et al., 2021)
CLIP on LAION-2B	open web pairs kept when an existing CLIP model scores them as matching (Schuhmann et al., 2022)
ConvNeXt CLIP	a ConvNeXt image encoder (Liu et al., 2022) in place of the ViT
SigLIP	an independent sigmoid loss on each image-text pair, so no softmax over the batch as in Equation (48) (Zhai et al., 2023)
ALIGN	1.8 billion noisy alt-text pairs with light filtering (Jia et al., 2021)
CoCa	a caption-generating decoder added to the contrastive loss; the image encoder is still read out by Equation (49) (Yu et al., 2022)

All of these are *discriminative* encoders, which map an image to a vector used for classification or retrieval. Much recent writing uses "VLM" for *generative* models instead, large VLMs (LVLMs) that pass image features to a language model and answer in text (Miyai et al., 2025). These notes treat only the discriminative kind; a language model on top adds errors of its own.

10.5 DINOv2 and a recognizer built on it

DINOv2 learns image features with no labels and no text, on LVD-142M, a curated set of 142 million images (Oquab et al., 2024). It trains by self-distillation. Two copies of the network, a *student* and a *teacher*, see different crops of the same image, and the student is trained to produce the teacher's output. The teacher is not trained by gradients; it is kept as a slowly updated average of the student. A patch-level term joins this image-level one, in which the student predicts the teacher's features for patches that were masked in its own input. ViT-S/14 is distilled from a ViT-g/14, and the features are meant to be used frozen, read by linear probes (Oquab et al., 2024). It is the ViT-S/14 of Section 5.2, whose pooled 384-dimensional output feeds the head.

The running example's recognizer, built in notebook Part 5, is such a probe with a quantized backbone. It is built in four steps.

1. Train a linear head, Equation (47) with $K = 4$ and $d = 384$, to convergence on frozen FP32 features.

2. Quantize the 48 linear layers inside the 12 blocks to W8A8 as in Section 5.3, with per-channel weight steps and a per-tensor LSQ step on each layer input, set by range calibration on clean crops. Patch embedding, LayerNorms, attention softmax, and head stay in higher precision.

3. Run frozen-backbone QAT, which trains the 48 activation steps and the head on clean crops (Figure 23).

4. Give an FP32 reference, the same starting head on the unquantized backbone, the same number of extra training steps, so that any difference between the two is due to quantization and not to the extra training.

With the weight steps fixed, as in the notebook, the QAT trains the head, $4 \times 384 + 4 = 1540$ parameters, and 48 steps, 1588 numbers or 7.3×10^{-5} of the 21,628,800 parameters of ViT-S/14 at a 224-pixel input. The quantization residual of such a recognizer is therefore rounding and clipping on fixed weights, with no weight drift (Section 6.5).

In the hands-on session

Notebook Part 5 builds this recognizer from the timm model `vit_small_patch14_dinov2.lvd142m` and a head `nn.Linear(384, 4)`. Predict the number of parameters its QAT phase trains, then count those with `requires_grad` set. Part 5 needs a GPU runtime.

Resolving an ambiguity, fine-tuned and head

- **Fine-tuned.** Writing on quantization sometimes calls a frozen-backbone QAT "fine-tuning", although no backbone weight changes. Before comparing two such results, check which parameters were trained.

- **Head.** A *classifier head* (Equation (47)) is not the *spectral head* of Section 7, nor a detector or attention head.

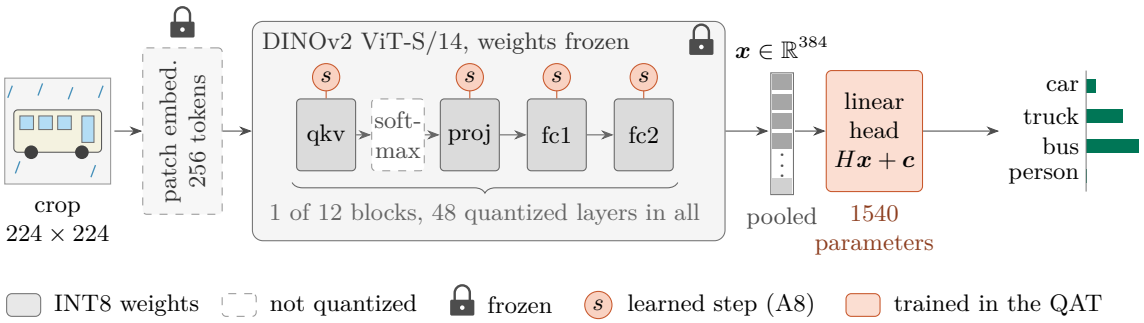


Figure 23: The running example's recognizer as a linear probe, and what its frozen-backbone QAT trains. The DINOv2 ViT-S/14 backbone is frozen; in each of its 12 blocks the four linear layers are quantized (gray) and each layer input has a learned LSQ step s (orange circle), 48 in all. Patch embedding, LayerNorms (not drawn), attention softmax, and head stay in higher precision. Only the steps and the head train, and a softmax turns the head's logits into class probabilities. The pooled vector $\boldsymbol{x}$ comes from the quantized backbone; in the notebook the per-channel weight steps stay fixed. Class bars are the running example's toy numbers.

Running example: the bus in the rain

The zero-shot example gave bus $p = 0.880$, and 0.622 after a small quantization shift of the cosines. With a linear probe on DINOv2 the classifier is trained instead. Let three coordinates stand in for the 384 of $\boldsymbol{x}$,

$$H = \begin{pmatrix} 0.8 & 0.6 & 0.2 \\ 0.2 & -0.4 & 1.1 \\ 0.1 & -0.9 & 1.3 \\ -0.5 & 0.9 & -0.6 \end{pmatrix}, \quad \boldsymbol{c} = \begin{pmatrix} 0.3 \\ 0 \\ -0.2 \\ 0.1 \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} 1.0 \\ -0.5 \\ 2.0 \end{pmatrix} \implies H\boldsymbol{x} + \boldsymbol{c} = \begin{pmatrix} 1.20 \\ 2.60 \\ 2.95 \\ -2.05 \end{pmatrix}.$$

The softmax gives $0.092, 0.374, 0.530, 0.004$ for car, truck, bus, person, a bus with the truck close behind, a narrow margin of the kind blur can overturn. Multiplying H and $\boldsymbol{c}$ by one positive number plays the role of κ. In the frozen-backbone QAT, $\boldsymbol{x}$ comes from the quantized backbone, and only H, $\boldsymbol{c}$, and the 48 steps adapt to it.

Exercises

Exercise I.22: InfoNCE at its limits

For the matrix S of the InfoNCE worked example, show that the loss tends to $\ln 3$ as $\kappa \to 0$ and to 0 as $\kappa \to \infty$, naming the property of S the second limit needs. Then compute the row-2 loss at $\kappa = 1/0.07$.

Solution sketch. As $\kappa \to 0$ every softmax tends to uniform, so each log term tends to $\ln(1/3)$. As $\kappa \to \infty$, dividing by $e^{\kappa S_{ii}}$ leaves terms $e^{-\kappa(S_{ii}-S_{ij})} \to 0$, because each diagonal entry is strictly the largest in its row and column. At $\kappa = 14.3$ the row-2 logits are $1.71, 4.00, 2.86$, so $p = 54.60/77.56 = 0.704$ and the loss is 0.351.

Exercise I.23: How large a logit scale?

For the bus-crop cosines $0.24, 0.29, 0.31, 0.12$, find the κ that gives $p(\text{bus}) = 0.95$ and compare it with CLIP's cap of 100.

Solution sketch. Neglecting car and person, $1/(1 + e^{-0.02\kappa}) = 0.95$ gives $\kappa = \ln 19/0.02 \approx 147.2$ (147.3 with car and person kept), above the cap, so CLIP at its cap cannot be this confident here.

Check yourself before moving on

1. Which parameters change during the recognizer's frozen-backbone QAT?[a]
2. Why does the visual-only scope keep the text encoder in FP32 at no inference cost?[b]
3. Why must the FP32 reference get as many extra training steps as the QAT variant?[c]

[a]The linear head (1540 numbers) and the 48 LSQ activation steps. The backbone weights stay frozen, and in the notebook so do the per-channel weight steps.

[b]For a fixed class list the prompt embeddings are computed once and cached, so the text encoder does not run at inference.

[c]Otherwise the comparison mixes the effect of quantization with the effect of the extra training.

Where this is used. Sections 15 and 16 read a large grid of quantized CLIP-family models in both scopes, with and without logit tuning, and Section 18 reads the recipe of a DINOv2 recognizer like this one.

11 Object detection in one section

What you will be able to do after this section

Compute the intersection over union of two boxes and the COCO thresholds it clears; build a precision-recall curve and an average precision from a ranked list; explain why mAP@[.5:.95] credits a correct label only in proportion to its box's tightness, while mAP50 gives full credit from an overlap of 0.5; solve a small one-to-one matching by hand and say which duplicate it removes; and state what ground-truth crops isolate and what they give up.

A camera perception stack often has two stages, a detector that boxes the objects in a frame and a recognizer that classifies each crop. This section builds the metrics of the first and the oracle-box test of the second. "Detection" means object detection here, not the out-of-distribution detection of Section 12.

11.1 Boxes, intersection over union, and matching

A detector maps an image to a list of *detections*. Each detection has three parts.

- A box $b = (x_1, y_1, x_2, y_2)$ in pixels, given by its top-left and bottom-right corners, with y pointing down.
- A class c, one of the K classes the detector knows.
- A score $p \in [0, 1]$, the detector's confidence in the detection.

Scoring the detections against the annotated *ground-truth* boxes asks two questions at once, whether a box is in the right place (*localization*) and whether it carries the right label (*classification*). Some annotated objects are too small, crowded, or ambiguous to score fairly. They are marked as *ignore regions*, like the "difficult" objects of the PASCAL Visual Object Classes (VOC) benchmark (Everingham et al., 2010). A detection on an ignore region is neither rewarded nor penalized, and missing one costs nothing. A deployed detector keeps the detections above one score threshold, while the metrics below sweep every threshold.

Intersection over union (IoU). For two axis-aligned boxes A and B with areas $|A|$ and

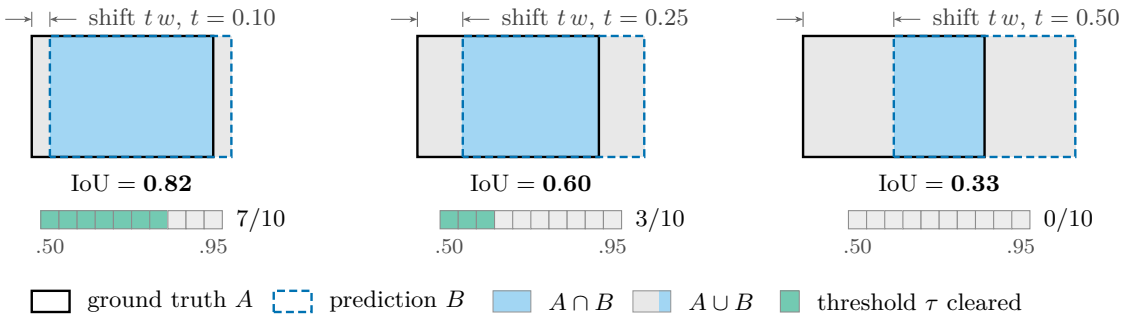


Figure 24: IoU of a predicted box (dashed) shifted sideways by $t\,w$ from a same-size ground-truth box (solid), by Equation (51), with the COCO thresholds it clears. A shift of a tenth of the width already fails the three strictest, and half the width fails them all.

$|B|$,

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}, \tag{50}$$

where $|A \cap B| = \max\big(0, \min(x_2^A, x_2^B) - \max(x_1^A, x_1^B)\big) \cdot \max\big(0, \min(y_2^A, y_2^B) - \max(y_1^A, y_1^B)\big)$, and it equals 1 only for identical boxes. Two boxes of width w and height h, one shifted sideways by $t\,w$ $(0 \le t \le 1)$, have

$$\mathrm{IoU} = \frac{(1-t)\,wh}{2wh - (1-t)\,wh} = \frac{1-t}{1+t}, \qquad \text{equivalently} \qquad t = \frac{1-\mathrm{IoU}}{1+\mathrm{IoU}}. \tag{51}$$

Equation (51) turns thresholds into tolerances. IoU 0.5 allows a shift of a third of the width, IoU 0.95 only $t = 0.05/1.95 = 0.026$. Figure 24 draws three shifts against the ten thresholds $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$ of the COCO (Common Objects in Context) metric (Lin et al., 2014).

Matching at a threshold τ. Detections are scored one class at a time, by a greedy walk down the scores.

1. Sort all detections of the class, over the whole test set, by decreasing score.
2. Take them in this order. A detection is a *true positive* (TP) if its image holds a ground-truth box of the class, not yet claimed, with IoU at least τ; the detection then claims the one of highest IoU. Otherwise the detection is a *false positive* (FP).
3. At the end, every unclaimed ground-truth box is a *false negative* (FN), a missed object.

A second detection of an already claimed object is thus an FP, so duplicates are errors (Everingham et al., 2010). The verdict depends on τ, and one detection can be a TP at $\tau = 0.5$ and an FP at $\tau = 0.9$.

11.2 Precision, recall, and average precision

Walk down the ranked list. After the n-th detection, with TP_n true positives among the first n and G ground-truth objects of the class,

$$P_n = \frac{\mathrm{TP}_n}{n} = \frac{\mathrm{TP}_n}{\mathrm{TP}_n + \mathrm{FP}_n}, \qquad R_n = \frac{\mathrm{TP}_n}{G} = \frac{\mathrm{TP}_n}{\mathrm{TP}_n + \mathrm{FN}_n}. \tag{52}$$

Precision is the fraction of kept detections that are right, recall the fraction of objects found. Each TP raises recall and each FP lowers precision, so the pairs (R_n, P_n) trace a sawtooth, the precision-recall curve.

Bus detections, ranked by score

n	score	match	P_n	R_n
1	0.95	TP	1.00	0.20
2	0.91	TP	1.00	0.40
3	0.85	FP	0.67	0.40
4	0.80	TP	0.75	0.60
5	0.72	FP	0.60	0.60
6	0.64	TP	0.67	0.80
7	0.50	FP	0.57	0.80
8	0.41	FP	0.50	0.80

ground-truth buses: $G = 5$

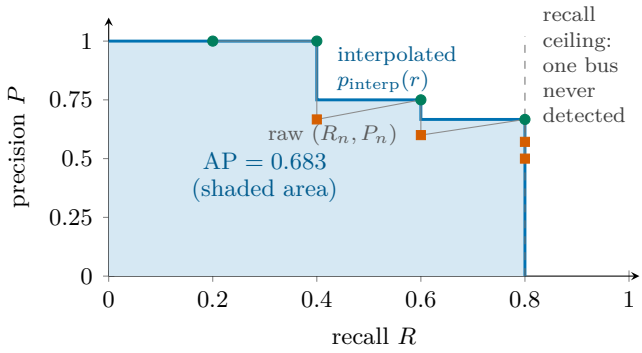


Figure 25: Precision-recall curve of a synthetic ranked list of eight bus detections against five buses (left). The average precision (AP) is the area under the monotone envelope (blue) of the raw sawtooth (gray), 0.683; the bus never detected caps recall, and so AP, at 0.8.

Average precision (AP). Replace the sawtooth by its monotone envelope, the *interpolated precision* $p_{\text{interp}}(r) = \max\{P_n : R_n \geq r\}$ (zero if no R_n reaches r), and take the area under it,

$$\text{AP} = \int_0^1 p_{\text{interp}}(r)\, dr \;\approx\; \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} p_{\text{interp}}(r). \tag{53}$$

The integral is evaluated exactly, as $\sum_n (R_n - R_{n-1})\, p_{\text{interp}}(R_n)$ ("all points"), or on a grid $\mathcal{R}$ of eleven (VOC) or 101 (COCO evaluation code) recall levels. The mean average precision (mAP) averages AP over the K classes; COCO's primary metric also averages over the thresholds $\mathcal{T}$, while mAP50 uses $\tau = 0.5$ only,

$$\text{mAP@[.5:.95]} = \frac{1}{10} \sum_{\tau \in \mathcal{T}} \frac{1}{K} \sum_{c=1}^{K} \text{AP}_c(\tau), \qquad \text{mAP50} = \frac{1}{K} \sum_{c=1}^{K} \text{AP}_c(0.5). \tag{54}$$

Detection tables print these in percent, so an mAP of 40.0 means an average AP of 0.400.

Worked example (AP by hand)

Take the list of Figure 25, with $G = 5$. The envelope is 1 on $[0, 0.4]$; on $(0.4, 0.6]$ it is the best precision at recall 0.6 or more, $\max(0.75, 0.6, 0.667, 0.571, 0.5) = 0.75$; it is 0.667 on $(0.6, 0.8]$ and 0 above. The three conventions then give three values.

- **All points.** $\text{AP} = 0.4(1) + 0.2(0.75) + 0.2(0.667) = 0.683$.
- **COCO grid.** Of the 101 levels $0, 0.01, \ldots, 1$, the counts 41, 20, 20 and 20 fall in the four bands, so $\text{AP}_{101} = (41 + 15 + 13.33)/101 = 0.686$.
- **VOC grid.** Of the eleven levels $0, 0.1, \ldots, 1$, the counts 5, 2, 2 and 2 do, so $\text{AP}_{11} = (5 + 1.5 + 1.33)/11 = 0.712$.

The conventions differ by up to 0.03 on one list, so AP values compare only within one protocol, and the missed fifth bus caps all three at 0.8.

11.3 In what sense mAP@[.5:.95] is dominated by localization

A detection with the right label and IoU u with its object is a TP at exactly the thresholds $\tau \le u$ and an FP, with its object missed, at the others. Its share of the ten thresholds is

$$f(u) = \frac{\#\{\tau \in \mathcal{T} : \tau \le u\}}{10}, \qquad f(0.62) = 0.3, \quad f(0.81) = 0.7, \quad f(0.97) = 1. \tag{55}$$

A detection with the wrong label is an FP at every threshold, and its object is missed. Two kinds of improvement therefore earn different credit.

- *A label correction* on a box of IoU u earns credit only at the share $f(u)$ of thresholds that the box already clears, 30% for $u = 0.62$ and 70% for $u = 0.81$. Under mAP50 the same correction earns full credit as soon as $u \ge 0.5$.
- *A tighter box* with the right label earns credit at each threshold it newly clears. From $u = 0.62$ to 0.81 that is four more thresholds, 0.65 to 0.80, while under mAP50 it earns nothing, since both boxes already clear 0.5.

Nine of the ten thresholds demand more than half overlap, and the strictest demands, by Equation (51), a box within 2.6% of its width. This is the sense in which mAP@[.5:.95] is dominated by localization. It rewards box tightness that mAP50 ignores. The counting is the notes' simplification, since AP is not linear in the number of TPs, and how much of a real detector's error is localization stays an empirical question that the metric alone does not answer.

Reading a gap between the two metrics. A larger gain in mAP50 than in mAP@[.5:.95] is ambiguous, because two different changes produce it.

- *Better labels* on moderately tight boxes, a classification gain. Each correction earns full credit in mAP50 and only the share $f(u)$ in mAP@[.5:.95].
- *Loose boxes* that move from just below IoU 0.5 to just above it, a localization gain. Each turns a miss into a TP at the one threshold of mAP50, but at only the first of the ten thresholds of mAP@[.5:.95].

Telling them apart needs AP at each threshold, or a classification test on ground-truth boxes (Section 11.5). A small difference also needs a test against the spread over training seeds (Section 13).

11.4 Set prediction with DETR and RT-DETR

Most detectors score a dense grid of candidate boxes, so one object receives several overlapping high-scoring boxes, all but one of them false positives. Classic pipelines therefore end with *non-maximum suppression* (NMS), which keeps the highest-scoring box, deletes every same-class box whose IoU with it exceeds a hand-set threshold, and repeats. NMS adds a tuning knob and latency after the network, which is what the real-time detector below sets out to remove (Zhao et al., 2024).

The detection transformer (DETR) of Carion et al. (2020) removes duplicates during training instead. The network emits a fixed number of predictions, one per learned *object query*, each a distribution over the K classes plus "no object" $\varnothing$, and a box. In training, the G annotated objects of an image are paired with queries by the one-to-one assignment of lowest total cost,

$$\hat{\pi} = \arg\min_{\pi} \sum_{j=1}^{G} \mathcal{L}_{\text{match}}\big(y_j, \hat{y}_{\pi(j)}\big), \tag{56}$$

where y_j is the j-th annotated object, $\hat{y}_{\pi(j)}$ the prediction of the query that π gives it, and π gives each object its own query. The matching cost $\mathcal{L}_{\text{match}}$ is low when the query gives the

object's class a high probability and its box lies close to the annotated box. The loss is computed on the matched pairs, and every unmatched query is trained to predict $\varnothing$. A second query on an already matched object is thus taught to say "no object", and the trained detector needs no NMS (Carion et al., 2020). Equation (56) is an assignment problem, solved in polynomial time by the Hungarian method (Kuhn, 1955). Trying every assignment is hopeless, since 100 queries and 10 objects already allow $100!/90! \approx 6.3 \times 10^{19}$ of them.

Worked example, three queries and two objects

A frame holds a bus g_1 and a car g_2. Take the simplified cost $c_{ij} = -p_i(\text{class of } g_j) + (1 - \text{IoU}_{ij})$, which keeps the two ingredients of DETR's cost (the notes' simplification). For example, $c_{11} = -0.70 + (1 - 0.80) = -0.50$.

query	p(bus)	p(car)	IoU with g_1	IoU with g_2	c_{i1}	c_{i2}
q_1	0.70	0.10	0.80	0	-0.50	0.90
q_2	0.60	0.20	0.70	0.10	-0.30	0.70
q_3	0.10	0.80	0	0.75	0.90	-0.55

The $3 \cdot 2 = 6$ ways to give g_1 and g_2 distinct queries cost $c_{11} + c_{22} = 0.20$, $c_{11} + c_{32} = -1.05$, $c_{21} + c_{12} = 0.60$, $c_{21} + c_{32} = -0.85$, $c_{31} + c_{12} = 1.80$, and $c_{31} + c_{22} = 1.60$. The optimum pairs q_1 with the bus and q_3 with the car, and it leaves q_2, a near duplicate of q_1, unmatched, so training pushes q_2 toward $\varnothing$. A classic detector would output both q_1 and q_2 and rely on NMS to delete q_2.

The real-time DETR (RT-DETR) of Zhao et al. (2024) keeps set prediction without NMS and adds an efficient hybrid encoder over multi-scale convolutional features. It comes in variants named by backbone, such as R18 for a ResNet-18 (He et al., 2016).

11.5 Recognition on ground-truth crops

mAP cannot say which of the two questions of Section 11.1 a detection failed. To test the second stage alone, crop each ground-truth box as a square with a margin, resize it to 224×224 (Section 10), and score the classifier by balanced accuracy, the mean per-class recall (Equation (57)).

Definition 11.1: Oracle-box classifier

A classifier evaluated on crops cut from the ground-truth boxes rather than from a detector's predicted boxes. Every object is present, centered, and correctly framed, so missed objects, background false positives, and localization error are removed by construction, and accuracy measures classification given perfect localization.

The isolation has a price. Oracle-box accuracy gives up three things.

1. *Realistic crops.* A deployed recognizer sees the crop of a *predicted* box, with IoU below 1, so its object can be off center or cut off.
2. *A link to the detector.* A recognizer that is a separate system, with its own network, says nothing direct about the labels the detector itself assigns.
3. *A common blur scale.* Blur on the frame and blur on the resized crop are different corruptions, since only σ relative to object size matters (Section 9).

For the third item, let the crop side be 1.1 times the object's longer side and the blur come after resizing, as in the worked example of Section 9. Every object then spans about $224/1.1 \approx 204$ crop pixels along its longer side. Blur $\sigma = 6$ is thus $6/204 = 0.029$ of the length of a car 60

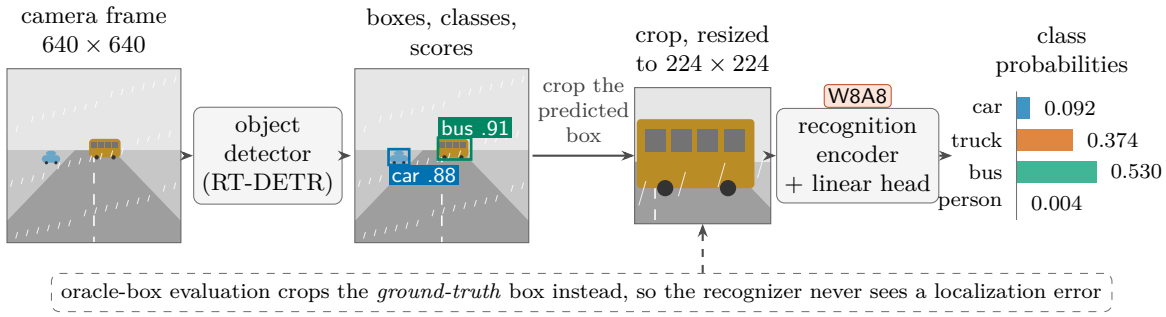


Figure 26: The running example as a two-stage pipeline. The predicted bus box is cropped, resized, and classified by an 8-bit encoder (probabilities of Section 10). Oracle-box evaluation crops the ground-truth box instead (dashed) and so measures classification alone.

frame pixels long on the crop, but $6/60 = 0.10$ of it on the frame. Only objects longer than 204 frame pixels are blurred harder on the crop.

Running example: the bus in the rain

In a 640×640 frame the annotated bus is $A = (300, 250, 420, 330)$, 120×80 pixels, and the detector returns $B = (306, 254, 428, 334)$, bus, score 0.91 (Figure 26). By Equation (50), $|A \cap B| = 114 \cdot 76 = 8664$, $|A| = 9600$, $|B| = 122 \cdot 80 = 9760$, $|A \cup B| = 10696$, and IoU $= 0.81$, so $f(0.81) = 0.7$, a TP at $\tau \leq 0.80$ and an FP above. With margin 1.1 the deployed crop is a square of side 134 around B's center, the oracle crop one of side 132 around A's. Had the detector labeled B a truck, the runner-up of Section 10, the bus would be missed at every threshold; fixing the label recovers a TP at seven of the ten thresholds of mAP@[.5:.95] and at the single one of mAP50, 70% of the credit against 100% in the counting of Equation (55).

In the hands-on session

The set-up code of notebook Part 5, before Part 5.1, builds oracle-box crops from DAWN (Kenk & Hassaballah, 2020), squares of side $1.1 \max(w, h)$ around each box of shorter side at least 40 pixels, resized to 224×224 and blurred after resizing. The notebook has no detector.

Exercise I.24: IoU beyond sideways shifts

A ground-truth box is 100×100 pixels. Compute the IoU, and the thresholds in $\mathcal{T}$ it clears, for (a) a same-size prediction shifted 10 pixels right and 10 down, and (b) a prediction with the same center and 10% longer sides. (c) Both predictions are off by 10 pixels somewhere. Why is IoU harsher on the shift?

Solution sketch. (a) $|A \cap B| = 90^2 = 8100$, $|A \cup B| = 20000 - 8100 = 11900$, IoU $= 0.681$, four of ten thresholds. (b) The prediction contains the ground truth, so only the union grows, IoU $= 100^2/110^2 = 0.826$, seven of ten. (c) A shift both removes area from the intersection and adds it to the union. A centered enlargement keeps the whole ground truth in the intersection and only enlarges the union.

Exercise I.25: Matching by hand

Use the costs of the worked example in Section 11.4. (a) How many one-to-one assignments do N queries and $G \leq N$ objects allow? Check your formula against 6 and 100!/90!. (b) Remove q_3. Which query is matched to the car, and what does training then do to it? (c) Why does the simplified cost carry a minus sign in front of the probability?

Solution sketch. (a) The first object has N queries to choose from, the second $N-1$, and so on, so $N!/(N-G)!$. This is $3 \cdot 2 = 6$ for $N = 3$ and $G = 2$, and $100!/90!$ for $N = 100$ and $G = 10$. (b) Only $c_{11} + c_{22} = 0.20$ and $c_{21} + c_{12} = 0.60$ remain, so q_1 takes the bus and q_2 the car, although q_2 gives the car only 0.20 and overlaps it with IoU 0.10. Every object gets a query, and the loss on the matched pair pulls q_2 toward the car's class and box. (c) The assignment minimizes the cost, and a high probability for the right class should make a pair cheaper.

Exercise I.26: What a duplicate costs

In the list of Figure 25, (a) delete detection 3 and recompute the all-points AP; (b) instead insert a duplicate of detection 1 with score 0.93, a false positive at rank 2, and recompute it. (c) Which mechanism of Section 11.4 prevents (b)?

Solution sketch. (a) The envelope is 1 up to recall 0.6, then 0.8, so AP $= 0.6 + 0.2(0.8) = 0.760$. (b) The pattern is TP, FP, TP, FP, TP, FP, TP, FP, FP; the envelope is 1, 0.667, 0.6, 0.571 on the four recall bands of width 0.2, so AP $= 0.568$, a loss of 0.116 from one duplicate. (c) NMS deletes it after the network; a DETR-style detector is trained by one-to-one matching not to emit it.

Check yourself before moving on

1. Two same-size boxes are shifted sideways by a third of their width. What is their IoU?[a]
2. Why does AP need no score threshold, while a deployed detector does?[b]
3. mAP50 rises by 2 and mAP@[.5:.95] by 0.5. Better labels, or better boxes?[c]
4. Why does DETR need no NMS?[d]
5. What does oracle-box accuracy say about a deployed detector?[e]

[a] $(1 - 1/3)/(1 + 1/3) = 0.5$, by Equation (51), so the pair just clears the loosest COCO threshold.

[b] AP integrates precision over all recall levels, which sweeps the threshold through the whole ranked list; a deployed detector runs at one threshold, one point of that curve.

[c] Either. Better labels on moderately tight boxes fit, since a label fix earns full credit in mAP50 but only $f(u)$ in mAP@[.5:.95]. Loose boxes crossing IoU 0.5 fit too, and a small difference needs a test against the seed spread (Section 13).

[d] Its loss follows a one-to-one matching, so a second query on an already matched object is trained toward "no object".

[e] How well classes are read from perfect boxes, and nothing about misses, background false positives, or loose boxes.

Where this is used. Section 18 meets a real detector and an oracle-box recognizer, and Section 19 reads their tables with these tools.

12 Reliability beyond accuracy

What you will be able to do after this section

Compute plain and balanced accuracy, and say why a car-dominated test set needs the second. Convert between a relative change, an absolute difference, and a retention gain with one identity. Compute the expected calibration error and say what temperature scaling can and cannot repair. Score out-of-distribution detection in two ways that can disagree. Name the kind of shift a test set contains, measure reliance on spurious context, and explain why an effect inside a severity window must be reported severity by severity.

Top-1 accuracy says how often a prediction is right on data like the training data. A vehicle needs to know more, and this section builds one metric for each further question.

- Balanced accuracy (Section 12.1) asks whether the model is right on rare classes.
- Differences and retention (Section 12.2) compare two models on clean and corrupted data.
- Probability calibration (Section 12.3) asks whether the model's confidence can decide a hand-over to a fallback.
- Out-of-distribution detection (Section 12.4) asks whether the model notices an object of a class it was never taught.
- The spurious gap (Section 12.6) asks whether the model leans on the context instead of the object.
- Severity windows (Section 12.7) ask how a difference between two models changes as the image degrades.

Section 13 then decides when a difference in any of them is real.

12.1 Plain accuracy, per-class recall, and balanced accuracy

Evaluate a K-class classifier on N labeled examples, N_i of class i, and let C_{ij} count the examples of true class i predicted as class j. The recall of class i is $r_i = C_{ii}/N_i$, the recall of Section 11 with crops in place of boxes.

Plain and balanced accuracy are two averages of the same recalls.

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=1}^{K} C_{ii} = \sum_{i=1}^{K} \frac{N_i}{N}\, r_i, \qquad \mathrm{BAcc} = \frac{1}{K}\sum_{i=1}^{K} \frac{C_{ii}}{N_i} = \frac{1}{K}\sum_{i=1}^{K} r_i. \tag{57}$$

Chance level. A classifier that ignores its input and predicts class j with probability q_j has expected recall $\mathbb{E}[r_i] = q_i$, so its expected balanced accuracy is $\frac{1}{K}\sum_i q_i = 1/K$ whatever the q_j and the class mix, 25% for four classes. Its plain accuracy $\sum_i (N_i/N)\, q_i$ reaches the majority share under the rule "always answer the majority class". □

Worked example, a car-dominated test set

Take 1000 synthetic crops, 700 cars, 150 trucks, 100 buses, and 50 persons. "Always car" scores Acc = 70% and BAcc = $(100 + 0 + 0 + 0)/4 = 25\%$, exactly chance. Two models on blurred crops have the correct counts below.

class	N_i	FP32 correct	recall	W8A8 correct	recall
car	700	665	95.0%	637	91.0%
truck	150	96	64.0%	102	68.0%
bus	100	48	48.0%	60	60.0%
person	50	35	70.0%	38	76.0%
Acc		844/1000 = 84.4%		837/1000 = 83.7%	
BAcc		2.77/4 = 69.25%		2.95/4 = 73.75%	

Plain accuracy *falls* by 0.7 points while balanced accuracy *rises* by 4.5. One crop of class i is worth $100/(KN_i)$ points of balanced accuracy, 0.036 for a car and 0.5 for a person, so the 28 lost cars cost 1.0 point while the 21 extra trucks, buses, and persons gain 5.5. Only the per-class recalls show where such a change lives, so report them next to both averages.

12.2 Differences, ratios, and retention

Write f for the full-precision reference, q for the quantized model, and $A(m, \mathcal{D})$ for the accuracy of model m, in percent, on test set $\mathcal{D}$. For a corruption of strength σ, such as a blur of σ pixels (Section 9), write $A_m(\sigma) = A(m, \mathcal{D}_\sigma)$, with $\sigma = 0$ the clean set.

Relative change, absolute difference, and retention.

$$\delta(\mathcal{D}) = \frac{A(f, \mathcal{D}) - A(q, \mathcal{D})}{A(f, \mathcal{D})}, \tag{58}$$

$$\Delta\mathrm{Acc}(\sigma) = A_q(\sigma) - A_f(\sigma). \tag{59}$$

Equation (58) is a fraction, positive when the quantized model is *worse*. Equation (59) is a difference in percentage points (pp), positive when it is *better*. On one test set, $\delta = -\Delta\mathrm{Acc}/A_f$, and a fall from 60% to 57% is 3 pp but 5% relative, so never give the two the same unit. The *drop* of model m is $D_m(\sigma) = A_m(0) - A_m(\sigma)$, and the *retention gain* $D_f(\sigma) - D_q(\sigma)$ is positive when the quantized model loses less to the corruption. For any two models f, q,

$$D_f(\sigma) - D_q(\sigma) \;=\; \Delta\mathrm{Acc}(\sigma) - \Delta\mathrm{Acc}(0). \tag{60}$$

Proof. $[A_f(0) - A_f(\sigma)] - [A_q(0) - A_q(\sigma)] = [A_q(\sigma) - A_f(\sigma)] - [A_q(0) - A_f(0)]$. □

Worked example, one comparison in three currencies

Synthetic. A full-precision recognizer scores $A_f(0) = 80.0\%$ on clean crops and $A_f(\sigma) = 50.0\%$ under heavy blur; a QAT model scores 79.0% and 54.0%. Then

$$\Delta\mathrm{Acc}(0) = -1.0 \text{ pp}, \qquad \Delta\mathrm{Acc}(\sigma) = +4.0 \text{ pp}, \qquad \delta(\mathcal{D}_\sigma) = (50.0 - 54.0)/50.0 = -0.08,$$
$$D_f - D_q = (80.0 - 50.0) - (79.0 - 54.0) = 30.0 - 25.0 = +5.0 = 4.0 - (-1.0) \text{ pp}.$$

Under blur the QAT model is 4.0 points more accurate, 8% better in the relative sense, and retains 5.0 points more, three true statements with three different numbers. Plots draw relative changes in either direction, so read the axis label before the number.

Retention can flatter. A PTQ model at 75.0% clean and 52.0% blurred is only 2.0 points ahead under blur, yet it retains $2.0 - (-5.0) = 7.0$ points more, because it starts lower and has less to lose. Report $\Delta\mathrm{Acc}(\sigma)$ with the clean offset, not retention alone.

12.3 Probability calibration

A softmax classifier (Equation (20)) predicts $\hat{y} = \arg\max_y p(y \mid x)$ with *confidence* $\hat{p} = \max_y p(y \mid x)$. It is *calibrated* when, among inputs that receive confidence c, a fraction c are right, $\Pr(\hat{y} = Y \mid \hat{p} = c) = c$. Of 100 crops that a calibrated model labels with confidence 0.8, about 80 are labeled correctly. A reliability diagram checks this by sorting the confidences into M bins $B_1, \ldots, B_M$ and plotting each bin's accuracy against its mean confidence (Figure 27).

Expected calibration error (ECE). With n_b examples in bin B_b,

$$\mathrm{ECE} = \sum_{b=1}^{M} \frac{n_b}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|, \tag{61}$$

where $\mathrm{acc}(B_b) = \frac{1}{n_b} \sum_{i \in B_b} \mathbf{1}[\hat{y}_i = y_i]$ is the fraction of correct predictions in the bin and $\mathrm{conf}(B_b) = \frac{1}{n_b} \sum_{i \in B_b} \hat{p}_i$. The number of bins and the binning rule are choices, and the value

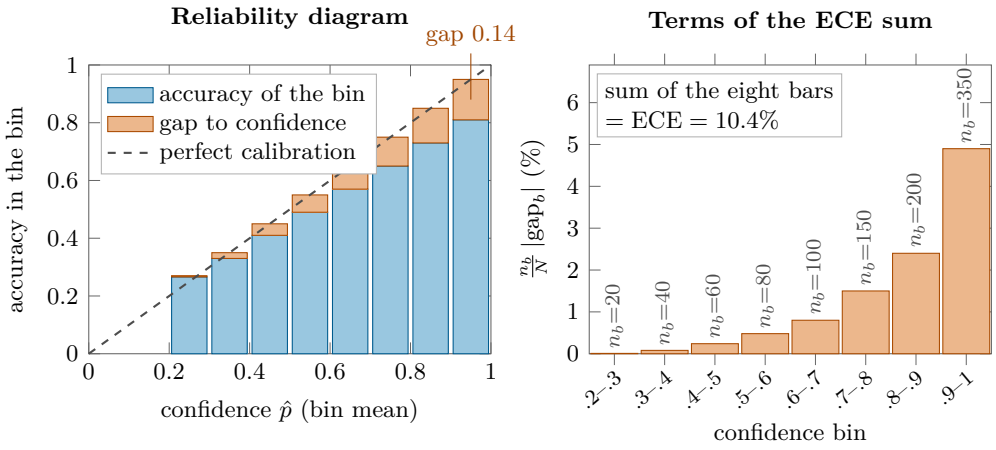


Figure 27: An overconfident model, read two ways. Left, the reliability diagram, with bin accuracy in blue, the gap to the bin's mean confidence in orange, and perfect calibration dashed. Right, the terms of Equation (61) and the bin counts n_b. Synthetic model, 1000 crops of a four-class problem in ten equal-width bins, accuracy $0.05 + 0.8c$ at mean confidence c; the two bins below 0.2 are empty. No bar lies above the diagonal, so no term cancels and the expected calibration error (ECE), 0.104, is mean confidence minus accuracy, $0.770 - 0.666$, almost half of it from the top bin.

depends on them (Exercise I.27).

The binned form goes back to Naeini et al. (2015), and Guo et al. (2017) made it the standard check for deep networks, which they found to be overconfident. Two ECE values are comparable only when they were computed with the same bins.

▷ **Check yourself.** Three bins hold 40, 40, and 20 of 100 predictions, at mean confidence 0.55, 0.75, and 0.95, with accuracy 0.50, 0.70, and 0.80. Compute the ECE, and the share of it that comes from the top bin.[19]

Temperature scaling. For a temperature $T > 0$ applied to the logits z,

$$p_T(y \mid x) = \frac{\exp(z_y/T)}{\sum_{y'} \exp(z_{y'}/T)}. \tag{62}$$

$T > 1$ lowers the confidence, $T < 1$ raises it. Dividing all logits by one positive number never changes the largest, so predictions, accuracy, and balanced accuracy are untouched. T is fitted on held-out labeled data by minimizing the negative log-likelihood (Guo et al., 2017).

The learned logit scale of CLIP (Equation (49)) plays the role of $1/T$ (Section 10). A temperature is one number fitted on one data set, and nothing guarantees that a value fitted on clean crops suits blurred ones.

[19] ECE $= 0.4(0.05) + 0.4(0.05) + 0.2(0.15) = 0.02 + 0.02 + 0.03 = 0.07$, and the top bin gives $0.03/0.07 \approx 43\%$ of it. Every bin is overconfident, so no term cancels, and the ECE equals mean confidence minus accuracy, $0.71 - 0.64$.

Range calibration versus probability calibration

In quantization work "calibration" names two unrelated operations. *Range calibration* sets a quantizer's step from a small unlabeled "calibration set" (Section 6). *Probability calibration* is the agreement that Equation (61) measures. These notes qualify the word wherever the two senses could meet, and a *calibration set* always serves range calibration.

Selective prediction. A deployed classifier may abstain when $\hat{p} < \tau$ and hand the input to a fallback. The *coverage* is the fraction of inputs accepted, the *selective accuracy* the accuracy on them. In the model of Figure 27, $\tau = 0.8$ keeps 550 crops (55% coverage) at $(200 \times 0.73 + 350 \times 0.81)/550 = 78.1\%$ selective accuracy, against 66.6% overall, where their mean confidence promised $(200 \times 0.85 + 350 \times 0.95)/550 = 91.4\%$. An overconfident model delivers less than its threshold suggests, so recalibrate before choosing τ.

Running example: the bus in the rain

The bus in the rain, blurred by a fogged lens, reaches both encoders (synthetic logits for car, truck, bus, person). Full precision gives logits (0.6, 2.3, 1.9, −1.2) and softmax (0.097, 0.531, 0.356, 0.016), the answer "truck" at confidence 0.53. The 8-bit encoder gives (0.4, 1.7, 2.4, −1.3) and (0.082, 0.300, 0.604, 0.015), "bus" at 0.60. At $T = 1.5$ it stays "bus" at 0.51. In the test set of Section 12.1 this one crop is worth 0.1 points of plain accuracy and 0.25 of balanced accuracy.

12.4 Out-of-distribution detection

A K-class classifier names one of its classes even when the input shows none. *Out-of-distribution (OOD) detection* adds a score $S(x)$, high for in-distribution (ID) inputs and low for OOD inputs, which contain classes never taught. The OOD set is *near* when it resembles the ID set (CIFAR-100 images for a CIFAR-10 model) and *far* when it does not (texture images for an ImageNet model) (Yang et al., 2022; Zhang et al., 2024). Three common scores are computed from the classifier's own logits z and softmax $p(y \mid x)$.

- The *maximum softmax probability* (MSP), $S(x) = \max_y p(y \mid x)$, which is the confidence $\hat{p}$ of Section 12.3 (Hendrycks & Gimpel, 2017).
- The *negative energy*, $S(x) = T \log \sum_y \exp(z_y/T)$ (Liu et al., 2020). Adding one constant to every logit leaves the softmax, and so the MSP, unchanged, but raises this score by the same constant. The negative energy thus keeps the common level of the logits, which the MSP discards.
- The *negative entropy* of the softmax, $S(x) = \sum_y p(y \mid x) \log p(y \mid x)$, which is 0 for a one-hot softmax and lowest, $-\log K$, for a uniform one.

Scores built for vision-language models, maximum concept matching (MCM, Ming et al., 2022), NegLabel (Jiang et al., 2024), and envisioning outlier exposure (EOE, Cao et al., 2024), are only named here.

Images and patches. The scores above judge a whole image from the classifier's outputs. A score can also use the embeddings. The distance from an image's embedding to its nearest neighbors among the training embeddings is large for an unusual image (Sun et al., 2022). Computed for the features of local patches, the same distance points to the unusual region, as PatchCore does with a memory bank of patch features from normal images (Roth et al., 2022); in a vision transformer the patch features are the token embeddings of Section 5. Road-scene benchmarks score every pixel for the same reason, since an unknown object often covers only

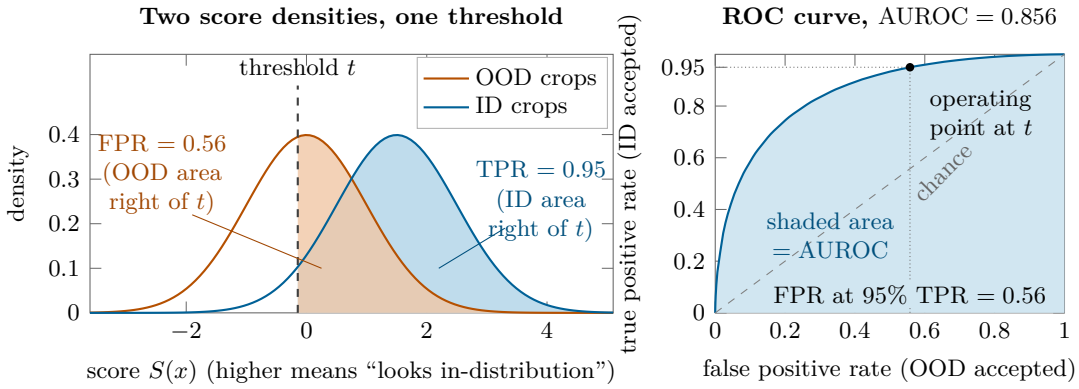


Figure 28: One detector, two summaries. Left, synthetic scores, OOD $S \sim \mathcal{N}(0,1)$ and ID $S \sim \mathcal{N}(1.5,1)$; the dashed threshold accepts 95% of ID inputs and, with them, 56% of OOD inputs. Right, the ROC curve. The shaded area under it is the AUROC (area under the ROC curve), $\Phi(1.5/\sqrt{2}) = 0.856$, since $S_{\text{ID}} - S_{\text{OOD}} \sim \mathcal{N}(1.5, 2)$. A respectable AUROC coexists with a false positive rate above one half at the operating point a vehicle would use.

a small part of the scene (Blum et al., 2021). In this sense an outlier is an unusual input or region, not the outlier channel of quantization (Section 6.2).

From a score to a decision. Accept an input as ID when $S(x) \geq t$. Two rates describe the threshold t, the *true positive rate* $\text{TPR}(t) = \Pr(S_{\text{ID}} \geq t)$, the fraction of ID inputs accepted, and the *false positive rate* $\text{FPR}(t) = \Pr(S_{\text{OOD}} \geq t)$, the fraction of OOD inputs wrongly accepted. Sliding t from high to low traces the receiver operating characteristic (ROC) curve of the pairs (FPR, TPR), from $(0,0)$, where nothing is accepted, to $(1,1)$, where everything is.

Area under the ROC curve (AUROC) and FPR at 95% TPR. For independent ID and OOD scores,

$$\text{AUROC} = \int_0^1 \text{TPR} \; d\,\text{FPR} = \Pr(S_{\text{ID}} > S_{\text{OOD}}) + \tfrac{1}{2}\Pr(S_{\text{ID}} = S_{\text{OOD}}). \tag{63}$$

AUROC $= 1$ is perfect separation, 0.5 is chance. FPR@95 is the false positive rate at the threshold that accepts 95% of ID inputs, lower being better. AUROC summarizes all thresholds; FPR@95 reports the one a safety case cares about.

▷ **Check yourself.** Four ID crops score $0.9, 0.8, 0.6, 0.4$ and three OOD crops $0.7, 0.5, 0.4$. Count the 12 pairs to find the AUROC, then find FPR@95.[20]

12.5 Kinds of shift

A test set can differ from the training data in ways that call for different metrics. The first question is whether the label set changed.

- *Covariate shift.* The rain changes how the bus looks, but a bus is still one of the four classes. The right output is still the class, and the metric is accuracy on the shifted set, through Equation (58) or Equation (59).

[20] The ID crops at 0.9 and 0.8 win 3 pairs each, the one at 0.6 wins 2, and the one at 0.4 ties once ($\frac{1}{2}$), so AUROC $= 8.5/12 = 0.708$, which the ROC area confirms. Accepting 95% of four ID crops means accepting all four, so $t = 0.4$, all three OOD crops pass, and FPR@95 $= 100\%$.

- *Semantic shift* (Yang et al., 2024). A tractor in the same rain belongs to none of the four classes. The right output is not a confident "truck" but a low score $S(x)$ that flags the input as OOD, and the metrics are Equation (63) and FPR@95.

The second question is where the shift comes from.

- *Natural* shift is collected, such as ImageNet-A, -R, -V2, and -Sketch (Hendrycks, Basart, et al., 2021; Hendrycks, Zhao, et al., 2021; Recht et al., 2019; Wang et al., 2019) for an ImageNet model, or the real fog of DAWN (Kenk & Hassaballah, 2020) for a model trained in clear weather.
- *Synthetic* shift is a corruption applied to clean images, such as the blur of Section 9 or ImageNet-C (Hendrycks & Dietterich, 2019).

Whether weather is a shift at all depends on the training data. For a model trained on foggy crops, fog is in-distribution.

"Semantic" robustness

"Semantic" is also used loosely, for instance for robustness to renditions and sketches whose classes do not change. The test is whether the label set changed. If it did not, the shift is covariate. These notes write *covariate shift* and *semantic shift* only, and *natural* versus *synthetic* for the source.

12.6 Spurious correlations and group robustness

A feature is *spurious* when it predicts the label in the training data without being part of what the label means. Buses are usually seen on city streets, yet a bus on a rural road is still a bus. A model that leans on the street does well on average and fails on the rare group where context and class disagree, which the *worst-group accuracy* $\min_g A_g$ over groups g (class crossed with context) exposes (Sagawa et al., 2020). To measure the reliance, split each class into a subset $\mathcal{D}_N$ in typical context and a subset $\mathcal{D}_C$ in atypical context, as CounterAnimal does for animal photographs (Wang, Lin, et al., 2024).

Relative spurious gap (RSG) and added vulnerability.

$$\mathrm{RSG}(m) = \frac{A(m, \mathcal{D}_N) - A(m, \mathcal{D}_C)}{A(m, \mathcal{D}_N)}, \qquad \Delta\mathrm{RSG} = \mathrm{RSG}(q) - \mathrm{RSG}(f), \tag{64}$$
$$\mathrm{Vuln}_{\mathrm{add}} = \delta_C - \delta_N,$$

with δ_C, δ_N from Equation (58) on each subset. Positive values mean that quantization widened the gap or hurt the atypical subset more. Both are differences of relative quantities, so a value printed as "5%" means 5 points of a relative gap.

▷ **Check yourself.** On buses in typical street scenes full precision scores 80% and the quantized model 79%; in atypical scenes, 60% and 55% (synthetic). Compute ΔRSG, $\mathrm{Vuln}_{\mathrm{add}}$, and the change in worst-group accuracy.[21]

[21] $\mathrm{RSG}(f) = 20/80 = 0.250$ and $\mathrm{RSG}(q) = 24/79 = 0.304$, so $\Delta\mathrm{RSG} = +5.4$ points; $\delta_N = 1.25\%$ and $\delta_C = 8.33\%$, so $\mathrm{Vuln}_{\mathrm{add}} = +7.1$ points. The worst-group accuracy falls from 60% to 55%, the equal-weight average only from 70% to 67%.

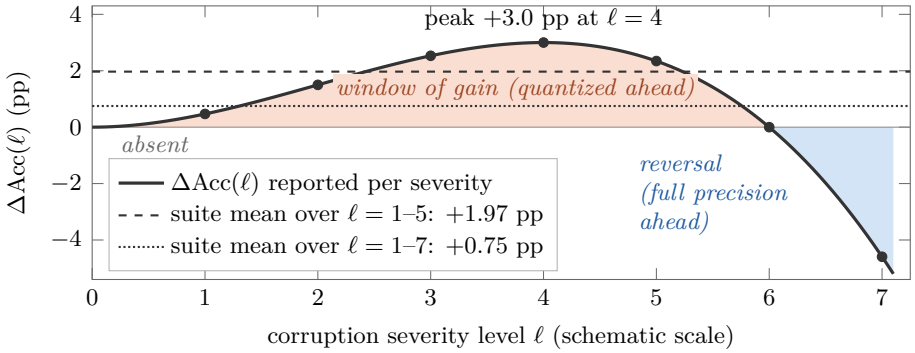


Figure 29: A severity window, schematic. The curve is the stated model $\Delta\mathrm{Acc}(\ell) = \frac{3}{32}\ell^2(6-\ell)$ points at severity level ℓ, chosen only to rise, peak, and reverse. A suite averaging $\ell = 1$–5 reports +1.97; one that also samples $\ell = 6, 7$ reports +0.75, a quarter of the peak of +3.0. Both averages are correct, and neither shows the window or the reversal.

12.7 Severity windows, and why averages hide them

Plot Equation (59) against the strength of the corruption, a blur σ in pixels of the image it acts on (Section 9) or a benchmark severity level ℓ, and one comparison becomes a curve. The gain can be absent at mild corruption, rise, peak, and reverse, and the range where it is positive is a *severity window* (Figure 29). A suite average over severities and corruption families dilutes the window, which occupies only part of the axis, and a reversal can cancel it. Report such an effect per severity, with any average next to the curve, never instead of it.

▷ **Check yourself.** Tabulate the model of Figure 29 at $\ell = 1, \ldots, 7$ and recompute the two suite averages of its caption.[22]

A window says where an effect lives, not why it ends; the cause of a reversal needs an experiment of its own. Nor does a window found at one image size carry over to another, since a blur of σ pixels removes more of a small image than of a large one (Section 9).

In the hands-on session, notebook Part 5

Part 5 evaluates a frozen DINOv2 ViT-S/14 with a linear head on DAWN crops, in FP32 and W8A8 (PTQ and QAT), under Gaussian blur $\sigma \in \{0, 1, 2, 4, 6, 8, 10, 14\}$ on the 224×224 crop. Its `bal_acc` is Equation (57) and its `QAT−FP32` column Equation (59). Extend it three ways. Add plain accuracy; compute a ten-bin ECE from the softmax outputs of `predict` at $\sigma = 6$; average the seven blurred rows ($\sigma \geq 1$) of `QAT−FP32` and compare the mean with the peak. With one seed, single points move by about ±2 points between runs. Part 5 needs a GPU.

Exercises

Exercise I.27: ECE depends on the bins

A model puts 100 crops at confidence 0.6 with accuracy 0.7 and 100 crops at confidence 0.8 with accuracy 0.7. Compute the ECE with the bins $[0.5, 0.7)$ and $[0.7, 0.9)$, then with the single bin $[0.5, 0.9)$. Can temperature scaling change this model's accuracy?

Solution sketch. Two bins give $\frac{1}{2}\cdot 0.1 + \frac{1}{2}\cdot 0.1 = 0.10$. The single bin has mean confidence and accuracy 0.7, so ECE = 0, because underconfidence and overconfidence cancel inside a wide bin. Temperature cannot change accuracy, since Equation (62) preserves each arg max.

[22] $\Delta\mathrm{Acc}(\ell) = 0.47$, 1.50, 2.53, 3.00, 2.34, 0, and −4.59 points. The first five sum to 9.84, a mean of +1.97; all seven sum to 5.25, a mean of +0.75.

Exercise I.28: What AUROC ignores, and what temperature changes

(a) Show that replacing a score S by $g(S)$, with g strictly increasing, leaves AUROC and FPR@95 unchanged. (b) Is the maximum softmax probability at temperature T an increasing function of the one at $T = 1$, across inputs? Test $z_1 = (2, 0, 0, 0)$ and $z_2 = (1, 1, -10, -10)$ at $T = 1$ and $T = 10$.

Solution sketch. (a) g keeps every pairwise order and tie, so Equation (63) is unchanged, and the threshold $g(t)$ has the TPR and FPR of t. (b) No. The MSP of z_1 and z_2 is 0.711 and 0.500 at $T = 1$, but 0.289 and 0.375 at $T = 10$, so temperature can change the AUROC of MSP and what a confidence threshold accepts.

Check yourself before moving on

1. Why is the chance level of balanced accuracy $1/K$ whatever the class mix, while that of plain accuracy is not?[a]
2. Quantized model A is 1 point ahead of full precision under blur and 4 behind on clean data; model B is 3 ahead and 1 behind. Which has the larger retention gain, and which is more accurate under blur?[b]
3. What does ECE = 0 guarantee, and what does it not?[c]
4. Can a detector with AUROC = 0.86 accept more than half of the OOD inputs at the threshold that keeps 95% of ID inputs?[d]
5. A bus in fog and a tractor in fog. Which is covariate shift, which is semantic shift, and which metric belongs to each?[e]

[a] Under an input-independent rule each recall equals the probability of predicting that class, and balanced accuracy averages these with equal weights. Plain accuracy weights them by class shares, so favoring the majority class scores its share.

[b] By Equation (60), A retains $1 - (-4) = 5$ points more than full precision and B $3 - (-1) = 4$. B is the more accurate under blur; A only started lower.

[c] Agreement of accuracy and mean confidence in every bin, on the data measured. Nothing about accuracy itself, and it can hide cancellation inside a bin (Exercise I.27).

[d] Yes. The model of Figure 28 has AUROC = 0.856 and FPR@95 = 0.56.

[e] The bus is covariate shift, scored by accuracy or ΔAcc on the shifted set. The tractor is semantic shift, scored by OOD detection with AUROC and FPR@95.

Where this is used. Sections 15 and 16 read calibration, OOD detection, shift, and spurious reliance with these metrics, and Section 15 lists the sign convention of every relative change it meets in one table. Sections 18 and 19 read balanced accuracy, ΔAcc, retention, and a measured severity window.

13 Statistics for comparing models

What you will be able to do after this section

Say what a mean, a median, a standard deviation, a standard error, and a confidence interval each describe, and which spread a table's "±" is. Build a paired, class-stratified bootstrap interval and say which variation it covers. Keep a small p-value apart from a large effect, and apply the Benjamini–Hochberg and Holm corrections by hand. Explain why "no significant difference" is not "no difference", why group means can correlate far more strongly than the runs inside them, and why selecting runs by their outcome biases what you see. Name the elements of a comparison that can be read causally.

Every result in Part II is a difference between two models, measured on a finite test set after a finite number of training runs. Three questions decide what such a difference means. How large is it? How sure is it, given everything that could have come out otherwise? And what caused it? The first two are statistics and the third is experimental design. All numbers in this section are synthetic.

13.1 Describing a set of runs

Training involves random choices, such as the initial head weights and the order of the mini-batches. A *seed* fixes them, and a *run* is one trained model from one seed, one draw of a random outcome. Scoring one checkpoint on many test sets gives many evaluations but not independent repetitions of training. Every accuracy computed from a run inherits the training randomness, and the finite test set adds a second, separate source. Much of this section is about keeping the two apart.

The *mean* $\bar{a} = \frac{1}{n}\sum_i a_i$ and the *median*, the middle value after sorting, agree for a symmetric distribution and part ways under skew, because the mean follows a long tail and the median does not. Nine runs that lose 5% and one that loses 80% have mean -12.5% and median -5%; a mean far below the median is the signature of a long tail of large losses. Neither number is wrong. The mean answers "how much is lost on average", the median "what does a typical run look like".

Three spreads for three questions. For run scores $a_1, \dots, a_n$ with mean $\bar{a}$,

$$\mathrm{SD}_{\mathrm{pop}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(a_i - \bar{a})^2}, \qquad \mathrm{SD} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(a_i - \bar{a})^2} = \mathrm{SD}_{\mathrm{pop}}\sqrt{\frac{n}{n-1}}, \tag{65}$$

$$\mathrm{SE}(\bar{a}) = \frac{\mathrm{SD}}{\sqrt{n}}.$$

The *population standard deviation* $\mathrm{SD}_{\mathrm{pop}}$ describes the n runs at hand. The *sample standard deviation* SD estimates how far one new run lands from the long-run mean of the recipe. The *standard error* estimates how far the mean of n runs lands from it; it shrinks as $1/\sqrt{n}$, while the spread of single runs does not.

A "$\pm$" without a definition cannot be interpreted, so check which spread a table prints before reading it.

A 95% *confidence interval* (CI) is a range computed from the data by a rule that covers the true value in 95% of repetitions of the whole experiment. The 95% belongs to the rule, not to one computed interval. Under a normal model the rule for a mean is

$$\bar{a} \pm t_{n-1}\,\mathrm{SE}(\bar{a}),$$

where t_{n-1} is the 97.5th percentile of *Student's t distribution* with $n-1$ *degrees of freedom*, the number of independent deviations left once the mean is estimated. It replaces the normal 1.96 because the SE is itself estimated from few runs, and it is 4.30 for $n = 3$, 2.78 for $n = 5$, and 2.26 for $n = 10$.

Worked example, the interval behind 72.0 ± 1.0

A table prints 72.0 ± 1.0 over three seeds, with $\pm$ the population standard deviation.

- By Equation (65), $\mathrm{SD} = 1.0\sqrt{3/2} = 1.22$.
- The standard error of the mean is $\mathrm{SE} = 1.22/\sqrt{3} = 0.71$.

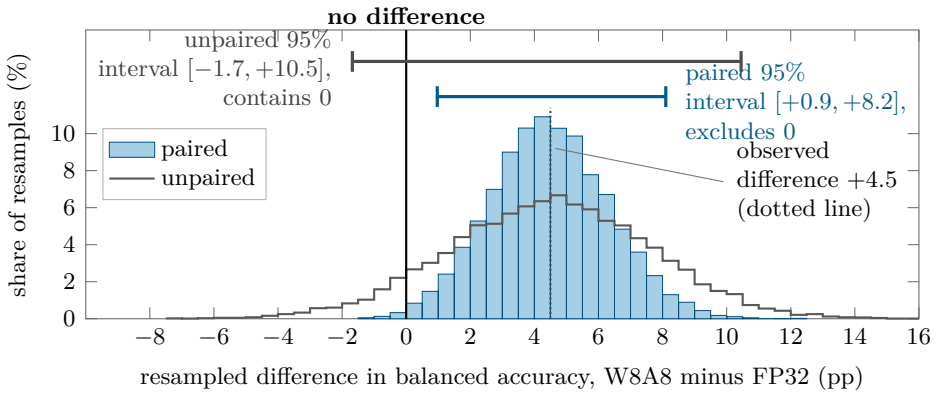


Figure 30: Pairing narrows the interval. Bootstrap distributions of the difference in balanced accuracy between two synthetic recognizers on the car-dominated test set of Section 12, 10,000 resamples each. The blue bars draw crops within each class and score both models on the same draw; the gray outline draws the two models' crops independently. Both center on the observed +4.5 pp, but only the paired percentile interval of Equation (66) excludes zero, because pairing cancels the difficulty the two models share crop by crop.

- With $t_2 = 4.30$, the 95% CI for the long-run mean is $72.0 \pm 4.30 \times 0.71 = [69.0, 75.0]$.

Three seeds leave the mean uncertain by three points either way, three times the printed $\pm$.

Comparing two variants. An interval for one variant's mean is not an interval for the difference between two variants, and setting it against the other variant's point estimate is not a test. The difference needs an interval of its own. For two independent means,

$$\mathrm{SE}(\bar{a} - \bar{b}) = \sqrt{\mathrm{SE}(\bar{a})^2 + \mathrm{SE}(\bar{b})^2}.$$

When the two variants share seeds, a *paired t-test* applies the rule above to the per-seed differences d_i, and rejects "no difference" at the 5% level when $\bar{d} \pm t_{n-1}\,\mathrm{SE}(\bar{d})$ excludes zero. Pairing usually gives the smaller SE, because a seed that suits one variant tends to suit the other (Exercise I.29).

13.2 Paired differences and the bootstrap

Two models scored on the same test crops are not two independent samples. A sharp, typical crop is usually right for both and a hopeless one wrong for both; only the *discordant* crops, right for one model and wrong for the other, carry information about the difference. A *paired* analysis works with the per-crop differences $d_i \in \{-1, 0, +1\}$ and cancels the difficulty the two models share, while an unpaired one charges the difference with the full variance of both.

The *bootstrap* (Efron, 1979) estimates how a statistic would vary over fresh test sets by recomputing it on resamples of the one test set, drawn with replacement and of the original size.

Percentile bootstrap interval (Efron & Tibshirani, 1993). Let $\hat{\theta}$ be a statistic of the test set, such as the difference in balanced accuracy. Draw B resamples, compute $\hat{\theta}^*_1, \ldots, \hat{\theta}^*_B$, sort them, and take

$$\mathrm{CI}_{1-\alpha} = \left[\hat{\theta}^*_{(\lceil B\alpha/2 \rceil)},\ \hat{\theta}^*_{(\lceil B(1-\alpha/2) \rceil)} \right], \tag{66}$$

the 2.5th and 97.5th percentiles for $\alpha = 0.05$. *Paired* means that one resample draws crop indices once and scores both models on them. *Class-stratified* means that the draw happens within each class, n_c crops from the n_c of class c, so every resample keeps the class counts n_c, the denominators of the per-class recalls that balanced accuracy (Equation (57)) averages with equal weights.

An interval covers the variation that the resampling reproduces, and nothing else. Resampling crops imitates a new draw of test crops with the trained models held fixed. Nothing is retrained, so the interval says nothing about how far a new seed would move the result, and a study with several seeds per variant must also say how the seeds enter each resample (scored separately, averaged, or pooled).

Worked example, a paired class-stratified bootstrap by hand

Take the two recognizers of the worked example in Section 12, FP32 with BAcc = 69.25% and W8A8 with 73.75%, and fix how their outcomes pair up.

class	n_c	both right	FP32 only	W8A8 only	both wrong	recall difference
car	700	630	35	7	28	$(7-35)/700 = -0.040$
truck	150	90	6	12	42	$(12-6)/150 = +0.040$
bus	100	44	4	16	36	$(16-4)/100 = +0.120$
person	50	33	2	5	10	$(5-2)/50 = +0.060$

The four recall differences average to +0.045, the +4.5 pp of Section 12, and only the 87 discordant crops enter them. With discordant shares p^+_c (W8A8 only) and p^-_c (FP32 only), the per-crop difference in class c has variance $v_c = p^+_c + p^-_c - (p^+_c - p^-_c)^2$, so the bootstrap standard deviation is about

$$\tfrac{1}{4}\sqrt{\textstyle\sum_c v_c/n_c} = \tfrac{1}{4}\sqrt{\tfrac{0.0584}{700} + \tfrac{0.1184}{150} + \tfrac{0.1856}{100} + \tfrac{0.1364}{50}} = 0.0185,$$

or 1.85 pp. Resampling the models separately replaces v_c by the sum of two binomial variances and raises it to 3.15 pp. Only 0.51% of the paired resamples of Figure 30 fall at or below zero, and doubling that share is one simple way to report a two-sided $p \approx 0.01$. The 50 persons supply half of $\sum_c v_c/n_c$, so the rare classes set the width.

Running example: the bus in the rain

In the worked test set the bus in the rain is one of the 16 buses that only the 8-bit encoder gets right. Its crop carries $d = +1$, worth 0.25 pp of balanced accuracy. A class-stratified resample draws 100 buses with replacement, so the crop appears 0, 1, 2, or 3 times with probabilities 0.366, 0.370, 0.185, and 0.061; in about a third of the resamples its 0.25 pp is missing. Summed over the 87 discordant crops, this jitter is the width of the blue histogram. The bootstrap never asks whether a second training seed would still get the bus right; only retraining answers that.

13.3 p-values, effect sizes, and many tests

A *p-value* is the probability, computed as if there were no effect (the *null hypothesis*), of a result at least as extreme as the one observed. A small p says the data would be surprising

without an effect. It does not say the effect is large, and for any fixed nonzero effect it shrinks as observations accumulate. The *effect size* is the quantity itself, such as a difference in points with its interval.

The *Pearson correlation* of paired values (x_i, y_i),

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \, \sum_i (y_i - \bar{y})^2}} \in [-1, 1],$$

measures how closely the points follow a straight line, and r^2 is the share of the variance of y that the line explains. The usual test of $r = 0$ compares $t = r\sqrt{n-2}/\sqrt{1-r^2}$ with Student's t at $n-2$ degrees of freedom, so p depends on n as much as on r. For $r = 0.03$ it gives $p = 0.77$ with $n = 100$ points, 0.034 with $n = 5000$, and about 7×10^{-100} with $n = 500{,}000$, while $r^2 = 0.0009$ stays at 0.09% of the variance. With enough runs every nonzero effect becomes significant, so report the effect and its interval before the p-value.

Testing many cells multiplies the chances of a false alarm. With $m = 10$ independent tests at level 0.05 and no true effect anywhere, the probability of at least one $p < 0.05$ is $1 - 0.95^{10} = 0.40$. Two standard corrections sort the p-values, $p_{(1)} \le \cdots \le p_{(m)}$, and write $H_{(k)}$ for the hypothesis with the k-th smallest (here k is a position among sorted p-values, not a rank index).

Benjamini–Hochberg (BH) and Holm. For a false discovery rate q,

$$k^\star = \max\Big\{k : \; p_{(k)} \le \frac{k}{m}\, q\Big\}, \qquad \text{reject } H_{(1)}, \ldots, H_{(k^\star)}. \tag{67}$$

For a family-wise error level α,

$$k^\dagger = \min\Big\{k : \; p_{(k)} > \frac{\alpha}{m - k + 1}\Big\}, \qquad \text{reject } H_{(1)}, \ldots, H_{(k^\dagger - 1)}, \tag{68}$$

and every hypothesis when no such k exists. BH is a *step-up* rule. It finds the largest k whose p-value passes its bound and rejects everything up to it, even a hypothesis that fails its own bound on the way. For independent tests it keeps the expected share of false discoveries among the rejections at or below q (Benjamini & Hochberg, 1995), and the guarantee extends to the positive dependence expected when the tests share models and test crops (Benjamini & Yekutieli, 2001). Holm is a *step-down* rule. It walks up from the smallest p-value and stops at the first that fails. It keeps the probability of even one false rejection at or below α under any dependence (Holm, 1979). Bonferroni compares every p with α/m, the first and strictest of Holm's thresholds.

Worked example, ten cells corrected two ways

A comparison of two recognizers tests their difference in ten corruption cells and finds five p-values below 0.05 and five above it (synthetic). With $m = 10$ and $q = \alpha = 0.05$,

k	$p_{(k)}$	BH bound $k\,q/10$	Holm bound $\alpha/(11-k)$
1	0.0004	0.005 pass	0.0050 pass
2	0.003	0.010 pass	0.0056 pass
3	0.009	0.015 pass	0.0063 fail, stop
4	0.019	0.020 pass	0.0071 not reached
5	0.041	0.025 fail	0.0083 not reached

Rank 5 fails BH ($0.041 > 0.025$), and so do ranks 6 to 10, whose p-values exceed 0.05, the largest bound, so $k^\star = 4$ and BH rejects four hypotheses. Holm stops at $0.009 > 0.0063$ and rejects two. An uncorrected reading would call five cells significant, although ten tests with no effect anywhere raise at least one false alarm with probability 0.40.

13.4 No significant difference is not no difference

A test that fails to reject says the data are compatible with no effect, and they may be equally compatible with a sizable one. An *equivalence test* turns the question around. It fixes a margin δ_{eq}, the smallest difference that would matter, and asks whether the data rule out every true difference θ at least that large. The two one-sided tests (TOST) procedure tests against $\theta \le -\delta_{\text{eq}}$ and against $\theta \ge \delta_{\text{eq}}$ and declares equivalence when both reject at level α, which is the same as the $1 - 2\alpha$ confidence interval lying inside $(-\delta_{\text{eq}}, \delta_{\text{eq}})$ (Lakens, 2017).

Three seeds limit how narrow such a bound can be.

Worked example, how narrow a bound can three seeds certify?

A quantized detector scores 20.0 ± 0.8 mAP and its full-precision control 21.0 ± 0.8, each over three seeds with $\pm$ the population standard deviation, so $\hat{\theta} = -1.0$ (synthetic).

1. *Spreads.* SD $= 0.8\sqrt{3/2} = 0.98$, and each mean has standard error $0.98/\sqrt{3} = 0.57$.
2. *The difference.* Two independent means give $\text{SE}(\hat{\theta}) = \sqrt{0.57^2 + 0.57^2} = 0.80$, with $3 + 3 - 2 = 4$ degrees of freedom.
3. *Significance.* The 95% CI is $-1.0 \pm 2.78 \times 0.80 = [-3.2, +1.2]$. It contains zero, so the difference is not significant.
4. *Equivalence.* The 90% CI uses the one-sided 5% value 2.13 and gives $-1.0 \pm 1.7 = [-2.7, +0.7]$, so TOST at $\alpha = 0.05$ declares equivalence only for margins above 2.7 mAP.

The comparison bounds the difference at the protocol's resolution, here about three points. It does not show that the two models are equivalent within any smaller margin.

Pairing by seed, when the variants share seeds, can narrow the bound, as in Exercise I.29.

13.5 Aggregation and selection

A correlation computed over group averages, such as classes or quantization methods, can be much stronger than the correlation over the individuals inside the groups, and can even have the opposite sign. Robinson (1950) called the first kind an *ecological* correlation. Averaging n runs divides the within-group scatter variance by n and leaves the spread of the group centers untouched, so whatever relation the centers share dominates the averages (Figure 31).

A report that prints the number of raw runs beside a correlation over group means invites the wrong reading, and the p-value gives it away. The right panel of Figure 31 has $p = 8 \times 10^{-7}$ at $n = 12$, while the same r over 360 runs would have about 5×10^{-200}.

Selection on the outcome is the second trap. When a subset is chosen by how its runs turned out, every metric correlated with the selection criterion looks better inside it, with or without an effect. If a thousand quantized runs have a true accuracy change of zero and measurement noise of standard deviation 1 pp, keeping the "successful" runs measured above -1 pp keeps 84% of them and reports a mean gain of $\varphi(1)/\Phi(1) = +0.29$ pp from nothing, with φ and Φ the standard normal density and distribution function. When two subsets differ in the filter and in a second factor at once, such as the bit widths they admit, their contrast isolates neither.

Regression to the mean is selection's twin. When a measurement is noisy, units chosen for an extreme first value score closer to the average on a second one, with no intervention at all. If the true expected calibration errors (Section 12) of a set of models have variance 1 and each measurement adds independent noise of variance 1, two measurements correlate at 1/2, and models selected two standard deviations above average on the first sit, on average, one above on the second. A comparison that splits models by their starting value, such as "the

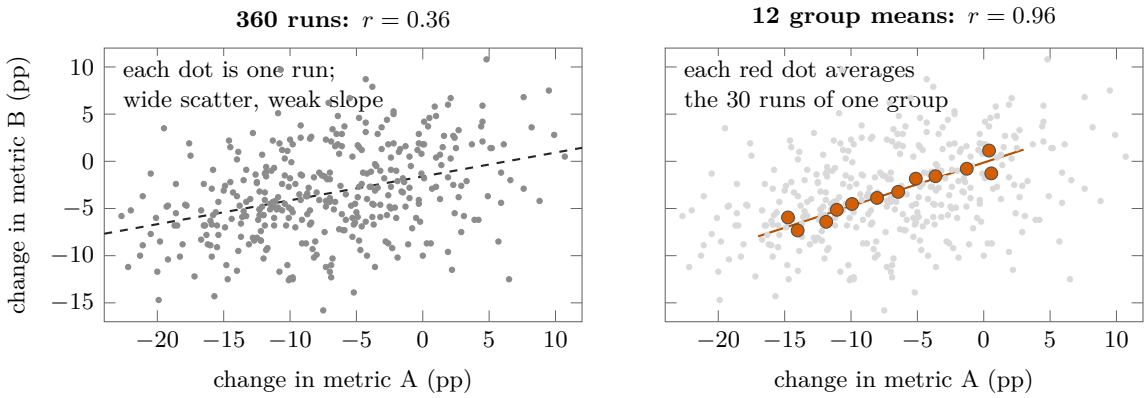


Figure 31: Same runs, two correlations. Left, 360 synthetic runs in 12 groups (think of a group as one quantization method), correlated run by run, $r = 0.36$. Right, the 12 group means of the same runs, $r = 0.96$. The group centers m_g are evenly spaced from -15 to 0, and one run changes A by $m_g + u$ and B by $0.45\, m_g + v$, with $u \sim \mathcal{N}(0, 5^2)$ and $v \sim \mathcal{N}(0, 4^2)$. Averaging removes the run-level noise but not the shared dependence on m_g, so a correlation over means says how the groups line up, not how tightly one run's B follows its A.

worst-calibrated models improved most after quantization", inherits part of this pattern for free.

13.6 Designing a comparison that can be read causally

Statistics says how sure a difference is, and design says what it can be attributed to. A comparison of a quantized model with its full-precision control can be read causally when it has four elements.

- *One factor changed.* Same data, schedule, and seeds; only quantization differs.
- *The same start and budget.* One trained checkpoint for both, and the same extra training steps for the control without quantizers, so a gain cannot come from training longer.
- *The same randomness.* Identical seeds, data order, and corruption draws, so every crop and every seed can be paired.
- *Controls for simpler explanations.* A variant quantized without training tests the arithmetic alone, and one trained with added noise tests whether any noise would do.

A control rules out only what it matches. A full-precision model trained with uniform noise of variance $s^2/12$ (Section 4) matches the size of the rounding error, but not its being a deterministic function of the input (Section 4.4), nor the clipping, nor a step that changes during training. If it fails to reproduce a gain, noise of that size is ruled out as the whole cause and the other properties are not. Its own result has a seed spread and needs an interval like any other.

Three distinctions complete the toolkit.

- *Intervention versus correlation.* An intervention sets a factor and watches the outcome, as the variants above do. A correlational finding observes two quantities together, such as an alignment between two residuals, and can neither say which causes which nor rule out a third factor.
- *Necessary versus sufficient.* A condition is necessary for an effect if the effect never occurs without it, and sufficient if the effect always occurs with it. Finding the condition in

perturbations that lack the effect shows it is not sufficient; showing it necessary takes an intervention that removes it.

- *A falsifiable prediction.* An account earns trust by risking a prediction that could fail. One that credits a gain to the size of the rounding error predicts that the matched-noise control reproduces it, and is falsified in that form if it does not.

In the hands-on session, notebook Part 5

Part 5 trains FP32, PTQ, and QAT recognizers with `SEED = 0` and prints balanced accuracy per blur σ with a `QAT−FP32` column. Its test split has its own fixed seed (`split_by_image`), so changing `SEED` retrains without changing the test crops. Rerun with `SEED` set to 0, 1, 2 and report $\mathrm{SD}_{\mathrm{pop}}$, SD, and the paired SE of `QAT−FP32` at $\sigma = 6$ (Equation (65)). Then, for one seed, keep the per-crop correctness `predict(model, x).argmax(1) == TE_Y` of FP32 and QAT at $\sigma = 6$ and compute the paired, class-stratified interval of Equation (66) with 2000 resamples. Part 5 needs a GPU.

Exercises

Exercise I.29: Three seeds, paired and unpaired

Three seeds give balanced accuracies 70.7, 72.1, 73.2 for a W8A8 recognizer, and the FP32 control trained with the same three seeds gives 67.2, 68.3, 68.5 (synthetic). (a) Compute the mean, $\mathrm{SD}_{\mathrm{pop}}$, SD, and SE of each. (b) Compute the standard error of the difference of means treating the two as independent. (c) Compute the three per-seed differences, their mean, their SE, and the paired 95% CI. Why is the SE in (c) smaller than in (b)?

Solution sketch. (a) W8A8 mean 72.0, $\mathrm{SD}_{\mathrm{pop}} = 1.02$, SD = 1.25, SE 0.72; FP32 mean 68.0, $\mathrm{SD}_{\mathrm{pop}} = 0.57$, SD = 0.70, SE 0.40. (b) $\sqrt{0.72^2 + 0.40^2} = 0.83$. (c) Differences 3.5, 3.8, 4.7, mean 4.0, SD = 0.62, SE 0.36, less than half of (b). The two variants' seed scores correlate at 0.95, because a seed that suits one variant suits the other, and pairing cancels that shared part. With $t_2 = 4.30$ the paired 95% CI is 4.0 ± 1.6, which excludes zero.

Exercise I.30: Step-up and step-down

Use the ten cells of the worked example, with $m = 10$ and $q = \alpha = 0.05$. (a) The cell at 0.041 had instead been 0.024. Which cells pass BH, and which pass Holm? (b) Instead, the cell at 0.019 had been 0.022. Which pass BH now? (c) With $m = 2$, show that $p = (0.03, 0.04)$ rejects both hypotheses under BH although 0.03 exceeds its own bound.

Solution sketch. (a) The fifth bound is $5 \times 0.05/10 = 0.025 \geq 0.024$, so BH rejects all five; Holm still stops at rank 3 and rejects two. (b) Rank 4 fails ($0.022 > 0.020$), rank 5 fails ($0.041 > 0.025$), and rank 3 passes, so BH rejects three. (c) The bounds are 0.025 and 0.05; $p_{(2)} = 0.04 \leq 0.05$ gives $k^\star = 2$, and a step-up rule then rejects every hypothesis ranked at or below $k^\star$.

Exercise I.31: A stricter success filter

Take the thousand runs of Section 13.5, with true change zero and measurement noise of standard deviation 1 pp. (a) A report keeps the runs measured above 0 pp. What share does it keep, and what mean change does it report? (b) The same for the runs above +1 pp. (c) Which number answers whether quantization helps?

Solution sketch. For a standard normal truncated below at c, the share kept is $1 - \Phi(c)$ and the mean is $\varphi(c)/(1 - \Phi(c))$; the text's filter is $c = -1$. (a) 50% kept, mean $0.3989/0.5 = +0.80$ pp. (b) 16% kept, mean $0.2420/0.1587 \approx +1.5$ pp. The stricter the filter, the larger the gain it manufactures. (c) Only the mean over all runs, 0.

Check yourself before moving on

1. A table prints "±" over three seeds and says it is the population standard deviation. How do you get the standard error of the mean from it?[a]
2. Which variation does a paired, class-stratified crop bootstrap cover, and which does it not?[b]
3. Why can the same two models give a paired interval that excludes zero and an unpaired one that does not?[c]
4. BH rejects four cells of the worked example and Holm only two. What does each procedure control?[d]
5. Does $r = 0.96$ over 12 group means say how tightly one run's B follows its A?[e]

[a] By Equation (65), $\mathrm{SE} = \mathrm{SD}/\sqrt{n} = \mathrm{SD}_{\mathrm{pop}}/\sqrt{n-1}$, so divide by $\sqrt{2}$.

[b] A new draw of test crops with the trained models of every seed held fixed. Not retraining, so a significant interval is conditional on the seeds that were run.

[c] Pairing cancels the difficulty the two models share on each crop, so only the discordant crops vary, as in Figure 30.

[d] BH bounds the expected share of false discoveries among the rejected hypotheses; Holm bounds the probability of any false rejection, a stricter demand.

[e] No. It is an ecological correlation; the same runs correlate at only 0.36 one by one (Figure 31).

Where this is used. Sections 15 and 18 read the two papers' error bars, intervals, and tests with these tools, Sections 16 and 19 their correlations, filters, and bounds, and Section 20 their controls.

Part II

Reading the Two Papers

14 The question, and why the answer is surprising

What you will be able to do after this section

Read a paper-result box and the three claim tags; state the trade-off both papers test, prove that independent noise cannot raise the best achievable accuracy, and name the three premises the papers break; translate each paper's vocabulary; and plan a route through Part II.

Part II reads two papers with the tools of Part I, the ICML paper (Bouguerra et al., 2026a) on vision-language models (VLMs) of the CLIP family (contrastive language-image pre-training), and the accepted ICVES 2026 paper (Bouguerra et al., 2026b), the ICVES paper from here on, on a road-scene detector and recognition encoder. Both compare a model in 32-bit floating point (FP32) with the same model quantized to 8-bit integer (INT8) weights and activations (W8A8), by post-training quantization (PTQ, no gradient training) or quantization-aware training (QAT, training through the quantizers).

14.1 How Part II grades what the papers say

A gray paper-result box, titled with its locator such as [ICVES paper, Table V], reports what a paper says, with numbers as printed and quotations verbatim. The prose around it judges how far the evidence reaches, never passes off a plot reading or the notes' own arithmetic as the paper's, and tags a claim wherever the distinction matters.

established marks a claim shown by a cited result or proved on the page, such as the ICML paper's INT8 TensorRT engine of CLIP ViT-B/32, trained with learned step size quantization (LSQ), beating the pretrained FP32 model's zero-shot accuracy on clean CIFAR-100 by $+1.83$ percentage points (pp) over ten seeds, $p < 10^{-7}$ (Sec. 6, Table 2). The tag covers the measurement, not its cause.

suggestive marks consistent evidence without a control that isolates the cause. The ICVES paper measures that quantization and blur perturb the same low-variance feature directions, relative to the signal (Fig. 6); that the recognizer gains *because* of this overlap is suggestive, since sensor noise and contrast co-align comparably, "a necessary condition, not a fingerprint" (Sec. VII). open marks what is untested, such as the ICVES paper's leading candidate mechanism, clipping of activations at the range set by the learned step size (Sec. VII; limitation L5 in Sec. VIII).

14.2 The assumed trade-off

ICML paper, Abstract and Sec. 1; ICVES paper, Abstract and Sec. I

The ICML paper asks "How does quantization impact VLMs' reliability?" (Sec. 1, p. 2) and presents its findings as "contrary to the assumption that quantization's noise degrades performance" (Abstract). The ICVES paper states that quantization "is assumed to cost robustness to degraded camera inputs", a cost that for road-scene perception "has never been measured under matched training budgets" (Abstract, p. 1), and asks "whether quantization changes how quickly that accuracy falls as corruption grows" (Sec. I, p. 1).

The expectation has three good reasons behind it.

Two errors stack. A quantized encoder carries rounding error from every quantized layer before any corruption arrives, and blur adds a second perturbation. In the idealized model of Section 8, independent perturbations add their variances (Equation (37)), so the quantized model meets the blur with less margin.

Compression has hurt before. Compression concentrates its damage on rare examples (Hooker et al., 2019), and quantized ImageNet classifiers proved more vulnerable to natural corruptions than floating-point ones (Xiao, Liu, et al., 2023); the regularizer view (AskariHemmat et al., 2022; Javed et al., 2025) covers only weight-only quantization and accuracy or domain generalization (ICML Sec. 2).

Noise cannot create information. The third reason is a theorem, worth stating exactly because the places where its premises fail are the places Part II searches.

Proposition 14.1: Independent noise cannot raise the best achievable accuracy

Let $\boldsymbol{z}$ be a feature vector with label y, drawn from any joint distribution (of clean or of blurred inputs alike), and let $\boldsymbol{n}$ be a random vector independent of $(\boldsymbol{z}, y)$. For every decision rule g applied to $\boldsymbol{z}+\boldsymbol{n}$ there is a rule h applied to $\boldsymbol{z}$, using random draws of its own, with the same accuracy. Hence the best accuracy achievable from $\boldsymbol{z}+\boldsymbol{n}$ is at most the best accuracy achievable from $\boldsymbol{z}$.

Proof. Let h draw $\boldsymbol{n}'$ from the distribution of $\boldsymbol{n}$, independently of everything else, and output $g(\boldsymbol{z}+\boldsymbol{n}')$. The pair $(\boldsymbol{z}+\boldsymbol{n}', y)$ has the same joint distribution as $(\boldsymbol{z}+\boldsymbol{n}, y)$, so $P(h(\boldsymbol{z})=y) = P(g(\boldsymbol{z}+\boldsymbol{n})=y)$. The accuracy of h is an average, over $\boldsymbol{n}'$, of the accuracies of the deterministic rules $\boldsymbol{z} \mapsto g(\boldsymbol{z}+\boldsymbol{n}')$, so it cannot exceed the best deterministic rule on $\boldsymbol{z}$. □

The proposition is proved on the page established. If the quantized features were the FP32 features plus independent noise, as in the flat floor of Section 8, and each model used the best rule for the data it is scored on, the quantized model could never win, clean or blurred.

What the naive picture predicts, in numbers

Along one feature direction let bus crops sit at $+1$ and truck crops at -1, in equal numbers, with within-class variance $\tau^2 = 0.16$. Blur adds a variance β, and the quantized encoder an independent residual of variance 0.09 (all synthetic). For every β the best rule is "bus if positive", with accuracy $\Phi(1/\sqrt{v})$ for total within-class variance v and Φ the standard normal distribution function. Clean, FP32 scores $\Phi(1/\sqrt{0.16}) = \Phi(2.5) = 99.4\%$ and the quantized encoder $\Phi(1/\sqrt{0.25}) = \Phi(2) = 97.7\%$, so $\Delta\text{Acc}(0) = -1.7$ pp (Equation (59)). Across the sweep, with differences from unrounded values,

blur variance β	0	0.25	0.5	1	2	4
FP32 accuracy (%)	99.4	94.1	89.1	82.3	75.2	68.8
quantized accuracy (%)	97.7	92.1	87.6	81.4	74.8	68.6
ΔAcc (pp)	−1.7	−1.9	−1.5	−0.9	−0.4	−0.2

The difference is negative everywhere, as Proposition 14.1 requires, and fades toward chance. The ICVES Gaussian ΔAcc is instead -0.2 clean, $+4.1$ at $\sigma = 6$, and -6.0 at $\sigma = 14$ (Table VIII), a positive window and a growing reversal. The toy's β has no calibrated relation to σ, so compare shapes only.

Where the argument leaks. A measured gain means that a premise of the proposition fails in the experiment, and three fail in both papers.

1. *Garbling.* The proof covers any quantized features computable from the FP32 features $\boldsymbol{z}$ plus independent randomness, so even error that depends on $\boldsymbol{z}$, such as a dead zone or clipping applied to $\boldsymbol{z}$ itself, cannot help. Rounding instead acts on every quantized layer's activations, which carry information $\boldsymbol{z}$ may have discarded, with error tied to that signal (Section 4; Proposition 8.2). The quantized features are another function of the image, not a garbled copy, and the proposition does not order the two.

2. *Optimality.* Neither model is the best rule for the data it is scored on. The ICVES models see blur only at evaluation (Sec. III), the ICML models are scored zero-shot (Sec. 3.2), and Exercise II.2 shows a quantized model winning under blur below the bound.

3. *Sameness.* The quantized model is trained around its own perturbation, the weights in ICML QAT against an FP32 model given no such training (Sec. 3.2; App. C.2), the step sizes and head in the ICVES encoder (Sec. III; Section 6).

That the premises fail is established; which failure produces the gains is not settled. The ICVES matched-noise variant keeps the second and third leaks but replaces rounding and clipping with independent noise sized to each quantizer's calibrated, not learned, step. It stays flat to negative where QAT gains (Table VII), which points away from noise magnitude alone suggestive (Exercise II.1); what the quantizer adds beyond it is open.

Resolving an ambiguity about "noise" in the two papers

The ICML paper calls the residual energy "Quantization Noise Power" (App. D.1, Eq. (6)) and a pixel corruption "Gaussian Noise" (App. C.4), and credits quantization noise with "stochastic regularization" (Sec. 4.2) while calling its grid "deterministic" (Sec. 5). The ICVES paper uses the word for its residual (Fig. 6), for sensor noise (Fig. 6C), and for its matched-noise control (Sec. VII). Only the control is noise in the sense of Proposition 14.1.

Running example: the bus in the rain

Of 100 bus crops of the worked example at $\beta = 1$, a numerical integration of the joint Gaussian gives about 79.0 that both encoders call bus, 15.2 that both call truck, 3.3 that only FP32 gets right, and 2.4 that only the quantized encoder gets right, the net -0.9 of the table. The ICVES paper reports the other balance, without counts. In the six crops of its Fig. 5 the W8A8 encoder corrects a readable object that the FP32 control misreads, in all three seed pairs, and across the test set such corrections outnumber the reverse errors under heavy Gaussian blur but not under defocus (Sec. V). Part II asks which failed premise turns 2.4 against 3.3 around.

14.3 What reliability means in each paper

The ICML paper spreads reliability over five dimensions (Sec. 1, p. 2), not the "three dimensions" of its Sec. 3.2 (Section 15), so one run can gain calibration and lose accuracy, as 31.0% of the 6720 significant W8A8 runs do, while 54.8% lose both (Fig. 1). The ICVES paper asks how fast accuracy falls as blur grows, for the recognizer in balanced accuracy (Equation (57)) since rare classes "carry the safety case". Its confidence-thresholded reliability is slightly worse under quantization (Sec. V), a sign it contrasts with ICML calibration gains that come from another regime (Section 16). Equation (60) converts the drops of ICML Table 3 into the ICVES ΔAcc.

14.4 Two papers, two scales, one claim

Two scales. The ICML paper measures broadly, about 708,480 evaluation runs over ten discriminative CLIP-family models and 16 methods (Section 15), buying a distribution of outcomes but no matched control. The ICVES paper measures one DAWN pipeline tightly, each variant differing from its FP32 control only in quantization (Section 18), buying attribution, not generality.

One claim. Both papers argue that quantization need not cost reliability and can add robustness to some corruptions. For the ICML paper, which also scores out-of-distribution (OOD)

Table 8: What the same words mean in the two papers, with locators into the paper heading each column (designs in full in Sections 15 and 18). The residual row limits how far the spectral figures of the two papers can be compared.

	ICML paper (Bouguerra et al., 2026a)	ICVES paper (Bouguerra et al., 2026b)
Quantizer	W8A8, W6A8, W4A8; all linear and convolution layers but each encoder's last projection, granularity not stated; simulated, plus one TensorRT INT8 engine (Sec. 3.1, 6; App. C)	W8A8 only; 48 block linear layers (encoder), 37 linear and 69 convolution layers (detector); per-channel weights, per-tensor activations; LSQ named, no formula printed; simulated, then true-INT8 kernels (Sec. II, III, IV.B, VI)
Calibration data	1000 image–caption pairs of CC3M, YFCC, or SBU (Sec. 3.1)	64 clean images or 256 clean crops (Sec. III)
PTQ	8 methods, from MinMax to rotation (Sec. 3.1)	the QAT quantizers and calibration without training (Sec. III, VII)
QAT and what it trains	8 variants; weights (AdamW, 10^{-6}) and steps (10^{-4}) by distillation on the proxy pairs (Sec. 3.1; App. C.2)	LSQ with the backbone frozen; step sizes and heads only, on clean data, 600 steps (detector) or 400 (encoder) (Sec. III)
Reference model	the pretrained FP32 model f; none given the same proxy training (Sec. 3.2)	an FP32 control with the same continuation steps from a shared start (Sec. III)
Residual	$X - X_Q$, penultimate visual features (App. D.1, Eq. (6)); for QAT runs it mixes rounding with weight drift, while PTQ runs retrain no weights	$R_q = X_{\mathrm{W8A8}} - X_{\mathrm{fp}}$, pooled features of frozen weights, a pure quantization residual (Sec. VII)
Robustness	relative change δ, Eq. (1), positive when q is worse, under natural shift and CIFAR-10-C severity 3; drops in pp for the engines (Table 3)	$\Delta\mathrm{Acc}(\sigma)$ in pp of balanced accuracy on corrupted inputs (Sec. I, III)
Evidence	runs filtered as "significant" (Fig. 1) or "successful" (Fig. 7)	three seeds, paired bootstrap intervals (Sec. III, V)

detection, it "can simultaneously improve accuracy, calibration, OOD detection, and robustness to noise, though not to covariate shift or spurious correlations" (Abstract); for the ICVES paper recognition-stage quantization is "a possible source of corruption robustness rather than a cost" (Sec. IX). The ICVES paper cites the ICML paper as [20] for "the spectral-filtering view" (Sec. II), though the "selective prediction" it also credits occurs nowhere in the ICML text. Read with the tags, both claims are bounded. The effect is real in places and absent or negative in others established, its cause is suggestive at best, and its transfer to other tasks and models is open; Table 20 sorts the claims one by one.

14.5 A reading guide to Part II

Part II reads each paper in three steps, design, results, and mechanism, for the ICML paper in Sections 15 to 17 and for the ICVES paper in Sections 18 to 20. Section 21 then asks what the two accounts share and what stays open. Part 5 of the hands-on notebook, a one-seed reimplementation of the ICVES recognizer protocol, belongs after Section 19.

Exercises

Exercise II.1: Grading claims with the three tags

Tag each statement about the ICVES paper as established, suggestive, or open, and name the evidence that would move it up. (a) W8A8 quantization does not change the detector's robustness to blur. (b) Noise of the right size cannot produce the recognizer's gain.

Solution sketch. (a) Not established as worded. What is established is a bound, all variants within 1.7 points of mean average precision (mAP) over three seeds (Table I), with no equivalence test (Sec. IV.A);

a test against a stated margin, with more seeds, would support the wording. (b) The control as run is established, -1.2 and -0.9 pp at $\sigma = 4$ and 6 where QAT gains $+2.5$ and $+4.1$ (Table VII), and the paper calls the noise alternative "tested and excluded" (L5). The general statement is suggestive, since the control is a three-seed point estimate matched to the calibrated, not the learned, step; a control matched to the learned step, with more seeds, would upgrade it.

Exercise II.2: A loophole in numbers

Two independent features separate bus from truck, a coarse one with class means ± 1 and within-class variance 1, and a fine one with class means ± 0.2 and within-class variance 0.01 (synthetic). For class means $\pm(m_1, m_2)$ and variances (τ_1^2, τ_2^2) the rule $\mathrm{sign}(w_1 z_1 + w_2 z_2)$ has accuracy $\Phi\big((w_1 m_1 + w_2 m_2)/\sqrt{w_1^2\tau_1^2 + w_2^2\tau_2^2}\big)$, best at $w_i = m_i/\tau_i^2$. (a) Find the best clean rule $\boldsymbol{w}_{\mathrm{FP}}$, and $\boldsymbol{w}_Q$ for a quantized encoder that adds independent noise of variance 0.01 to each feature and trains on its own clean features, with both clean accuracies. (b) Heavy blur erases the fine cue (class means 0, within-class variance 0.03). Score both rules on blurred inputs, the quantized one with its noise. (c) Why does (b) not contradict Proposition 14.1?

Solution sketch. (a) $\boldsymbol{w}_{\mathrm{FP}} = (1, 20)$ and $\boldsymbol{w}_Q = (1/1.01,\ 0.2/0.02) = (0.990, 10)$, a shrunken readout. Clean FP32 has argument $5/\sqrt{5} = 2.236$, accuracy 98.7%; the quantized model has $\sqrt{0.990 + 2} = 1.729$, accuracy 95.8%, so $\Delta\mathrm{Acc}(0) = -2.9$. (b) FP32 gives $1/\sqrt{1 + 400 \times 0.03} = 1/\sqrt{13} = 0.277$, accuracy 60.9%; the quantized model gives $0.990/\sqrt{0.990 + 100 \times 0.04} = 0.443$, accuracy 67.1%, so $\Delta\mathrm{Acc} = +6.2$. The gain is the retrained readout's, since the FP32 rule on the noisy features scores $\Phi(0.242) = 59.6\%$ and the quantized rule without its noise $\Phi(0.496) = 69.0\%$. (c) The best rule for blurred features drops the fine cue and scores $\Phi(1) = 84.1\%$, above both. The proposition bounds best rules, and a rule trained on clean data is not one. The ICVES matched-noise variant resembles this toy (whether its noise stays on at test time is not stated) but did not gain in the window.

Check yourself before moving on

1. Which three premises must all hold for Proposition 14.1 to forbid a quantized gain?[a]
2. The ICML engine's $+1.83$ pp over FP32 is tagged established. What does the tag establish, and what does it leave open?[b]
3. Why is the ICML residual of a QAT run not a pure quantization residual, while the ICVES R_q is?[c]

[a] That the quantized features be computable from the FP32 features plus independent randomness, that the compared rules be optimal for the data they are scored on, and that the two models be the same apart from that randomness.

[b] That this engine beats the pretrained FP32 model as printed (Table 2), not that rounding causes it, since the FP32 reference was never trained with LSQ.

[c] ICML QAT retrains the weights (App. C.2), so for those runs $X - X_Q$ mixes rounding error with weight drift. ICVES freezes the backbone, so R_q compares two computations on the same stored weights, rounded or not.

15 The ICML paper, design and metrics

What you will be able to do after this section

Rebuild the ICML grid and its 708,480 runs, including the line that does not add up, and say why they rest on ten pretrained models. Name the metric and Part I equation behind each reliability dimension. Convert results between the sign conventions of both papers. Say what the "significant", "successful", and "failure" filters select, and what the paper leaves unstated.

The ICML paper asks "How does quantization impact VLMs' reliability?" (Bouguerra et al., 2026a, Sec. 1), where VLM stands for vision-language model, and answers by breadth. It crosses ten models, sixteen quantization methods, three bit widths, two scopes, three proxy data sets, and three seeds, and scores every checkpoint 82 times. A pattern that survives across ten models and sixteen methods is not an accident of one configuration, which is the design's strength. The accepted ICVES 2026 paper (Bouguerra et al., 2026b) measures one road-scene pipeline tightly instead (Section 18); the ICML results follow in Sections 16 and 17.

15.1 The grid

Models and scopes. The ten models are contrastive, non-generative VLMs of the CLIP family (Radford et al., 2021), whose families Section 10 describes. Fig. 4 groups them as curated (ViT-L/14 and ViT-B/32 on WIT-400M), filtered or distilled (ALIGN-1.8B, SigLIP-WebLI-10B, ViT-B/32-DFN-2B), and noisy web scale (ConvNeXt on LAION-400M; ViT-L/14, ViT-B/32, EVA-02, and CoCa on LAION-2B). The grouping is not consistent, since Sec. 4.1 puts SigLIP among models trained on "massive but uncurated, noisy datasets" and the ALIGN checkpoint carries a different label in each of Figs. 2, 4, and 7. CoCa counts as non-generative (Sec. 1) though Sec. 4.4 calls it "generative-based"; generative models such as LLaVA are out of scope (footnote 1). The *visual-only* scope keeps the text encoder in FP32, and the *joint visual-text* scope quantizes both encoders alike (Sec. 3.1).

Methods. All sixteen are "simulated" (Sec. 3.1), fake quantization in floating point (Section 2), since GPUs lack INT6 and INT4 arithmetic (App. G.3); a check of at most 256 distinct values per INT8 tensor confirms the grid, not what a deployed kernel computes. Eight are post-training quantization (PTQ) methods, Simple MinMax (Jacob et al., 2018), SmoothQuant (Xiao, Lin, et al., 2023), IGQ-ViT (Moon et al., 2024), QwT (Fu et al., 2025), APQ-ViT (Ding et al., 2022), QuaRot rotation (Ashkboos et al., 2024), Outlier Suppression (Wei, Zhang, et al., 2022), and Q-VLM (Wang, Wang, et al., 2024). Simple MinMax has no printed formula, and its cited scheme uses asymmetric ranges, not the symmetric max-abs rule of Section 6.1; the rotations are random orthogonal matrices (App. F), not Hadamard ones. Eight are quantization-aware training (QAT) variants (Fig. 18, right), learned step size quantization, LSQ (Esser et al., 2020), QAT with the straight-through estimator (Courbariaux et al., 2015), and Rotation+LSQ, each in a *Contrastive-Only* and a *Hybrid* regime ("Contrastive + Feature MSE", a feature distance, not the quantizer error), plus QAT-LoRA (Xu et al., 2024) and Q-ViT (Li et al., 2022). They train the weights as well as the steps (App. C.2), unlike the ICVES QAT (Section 6), though App. F reads otherwise for LSQ (Section 15.5).

Bits, data, and the reference. The precisions are W8A8, W6A8, and W4A8 (App. C.5). Only the weight width varies, so the grid is silent on the ICVES prediction that coarser *activation* quantization widens the severity window (Bouguerra et al., 2026b, Sec. VII, L2 in Sec. VIII). "All quantizations are performed on 1000 unique image-caption pairs" of CC3M (Sharma et al., 2018), YFCC (Thomee et al., 2016), or SBU (Ordonez et al., 2011) (Sec. 3.1), a budget chosen by a scaling study from 1 to 10,000 samples (App. G.2). Although Sec. 2 calls PTQ "data-free", these pairs calibrate the PTQ ranges, train the QAT variants (App. C.2), and fit the *logit tuning*, one refitted logit scale acting as the temperature of Equation (62) (Secs. 3.1, 4.2); Sec. 3.1 calls both steps "calibration". All linear and convolution layers are quantized except "the absolute last projection layers of each encoder" (App. C). One configuration, CLIP ViT-B/32 (WIT) with LSQ, also ran as a real TensorRT INT8 engine with explicit quantize and dequantize nodes (Sec. 6; App. G.5). Every metric compares a quantized checkpoint q with the pretrained FP32 model f (Sec. 3.2), and the paper describes no FP32 model given the same proxy training or logit tuning, so a QAT or tuned difference also contains that extra step, which

Table 9: The ICML grid as App. C.5 prints it (Bouguerra et al., 2026a, App. C.5), factors (top) and evaluations per checkpoint (bottom). OOD is out-of-distribution, ID in-distribution; the last column names the section that builds each item. The spectral line is printed as 4 although its factors give 3.

Factor	Levels	Count	Built in
Model	ten CLIP-family models (Sec. 3.1; Fig. 4)	10	Section 10
Precision	W8A8, W6A8, W4A8	3	Section 2
Scope	visual-only, joint visual-text (Sec. 3.1)	2	Section 10
Seed	three seeds	3	Section 13
Proxy data	1000 image–caption pairs of CC3M, YFCC, or SBU	3	Section 6
Method	8 PTQ and 8 QAT variants (Sec. 3.1)	16	Section 6
Checkpoints, $10 \cdot 3 \cdot 2 \cdot 3 \cdot 3 \cdot 16$		8,640	
Evaluation	Composition as printed	Count	
Zero-shot accuracy	7 data sets $\times 2$ (logit tuning)	14	Section 12
OOD detection	9 ID–OOD pairs $\times 6$ scoring functions	54	Section 12
Robustness	4 natural $+ 4$ synthetic $+ 2$ baselines $- 2$ (ViT-L-14)	8	Section 12
Spurious	2 spurious splits	2	Section 12
Spectral	1 data set $\times 10$ rank intervals $\times 0.3$	4	Section 17
Evaluation runs, $8{,}640 \times (14 + 54 + 8 + 2 + 4)$		$\approx$ 708,480	

the ICVES paper controls (Section 14).

A *checkpoint* is one cell of the six-factor product in Table 9, and a *run* is one evaluation of it in one setting of App. C.5 (a data set in one logit-tuning state, an ID–OOD pair under one scoring function, and so on). "8k+ quantized models and 700k+ evaluation runs" (Sec. 1) thus means 8,640 checkpoints scored 82 times each, for about 12,000 GPU-hours on an Nvidia H200 cluster (App. C.5), some 1.4 per checkpoint.

Worked example, the grid by hand

By the notes' own arithmetic, $10{\cdot}3{\cdot}2{\cdot}3{\cdot}3{\cdot}16 = 8{,}640$ checkpoints times $14{+}54{+}8{+}2{+}4 = 82$ evaluations give 708,480 runs. The spectral line multiplies to $1 \times 10 \times 0.3 = 3$, not the printed 4, and with 3 the total is $8{,}640 \times 81 = 699{,}840$, just under 700,000. The robustness line subtracts 2 "(ViT-L-14)" unexplained (Fig. 7 lists no ViT-L/14 model). The OOD line gives $8{,}640 \times 54 = 466{,}560$, the count printed on Fig. 20B and two thirds of all runs.

Now count what is independent. Every run descends from one of 10 pretrained models, and a cell that fixes model, bit width, scope, method, and proxy set holds 3 checkpoints (one per seed) and $3 \times 82 = 246$ runs. The two logit-tuning copies of a zero-shot evaluation rescale all logits by one positive scalar, which cannot change a top-1 class (Equation (62)), so they can differ in calibration only. A claim about models, such as "curated data makes a model easier to quantize", rests on 10 points however many runs sit behind each (Section 13).

15.2 Five dimensions and their metrics

Each of the five reliability dimensions of Sec. 1 rests on a metric built in Part I (Table 10), and the one numbered equation of the main body, Eq. (1), is Equation (58). "Detection" always means OOD detection here (Secs. 3.2, 4.3).

The zero-shot sets (App. C.3; Fig. 25) are CIFAR-10 and CIFAR-100 (Krizhevsky, 2009), ImageNet-1k (Deng et al., 2009) with 10-, 20-, and 500-class subsets, and SUN397 (Xiao et al., 2010). The OOD pairs include near-OOD swaps such as CIFAR-10 against CIFAR-100 and far-OOD pairs of ImageNet-1k after the MOS protocol of Huang and Li (2021) (App. C.3).

Table 10: The five reliability dimensions of ICML Sec. 1, the metric behind each, and the equation of these notes that defines it. Locators refer to Bouguerra et al. (2026a).

Dimension	Metric and locator	Defined in
(1) Robustness to quantization noise	top-1 zero-shot accuracy $A(m, \mathcal{D})$ on seven sets and its relative change, Eq. (1) (Sec. 3.2; App. C.5)	Equation (58)
(2) Calibration	expected calibration error (ECE), bin count unstated (Sec. 3.2); Fig. 5	Equation (61)
(3) OOD detection	area under the ROC curve (AUROC) and FPR95 (FPR@95 in Part I) with six scoring functions on nine ID–OOD pairs (App. C.5); δ_{OOD}, the relative AUROC change in the form of Eq. (1) (Sec. 3.2)	Equation (63)
(4) Distribution shift	$\delta(\mathcal{D})$ on each shifted set (Sec. 3.2); Fig. 7's "Relative Gap Change" has no formula	Equation (58)
(5) Spurious correlations	ΔRSG and Vuln_{add} on CounterAnimal (Sec. 3.2)	Equation (64)

Table 11: Signs and units in the two papers. Read "positive means" before quoting any relative change; locators refer to Bouguerra et al. (2026a) and Bouguerra et al. (2026b).

Where	Quantity and unit	Positive means
ICML paper (Bouguerra et al., 2026a)		
Eq. (1), Sec. 3.2	$\delta(\mathcal{D})$ and δ_{OOD}, a fraction	worse
Figs. 1, 3, 15–18	"Relative Accuracy Change (%)", presumably $-100\,\delta$ (not printed)	better
Fig. 2	"Relative Degradation (%)", drawn as negative numbers	never positive; lower is worse
Figs. 1 and 4	"Relative ECE Change (%)", formula not printed	worse (ECE is an error)
Fig. 19, left	"Relative Improvement in ECE vs FP32 (%)", boxes printed "Mean ECE Δ"	better, so the printed -36.9% is a worsening
Fig. 7	"Relative Gap Change (%)", left panel on a reversed axis	more robust (Sec. 4.4)
Table 1	ΔRSG and Vuln_{add}, differences of relative quantities, printed in "%"	more spurious reliance
App. D, Eq. (11)	relative subspace accuracy change	better
App. D, Eq. (6)	residual $X - X_Q$ (the notes write $X_Q - X$)	sign-free for energies
Tables 2 and 3	accuracies and drops in points, and "Added rob."	INT8 loses less than FP16
ICVES paper (Bouguerra et al., 2026b)		
Tables III and V	Δ, quantized minus FP32, in mAP or balanced-accuracy points (ΔAcc)	better
Sec. VII	residual $X_{\text{W8A8}} - X_{\text{fp}}$, the notes' sign	sign-free for energies

Natural shift uses ImageNet-A (Hendrycks, Zhao, et al., 2021), -R (Hendrycks, Basart, et al., 2021), -V2 (Recht et al., 2019), and -Sketch (Wang et al., 2019), and CIFAR-10-C (Hendrycks & Dietterich, 2019) enters at severity 3 of 5 with Gaussian noise of standard deviation $\nu = 0.08$ (App. C.4 writes σ, which these notes keep for blur), disk defocus blur, and brightness and contrast factors of 1.5 (Fig. 10 shows CIFAR-100-C instead, unreconciled). Spurious reliance uses CounterAnimal (Wang, Lin, et al., 2024).

15.3 Signs and units

Eq. (1) is positive for a loss, but most figures draw improvement upward, so read the axis label first. Table 11 is the notes' single reference for the signs of both papers.

Running example: the bus in the rain

Suppose the bus-crop recognizer were a CLIP-family encoder scored zero-shot against the prompts car, truck, bus, and person, in the visual-only scope, with synthetic numbers. On

clean crops FP32 scores 80.0% with ECE = 0.070, and seed 1 scores 78.6% with ECE = 0.049, so $\delta = (80.0 - 78.6)/80.0 = 1.75\%$. In Fig. 1 it sits at a relative accuracy change of -1.75% and a relative ECE change of $(0.049 - 0.070)/0.070 = -30\%$ (a formula the paper does not print), worse accuracy with better calibration. Seed 2 scores 78.2%, $\delta = 2.25\%$, so of two seeds 0.4 points apart only seed 1 is "successful".

In rain FP32 scores 60.0% and seed 1 61.2%, so $\delta = -2.0\%$ while the ICVES currency gives $\Delta\text{Acc} = +1.2$ points, the same gain with opposite signs; by Equation (60) seed 1 also loses $1.2 - (-1.4) = 2.6$ points less to rain than FP32.

15.4 Which runs are counted

Three filters recur, each selecting runs by an outcome. Fig. 1 plots "6720 significant W8A8 quantized evaluation runs" without naming the test. A *successful* run has "less than 2% degradation on the non-shifted dataset" (Fig. 7 caption; "$\leq 2\%$" in App. G), a word Figs. 4 and 6 use without the threshold, and a *failure* is "a $> 5\%$ relative accuracy drop" (Table 4 caption; App. F). A statistic over successful runs answers "what does quantization do when it has not cost accuracy?", not "what does quantization do?", and the two differ because accuracy moves with the other metrics.

ICML paper, App. G and Fig. 20B, 20D

The separability gap, mean ID minus mean OOD confidence score, is 1.29 at full precision. Fig. 20B (no constraint, all bit widths, 466,560 runs) prints a quantized gap of 1.02; Fig. 20D (accuracy drop at most 2.0%, W8A8 only, 60,738 runs) prints 1.53, which App. G calls a *widening* of the gap in successful W8A8 quantization.

Both panels describe their subsets correctly established, but they differ in bit widths and in the filter at once, so their contrast isolates neither. The filter keeps 60,738 of the $2{,}880 \times 54 = 155{,}520$ W8A8 OOD runs, about 39% (the notes' arithmetic, assuming both panels count runs alike), while the unfiltered panel shows the gap narrowing on average. Quote such figures as "among W8A8 runs that lost less than 2% accuracy". No correction for the many tests is mentioned (Section 13), and Exercise II.3 shows a filter manufacturing a gain.

15.5 What the paper leaves unstated

Choices a replication would still have to make, listed neutrally

- *Quantizer.* Each method's cited paper fixes its rule (Sec. 3.1), such as minimum and maximum statistics for Simple MinMax or a learned step for LSQ, and App. G.6 prints the affine form with a zero point and the INT8 range $[-128, 127]$ (Equation (15)). Unstated are the granularity, symmetric or affine form, which methods fix $z = 0$, the LSQ step initialization, and signed, static, or dynamic activation ranges.
- *Training.* QAT length, batch size, schedule, Hybrid loss weights, and teacher; App. C.2 gives a weight learning rate, while App. F says LSQ updates the scaling factors "rather than the weights themselves".
- *Statistics.* The test behind "significant" and the p-values of Figs. 4 and 6, the error type of Fig. 7, the $\pm$ of Table 1, and why seeds number three in the grid but ten in Tables 1 to 3.

App. A fixes the scope, with no fine-tuned models, vision foundation models, or generative VLMs, so the ICVES DINOv2 encoder lies outside the grid, and the transfer to generative

models with a CLIP-family vision branch, which App. A expects, is measured by neither paper open. The link between pretraining data and quantizability (Sec. 4.1) is suggestive suggestive, since the compared checkpoints differ in more than their data (Exercise II.4).

Exercises

Exercise II.3: A filter that manufactures a gain

Synthetic W8A8 runs are of two kinds. In 60% quantization is benign, with a clean relative drop $\delta = 1\%$ and a synthetic-robustness change of $+10$ points; in 40% it is damaging, with $\delta = 8\%$ and a change of -15. Compute the mean change over all runs and over the successful runs ($\delta < 2\%$). What should a report state next to the second number?

Solution sketch. All runs, $0.6 \cdot 10 + 0.4 \cdot (-15) = 0$; successful runs, $+10$, because the filter keeps only the benign kind. Quantization has no average effect, yet the filtered mean reports a 10-point gain established. State the filter, the share it keeps (60%), and the unfiltered mean.

Exercise II.4: What the grid can and cannot test

Can the ICML grid test each claim, and by comparing which cells? (a) Quantizing the text encoder hurts probability calibration. (b) Coarser activation quantization widens a severity window. (c) Curated pretraining data makes a model easier to quantize. (d) Rotation before LSQ lowers the failure rate.

Solution sketch. (a) Yes, visual-only against joint cells with the rest fixed. (b) No, activations are 8-bit throughout and corruptions are scored at severity 3 only (App. C.4). (c) Only partly. ViT-B/32 on WIT, LAION-2B, and DFN-2B fixes the architecture but not the data size (400M, 2B, 2B in the Fig. 4 labels) or, presumably, the training recipe, and App. A rules out a fully crossed design. (d) Yes, Rotation+LSQ against LSQ in one regime, with the 5% threshold as a stated choice.

Check yourself before moving on

1. A run has $\delta = +3\%$ by Eq. (1). Where does it sit on the vertical axis of Fig. 1?[a]
2. What does "successful W8A8" select, and what can a statistic over those runs not tell you?[b]
3. Name three choices a replication must make that the paper does not report.[c]

[a] At -3%, below the origin, since Fig. 1 draws improvement upward.
[b] Runs that lost less than 2% on the unshifted data. Not the average effect of quantization, since the selecting outcome moves with the other metrics (App. G; Fig. 20D).
[c] Any three of granularity, symmetric or affine form, LSQ step initialization, activation ranges, QAT schedule, Hybrid loss weights, and the significance test.

16 The ICML paper, results axis by axis

What you will be able to do after this section

Read ICML Fig. 1 as a two-by-two table, report all four quadrants, and compute from the printed counts how often the two improvements occur together. For each reliability dimension, state the direction of the effect, its number and locator, its population (bit width and filter), and its evidential tier. Tell which PTQ failure rate belongs to which population, split the deployment speed-up into the part owed to the engine and 16-bit arithmetic and the part owed to going from 16 to 8 bits, and say where the paper's text

and tables disagree.

This section reads the results of the ICML paper (Bouguerra et al., 2026a) one reliability dimension at a time (Secs. 4.1 to 4.5), then the method axis (App. D.4, F) and the deployed engine (Sec. 6). For every number, read the axis label before the sign (Table 11) and name the population (Section 15), since many figures pool bit widths or keep only runs that held their accuracy. A tier here grades the measured effect, not the paper's explanation of it, which Section 17 weighs.

16.1 All four quadrants of Figure 1

ICML Fig. 1 places each quantized evaluation run relative to its FP32 baseline, with the relative change in expected calibration error (ECE, Equation (61)) horizontally and in top-1 zero-shot accuracy vertically, on symmetric-log scales (logarithmic away from zero, linear near it). Improvement lies up and to the left, since ECE is an error. The figure prints the share and count of each quadrant (Table 12).

Table 12: All four quadrants of ICML Fig. 1, the 6720 "significant" W8A8 evaluation runs (Bouguerra et al., 2026a, Fig. 1, p. 1). Names, shares, and counts as printed; the shares sum to 99.9% by rounding. From the counts, the bottom block's shares are 38.9% and 14.2%, and at least one metric improves in $3038/6720 = 45.2\%$ (the notes' arithmetic). The caption's "nearly 40%" with improved calibration is rows one and three.

Quadrant (as printed)	Position	Accuracy	Calibration	Share	N
Improvement	top left	better	better	7.8%	527
Compromise (worse calibration)	top right	better	worse	6.3%	425
Compromise (worse accuracy)	bottom left	worse	better	31.0%	2086
Simultaneous degradation	bottom right	worse	worse	54.8%	3682
Better calibration, rows one and three				38.8%	2613
Better accuracy, rows one and two				14.1%	952

The caption's headline, that quantization "is not systematically destructive", holds. About one run in thirteen improves both metrics, and more than a third improve calibration established. Yet the largest quadrant, 54.8%, degrades both, and accuracy falls in $31.0 + 54.8 = 85.8\%$ of the runs. The test behind "significant" is not named (Section 15), so every share describes a filtered set, not all W8A8 runs.

Worked example, do the two improvements travel together?

Calibration improved in $527+2086 = 2613$ runs and worsened in $425+3682 = 4107$. Among the first, accuracy also improved in $527/2613 = 20.2\%$; among the second, in $425/4107 = 10.3\%$. A calibration gain thus comes with an accuracy gain about twice (1.95 times) as often as a calibration loss does. Had the two outcomes been unrelated, the top-left cell would hold the accuracy share times the calibration share times the total, $(952/6720)(2613/6720) \cdot 6720 = 370$ runs, not 527. This is the notes' arithmetic on these 6720 runs only; runs sharing a model or a checkpoint are not independent, and an undefined filter chose them, so no textbook test of association applies.

16.2 Accuracy, with redundancy and fidelity

ICML paper, Sec. 4.1, Figs. 2 and 3, p. 4; Fig. 16, p. 21

Across the ten models, the relative accuracy loss under PTQ, each point a mean over the W8, W6, and W4 PTQ runs, correlates with the loss under random weight dropout (1%, 1.5%, 2.5%), $r = 0.91$ ($p = 2.4 \times 10^{-4}$, Fig. 2). For embedding change against accuracy change, Fig. 3 prints $r = 0.32$ for PTQ ($p = 6.4 \times 10^{-96}$), $r = -0.03$ for QAT ($p = 0.028$), and $r = -0.32$ for the QAT runs that beat FP32 ($p = 5.5 \times 10^{-28}$). Per model, over $N = 1008$ runs per cell at a bit width the caption does not state, the mean relative accuracy change is negative for all ten models under both PTQ and QAT, and the loss is smaller for QAT in every model; ConvNeXt under PTQ has a mean of -19.1% and a median of -8.8% (Fig. 16).

Fig. 2 correlates ten model-level points established; that dropout and PTQ both probe spare capacity, the authors' "redundancy", is suggestive suggestive. The QAT $r = -0.03$ is significant only because the sample is large, about 5400 points by its p-value (the notes' arithmetic; Section 13). QAT runs that beat FP32 depart further from the FP32 embedding the more they gain, which the authors read as "shedding brittle features", an interpretation of an outcome-chosen subset suggestive; a QAT residual here thus mixes rounding error with weight drift, unlike the frozen-backbone residual of the ICVES paper (Table 8). "Less precise can be more accurate" (Sec. 4.1) holds for the 14.1% of Fig. 1 runs above the axis, not on average, where every model loses (Fig. 16) established and a mean of -19.1% against a median of -8.8% signals a long tail. Part of any QAT gain may be the proxy-data training itself (Section 14).

16.3 Calibration and a one-parameter repair

ICML paper, Sec. 4.2, Figs. 4 and 5, p. 5

Over successful W8A8 runs of all methods, the relative ECE change (negative is better) is $+9\%$ for visual-only quantization ($N = 5543$, $p = 1 \times 10^{-50}$) and $+98\%$ with the text encoder also quantized ($N = 3893$, $p = 1 \times 10^{-1}$) (Fig. 4A); for Rotation+LSQ alone it is -3% ($N = 1244$, $p = 2 \times 10^{-48}$) and -5% ($N = 1169$, $p = 4 \times 10^{-69}$) (Fig. 4B). The caption does not say whether each value is a mean or a median. Averaged over ImageNet, CIFAR-10, CIFAR-100, and SUN397, the ECE moves from FP32 to QAT to QAT with logit-scale tuning as $5.8 \to 2.2 \to 2.1\%$ (WIT) and $6.9 \to 3.1 \to 1.1\%$ (LAION) with the visual encoder quantized (Fig. 5; with both encoders quantized, LAION ends at 1.4%).

Three readings follow. First, the sign depends on the method. Over all methods the relative ECE rises, while Rotation+LSQ, a QAT method, lowers it established; the Fig. 4 caption blames PTQ, which "generally renders the model's logit scales obsolete". Second, the text encoder is fragile. The $+98\%$ gathers per-model values from $+7\%$ (DFN) to $+298\%$ (EVA-02) at $p = 10^{-1}$, not significant at 0.05 suggestive; that the learned logit scale no longer fits two independently quantized embeddings is untested open. Third, pretraining data matter. In Fig. 4A the two WIT models improve under visual-only quantization (-6%, -5%) and degrade once the text encoder is quantized too, DFN degrades in both scopes ($+8\%$, $+7\%$) and improves only under Rotation+LSQ (Fig. 4B), and every LAION cell degrades suggestive. Sec. 4.2 and Table 5 count DFN among the curated models that improve, which Fig. 4A does not show, and the cause they give, "stochastic regularization", is weighed in Section 17.

Logit tuning (Section 15) is the temperature of Equation (62). In Fig. 5, QAT alone already lowers the ECE, and tuning brings visual-only LAION models to the 1.1% that Sec. 4.2 quotes for "fully quantized models". No FP32 baseline got the same fit, so how much of the gain is recalibration is open open. Patterns split by a starting value, larger tuning gains where ECE

was worst (Fig. 19, right, p. 22) and calibration improving more often from an "Originally Poor" state (Fig. 27, p. 29), inherit some regression to the mean (Section 13.5); the paper does not discuss how much open. Proxy data barely move accuracy but do move calibration, CC3M ahead of YFCC at $p = 4.5 \times 10^{-4}$ (Sec. 4.2; Fig. 19, the subject of Exercise II.5).

Which ICML calibration result the ICVES paper compares with

The ICVES paper finds confidence-thresholded reliability slightly worse under quantization, "the opposite sign from the calibration improvements reported for large-scale post-training-quantized recognition" (Bouguerra et al., 2026b, Sec. V). Yet the ICML gains come from Rotation+LSQ and QAT (Figs. 4B and 5), PTQ breaks the logit scale (Fig. 4), and the ICVES recognizer is not recalibrated and names no calibration metric. Method, recalibration, and metric all differ, as the ICVES paper concedes ("a different regime we do not reconcile here").

16.4 OOD detection follows accuracy

ICML paper, Sec. 4.3, Fig. 6, p. 6; App. G, Fig. 20A, p. 22

Fig. 6 plots, for ten models and six scoring functions, the average AUROC (Equation (63)) of successful W8A8 runs against FP32. Specific configurations give "small but statistically significant AUROC improvements" for SigLIP and DFN-B32, and ConvNeXt "benefits significantly more from quantization than Transformer-based architectures" (Sec. 4.3). Relative in-distribution (ID) accuracy change and relative AUROC change correlate with $r = 0.994$ and slope 0.428 ($p = 3.4 \times 10^{-91}$); the plot prints "Methods: 96" and "Raw Samples: 466560" (Fig. 20A), the count that Fig. 20B labels "All Bit-Widths" with no constraint.

By that count, Fig. 20A pools all bit widths without the filter of Fig. 6, and its 96 points are aggregated groups, although Sec. 4.3 cites "$N = 466k$". So $r = 0.994$ is an ecological correlation (Section 13.5), and the printed p fits 96 points, not 466,560 (the notes' reading and arithmetic). The slope says more, since on these averages OOD detection moves less than half as much as accuracy established. The paper's hypothesis that quantization damps ConvNeXt's high-frequency textural features (Sec. 4.3; App. G.1) echoes the texture bias of ImageNet-trained CNNs (Geirhos et al., 2019), but this ConvNeXt is CLIP-trained on LAION, and no spectrum or texture test is shown open.

16.5 Corruptions versus natural distribution shift

ICML paper, Sec. 4.4, Fig. 7, p. 7; Sec. 6, Table 3, p. 9

Over successful W8A8 runs, the average relative gap change (positive is more robust) is −0.3 under natural covariate shift ($n = 1632$) and +8.9 under synthetic corruption ($n = 2182$), both at $p < 0.001$ (Fig. 7). In Table 3 (CLIP ViT-B/32 (WIT), CIFAR-100, $n = 10$ seeds) the TensorRT INT8 engine loses 1.71, 7.99, and 30.72 points under Gaussian blur of $\sigma = 2$, 3.8, and 8 ($k = 11$, 23, 51), against 3.92, 12.28, and 35.64 for FP16, an added robustness of +2.21, +4.29, and +4.92 points (all paired t-tests $p < 10^{-8}$).

For a camera this contrast is the paper's most useful result. Per model in Fig. 7, corruption gains run from 0.2 (EVA02 B-16, not significant) to 14.8 (SigLIP B-16), significant for seven of eight models from 3.3 up, while natural-shift changes stay between +0.8 (CoCa B-32) and −1.3 (EVA02 B-16), with four significant losses and one significant gain established. (Fig. 7 calls natural shift "Semantic Robustness", unrelated to the semantic shift of OOD detection, and all four corruptions "Noise Injection".) Three limits apply. The runs are those that kept their clean

accuracy (Section 15). How the "relative gap change" follows from $\delta(D)$ of Sec. 3.2's Eq. (1) is not written, so +8.9 cannot be turned into points. And the four corruptions (App. C.4) are not reported separately; since brightness and contrast single out no frequency band relative to the image (Section 9), the paper's appeal to "coarse, low-frequency" components (Sec. 4.4) predicts no benefit for them, which a per-corruption table would test open.

The deployed check agrees on one model, data set, and corruption, the advantage growing with blur established. Adding the clean offset over TensorRT FP16, $66.30 - 64.43 = +1.87$ points (Table 2), Equation (60) gives $\Delta\text{Acc} = +4.08$, +6.16, and +6.79 points for the three blur levels (the notes' arithmetic, assuming the drops start from the Table 2 means). Growth up to $\sigma = 8$ does not contradict a severity window (Figure 29), since σ counts pixels of an unnamed image; a 51-pixel kernel, wider than a 32×32 CIFAR image, suggests enlargement to the encoder input first open.

16.6 Spurious correlations, where quantization hurts

Table 13: ICML Table 1 as printed, CounterAnimal (Wang, Lin, et al., 2024), "averaged across 10 independent runs" (Bouguerra et al., 2026a, Table 1, p. 7). Both metrics are defined in Equation (64); positive means more reliance on spurious context. The printed unit "%" is best read as points of a relative gap. The ± is not defined.

Method	Metric	W8A8	W6A8	W4A8
Simple PTQ	ΔRSG	2.6 ± 0.2	2.8 ± 0.3	12.5 ± 0.6
	Vuln_{add}	3.0 ± 0.3	3.2 ± 0.3	10.3 ± 0.4
QAT (Contr.)	ΔRSG	1.6 ± 0.2	2.1 ± 0.3	9.1 ± 0.5
	Vuln_{add}	1.9 ± 0.3	2.4 ± 0.3	9.1 ± 0.4
Rot+LSQ	ΔRSG	-0.1 ± 0.1	0.2 ± 0.1	4.0 ± 0.3
	Vuln_{add}	-0.2 ± 0.1	0.2 ± 0.1	4.4 ± 0.3

Every PTQ and QAT cell of Table 13 is positive. Both families widen the relative spurious gap (RSG) between typical and atypical contexts, several times more at W4A8 than at W8A8 established; Rotation+LSQ keeps it near zero at 8 and 6 bits but not at 4. The paper's reading, that naive PTQ suppresses "subtle invariant features" (Sec. 4.5), is weighed with its truncation intervention in Section 17 suggestive. Its account of the W4A8 jump, "large numerical imprecision" that "erodes the fine-grained features" (Sec. 4.5), is verbal and untested open, like the notes' own dead-zone conjecture (Section 4.3; Exercise II.6). The Impact Statement warns that the effect can amplify stereotypes (Birhane et al., 2021; Hooker et al., 2020).

Quote the table, not the prose. Sec. 4.5 gives "+2.7% at W8A8" for "PTQ and QAT (families)", which no cell holds, "+3.7%" for robust methods at W4A8 against 4.0 and 4.4, and "−0.2 n.s." at both W8A8 and W6A8 where the Rot+LSQ vulnerability is −0.2 and 0.2. The caption even disagrees with itself, saying that Rot+LSQ "does not significantly impact spurious correlations at 8-bit and 6-bit" and that "All results are significant".

16.7 Failure rates by method

A run fails when it shows "a relative zero-shot accuracy drop exceeding 5%" (App. F). The paper reports failure rates pooled by family and per method, and neither Fig. 15 nor Fig. 18 states a bit width.

ICML paper, App. D.4, p. 18; Figs. 15 and 18, pp. 20–21; Table 4, p. 15

Pooled, PTQ "fails the 5% accuracy tolerance threshold in 44.1% of evaluations" and QAT in 25.0% (App. D.4; Fig. 15, "60k runs"). Per method (Fig. 18), the PTQ rates are IGQ-

ViT 5.6%, Rotation PTQ 5.6%, Q-VLM 44.0%, QwT 57.6%, SmoothQuant 58.9%, OutlierAware 61.2%, APQ-ViT 61.9%, and Simple PTQ 62.0%. The QAT rates are 0.0% for Rotation+LSQ (both distillation regimes), 28.7% and 29.4% for LSQ, 32.7% and 33.5% for QAT (Hybrid, Contrastive Only), 33.5% for Q-ViT, and 42.4% for QAT-LoRA. Table 4, titled for W8A8, prints 62.0% for "PTQ", 25.0% for "QAT", and 0.0% for "Rotation+LSQ".

Worked example, which 62.0%?

The "60k" of Fig. 15 matches 8,640 checkpoints times 7 zero-shot data sets over all three bit widths. The eight PTQ rates average $356.8/8 = 44.6\%$, near the pooled 44.1% (the pool weights methods by unprinted counts), and the QAT rates $200.2/8 = 25.0\%$. The 62.0% of Table 4 matches neither pool but equals Simple PTQ in Fig. 18, the one method its PTQ column most likely holds, so quote 44.1% for PTQ and 62.0% for Simple PTQ. All of this is the notes' arithmetic.

Rotation takes plain PTQ from 62.0% to 5.6% and LSQ from about 29% to 0.0% established, though IGQ-ViT PTQ also reaches 5.6%. The paper credits the smaller step that rotation buys by spreading outliers (App. D.4), the argument of Section 6.2 (Ashkboos et al., 2024), but no control isolates it suggestive. App. F reads the 42.4% of QAT-LoRA as catastrophic forgetting by an adapter (Hu et al., 2022; Xu et al., 2024) too low-rank to absorb quantization error. Yet LSQ, which App. F describes as updating only the scales, fails less (28.7%, 29.4%), and no rank is varied, so this stays a hypothesis open. Bit width matters as much. At 4 weight bits every method has a win rate ("Acc $\geq$ 99% Base + ECE Improved") of 0% or 2%, against up to 55% (LSQ + Rotation [Hybrid]) at 8 (Fig. 28, p. 29) established.

16.8 The deployed engine

ICML paper, Sec. 6, Table 2, p. 9

CLIP ViT-B/32 (WIT) on CIFAR-100, RTX 4070, $n = 10$ seeds, batch size 64; size counts the visual backbone only. Accuracy, latency, speed-up, and size are 64.47%, 120.89 ms, 1.00×, 302.1 MB for PyTorch FP32; $64.43 \pm 0.05\%$, 22.70 ms, 5.33×, 178.7 MB for TensorRT FP16; and $66.30 \pm 0.39\%$, 15.66 ms, 7.72×, 118.4 MB for TensorRT INT8 (LSQ). The INT8 engine exceeds FP32 accuracy by +1.83 points ($p < 10^{-7}$).

The headline 7.72× is measured against PyTorch FP32, so it bundles an optimized TensorRT engine, 16-bit arithmetic, and 8-bit arithmetic. The engine with FP16 gives $120.89/22.70 = 5.33\times$; INT8 against FP16 in the same engine, the baseline App. G.5 names, gives $22.70/15.66 = 1.45\times$ in latency, $178.7/118.4 = 1.51\times$ in size, and +1.87 points (the notes' arithmetic). Neither halving of the format halves the size ($302.1/178.7 = 1.69\times$, $1.51\times$; Equation (7)), so the sizes hold more than weights in the nominal format. INT8 kernels that dequantize weights on the fly (bitsandbytes), by contrast, run CLIP ViT-B/32 at 81.1 ms per batch against 19.6 ms in FP32 (App. G.4, Table 6, p. 24), so the engine matters as much as the bits. The deployed result is one model, one data set, and one GPU established, yet the transfer of the simulated results to real engines rests on it (App. G.5).

Running example: the bus in the rain

The bus encoder lies outside the ICML grid of CLIP-family models scored zero-shot, so the results offer the bus in the rain priors, not predictions. *Blur from a fogged lens* falls under the corruption suite, which contains defocus blur, and there robustness rose on average (Fig. 7). *Real rain* falls under neither axis. The natural-shift sets are renditions, sketches,

re-collected photos, and adversarially filtered images, and the corruption suite holds no rain, snow, or fog (App. C.4). Its "weather" (Sec. 4.4) can only be brightness, which Hendrycks and Dietterich (2019) file under weather. Whether +8.9 or −0.3 applies to rain is unknown open; the ICVES paper tests native weather (Section 18).

Exercises

Exercise II.5: Two figures, one population

Fig. 4A prints all-method totals of +9% ($N = 5543$) and +98% ($N = 3893$), negative meaning better; Fig. 19 prints mean ECE changes of −36.9% ($N = 3291$, CC3M), −50.5% ($N = 3124$, YFCC), and −49.1% ($N = 3021$, SBU), positive meaning better (Table 11). (a) Show that both count the same number of runs. (b) Compute each figure's run-weighted mean in the sign of Fig. 4A. (c) What does this suggest about the unnamed statistic of Fig. 4A, allowing for rounding to whole percents?

Solution sketch. (a) $5543 + 3893 = 9436 = 3291 + 3124 + 3021$. (b) $(5543 \cdot 9 + 3893 \cdot 98)/9436 = +45.7\%$ and $(3291 \cdot 36.9 + 3124 \cdot 50.5 + 3021 \cdot 49.1)/9436 = +45.3\%$, both a worsening. (c) With the Fig. 4A values anywhere in $[8.5, 9.5]$ and $[97.5, 98.5]$ the weighted mean lies in $[45.2, 46.2]$, which contains 45.3, consistent with means over shared runs suggestive, though equal totals could be a coincidence. A mean worsening of about 45% coexists with the 38.8% of Fig. 1 runs that improve calibration, because a few very large relative degradations dominate a mean.

Exercise II.6: Bits and spurious gaps

For Simple PTQ, Table 1 prints ΔRSG = 2.8 at W6A8 and 12.5 at W4A8. (a) At a fixed clip range, by what factor does the step grow from 6 to 4 bits, and by how many decibels does the rounding-noise power rise (Equations (16) and (18))? (b) Compare with 12.5/2.8. (c) Give three reasons why nothing follows from the comparison.

Solution sketch. (a) The step is the range over $2^b - 1$ intervals, so it grows by $63/15 = 4.2$ and the power by $4.2^2 = 17.6$, or 12.5 dB. (b) $12.5/2.8 = 4.5$, and the vulnerability grows $10.3/3.2 = 3.2$-fold. (c) Ranges are recalibrated at each bit width; the error at the features compounds through many layers and, in QAT, through training; and ΔRSG is a difference of relative accuracy gaps, tied to noise power by no law. The paper's own account of the W4 break is verbal (Sec. 4.5), so any sharper mechanism is conjecture open.

Check yourself before moving on

1. Of the 6720 significant W8A8 runs of Fig. 1, what share improves calibration, what share improves both metrics, and what share degrades both?[a]
2. Is the PTQ failure rate 44.1% or 62.0%?[b]
3. On which dimensions does W8A8 evidence show reliability getting worse?[c]

[a] $7.8 + 31.0 = 38.8\%$, 7.8%, and 54.8% (Table 12).

[b] 44.1% pooled over the eight PTQ methods (App. D.4; Fig. 15); 62.0% is the Simple PTQ rate of Fig. 18, printed under "PTQ" in Table 4.

[c] Accuracy falls in 85.8% of the Fig. 1 runs. Over successful runs, calibration worsens for all methods pooled (+9%; +98% at $p = 0.1$, Fig. 4A) and natural covariate shift costs −0.3 (Fig. 7). Spurious reliance rises for Simple PTQ and QAT (Table 1).

17 The ICML paper, the spectral-filtering account

What you will be able to do after this section

- Map each of Eqs. (2) to (12) of the ICML appendix onto a Part I object and its basis.
- Read Fig. 8b with Fig. 8c to tell whether error buries a direction or erases part of it.
- Explain why "dampens high-rank spectral components" means a low per-rank signal-to-quantization-noise ratio (SQNR) SQNR_k in the tail, not energy removed.
- Convert Fig. 9's relative subspace accuracies into absolute ones, and say why the tail is not noise.
- Weigh the truncation and dropout controls, and sort the account by evidence.

Section 16 read the ICML paper's results one reliability axis at a time; its Sec. 5 proposes one mechanism behind several of them. The authors "hypothesize that although quantization injects uniform noise in the value domain, it operates as a non-uniform spectral filter in the feature domain" (Bouguerra et al., 2026a, Sec. 5, p. 8). The verdict, stated first, is that the per-rank measurements are sound descriptions established, their mechanistic reading is suggestive suggestive, and the bridge from feature rank to image frequency and texture is open open. "Eq. (2)" to "Eq. (12)" carry the paper's numbers; the notes' own equations are cited as "Equation".

17.1 Two stages on two bases

Both stages use "the penultimate visual features of the ImageNet-1k validation set" (App. D, p. 16), the input of the projection head as App. D.2 and D.3 imply, a layer that App. C leaves unquantized (Bouguerra et al., 2026a). They differ in the basis (Section 7.5). *Stage 1, passive filtering* (App. D.1, Fig. 8) takes the singular value decomposition (SVD) of the centered FP32 features once and projects the quantized features on those frozen directions $\boldsymbol{v}_k$, asking how much of each original direction survives. *Stage 2, active concentration* (App. D.2, Fig. 9) recomputes the SVD on the quantized features, asking how the model arranges its own representation (Figure 32).

The samples differ too. Fig. 8 pools "all models" into the families PTQ (post-training quantization), QAT (quantization-aware training) and ROT (rotation-based methods) over "3 independent runs" (Sec. 5, p. 8; App. C.5 says "2 seeds instead of 3"), without saying how models of different width share one rank axis; Fig. 9 uses one model, CLIP-ViT-B-32-WIT, over "10 full independent runs" (p. 9). Neither figure states its bit width.

17.2 Eqs. (2) to (12) as measurements

Table 14 lists the eleven definitions with the object each one computes. Write X and X_Q for the FP32 and quantized features of the same N images, $\boldsymbol{\mu}$ and $\boldsymbol{\mu}_Q$ for their means, and $z = (\boldsymbol{x} - \boldsymbol{\mu})^\top \boldsymbol{v}_k$ and $z_Q = (\boldsymbol{x}_Q - \boldsymbol{\mu})^\top \boldsymbol{v}_k$ for the clean and quantized components along a fixed FP32 direction, as in Equation (39).

Two details matter for reading the figures. Eq. (4) divides by "the baseline singular values" $\sigma^{(k)}_{\mathrm{FP32}}$ without stating their normalization; computed by Eq. (3) they are $S_{kk}/\sqrt{N} = \sqrt{\lambda_k}$, the reading taken here, while raw singular values would shift every rank by $\log_{10}\sqrt{N}$. And the band convention is unstated. Adjacent bands share endpoints ("0–8", "8–16", ...) and "64–768" would pass the last index, 767, if both ends counted, so these notes read the bands as half-open, "0–8" being $k = 1$ to 8 and "64–768" the last 704 directions; yet Eq. (10) as printed sums from $k = i$ to j inclusive, nine components for $[0, 8]$ (Bouguerra et al., 2026a, pp. 9 and 17).

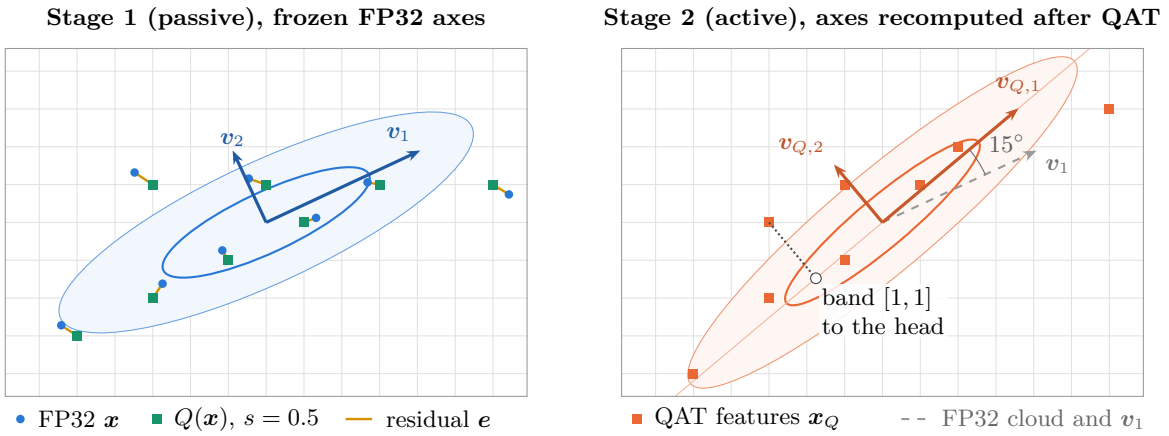


Figure 32: The two stages of the ICML spectral analysis on a synthetic two-dimensional model. FP32 features have principal variances 2.25 and 0.16 along $\boldsymbol{v}_1$ (25°) and $\boldsymbol{v}_2$, rounded on one grid of step $s = 0.5$. Left, Stage 1 keeps the FP32 axes; the rounding error has the same power $s^2/12 = 0.021$ on both, so the weak direction has the lower SQNR_k. Right, Stage 2 refits the axes after a QAT that also moved the weights (a linear map before rounding). The new first axis turns 15° and holds 0.961 of the variance against 0.934 for FP32, while the old axes hold only 0.899 of it; both clouds have zero mean, so Equation (36) explains why both readings hold. The model is stated in the figure file.

Eq.	In the paper	In these notes	Basis
(2)	SVD $X - \boldsymbol{\mu} = U\Sigma V^\top$	directions $\boldsymbol{v}_k$, $\lambda_k = S_{kk}^2/N$ (Equation (32))	FP32
(3)	projected magnitude $\sigma_Q^{(k)}$	$\sqrt{\mathbb{E}\left[z_Q^2\right]}$; $\sqrt{\lambda_k}$ for FP32	fixed
(4)	spectral distortion $\Delta\sigma^{(k)}$	$\frac{1}{2}\log_{10}\left(\mathbb{E}\left[z_Q^2\right]/\lambda_k\right)$, Equation (69)	fixed
(5), (6)	powers $P_S^{(k)}$, $P_N^{(k)}$	λ_k and the uncentered e_k (Definition 8.1); signs in Table 11	fixed
(7)	SQNR_k	Equation (38), up to the unstated ϵ	fixed
(8)	reconstruction $\hat{\boldsymbol{x}}_k$, Sim_k	rank-k reconstruction of $\boldsymbol{x}_Q$ (Equation (33)), cosine with $\boldsymbol{x}$	fixed
(9)	SVD $X_Q - \boldsymbol{\mu}_Q = U_Q\Sigma_Q V_Q^\top$	recomputed directions $\boldsymbol{v}_{Q,k}$	own
(10)	band reconstruction $\tilde{\boldsymbol{x}}_{[i,j]}$	ranks i to j of the own basis, fed to the zero-shot head	own
(11)	$\Delta\mathrm{Acc}_{[i,j]}$	relative change, positive is better, the sign opposite to Eq. (1) (Equation (58))	own
(12)	$F_Q(k)$	explained-variance fraction $F(k)$ (Equation (33))	own

Table 14: The App. D definitions of the ICML paper (Bouguerra et al., 2026a, pp. 16–17) and the objects they compute. "Fixed" is the frozen FP32 basis, "own" the basis recomputed on each model's features. Eqs. (3)–(8) feed Fig. 8, Eqs. (9)–(12) Fig. 9.

Spectral distortion in terms of SQNR_k and the gain. Eqs. (3), (5) and (6) project on the same $\boldsymbol{v}_k$ from the same FP32 mean, so Eqs. (4) and (7) are two views of one decomposition. With c_k, g_k and e_k of Equation (39), $\rho_k = e_k/\lambda_k = 10^{-\mathrm{SQNR}_k/10}$, and

$\sigma^{(k)}_{\mathrm{FP32}} = \sqrt{\lambda_k}$,

$$\Delta\sigma^{(k)} = \tfrac{1}{2}\log_{10}\frac{\mathbb{E}\left[z_Q^2\right]}{\lambda_k} = \tfrac{1}{2}\log_{10}\left(1 + \frac{2c_k}{\lambda_k} + \rho_k\right) = \tfrac{1}{2}\log_{10}\left(2g_k - 1 + \rho_k\right), \tag{69}$$

where the pair must satisfy $\rho_k \geq (1-g_k)^2$. *Proof.* Squaring Eq. (3) gives $\mathbb{E}\left[z_Q^2\right] = \lambda_k + 2c_k + e_k$ by Equation (39). Divide by λ_k and use $c_k = (g_k - 1)\lambda_k$. The constraint is $\mathbb{E}\left[n_k^2\right] = e_k - c_k^2/\lambda_k \geq 0$ divided by λ_k. □
Independent error ($g_k = 1$) gives $\Delta\sigma^{(k)} = \frac{1}{2}\log_{10}(1+\rho_k) > 0$ at every rank. The identity is the notes' own, derived from the paper's definitions established; it holds per model and per rank, not for curves averaged across models.

▷ **Check yourself.** Take $\mathrm{SQNR}_k = 10\,\mathrm{dB}$. What distortion does independent error give, and what does $g_k = 0.9$ give?[23]

Table 15 extends the computation. Along a row, a fixed gain gives a distortion that grows as SQNR_k falls toward the tail; down a column, one SQNR_k allows amplification or attenuation depending on g_k. Neither Fig. 8b nor Fig. 8c alone can tell burial from erasure; the pair can.

	SQNR_k in dB (and ρ_k)				
gain g_k	20 (0.010)	10 (0.100)	6 (0.251)	0 (1.000)	−3 (1.995)
1 (independent)	+0.002	+0.021	+0.049	+0.151	+0.238
0.9	−0.046	−0.023	+0.011	+0.128	+0.223
0.75	–	−0.111	−0.062	+0.088	+0.199
0.5	–	–	−0.300	0.000	+0.150

Table 15: The spectral distortion $\Delta\sigma^{(k)}$ of ICML Eq. (4) predicted by Equation (69) for a given SQNR_k and gain g_k (the notes' arithmetic). A dash marks impossible pairs, $\rho_k < (1-g_k)^2$, since removing the fraction $1-g_k$ of the signal's amplitude takes an error of energy at least $(1-g_k)^2\lambda_k$.

17.3 Stage 1, passive filtering, read with both meters

Fig. 8b works as a volume meter on each FP32 direction and Fig. 8c as a clarity meter; only the pair says whether the error buried or erased a direction.

The clarity meter. Fig. 8c plots SQNR_k (Equation (38)), which "decays monotonically with increasing rank" for all three families (Bouguerra et al., 2026a, Sec. 5, p. 8), established for the aggregated curves. The reason offered, that with a fixed step components "with magnitudes comparable to the discretization grid (typically high-rank, low-variance features) are dominated by rounding errors", is the flat floor of Proposition 8.1, whose hinge, equal residual power in every direction, the paper neither states nor tests. These components are directions of the penultimate features, where the errors of every quantized layer add up (App. C), not one coordinate rounded to zero like coordinate B of Section 4; for QAT the residual also holds weight drift (App. C.2).

Whether the residual is flat or colored (Section 8.4) hinges on Fig. 8a's axis "Log_{10} Mag". From rank 10 to 600 that curve falls 0.8 to 0.9 decades while the PTQ and QAT SQNR_k falls

[23] $\rho_k = 0.1$, so $\frac{1}{2}\log_{10} 1.1 = +0.021$, a magnitude 1.05 times the FP32 one, and $\frac{1}{2}\log_{10}(0.8+0.1) = -0.023$, an attenuation, at the same SQNR_k.

about 8 dB (plot readings). As magnitudes, the signal loses 16 to 18 dB and the residual 8 to 10 dB, a colored residual; as energies, the signal loses 8 to 9 dB and the residual stays roughly flat. App. D.1 calls $\sigma_Q^{(k)}$ "the projected magnitude" (p. 17), which favors magnitudes, but no equation is tied to the plot open.

The volume meter. Fig. 8b plots Eq. (4). Read from the curves (no values are printed), the PTQ and QAT distortion starts slightly below zero and rises to clearly positive values toward the tail, while ROT stays small, positive and nearly flat. By the paper's convention "positive indicates signal amplification (noise injection)" (Bouguerra et al., 2026a, App. D.1, p. 17), so the abstract's "dampens high-rank spectral components" cannot mean that the tail loses energy. It means that the tail's signal is swamped by the error, a low SQNR_k established. Read as a gain $g_k < 1$ on the signal copy, "dampens" is possible, but that is the notes' open inference below, not what Fig. 8b shows.

Equation (69) applies Proposition 8.2 to these curves. The negative start requires $g_k < 1$, error anticorrelated with the signal. In the tail, independent error near 0 dB gives about +0.15 (Table 15), and values well below that again require $g_k < 1$ (Exercise II.7), a tail partly erased and not only buried. This is the notes' inference open, since averaging breaks the per-model identity. Two models at 20 and 0 dB average to 10 dB, but their residual-to-signal ratios 0.01 and 1 average to 0.505, or 3.0 dB, and the paper does not say which average it plots. The dead zone, clipping, the nonlinear layers after each quantizer and, for QAT, a weight update could each produce the anticorrelation; measuring g_k for PTQ alone, where no weight is retrained, would set the last apart.

Rotation. The Fig. 8 caption credits the higher SQNR_k of rotation-based methods to "their smaller step size" (Bouguerra et al., 2026a, p. 8), the outlier spreading of Section 6.2 (Ashkboos et al., 2024); under the flat floor, halving the step lifts every SQNR_k by $20 \log_{10} 2 = 6.02$ dB. The paper adds that rotation "prevents premature signal decay in the intermediate spectral region (Ranks 8–64)" (p. 9), ranks of the FP32 basis, unlike the "0–8" of Fig. 9. There the PTQ and QAT curves of Fig. 8c are still well above 0 dB (a reading), so rotation buys a graded gain, not a rescue from below a threshold suggestive. In the model of Figure 17, a band near 8–64 meets the floor only at about 4 bits, and halving the step moves the crossing rank by a factor $4^{1/1.2} = 3.2$. Whether filtering strength sets robustness is open open. ROT filters least, yet App. B, Table 4, marks synthetic-noise robustness as improved for PTQ, QAT and Rotation+LSQ (learned step size quantization) alike (Exercise II.16).

Running example: the bus in the rain

Take a synthetic bus-or-truck cue at rank 150 and, under the flat-floor model of Section 8 at 6 bits, give it $\mathrm{SQNR}_{150} = 1.7$ dB, $\rho_{150} = 0.676$. Two encoders can put this same point on a Fig. 8c curve. If the error ignores the cue ($g = 1$), $\Delta\sigma = \frac{1}{2}\log_{10} 1.676 = +0.112$, a direction 1.29 times louder than in FP32 with the cue intact but buried. If the dead zone scales the cue's copy by $g = 0.5$, $\Delta\sigma = \frac{1}{2}\log_{10} 0.676 = -0.085$, a direction 0.82 times as loud. A retrained head can undo the gain but not the lost signal-to-noise ratio. The part of z_Q that tracks the cue now stands at 0.59 (−2.3 dB) against its uncorrelated remainder, against 1.48 (+1.7 dB) when $g = 1$.

17.4 Stage 2, active concentration

Stage 2 leaves the FP32 basis for each model's own (Figure 32, right). The ICML QAT families also train the weights (App. C.2, p. 16), so their residual mixes rounding with training drift and a reorganized basis is possible. The recognizer of the accepted ICVES 2026 paper freezes its backbone, trains only LSQ scales and a linear head, and reports its effective rank unchanged,

90.4 against 90.7 (Bouguerra et al., 2026b, Fig. 6A, L3). The two describe different interventions and do not conflict (Table 8).

ICML paper, Sec. 5 and Fig. 9, pp. 8–9

With bases recomputed per model on CLIP-ViT-B-32-WIT and ImageNet-1k, quantized models "surprisingly achieve higher subspace accuracy than their FP32 baselines" in the band of ranks "0–8" (Fig. 9A), there is "a significant degradation in accuracy for high-rank subspaces (Rank 64+)" (Fig. 9B), and the cumulative energy of Eq. (12) shifts toward the leading ranks (Fig. 9C). The authors read this as "a 'coarse-grained compensation' effect with QAT", in which limited capacity "compels the model to concentrate discriminative information in the most robust, high-variance dimensions" (Bouguerra et al., 2026a).

The measurement is established for one model; in Fig. 9A the band 0–8 beats FP32 for all three families, though over 0–64 PTQ does not (Fig. 9B, a reading). The compensation mechanism, and any claim about quantized vision-language models in general, is suggestive (Table 20).

Worked example, relative into absolute

Eq. (11) divides by the FP32 accuracy of the same band, each model's band being its own top directions, so a relative change needs its base. Fig. 9B prints the bases, "FP32: 30.32%" (ranks 0–64) and "FP32: 28.52%" (64–768). Reading its bars, LSQ's +7.7% on the low band is $0.077 \times 30.32 = +2.3$ percentage points (pp), ROT+LSQ's +13.5% is +4.1 pp, and the −8% of PTQ and of LSQ on the high band is $-0.08 \times 28.52 = -2.3$ pp. The FP32 accuracy of the band 0–8 is not printed, but Fig. 12 puts an FP32 model truncated to $k = 8$ near 1% (a reading; the model is unnamed, its $k = 64$ point near 30%), so LSQ's relative +35% there (Fig. 9A) is of order +0.35 pp. Compare relative changes only after this conversion.

The tail is not noise. The FP32 band 64–768 alone reaches 28.52%, close to the 30.32% of the 64 leading directions (Fig. 9B), so this model's tail carries nearly as much discriminative information as its head established, the point of the heading "The Semantic Trade-off" in Sec. 5. A filter that lowers SQNR_k there costs information, not only noise. That this loss causes the covariate-shift and spurious-correlation costs of Section 16 is the authors' attribution, consistent with the bars but not tested against them suggestive; the bars are wide, of unstated type, and several span zero.

The endpoint of Eq. (12). By construction $F_Q(D) = F_{\mathrm{FP32}}(D) = 1$, with D the paper's feature width (d here, 768 for this model), so the plotted difference must end at zero. The curves of Fig. 9C end visibly below it, as do the PTQ and LSQ curves of Fig. 14 (near −0.01), while its dropout curves return to 0 (readings). Either the plotted quantity is normalized differently from Eq. (12) as printed, or the axis stops before D (Exercise II.8).

17.5 The truncation and dropout controls

App. D.3 opens by conceding that "correlation does not establish a causal chain to the downstream reliability effects" (Bouguerra et al., 2026a, p. 17). Two experiments aim at the gap.

Rank truncation (App. D.3, Figs. 11–13). The authors keep the top k principal components of an FP32 encoder's penultimate features, $k \in \{8, 16, 32, 64, 128, 256, 512\}$, and pass the reconstruction through the unchanged text-projection head. As the tail is removed, "accuracy initially holds, calibration improves, and noise robustness increases monotonically before collapsing once k drops below the discriminative core", a dependence on k they call "the smoking gun" (Bouguerra et al., 2026a, p. 18). The figures truncate the FP32 model and a ROT+LSQ W8A8

(8-bit weights and activations) model and plot the difference (Figs. 11, 12; Fig. 13 truncates PTQ and LSQ). On ImageNet-1k (Fig. 12, readings), truncation lowers the FP32 accuracy and raises its expected calibration error (ECE) as k shrinks, the opposite of "calibration improves" for FP32 alone. The quantized advantage peaks near $k = 64$ for clean accuracy and Gaussian noise, but for blur at severity 5 it is largest near full rank, so cutting the tail does not enlarge it. And truncation removes the tail, whereas quantization lowers its SQNR_k (Stage 1). The control is suggestive at best.

Weight dropout (App. D.4, Fig. 14). Fig. 14 compares the Eq. (12) shift of W8A8 PTQ and LSQ with that of weight dropout at 10, 20 and 30%. Quantization "concentrates energy in the low-rank head", while dropout, "being stochastic and full-rank, produces a markedly different and far weaker spectral shift" (Bouguerra et al., 2026a, p. 18). One unnamed model is shown and the dropout rates are not matched to the quantization error, so the figure shows different signatures, not that noise of matched size fails suggestive. The ICVES paper reaches a similar conclusion with activation noise matched to each quantizer's calibrated step (Section 20). The ICML paper is not consistent here, since its Sec. 4.2 credits a calibration gain to the "stochastic regularization" of quantization noise (p. 5).

17.6 What is derived, what is measured, what is asserted

Nothing in the chain is derived mathematically in the paper; Eqs. (2) to (12) define measurements, and the derivations here (Equation (69) and its use with Proposition 8.2) are the notes' own. Table 16 sorts the claims.

Claim	Locator	Status	Tier
SQNR_k falls with rank on the FP32 basis	Sec. 5; Fig. 8c	measured, aggregated	established
It falls because a fixed step gives every direction equal error	Sec. 5, p. 8	asserted; isotropy untested	suggestive
Quantization "dampens" the high-rank tail	Abstract; Fig. 8b–c	low SQNR_k; the tail's magnitude rises	established
The error is anticorrelated with the signal	Fig. 8b	the notes' inference (Proposition 8.2)	open
Rotation lifts SQNR_k, hence its smaller costs	Fig. 8c; Table 1	lift measured; causal link untested	suggestive
Own band 0–8 beats FP32 for all three families	Fig. 9A	measured, one model; over 0–64 PTQ does not	established
Tail suppression causes semantic and spurious costs	Sec. 5	attributed, consistent with Fig. 9B	suggestive
Concentration "directly correlates" with out-of-distribution detection	Sec. 5, p. 8	asserted; no coefficient given	open
Truncation reproduces the signature; dropout differs	App. D.3–D.4	partly supported; Fig. 14 one model	suggestive
QAT finds a flatter minimum	App. H; Fig. 23	"A conceptual illustration"; no curvature measured	open
Feature rank maps to image frequency and texture	Sec. 2	asserted via citations	open

Table 16: The ICML spectral-filtering account sorted by evidence (Bouguerra et al., 2026a). Table 20 sets these tiers beside the ICVES claims.

The last row carries the account from features to images. The paper's Sec. 2 recalls the "spectral bias" of networks toward low-frequency functions (Rahaman et al., 2019; Xu et al., 2020) and states that "low-rank spectral components typically encode coarse-grained, robust semantic structures (e.g., global shapes), while the high-rank tail captures fine-grained details, textures, and noise" (p. 2), citing prior work (Huh et al., 2023; Wang et al., 2020; Yin et al.,

2019). "Spectral" there is the Fourier spectrum of a function of the input, not the feature covariance of Fig. 8; the heading "Quantization as a Low-Pass Filter" (p. 6) joins the two by analogy. None of the paper's measurements links rank index k to spatial frequency (Section 8.7), and the ICVES profiles do not settle it. Blur's residual sits in the tail like the quantization residual, consistent with the link, but sensor noise and contrast co-align comparably (ICVES Fig. 6C, p. 7), so this does not isolate it. The band-limited-noise test of Section 9.5, one image octave at a time, could settle the question open.

Exercises

Exercise II.7: Reading Fig. 8b against Fig. 8c

Suppose that near the first ranks of the PTQ and QAT curves you read $\mathrm{SQNR}_k \approx 11\,\mathrm{dB}$ and $\Delta\sigma^{(k)} \approx -0.015$, and near the last ranks $\mathrm{SQNR}_k \approx -3\,\mathrm{dB}$ and $\Delta\sigma^{(k)} \approx +0.08$ (approximate plot readings, not printed values). (a) What would independent error give at each end? (b) Solve Equation (69) for g_k and the correlation $c_k/\sqrt{\lambda_k e_k}$ at each end. (c) Why are the answers indicative only?

Solution sketch. (a) $\rho = 10^{-1.1} = 0.079$ gives $+0.017$ at the head; $\rho = 1.995$ gives $+0.238$ at the tail. (b) Head, $2g - 1 + 0.079 = 10^{-0.03} = 0.933$, so $g = 0.93$ and the correlation is $(g-1)/\sqrt{\rho} = -0.26$. Tail, $2g - 1 + 1.995 = 10^{0.16} = 1.445$, so $g = 0.23$ and the correlation is -0.55, with $\rho \geq (1-g)^2 = 0.60$ satisfied. Both ends need anticorrelated error, and the tail would keep only a scaled copy of its signal of amplitude 0.23 (5% of its energy). (c) Averaging over models, runs and methods breaks the per-model identity, and the normalization of $\sigma^{(k)}_{\mathrm{FP32}}$ and ϵ are unstated.

Exercise II.8: Endpoints that must hold

(a) Show that Eq. (8) at $k = D$ returns $\hat{\boldsymbol{x}}_D = \boldsymbol{x}_Q$, and say what Sim_D measures. (b) Show that $F_Q(D) - F_{\mathrm{FP32}}(D) = 0$ for Eq. (12) as printed. (c) If each model's cumulative energy were divided by the FP32 total instead, what would the difference be at $k = D$?

Solution sketch. (a) V is a complete orthonormal basis, so $\sum_{j=1}^{D}((\boldsymbol{x}_Q - \boldsymbol{\mu}) \cdot \boldsymbol{v}_j)\boldsymbol{v}_j = \boldsymbol{x}_Q - \boldsymbol{\mu}$; Sim_D is the mean cosine between quantized and FP32 penultimate features, the penultimate-layer analogue of the fidelity in the paper's Fig. 3, and 1 for FP32. (b) Numerator and denominator coincide at $k = D$. (c) $\operatorname{tr} C_Q / \operatorname{tr} C - 1$, negative if the quantized features carry less total variance; the paper does not say which normalization it plotted.

Check yourself before moving on

1. Which of Eqs. (2) to (12) use the frozen FP32 basis?[a]
2. Why does a positive distortion in the tail not contradict "dampens high-rank components"?[b]
3. The FP32 model classifies at 28.52% from ranks 64–768 alone. What does that say about the tail?[c]

[a] Eqs. (3)–(8), on the basis of Eq. (2); Eqs. (10)–(12) use the basis of Eq. (9), recomputed on each model.

[b] "Dampens" is about SQNR_k, the signal swamped by the error, while the tail's projected magnitude rises. A gain $g_k < 1$ is the notes' open inference.

[c] It carries nearly as much discriminative information as the 64 leading directions (30.32%), so a lower SQNR_k there costs information (Fig. 9B).

18 The ICVES paper, a matched-budget road-scene testbed

What you will be able to do after this section

Describe DAWN and the paper's two testbeds by the numbers that define them; say what each variant trains and what its difference from the full-precision control isolates; name the alternative explanation each element of the matched protocol rules out; count the budgets and the evaluation grid; and say what the error bars cover and what the setting leaves unstated.

The accepted ICVES 2026 paper (Bouguerra et al., 2026b) measures one road-scene pipeline tightly, where the ICML paper measures many models broadly (Section 15). It takes robustness to mean accuracy on corrupted inputs and asks whether W8A8 quantization changes how fast that accuracy falls as corruption grows (Sec. I, p. 1), with budgets matched so that, by the paper's design, quantization is the only difference between the models compared. These notes quote it as the ICVES paper with locators; its numbers come from the accepted manuscript before the camera-ready and may differ slightly in the published version. Sections 19 and 20 read what the testbed measured and why.

18.1 DAWN and the two testbeds

Both testbeds use DAWN (Kenk & Hassaballah, 2020), 1026 driving images captured in real fog, sandstorm, rain, and snow and, as the paper uses it, annotated with four classes, car, truck, bus, and person. Every split is by image, and one rule is set in bold type, that no model sees a corrupted image during training or calibration (Sec. III, p. 2). Adverse weather is native to the images, and synthetic blur is added on top of it at evaluation only (Section 9).

The detector. RT-DETR (Zhao et al., 2024) with an R18 hybrid backbone, pretrained on COCO (Lin et al., 2014) and evaluated at 640×640 (Section 11). Its "20.2M parameters" are printed beside the backbone, but by the 40.5 MB of the FP16 detector (Table IV) they are the whole detector (the notes' reading). Boxes with a shorter side under 20 pixels are ignore regions, leaving 7347 boxes over 821 training and 205 test images. A base fine-tune on clean DAWN (AdamW, learning rate 10^{-5} for the backbone and 10^{-4} elsewhere, batch 8, 40 epochs, a cosine learning-rate schedule) gives one converged clean detector C, the start of every detector variant. Batch normalization (per-channel normalization with running statistics) is frozen, in a phase the paper does not name. The metric is COCO mean average precision, mAP@[.5:.95]; the COCO checkpoint scores 39.2 zero-shot on this test split and C 38.4 (Sec. III, p. 2).

The recognizer. To ask whether the detector's neutrality conceals a change inside the model, the paper evaluates the classification stage in isolation (Sec. I, p. 2). A frozen DINOv2 ViT-S/14 (Oquab et al., 2024) feeds 384-dimensional features to a linear head (Figure 23). Each ground-truth box with shorter side at least 40 pixels becomes a square context crop at 224×224, an oracle-box classifier (Section 11). The by-image split, independent of the detection split, holds out 25% of the images ($N = 1462$ test crops) and caps the car class at 700 training crops (Sec. III, p. 3). The metric is balanced accuracy (Equation (57)), because the crops are car-dominated; plain accuracy, also reported, slips by up to 2.9 points under W8A8, because the gain sits in the minority classes (Sec. V, p. 5).

What is quantized. W8A8 quantizes weights per channel and activations per tensor, and the activation steps are trained by learned step size quantization (LSQ) (Esser et al., 2020) with straight-through gradients (Courbariaux et al., 2015). The paper never says that the weight steps are trained, and L3, the third of the limitations it numbers L1 to L5 (Sec. VIII, p. 8), places the effect "in the learned activation scales and head". The detector quantizes all 37 linear and 69 convolution layers except its heads; the encoder quantizes the 48 linear layers inside its transformer blocks and keeps the classification head, patch embedding, layer norms, and attention softmax in higher precision (Sec. III, pp. 2–3; Section 5). No backbone weight trains

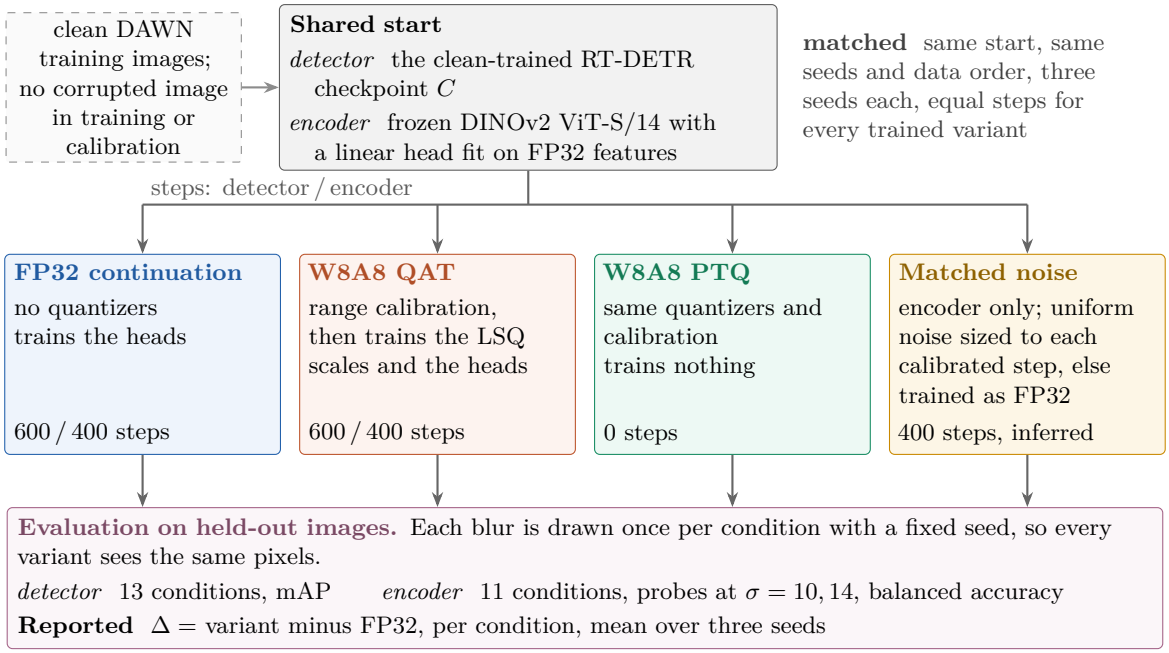


Figure 33: The matched-budget tree of the ICVES paper (Sec. III, pp. 2–3; Sec. VII, p. 7). Every trained variant gets the same steps (for the noise variant, inferred from "otherwise trained identically"), all variants are scored on the same corrupted pixels, and every accuracy Δ is taken against the FP32 continuation. By design, the branch a variant takes is then the only systematic difference behind its Δ, up to seed and sampling noise.

after the shared start. Plain LSQ was chosen so that an effect stays attributable to quantization itself and the scales export directly to integer kernels (Sec. II, p. 2). Table 8 contrasts this with the ICML paper's quantization-aware training (QAT), which moves the weights.

18.2 The variants and the matched protocol

From one clean start the paper derives the variants of Figure 33 and Table 17, which "share data, schedule, and seeds and differ only in quantization" (Sec. III, p. 2). They are a full-precision (FP32) continuation as the control, W8A8 QAT, W8A8 post-training quantization (PTQ), which trains nothing, and for the encoder a matched-noise variant (Sec. VII, p. 7); the QAT models are also exported to true integer kernels.

Each element of the protocol closes one alternative explanation of a QAT−FP32 difference; they are the design elements of Section 13.6 made concrete.

- **Shared start.** Neither variant starts from a better optimum. This fixes the starting model, C at 38.4 mAP (its operating point); neutrality at other operating points "is not established" (Sec. III, p. 2) open.
- **Equal continuation steps.** The FP32 control gets the same steps without quantizers, so "the only remaining difference is quantization" (Sec. III, p. 3). Otherwise QAT−FP32 would add the effect of 400 (or 600) extra head updates.
- **Same seeds and data order.** Every variant is "re-seeded identically (same data, same order)" (p. 2), so seed j of QAT and of FP32 form a *seed pair*, and a difference is not the luck of one shuffle.
- **Pixel-identical inputs.** A fixed corruption seed per condition makes "all three score on pixel-identical inputs" (p. 2). This matters for motion blur, whose kernel direction is random per image (Michaelis et al., 2019); Gaussian blur is deterministic.

Variant	Forward pass	What trains	Its contrast isolates
FP32 continuation	full precision	the heads, 600 steps (detector) or 400 (encoder)	nothing; it is the reference of every accuracy Δ
W8A8 QAT	fake-quantized W8A8, steps initialized by range calibration	LSQ steps and heads, straight-through gradients, same steps as FP32	QAT−FP32, the whole quantization intervention
W8A8 PTQ	the same quantizers and calibration	nothing	PTQ−FP32, quantized arithmetic without training (PTQ also lacks the control's head continuation, an untested difference)
Matched noise (encoder only)	full precision plus uniform noise at the quantizers' insertion points, sized to each calibrated step	"otherwise trained identically as a full-precision model"	noise−FP32, a perturbation matched to the calibrated step, without the quantizer (Table VII)
Deployed QAT (true INT8)	integer kernels; in the detector the convolution and linear layers are INT8, the rest FP16	nothing further	dep.−sim, integer kernels against their fake-quant simulation (Table VII); speed against the same pipeline in FP16 (Sec. IV.B; Tables IV, VI)

Table 17: The variants of the ICVES paper (Sec. III, pp. 2–3; Sec. IV.B, p. 4; Sec. VI, p. 5; Sec. VII, p. 7). AdamW is the optimizer wherever one is named (not for the encoder continuation). The detector continuations use learning rates 10^{-4} (heads) and 5×10^{-4} (scales), without weight decay. The encoder's shared start trains the head for 3000 steps at 10^{-3} and batch 256; its continuation runs 400 steps at batch 32 with "5×10^{-4} and 10^{-3}" for "the LSQ scales and head". Range calibration uses 64 clean training images (detector) or 256 clean crops (encoder).

- **Clean-only training.** No corrupted image and no corruption augmentation such as AugMix (Hendrycks et al., 2020) enter training or calibration, so any change in robustness is "emergent rather than trained in" (Sec. II, p. 2), at least with respect to synthetic blur. Whether the native weather contributes is open (L4) open.

Under these conditions QAT−FP32 at one blur level estimates the effect of the whole quantization intervention on this model, quantizers and trained steps together, up to seed and sampling noise established. Which ingredient carries it is open open. PTQ and the noise variant probe it from two sides, as separate ablations whose effects need not add up. PTQ keeps the quantizers and removes all training, the step training and the head continuation. The noise variant keeps a perturbation matched to the calibrated step (not the learned one; Section 20) and removes the quantizer. The trained parameters are not equalized. QAT trains its LSQ steps as well as the head, while the noise variant is "otherwise trained identically as a full-precision model" (Sec. VII, p. 7), which suggests the head alone; whether this asymmetry, the notes' observation, matters is untested open.

Three words, "matched", "clean", and "fine-tuned"

"Matched" names several equalities in the ICVES paper, among them training budgets (Sec. III), noise magnitude (Sec. VII), energy (Sec. VII, p. 6, not defined), and the deployment stack (Tables IV and VI); each time, read what is held equal before comparing two numbers. The legend "clean-QAT" means trained on clean images, the clean condition

evaluated on uncorrupted ones. Sec. I (p. 2) calls the encoder "fine-tuned with Learned Step Size Quantization", yet Sec. III freezes its backbone; only the LSQ steps and the head train.

18.3 Conditions, budgets, and the evaluation grid

The paper chooses blur because camera failure modes (defocus, motion, soiling, condensation) most commonly produce it (Sec. III, p. 3, citing Ceccarelli and Secci, 2023). That catalog lists blur among twenty-six failure types and does not rank them by frequency (Section 9), so "most commonly" rests on the paper's reading rather than on a count. The primary sweep is Gaussian blur, severities 1 to 5 mapped to $\sigma \in \{1, 2, 3, 4, 6\}$ after Hendrycks and Dietterich (2019), extended to $\sigma = 8$ and, for the recognizer, probed at $\sigma = 10$ and 14; defocus and motion blur check that a result is not specific to one family (Sec. III, p. 3). Detection blur acts on the frame and recognition blur on the crop, so the axes are "not directly comparable", and the paper says the same nominal blur removes "far more object detail in recognition" (Sec. IV.A, p. 4). By the count of Section 9 that holds only for objects longer than about 204 frame pixels, if crops are blurred after resizing, an order the paper does not state open.

Worked example, the budget and the grid counted

Budgets. The recognizer's shared start processes $3000 \times 256 = 768{,}000$ crops, counting repeats, and its continuation $400 \times 32 = 12{,}800$, or 1.7% of that. The detector's base fine-tune runs 40 epochs of about $821/8 \approx 103$ steps, 4100 in all, so the 600 continuation steps add about 15%. Both continuations are short adjustments of a converged start, and the encoder's trains only the 1540 head parameters and, for QAT, 48 activation steps (Section 10).

Grid. The detector is scored clean, at Gaussian $\sigma \in \{1, 2, 3, 4, 6, 8\}$, and at defocus and motion severities 3, 4, and 5, $1 + 6 + 3 + 3 = 13$ conditions (the "thirteen conditions" of Sec. IV.B), so three variants and three seeds make $13 \times 3 \times 3 = 117$ scored runs. The recognizer is scored clean, at the same six Gaussian levels, and at defocus and motion severities 4 and 5, $1 + 6 + 2 + 2 = 11$ conditions, the rows of Table VII and the natural reading of the unlisted "eleven cells" of Sec. V; the probes at $\sigma = 10$ and 14 appear only in Table VIII. All counts are the notes' arithmetic.

18.4 What the error bars and intervals cover

Every variant, PTQ included, runs three seeds, and "± is the population standard deviation over the three seeds" (Sec. III, p. 3), dividing by 3 rather than 2 (Section 13.1). Table V omits the FP32 spread (at most 0.8 points), and a printed Δ, computed from unrounded values, can differ by 0.1 from the difference of the printed entries (Exercise II.10). A t interval on one variant's seeds, such as W8A8's 70.8 ± 1.2 at $\sigma = 6$, describes that mean alone and is no test against FP32.

For the recognizer the paper adds paired, class-stratified bootstrap 95% confidence intervals over the test crops, with p-values where an interval excludes zero (Table V), controls the false discovery rate over the eleven cells at $q = 0.05$ (Benjamini & Hochberg, 1995), and reports the two strongest cells surviving the procedure of Holm (1979) (Sec. V, p. 5; Equations (66) to (68)). Pairing by crop is valid whenever both models are scored on the same crops; identical pixels make more crops agree, which narrows the interval, and keep kernel luck out of the difference (Table V's motion intervals use a fresh draw and so are not centered on the printed Δ). The intervals cover crop sampling at fixed seeds, not training-run variability, so the significance is "conditional on the three runs" (Sec. V, p. 5). The detector gets no test; with three seeds a difference below the seed spread is beneath the protocol's resolution, and the paper reads its

comparison as a bound, not an equivalence test (Sec. IV.A, p. 3; Section 13.4). The margin gap of Table VIII and the deployed detector's accuracy check use one seed (Sec. IV.B, p. 4).

18.5 Deployment, and the scope of the testbed

The QAT models also run as true integer models on custom CUTLASS kernels (Thakkar et al., 2023) that fold rescaling and activation into the epilogues and run as one CUDA graph (Section 2), because the vendor INT8 path ran slower than FP16 (Sec. IV.B, p. 4; Sec. VI, p. 5). The paper does not name the vendor, and neither do these notes. Latency is a median on one RTX 2000 Ada laptop GPU against the identical pipeline in FP16, a stack-matched baseline, with the general matrix multiplication (GEMM) layers in INT8 and the rest in FP16 (Tables IV and VI). The encoder's sweep is rerun on integer kernels and compared with its fake-quant simulation (dep.−sim, Table VII). Limitation L1 fixes the scope, one benchmark, one detector, one separate encoder, one laptop-class GPU, and no power measurement; Section 20 reads all five.

What the setting leaves unstated

The manuscript does not state the following (for the noise variant, see Section 20).

- The range-calibration statistic, whether the weight steps are learned (Sec. III and L3 suggest not), the training loss, and which encoder learning rate belongs to the scales.
- Which head the recognition PTQ variant uses, and which encoder operations beyond the 48 linear layers are quantized (GELU, LayerScale, products inside attention, biases).
- The crop margin, whether blur precedes resizing, how the detection evaluator treats ignore regions, and which modules the "heads" are.
- The per-class test counts, the number of bootstrap resamples, how the p-values are computed, and the list of the eleven cells.

Running example: the bus in the rain

The bus in the rain is now one of the $N = 1462$ test crops. At Gaussian $\sigma = 6$ it is blurred once, and the same blurred pixels go to the six FP32 and QAT models (three seeds each) and to the PTQ and noise variants. The paper's own bus in Fig. 1A is called a truck by the FP32 control and a bus by W8A8 "in all three seed pairs" (Fig. 1 caption; Sec. V, p. 5).

In the hands-on session

Notebook Part 5 builds the recognition testbed at small scale with one seed, the FP32, QAT, and PTQ variants on the budgets of Table 17, with no noise variant or bootstrap, blurring the crop at $\sigma \in \{0, 1, 2, 4, 6, 8, 10, 14\}$. Find where each variant starts, count its steps, and check that all three see the same blurred crops. The notebook warns that with one seed a point can move by about 2 points, so read its curves against the three-seed tables of Section 19. Part 5 needs a GPU runtime.

Exercise II.9: Which element answers which explanation

A skeptic offers five explanations for a QAT−FP32 gain of +4 points under blur. (a) QAT trained longer. (b) The QAT run happened to meet the rare buses early in its shuffle. (c) Under motion blur, the kernels drawn for the QAT evaluation happened to be milder. (d) QAT learned robustness from blurred examples. (e) Any perturbation of the right size

during training would have done the same. For each, name the element of the protocol or the variant that answers it, and say which one matching alone cannot answer.

Solution sketch. (a) Equal continuation steps from a shared start. (b) Identical seeds and data order, read over three seed pairs. (c) Pixel-identical inputs from the fixed corruption seed. (d) Clean-only training and calibration. (e) is not a confound of the comparison but a rival mechanism, so matching cannot answer it. The matched-noise variant is built for it (Table VII, Section 20), and even that control matches the size of the perturbation to the calibrated step, not everything else a quantizer does.

Exercise II.10: Differences from unrounded values

Table V prints FP32 and W8A8 balanced accuracies of 77.5 and 77.3 (clean), 66.8 and 70.8 (Gaussian $\sigma = 6$), 58.6 and 59.9 ($\sigma = 8$), 61.4 and 67.3 (defocus s5), and 69.2 and 71.4 (motion s4), with printed Δ of -0.2, $+4.1$, $+1.4$, $+5.9$, and $+2.3$. (a) Which rows disagree with your own subtraction? (b) Show that, with every number rounded to one decimal, the disagreement can never exceed 0.1.

Solution sketch. (a) $\sigma = 6$ (4.0 against 4.1), $\sigma = 8$ (1.3 against 1.4), and motion s4 (2.2 against 2.3). (b) Each printed accuracy lies within 0.05 of its unrounded value, so the difference of printed values lies within 0.10 of the true difference, and the printed Δ within 0.05 of it. Both sit on the 0.1 grid within 0.15 of each other, so they differ by 0 or 0.1. Nothing is wrong with the table; quote the printed Δ.

Check yourself before moving on

1. Why does the FP32 control of the recognizer receive 400 continuation steps and not zero?[a]
2. What does the fixed corruption seed add for Gaussian blur, and for motion blur?[b]
3. What do the contrasts PTQ−FP32 and noise−FP32 each remove from the QAT intervention?[c]

[a] So that QAT−FP32 excludes the effect of 400 extra head updates.

[b] Nothing for Gaussian blur, which is deterministic. For motion blur it gives every variant the same kernels, so the difference carries no kernel luck and the paired interval is as narrow as it can be.

[c] PTQ keeps the quantizers and removes all training, the step training and the head continuation. The noise variant keeps a perturbation matched to the calibrated step and removes the quantizer itself.

19 The ICVES paper, a neutral detector and a more robust recognizer

What you will be able to do after this section

Read the detector tables of the ICVES paper as a bound, and trace their one visible dip to a single rare class. Read the recognition table with its intervals and multiple-testing control, and say where the gain lives and where it does not. Describe the severity window, and separate what the tables show from what the paper infers about why it closes. Read the deployment tables by their baseline, and say which accuracy effect a deployment check can vouch for.

Section 18 built the testbed of the accepted ICVES 2026 paper (Bouguerra et al., 2026b). This section reads its detector (Sec. IV.A), recognizer (Sec. V), severity window (Sec. VII), and deployed integer models (Sec. IV.B and VI); Section 20 asks why the recognizer gains. Numbers not printed in the paper are labeled as the notes' arithmetic.

19.1 The detector, a bound and one rare class

ICVES paper, Tables I to III and Sec. IV.A, pp. 3–4

Over the thirteen conditions of Section 18, the COCO mAP@[.5:.95] of the FP32 RT-DETR detector falls from 38.4 on clean images to 4.2 at $\sigma = 8$, and the W8A8 QAT and PTQ variants track it; the Table I caption says quantization "stays within 1.7 mAP of FP32 in every condition". The largest QAT difference is -1.6 at $\sigma = 4$ (17.0 ± 1.0 against 18.6). The "single adverse entry" of Table III is Bus at $\sigma = 4$, -6.6 points, "the rarest class (24 test instances)", where every quantized run scores below every full-precision run; the three well-sampled classes stay "within 2.2 points in all" conditions. An open-vocabulary detector (Liu et al., 2024), run zero-shot, "loses accuracy at a similar relative rate" (no numbers printed). The authors read the comparison as "a bound on the difference rather than a formal equivalence test".

The bound is established as printed [established], and it is loose. Its 1.7 is set by PTQ under defocus severity 5 (9.8 against 11.5). If the unprinted FP32 population standard deviation (SD) at $\sigma = 4$ were also 1.0, the two three-seed means would differ with standard error 1.0, the 90% confidence interval (CI) of the two one-sided tests (TOST) procedure (Section 13.4) would be $-1.6 \pm 2.13 = [-3.7, +0.5]$ at four degrees of freedom, and only margins above 3.7 mAP could be certified, twice the paper's 1.7 (the notes' arithmetic and reading of Table I).

The mild-blur hint. The main metric is "dominated by localization" (Sec. I, p. 2), so Table II adds mAP50, "the loose-localization criterion". There the QAT differences at clean, $\sigma = 1$, and 2 grow from +0.2, +0.5, +0.8 to +0.8, +1.5, +2.2 (the last 3.7 times the QAT seed SD), as "expected if quantization aids classification rather than localization" (Sec. IV.A). They are "untested for significance", and loose boxes crossing IoU 0.5 leave the same signature (Section 11.3), so the classification reading is suggestive [suggestive]. From $\sigma = 4$ on, and in every defocus and motion cell but motion severity 3 (+0.1), the mAP50 differences are negative.

The dip is one class. COCO mAP is the unweighted mean of the per-class AP (Equation (54)), so a class with 24 test instances weighs as much as the car class.

Worked example, Table I is the mean of Table III

Average the four rows of ICVES Table III column by column and compare with the ΔQAT column of Table I (Bouguerra et al., 2026b, Tables I, III).

W8A8 QAT minus FP32 (mAP points)	clean	$\sigma = 2$	$\sigma = 4$	$\sigma = 6$
Car (Table III)	+0.2	+0.2	−0.4	−1.0
Truck	−0.1	+0.6	+0.7	+1.8
Person	+0.2	+0.2	+0.0	−2.2
Bus, 24 instances	+0.4	+2.1	−6.6	+0.4
mean of the four rows	+0.18	+0.78	−1.58	−0.25
ΔQAT printed in Table I	+0.2	+0.8	−1.6	−0.2

The columns agree to rounding. At $\sigma = 4$ the Bus cell contributes $-6.6/4 = -1.65$ and the three well-sampled classes together $+0.3/4 = +0.08$, so removing the rarest class removes the largest QAT difference of Table I; Bus also gives $2.1/4 = 0.53$ of the +0.8 at $\sigma = 2$. All values in this box are the notes' arithmetic on printed entries.

So the only feature of the detector sweep that looks like a quantization cost rests on 24 objects; Person (−2.2) and Car (−1.0) at $\sigma = 6$ lie inside the paper's 2.2-point band. The authors cannot separate "a genuine minority-class penalty", of the kind compression concentrates on under-represented data (Hooker et al., 2019), from sampling noise at this size, and flag it as open [open]. How often three runs fall entirely below three others by chance is Exercise II.11.

Table 18: The recognition gain as printed, from ICVES Table V (p. 4) (Bouguerra et al., 2026b, Table V). Balanced accuracy in %, three seeds; for ±, the omitted FP32 spread, and Δ from unrounded values, see Section 18. Intervals are paired, class-stratified crop-bootstrap 95% CIs (Equation (66)); the motion intervals use a fresh kernel draw and are not centered on Δ. A blank p is not printed. The last column reads Sec. V, Benjamini–Hochberg (BH) at $q = 0.05$ over eleven cells and "the two strongest surviving Holm", checked in the text.

Condition	FP32	W8A8	Δ (pp)	95% CI	p	survives
clean	77.5	77.3 ± 1.3	−0.2	$[-2.6, +2.4]$		
Gauss $\sigma = 4$ (s4)	72.6	75.1 ± 0.7	+2.5	$[+0.5, +4.7]$	.008	BH
Gauss $\sigma = 6$ (s5)	66.8	70.8 ± 1.2	+4.1	$[+1.0, +7.4]$	.004	BH, Holm
Gauss $\sigma = 8$ (s6)	58.6	59.9 ± 1.7	+1.4	$[-2.5, +5.2]$		
Defocus s4	70.1	73.7 ± 0.4	+3.6	$[+0.3, +7.0]$	.015	BH
Defocus s5	61.4	67.3 ± 1.8	+5.9	$[+2.8, +9.1]$	.0002	BH, Holm
Motion s4	69.2	71.4 ± 0.7	+2.3	$[-1.7, +4.4]$		
Motion s5	67.1	66.5 ± 0.1	−0.6	$[-3.9, +1.4]$		

19.2 The recognizer, a gain in balanced accuracy

The recognizer is the frozen DINOv2 ViT-S/14 with a linear head (Section 10) on ground-truth crops, which removes localization (Section 11.5). It is scored by balanced accuracy (Equation (57)), and its effect is ΔAcc of Equation (59), W8A8 QAT minus FP32, which the paper's tables print as ΔQAT (signs in Table 11). Table 18 transcribes the main recognition table.

Four cells have intervals that exclude zero, Gaussian $\sigma = 4$ and 6 and defocus severities 4 and 5; all four survive BH control (Equation (67)) and the two strongest survive Holm (Equation (68)). Clean crops show no detectable difference. The gain is largest under defocus (+5.9) and not significant in "the noisy motion family", whose kernel direction is drawn at random per image (Sec. III). The intervals cover crop sampling at fixed seeds, not training runs (Section 18), so the four gains are established against crop sampling on DAWN established and suggestive against training variability suggestive. The multiplicity claim checks out. Sorted, the printed p-values .0002, .004, .008, .015 lie below the BH bounds $k(0.05/11)$, from .0045 to .0182, whatever the seven unprinted values; the Holm bounds $0.05/(12-k)$ pass the first two and stop at $.008 > .0056$, if the printed four are the smallest (the notes' arithmetic).

Where the gain lives. The paper places it in "the minority road classes, whose recall improves, while the dominant car class is already near ceiling, so plain accuracy slips by up to 2.9 points" (Sec. V). Neither the condition of the 2.9 nor the per-class recalls are printed suggestive; Section 12.1 shows the pattern on synthetic counts. Fig. 5 shows six chosen objects, not a rate, that blur leaves "plainly readable to a human" and the quantized model, unlike FP32, gets right in all three seed pairs. Such corrections outnumber the reverse errors under heavy Gaussian blur but not under defocus, where the balanced gain "is carried by the minority classes" (Sec. V); no counts are printed.

Where it does not live. Confidence-thresholded reliability is "slightly worse under quantization, so a deployer relying on score thresholds should recalibrate" (Sec. V), with no metric values printed suggestive. One temperature (Equation (62)) would leave every prediction, and so the balanced-accuracy gain, unchanged. The authors call this the opposite sign from the ICML paper's calibration gains, "a different regime we do not reconcile here"; Section 16 shows how the regimes differ, and reconciling them is open open.

Running example: the bus in the rain

The bus in the rain belongs to a minority class in both halves. As one of 24 test buses, losing its box at every threshold would lower mAP, a mean over four classes, by at most

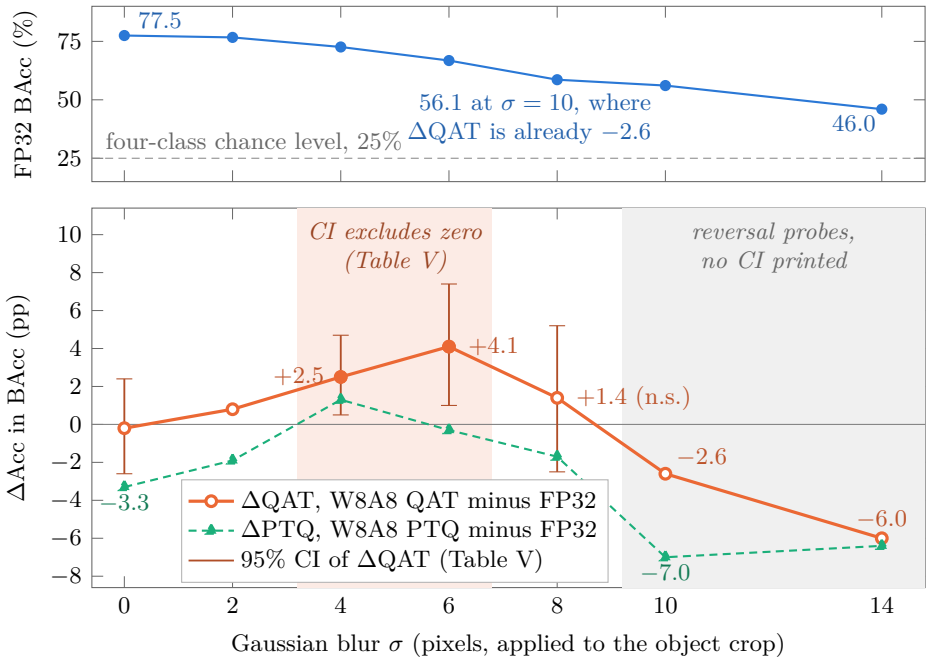


Figure 34: The ICVES severity window, transcribed from printed numbers, not redrawn from a paper figure. Top, FP32 balanced accuracy 77.5, 76.7, 72.6, 66.8, 58.6, 56.1, 46.0 at $\sigma = 0, 2, 4, 6, 8, 10, 14$, with the four-class chance level of 25%. Bottom, ΔQAT −0.2, +0.8, +2.5, +4.1, +1.4, −2.6, −6.0 and ΔPTQ −3.3, −1.9, +1.3, −0.3, −1.7, −7.0, −6.4 (Bouguerra et al., 2026b, Table VIII), with the 95% intervals of ΔQAT where Table V prints them; filled markers are the two cells whose interval excludes zero, and n.s. means not significant. The window of significant gain is two blur levels wide, and the reversal begins while the FP32 model is still more than 30 points above chance.

about $(100/24)/4 \approx 1.0$ point, more than ten of the thirteen |ΔQAT| of Table I. As a crop it is worth $100/(4N_{\text{bus}})$ points of balanced accuracy (Section 12.1); with the 100 buses of the synthetic set there (the paper prints no class counts), the +4.1 at $\sigma = 6$ would take about 16 corrected buses if all of it sat in that class (the notes' arithmetic). ICVES Fig. 1A shows a real DAWN bus at $\sigma = 6$ that FP32 calls a truck and W8A8 a bus in all three seed pairs. The kind of object that makes the detector's one adverse cell hard to read carries, by the paper's account, the recognizer's gain.

19.3 The severity window

Table VIII follows the Gaussian sweep to $\sigma = 8$ (severity 6) and adds two reversal probes, $\sigma = 10$ and 14 (Bouguerra et al., 2026b, Sec. III, Table VIII), transcribed in Figure 34. In the paper's row labels, ΔQAT is "rising" at $\sigma = 2$ and 4, at its "peak" at $\sigma = 6$, "positive (n.s.)", not significant, at $\sigma = 8$, and in "reversal" at $\sigma = 10$ and 14. ΔPTQ has no such rise; Section 20 reads what that says about training.

Retention. At $\sigma = 6$ FP32 loses 10.7 points from clean and W8A8 6.5, a retention gain of $\Delta\text{Acc}(6) - \Delta\text{Acc}(0) = 4.3$ (Equation (60); 4.2 from rounded entries), so the gain undoes about 38% of the blur damage. Retention can flatter. At $\sigma = 4$ PTQ is only 1.3 points ahead but starts 3.3 lower, so it retains 4.6 points more than FP32, against 2.7 for QAT. The paper's remark that the retention drop differs from $-\Delta\text{Acc}$ "only by the near-zero clean offset" (Sec. III) holds for QAT, not PTQ (the notes' arithmetic).

Three readings keep the figure honest. First, significance is narrower than the "severity 4–6"

shading of ICVES Fig. 4A, as the paper's Fig. 1 caption itself says ("bootstrap-significant at severities 4–5", $\sigma = 4$ and 6). Second, the reversal cells are three-seed means with no interval, a measured sign without a test. Third, the paper's reason, that the gain reverses "once the input is too degraded to classify at all" (Sec. VII, pp. 7–8), is interpretation, since at $\sigma = 10$ FP32 still scores 56.1%, 31 points above chance. The reversal is observed; its cause is open open (Exercise II.12).

A suite average hides the window (Section 12.7). Over severities 1 to 5 (Table VII) the mean ΔQAT is $8.2/5 = +1.64$, 2.5 times below the peak; with $\sigma = 8$ and the two probes it is $(8.2 + 1.4 - 2.6 - 6.0)/8 = +0.13$ (the notes' arithmetic). Hence the paper's advice to evaluate "per severity" (Sec. VIII).

19.4 Deployment, what it costs and what it can vouch for

ICVES paper, Tables IV and VI, Sec. IV.B and VI, pp. 4–6

Against "the identical pipeline in FP16" on one RTX 2000 Ada laptop GPU (stack-matched, median latency), the true-INT8 detector takes 3.42 instead of 5.33 ms at batch 1 (1.56×) and 65.7 instead of 99.0 ms at batch 16 (1.51×), with weights of 22.0 instead of 40.5 MB, "GEMM (conv. and linear) layers INT8, rest FP16, hence 1.8× not 2×". The DINOv2 encoder gains 1.36×, 1.58×, 1.85× at batch 1, 8, 32 for ViT-S/14, the only size evaluated for accuracy, and up to 2.13× for ViT-L/14; "The speed-up grows with model size" (Table VI caption).

Each ratio isolates precision in one stack, so the ICML paper's 7.72× against PyTorch FP32 (Bouguerra et al., 2026a, Table 2) is not comparable. Read backward, Equation (12) turns the size ratio $40.5/22.0 = 1.84$ into $f = 2(1 - 1/1.84) = 0.91$ of the FP16 bytes in INT8, leaving about 1.7M of the 20.2M parameters in FP16; the time ratio 1.56 needs $f \geq 0.36$ (0.72 if INT8 GEMMs are twice as fast). One GEMM's gain does not grow with its size (Section 2), so the growth in Table VI is no roofline effect; it reads as a growing GEMM share of the time, fixed overheads mattering less in wider models and larger batches suggestive. No kernel profile is printed, the detector's ratio falls slightly with batch, and 2.13× exceeds the 2× of halved bytes, which Equation (12) ($G \leq r$) allows only if the improved part gains more than 2.13×, from the INT8 arithmetic rate or from work the fused kernels remove. The f values are the notes' arithmetic; the speed-ups are established for that GPU and stack established, power unmeasured (L1).

The accuracy tables come from simulated (fake) quantization (Sec. VI; Definition 2.2). A rerun on true integer kernels vouches for an effect only if its deploy-versus-simulation drift (Equation (10)) is small compared with that effect.

Worked example, drift against effect

Encoder. The last column of ICVES Table VII, the deployed true-INT8 encoder minus its own fake-quant simulation, ranges from -1.0 ($\sigma = 6$) to $+0.9$ (defocus s5) over the eleven conditions (Bouguerra et al., 2026b, Table VII). The four significant gains of Table 18, $+2.5$ to $+5.9$, are at least 2.5 times the largest drift, and adding the drift to ΔQAT at those cells gives deployed gains of $2.5 + 0.0 = +2.5$ ($\sigma = 4$), $4.1 - 1.0 = +3.1$ ($\sigma = 6$), $3.6 - 0.3 = +3.3$ (defocus s4), and $5.9 + 0.9 = +6.8$ (s5), all positive if the two differences add (same FP32 reference, same inputs). In this sense the gain "survives into the deployed model" (Sec. VI). The small drift is established as printed established; the size of the deployed gains is suggestive, since the sums carry no interval suggestive.

Detector. No detector drift column is printed. The deployed detector scores "within -0.2 to $+2.0$ mAP of it", the FP16 pipeline, in all thirteen conditions (seed 0), a spread that

"matches the deploy-versus-simulation drift" (Sec. IV.B, p. 4). A difference of up to 2.0 exceeds every $|\Delta\text{QAT}|$ of Table I, at most 1.6, so this check shows unchanged accuracy only to within 2 mAP and cannot resolve an effect of Table I's size. The sums and ratios are the notes' arithmetic.

Does the recognition gain reach the deployed vehicle?

The abstract ends "The recognition gain therefore costs the deployed system nothing we can measure." The evidence supports less. The detector is neutral within a bound and the integer encoder keeps its simulated gain, but that encoder is "separate, unintegrated" (L1) and saw ground-truth crops, never a predicted box, within the scope of Section 18. The notes' proposed test feeds the detector's predicted boxes to the FP32 and W8A8 encoders under the same blur sweep and reports balanced accuracy per severity with the seed spread.

In the hands-on session, notebook Part 5

Compare Part 5's QAT−FP32 column with Figure 34, then rerun with two more values of `SEED` and take the spread. Part 5's closing text expects FP32 and QAT to sink together from $\sigma = 10$, whereas the paper reports QAT falling below FP32 there.

Exercises

Exercise II.11: How rare is complete separation?

Three FP32 runs and three W8A8 runs of the detector are scored on Bus at $\sigma = 4$, and quantization has no effect, so the six scores are exchangeable. (a) What is the probability that all three quantized runs score below all three full-precision runs? (b) Over four classes and thirteen conditions, treated as independent, how many adverse separations would chance give? (c) What would make the Bus result convincing?

Solution sketch. (a) One of the $\binom{6}{3} = 20$ equally likely placements puts the three quantized scores at the bottom, so $1/20 = 0.05$; if "every quantized run" also counts the three PTQ runs, it is $1/\binom{9}{3} = 1/84$. (b) $52 \times 0.05 = 2.6$ cells. The cells share runs, so this is only an order of magnitude, but one such cell is unremarkable. (c) More buses (a second dataset, which L1 names as future work), more seeds, and a per-class test fixed in advance with multiplicity control, since one bus moves Bus recall by 4.2 points.

Exercise II.12: Reading the reversal

From ICVES Table VIII, compute the W8A8 QAT balanced accuracy at each σ as FP32 plus ΔQAT, and each model's loss from one σ to the next. From which step on does QAT lose faster? Does "too degraded to classify at all" describe FP32 at $\sigma = 10$? Propose one measurement that could support or refute the paper's explanation.

Solution sketch. QAT gives 77.3, 77.5, 75.1, 70.9, 60.0, 53.5, 40.0. FP32 loses 0.8, 4.1, 5.8, 8.2, 2.5, 10.1 per step; QAT gains 0.2, then loses 2.4, 4.2, 10.9, 6.5, 13.5, faster from $\sigma = 6 \to 8$ on (from 8 to 10 FP32 loses only 2.5 while ΔQAT swings by 4.0). FP32 at $\sigma = 10$ is 31 points above chance, still classifiable on average. Measure per-class recall at $\sigma = 8$, 10, 14 and the share of crops FP32 reads correctly but QAT does not; a reversal on such crops refutes the explanation in its literal form.

Check yourself before moving on

1. Where does the −1.6 mAP difference of ICVES Table I at $\sigma = 4$ come from?[a]
2. Name three places where the quantized recognizer is not better than its FP32 control.[b]
3. The deployed encoder differs from its simulation by at most 1.0 pp, the deployed

detector from its FP16 reference by up to 2.0 mAP. Which accuracy effect can each check confirm?[c]

[a] Almost entirely from Bus, 24 test instances. mAP is the class mean, and Table III gives $-6.6/4 = -1.65$ from Bus and $+0.08$ from the other three classes together.

[b] Plain accuracy (a slip of up to 2.9 points), confidence-thresholded reliability (slightly worse), motion blur (-0.6 at severity 5), and the reversal at $\sigma = 10$ and 14 (-2.6, -6.0). Clean accuracy is unchanged within its interval.

[c] The encoder check confirms the window gains of $+2.5$ to $+5.9$, at least 2.5 times the drift. The detector check cannot resolve a difference of Table I's size, at most 1.6 mAP, below its 2.0.

20 The ICVES paper, the shared subspace and its controls

What you will be able to do after this section

- Read ICVES Fig. 6 with the per-rank energy e_k and ratio ρ_k of Section 8, and say what "concentrate in the high-rank tail" and "a common line" can and cannot mean.
- Compute a profile cosine and its shuffled-rank null by hand, and prove why any two profiles that rise with rank sit at the top of their null.
- Explain why the paper calls the spectral overlap necessary but not sufficient, and weigh each of its three exclusions.
- Read the matched-noise control and the post-training variant, state what they rule out and how firmly, and name the open candidates and the paper's five limitations.

Section 19 established what the ICVES encoder does. Sec. VII of the ICVES paper asks why training against quantization noise on clean images should confer robustness to blur (Bouguerra et al., 2026b, Sec. VII, p. 6), and answers with a spectral measurement and a set of controls. This section reads both with the tools of Sections 7 and 8.

20.1 Two residuals on one basis

ICVES paper, Sec. VII, p. 6

With X_{fp} the pooled FP32 features of the clean test set and V its right singular basis, the paper projects the quantization residual $R_q = X_{\text{W8A8}} - X_{\text{fp}}$ and the blur residual $R_b(\sigma) = X_{\text{fp}}(\text{blur}_\sigma) - X_{\text{fp}}$ (the names are the notes', the sign the paper's, Table 11) onto V and compares their per-rank energy with the signal's. "Relative to the clean signal, both residuals concentrate in the high-rank tail of the representation, and their per-rank energy profiles fall along a common line rank by rank (Fig. 6A,B; Table VIII). Spectrally, quantization noise resembles a scaled-down moderate blur. Clean-image QAT forces the features to stay usable despite quantization noise in that tail, and because blur perturbs the same tail, the learned invariance plausibly transfers" (Bouguerra et al., 2026b).

In the notation of Definition 8.1, $X = X_{\text{fp}}$ holds the features of the $N = 1462$ clean test crops (Sec. III), and every curve of Fig. 6 is an e_k or a ρ_k on the fixed FP32 basis (Section 7.5), whose plotted rank axis starts at 0 ($k = j + 1$). With the backbone frozen (Sec. III, p. 3), R_q is a pure quantization residual, the rounding and clipping of the 48 quantized layers at the learned steps, with none of the weight drift of the ICML QAT runs (Table 8). Centering, means or sums over crops, and the pooled token are unstated; centering changes e_k, and means versus sums do not change ρ_k.

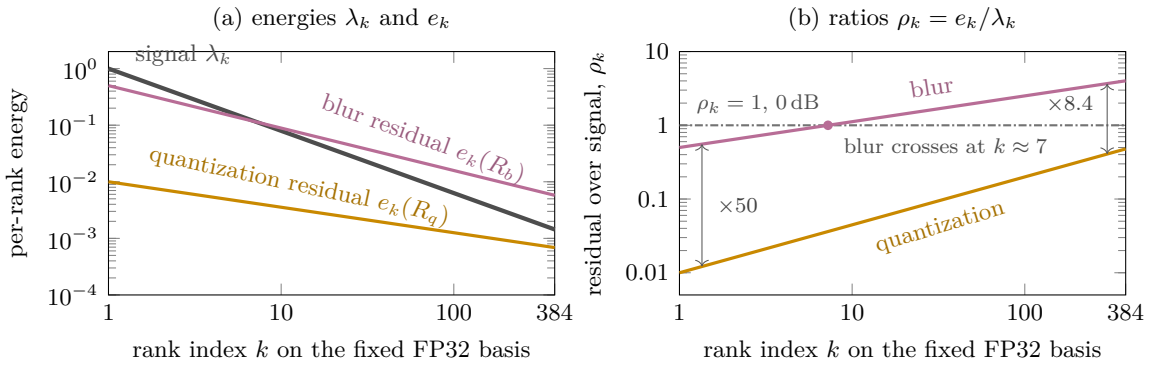


Figure 35: Two residuals on one FP32 basis (schematic, synthetic model with the qualitative shape of ICVES Fig. 6A, no paper numbers). (a) Signal $\lambda_k = k^{-1.1}$, quantization residual $e_k(R_q) = 0.01\,k^{-0.45}$, blur residual $e_k(R_b) = 0.5\,k^{-0.75}$. (b) Their ratios. Both rise toward the tail, which is all that "concentrate in the tail" means here. The quantization ratio climbs from 0.01 (20.0 dB) to 0.48 (3.2 dB) and never reaches the signal; the blur ratio crosses it at $k \approx 7$. The factor between the two falls from 50 at the head to 8.4 in the tail, so the blur profile is not a scaled copy of the quantization profile.

Read off Fig. 6A (no per-rank values are printed), the quantization residual is the colored residual of Section 8.4, largest at the head but decaying more slowly than the signal and staying below it, so its ρ_k rises toward the tail with no 0 dB rank. A quantizer after signal-shaping layers already gives this shape (Section 5); whether that explains the ViT residual is open **open**. The blur residual at $\sigma = 6$ lies above the signal over most ranks, so its ratio rises and exceeds 1 **established**, as a description of the plot. "Concentrate in the high-rank tail" is thus relative to the signal, as Figure 35 draws from a synthetic model.

Fig. 6B plots, one point per rank, $\rho_k(R_b(6))$ against $\rho_k(R_q)$ on log-log axes, with a dashed reference line the caption does not define. On such axes (Section 7) a slope-one line means $\rho_k(R_b) = c\,\rho_k(R_q)$, a scaled copy, and a shallower cloud a blur ratio that grows more slowly with rank. The head points lie above the dashed line and the tail points below it, so the cloud is shallower (a reading of the plot; no slope is printed). "A common line" and "scaled-down moderate blur" therefore mean the same ordering of ranks and a similar shape, not a scaled copy **suggestive**. The annotated cosine of 0.69 cannot settle this alone, since the paper does not say which profile it compares (Section 20.2).

20.2 The profile cosine and its shuffled-rank null

ICVES paper, Fig. 6B–C and Table VIII, p. 7

Fig. 6B is annotated "per-rank cosine 0.69", the value Table VIII prints for $\sigma = 6$. Fig. 6C plots the "profile cosine with quant." of blur, sensor noise and contrast against their residual energy, with "the quantization residual itself" at "the low-energy end where the match is tightest" (Bouguerra et al., 2026b). The paper defines neither the cosine nor the null.

Definition 20.1: Profile cosine and shuffled-rank null

Let $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^d$ be two profiles indexed by rank, for instance $a_k = e_k(R_q)$ and $b_k = e_k(R_b)$, or $a_k = \rho_k(R_q)$ and $b_k = \rho_k(R_b)$. Their *profile cosine* is

$$\cos(\boldsymbol{a}, \boldsymbol{b}) = \frac{\sum_{k=1}^{d} a_k b_k}{\|\boldsymbol{a}\|\,\|\boldsymbol{b}\|}.$$

For a permutation π of $\{1, \dots, d\}$ write $\boldsymbol{b}_\pi = (b_{\pi(1)}, \dots, b_{\pi(d)})$. The *shuffled-rank null* is the distribution of $\cos(\boldsymbol{a}, \boldsymbol{b}_\pi)$ over permutations drawn uniformly at random, and an observed cosine lies above the null when it exceeds, say, the null's 97.5th percentile.

These are the standard definitions, not the paper's, applied to vectors indexed by rank (Section 7). They leave open which profile enters (energies, ratios, logarithms, or normalized versions); the paper does not say, and the notebook's Part 5.2 uses raw energies.

Proposition 20.1: Co-monotone profiles beat every shuffle

Let $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^d$ be nonzero and *similarly ordered*, meaning $a_j > a_k$ implies $b_j \geq b_k$ (for instance, both nondecreasing in k). Then

1. $\cos(\boldsymbol{a}, \boldsymbol{b}) \geq \cos(\boldsymbol{a}, \boldsymbol{b}_\pi)$ for every permutation π, so the observed cosine is the maximum of its own shuffled-rank null;
2. $\cos(\boldsymbol{a}, \boldsymbol{b}) = 1$ if and only if $\boldsymbol{b} = c\,\boldsymbol{a}$ for some $c > 0$.

Proof. A permutation leaves $\|\boldsymbol{b}\|$ unchanged, so only the numerators need comparing. Suppose a pairing π has $a_j > a_k$ but $b_{\pi(j)} < b_{\pi(k)}$. Exchanging the two values of $\boldsymbol{b}$ changes the sum by $(a_j - a_k)(b_{\pi(k)} - b_{\pi(j)}) > 0$. Each exchange removes at least one such inversion and never lowers the sum, so repeated exchanges reach a pairing with no inversion. Every pairing without inversions gives the same sum as the identity, since it can differ from it only by exchanges between tied values, and this proves item 1 (the rearrangement inequality). Item 2 is the equality case of the Cauchy-Schwarz inequality. □

The proposition is proved on the page established; applying it to the paper is the notes' step. Beating the null certifies only that two profiles are ordered alike (any two rising ratio profiles, or any two falling energy profiles, pass), and a cosine below 1 only that the two vectors compared are not proportional. Jagged but nearly co-monotone real profiles sit near the top of their null.

Worked example, four ranks and three candidate corruptions

Take $\lambda = (16, 4, 1, 0.25)$ and $e(R_q) = (0.16, 0.08, 0.04, 0.02)$, so $\rho(R_q) = (0.01, 0.02, 0.04, 0.08)$, doubling at each rank. $A = (1, 2, 4, 8)$ is a scaled copy (slope 1 on log-log axes), and cos = 1. $B = (1, 1.5, 2.25, 3.375)$ grows by 1.5 per rank, a log-log slope of $\log 1.5 / \log 2 = 0.585$. Its numerator is $0.01\,(1 + 3 + 9 + 27) = 0.40$, the norms are $0.01\sqrt{85} = 0.0922$ and $\sqrt{19.70} = 4.439$, and cos = 0.977. $D = (2, 0.5, 3, 1)$, which does not rise, gives 0.661. Over the 24 permutations, B's null runs from 0.535 to 0.977 with mean 0.745, so the observed value is its maximum (Proposition 20.1); D's observed 0.661 lies below its null mean of 0.700. The profile choice also moves the number. On the energies, $e(R_q)$ against $\lambda \odot B$, cos = 0.989. On $\log_{10}$ profiles B gives -0.657, and so does the exact scaled copy A, because every $\log_{10} \rho_k(R_q)$ is negative. After centering the logarithms both give exactly 1. One pair thus reads 0.977, 0.989, -0.657 or 1, and a scaled copy can print -0.657, so a cosine below 1 excludes proportionality only for the profile it was computed on.

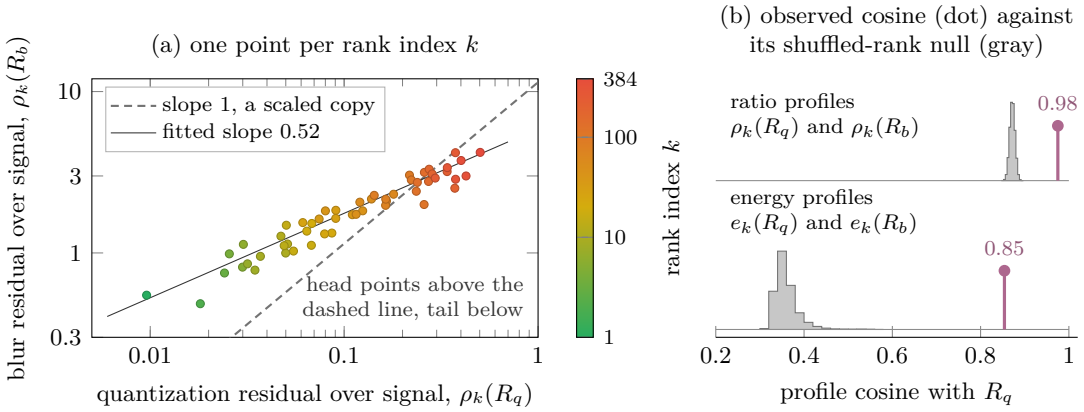


Figure 36: A common ordering, not a common scale (schematic, synthetic model of Figure 35 with a 12% log-normal jitter per rank). (a) Blur ratio against quantization ratio, one point per rank, 53 of 384 ranks shown, colored from head (green) to tail (red). The cloud has slope 0.52, shallower than the slope-one line of a scaled copy, so head points lie above that line and tail points below it. (b) Profile cosine over all 384 ranks against 20 000 random shuffles of the blur profile. On energies the observed 0.85 sits far above a null centered at 0.36; on ratios the observed 0.98 sits above a null centered at 0.87. Where the null lies depends on the profile chosen, and a pair that rises together clears it either way (Proposition 20.1).

20.3 Necessary, not sufficient

ICVES paper, Sec. VII, pp. 6–8

"As a control, sensor noise and contrast at matched energy co-align with the quantization profile comparably, far above a shuffled-rank null (Fig. 6C). The spectral overlap is therefore a necessary condition, not a fingerprint." The cosine "falls as the gain rises (Table VIII), so the overlap sets where transfer is possible rather than how much". Heavy sensor noise and pixelation "move only slightly positive" (p. 8; no numbers printed) (Bouguerra et al., 2026b).

Three observations make the overlap a precondition rather than an explanation. Other corruptions share it, since sensor noise co-aligns about as well as blur yet moves the gain only slightly. It does not track the gain, since the cosine falls with σ, 0.98, 0.83, 0.69, 0.62, 0.59, 0.58 at $\sigma = 2$ to 14, while ΔAcc rises from +0.8 to +4.1 at $\sigma = 6$ and then reverses to −6.0 (Table VIII). And it is "a co-alignment measurement rather than an intervention" (p. 7) that never changes the overlap to watch the gain respond (Section 13), so even necessity is part of the account rather than a tested result open. The null is also weak by construction (Proposition 20.1), the energy matching of Fig. 6C is not described, and the cosine may be coarse, since blur, sensor noise and contrast act at very different image frequencies (Figure 21) yet co-align comparably, and $1/k$ against $1/k^2$ over 384 ranks already gives 0.90 (the notes' reading open). The shared subspace is suggestive as a description of where the residuals sit and open as a cause.

20.4 Three simpler explanations and how firmly each is excluded

The paper sets aside three alternatives (Bouguerra et al., 2026b, Sec. VII, p. 7).

Input smoothing. If QAT had made the encoder a low-pass filter on images (Section 9), blur would remove less of what the encoder reads. The paper reports that "the quantized and

Condition	FP32	ΔQAT	Δnoise	ΔPTQ
clean	77.5	−0.2	+1.8	−3.3
Gaussian $\sigma = 1$	76.6	−0.6	+1.0	·
Gaussian $\sigma = 2$	76.7	+0.8	+0.2	−1.9
Gaussian $\sigma = 3$	74.6	+1.4	−0.6	·
Gaussian $\sigma = 4$	72.6	+2.5†	−1.2	+1.3
Gaussian $\sigma = 6$	66.8	+4.1†	−0.9	−0.3
Gaussian $\sigma = 8$	58.6	+1.4	−1.5	−1.7
Defocus s4	70.1	+3.6†	−1.3	+2.6
Defocus s5	61.4	+5.9†	+0.6	+5.1
Motion s4	69.2	+2.3	+1.0	·
Motion s5	67.1	−0.6	−2.0	·

Table 19: Three interventions against one FP32 control (balanced accuracy on DAWN ground-truth crops, 3 seeds, points). FP32, ΔQAT and Δnoise from ICVES Table VII; ΔPTQ from Table VIII and Sec. VII, p. 7 (defocus); · not printed; †bootstrap 95% interval excludes zero (Table V), none printed for Δnoise or ΔPTQ (Bouguerra et al., 2026b). Where QAT gains, the noise variant does not; PTQ keeps most of the defocus gain but no sustained Gaussian one.

full-precision models have near-identical frequency sensitivity, far too close to account for the gain", without printing the method or the numbers suggestive.

Boundary flattening. Table VIII prints a margin gap (the text calls it class-balanced) from a single-seed probe, +0.13, +0.00, +0.21, −0.39 at $\sigma = 0, 4, 8, 14$, with no entry at $\sigma = 2, 6, 10$. The paper does not define it; the notes' reading is the class-balanced mean of the margin of Section 5, W8A8 minus FP32. The paper reads it as "positive through the window" and negative only past it, "the sign structure corruption-invariant features imply and a uniformly flatter boundary does not". The row fits loosely, with a zero inside the window at $\sigma = 4$, a positive clean entry, no monotone decay despite the caption, one seed and no interval. This is the weakest exclusion suggestive.

Rank reduction. Fig. 6A prints "effective rank unchanged: 90.4 vs 90.7", from which the paper concludes that "the model gains *invariance* to the shared tail rather than discarding dimensions". Neither the formula (Equations (34) and (35) are the standard candidates) nor which value belongs to which model is stated suggestive. Yet the value is informative. Independent noise that added the synthetic $e_k(R_q)$ of Figure 35 to each λ_k would raise the entropy effective rank from 67.9 to 81.1, and Fig. 6B shows a real ratio at least as large as that model's (0.02 to about 0.6, against 0.01 to 0.48). An unchanged value points to error anticorrelated with the signal (Proposition 8.2) or to another formula or normalization, the notes' inference open. ICML Fig. 9 is a different measurement of a different intervention, so the two do not conflict (Section 17).

20.5 The matched-noise control and the untrained variant

ICVES paper, Sec. VII and Table VII, p. 7

"A fourth variant injects uniform noise matched to each quantizer's calibrated step size, at the same insertion points, and is otherwise trained identically as a full-precision model." The two effects "separate cleanly (Table VII)". The noise variant "improves clean accuracy, the signature of a regularizer, yet stays flat-to-negative exactly where QAT gains", and "the effect therefore requires properties of the quantizer beyond matched noise magnitude" (Bouguerra et al., 2026b).

Table 19 lines up the three interventions. The noise variant gains most on clean crops (+1.8 against −0.2 for QAT), the paper's signature of a regularizer, and scores −1.2, −0.9, −1.3 and +0.6 in the four cells where QAT gains significantly (+2.5, +4.1, +3.6, +5.9). Magnitude alone, at the calibrated step, does not reproduce the gain suggestive, which stays suggestive for three reasons.

- The noise is matched to the *calibrated* step, while LSQ goes on to learn the step and with it the clip range (Equation (14)), by an unreported amount.
- Δnoise is a three-seed mean with no interval, while the W8A8 seed spreads of Table V reach ± 1.8 points; the pattern across cells carries the evidence.
- The noise range, whether it reaches the weights or stays on at evaluation, and what the variant trains are unstated (Section 18). Uniform on $[-s/2, s/2]$ is the notes' reading (Definition 4.1). If its weights stay unrounded, it also lacks the fixed weight error of W8A8, a gain that a retrained head can undo (Exercise I.12).

ICML App. D.4 points the same way, contrasting quantization with weight dropout at 10, 20 and 30% (Bouguerra et al., 2026a, Fig. 14), an unmatched noise in one cumulative-energy comparison suggestive (Section 17).

The PTQ column tests the slogan "Training does the work, not the arithmetic" (Abstract; Sec. VII). Without training the encoder loses 3.3 points clean and shows "no sustained Gaussian gain", "though it inherits most of the defocus gain (+2.6/+5.1 vs +3.6/+5.9), so the arithmetic is not wholly inert" (p. 7). Training does the work for Gaussian blur and only in part for defocus suggestive; the ICML paper uses "arithmetic" differently (Section 21).

Running example: the bus in the rain

Return to the six activations of the bus crop at one quantized layer (Section 3), $x = (2.54, -0.731, 0.0093, -1.2468, 0.445, -0.0162)$, with the calibrated step $s = 0.02$ and clip range $[-2.56, 2.54]$. Three perturbations reach the next layer.

- *Calibrated quantizer.* Errors $(0, -0.009, -0.0093, +0.0068, -0.005, -0.0038)$, mean square 4.2×10^{-5}. The third entry sits in the dead zone and its error is minus its value.
- *Matched noise.* Uniform on $[-0.01, 0.01]$, mean square $s^2/12 = 3.3 \times 10^{-5}$ in expectation, redrawn at every pass and blind to x.
- *Learned quantizer.* Suppose (synthetic) LSQ shrinks the step to $s = 0.016$, so the clip range becomes $[-2.048, 2.032]$. Now $2.54/0.016 = 158.75$ is clamped to 127, and the errors are $(-0.508, -0.005, +0.0067, -0.0012, +0.003, +0.0002)$. The five unclipped entries have mean square 1.6×10^{-5}, close to $s^2/12 = 2.1 \times 10^{-5}$; the clipped one alone carries 99.97% of the error energy.

The control matches the first line in expected size and nothing else (Section 4.4). It lacks the dead zone (signal dependence), the repetition of the same error on every pass (determinism), and the -0.508 of the learned-quantizer line (clipping by the learned step), the three candidates below.

20.6 What remains open

Open questions on clipping (L5) and bit width (L2), the paper's own

"Activation clipping set by the learned step size is the leading candidate, and we did not isolate it here" (Bouguerra et al., 2026b, Sec. VII, p. 7). At fixed b the step sets both the rounding grid and the clip range (Equation (14)), and the matched noise imitates only the first; clipping without rounding and rounding without clipping, at the learned step, would separate them (Section 21) open. The paper also predicts that "coarser activation quantization ought to widen" the window, and that a simulated bit-width sweep "would falsify the account if the window did not move" (p. 8). A sweep with LSQ also moves the

clip range; Exercise II.14 separates size from clipping open.

Open question on three further candidates, the notes' own

A research note written for this tutorial, not the paper, adds three differences between a quantizer and matched noise open, each with a test. *Determinism*, since rounding repeats the same error for the same crop and the noise is redrawn (freeze one draw per crop). *Signal dependence*, since a gain $g_k < 1$ of Equation (39) at any rank of R_q means attenuation, which independent noise cannot produce (Proposition 8.2). *Trained capacity*, since QAT trains its steps as well as the head (Esser et al., 2020) and the noise variant may train the head alone (give it one trainable scale per insertion point that cannot change the noise size). Section 21 designs each experiment.

20.7 The paper's own limits

The paper lists five limitations (Bouguerra et al., 2026b, Sec. VIII, p. 8). The scope is one benchmark, detector and separate encoder, modest car-dominated test sets (24 Bus instances), one laptop GPU and no power measurement (L1). Synthetic corruptions on already adverse-weather imagery leave open whether real degradation seeds the invariance, which a clean-weather control would settle (L4); Section 18 reads both. The bit-width prediction is untested but simulable (L2). The precise claim is invariance to a shared subspace, with the effective rank unchanged and the effect in the learned activation scales and the head (L3). The account is correlational, with noise regularization excluded and clipping not (L5). Sec. IX states the mechanism more causally (training for tolerance to quantization noise "confers tolerance" to blur); these notes follow Sec. VII and L5.

In the hands-on session, notebook Part 5.2

Part 5.2 (GPU only) prints the cosine between the raw energy profiles of the quantization residual and the blur residuals at $\sigma \in \{2, 6, 14\}$ on the centered FP32 basis. Add the shuffled-rank null (1000 permutations) and the cosine on the ratios ρ_k. The closing cell calls rounding noise a "surrogate" for the corruption, quoting the cosine at $\sigma = 2$, where the ICVES gain is small (+0.8). Read "surrogate" as a shared direction, not a shared effect: this section shows the overlap is necessary but not sufficient. Exercises (c), `a_bits=6`, and (d), pixel noise, test prediction L2 and the sensor-noise comparison of Fig. 6C, and Exercise 3.2 is a small matched-noise control.

Exercise II.13: Slopes and cosines

Use the synthetic ratios $\rho_k(R_q) = 0.01\, k^{0.65}$ for $k = 1, \ldots, 384$, and let $\rho_k(R_b) = c\, \rho_k(R_q)^\beta$. (a) Show that the ratio-profile cosine does not depend on c. (b) Evaluate it for $\beta = 1$, 0.75, 0.5, 0.25 and 0. (c) What do the values say about how much a ratio-profile cosine can reveal, and what would a printed value of 0.69 then suggest about the profile a paper used?

Solution sketch. (a) The cosine is scale-invariant (Section 7), so "scaled-down" is invisible to it. (b) 1.000, 0.997, 0.986, 0.963 and 0.920. (c) Even $\beta = 0$, a flat blur ratio with no shape in common, gives 0.920 and equals its own null, so a ratio cosine near 1 reveals little. Here 0.69 needs opposite trends ($\beta = -0.5$ gives 0.714) or very jagged profiles, so a printed 0.69 suggests another profile (energies, logarithms, centered versions), which ICVES does not state open.

Exercise II.14: Designing the bit-width sweep

Take a per-tensor activation quantizer whose calibrated clip range tops out at 2.54, as in the running example. (a) Keep the range fixed and lower b from 8 to 6 to 4 ($q_{\max} = 127$, 31, 7). Find the step and the rounding variance relative to 8 bits. (b) Keep the step fixed at 0.02 and lower b the same way. Find the clip range and the error on the activation 2.54. (c) Which sweep tests the size of the residual and which tests clipping? What outcome of each would falsify the account of L2?

Solution sketch. (a) $s = 2.54/127 = 0.02$, $2.54/31 = 0.0819$ and $2.54/7 = 0.363$; the variance grows by 16.8 (12.2 dB) and 329 (25.2 dB), with no new clipping. (b) The clip range becomes $[-0.64, 0.62]$ and $[-0.16, 0.14]$, and the error on 2.54 is -1.92 and -2.40, with the rounding variance unchanged. (c) Sweep (a) varies magnitude alone, sweep (b) clipping alone. A window that widens under (a) supports the magnitude reading of L2, one that moves only under (b) points to clipping, and if neither moves it the account fails as L2 says. Retraining LSQ at each b moves both.

Check yourself before moving on

1. What does "both residuals concentrate in the high-rank tail" mean for a quantization residual whose energy is largest at the head?[a]
2. Give two reasons from the paper why the overlap is necessary but not sufficient.[b]
3. What does the matched-noise control rule out, how firmly, and which candidate is left?[c]

[a] Its ratio to the signal, ρ_k, rises toward the tail; ICVES Fig. 6A–B.

[b] Sensor noise and contrast co-align comparably (Fig. 6C) while noise gains only slightly, and the cosine falls as the gain rises (Table VIII).

[c] Noise magnitude alone at the calibrated step, suggestively (three-seed means without intervals, calibrated rather than learned step, unstated range). Activation clipping by the learned step (L5) is left, not isolated.

21 Reading the two papers together

What you will be able to do after this section

- Read ICML Fig. 8c and ICVES Fig. 6A–B as two measurements of one object, the per-rank ratio ρ_k on a fixed FP32 basis, and say where the two papers' accounts part.
- Sort the claims of both papers into established, suggestive, and open, with locators.
- Say where a quantization robustness gain holds and where it reverses, and turn that into evaluation practice.
- Design a controlled experiment that could settle one of the open questions.

After Sections 15 to 20 read each paper on its own terms, this section reads the ICML paper (Bouguerra et al., 2026a) and the accepted ICVES 2026 paper (Bouguerra et al., 2026b) side by side.

21.1 One object, two papers

Both papers compute the principal directions $\boldsymbol{v}_k$ of clean full-precision features and ask how much of a residual, between two models or between a clean and a corrupted input, lands on each direction compared with the signal's variance there, the ratio $\rho_k = e_k/\lambda_k$ of Definition 8.1. ICML plots it in decibels, $\mathrm{SQNR}_k = -10 \log_{10} \rho_k$ (Equation (38)), for the FP32-to-quantized residual of penultimate features (App. D, Eq. (7); Fig. 8c). ICVES plots the energies (Fig. 6A)

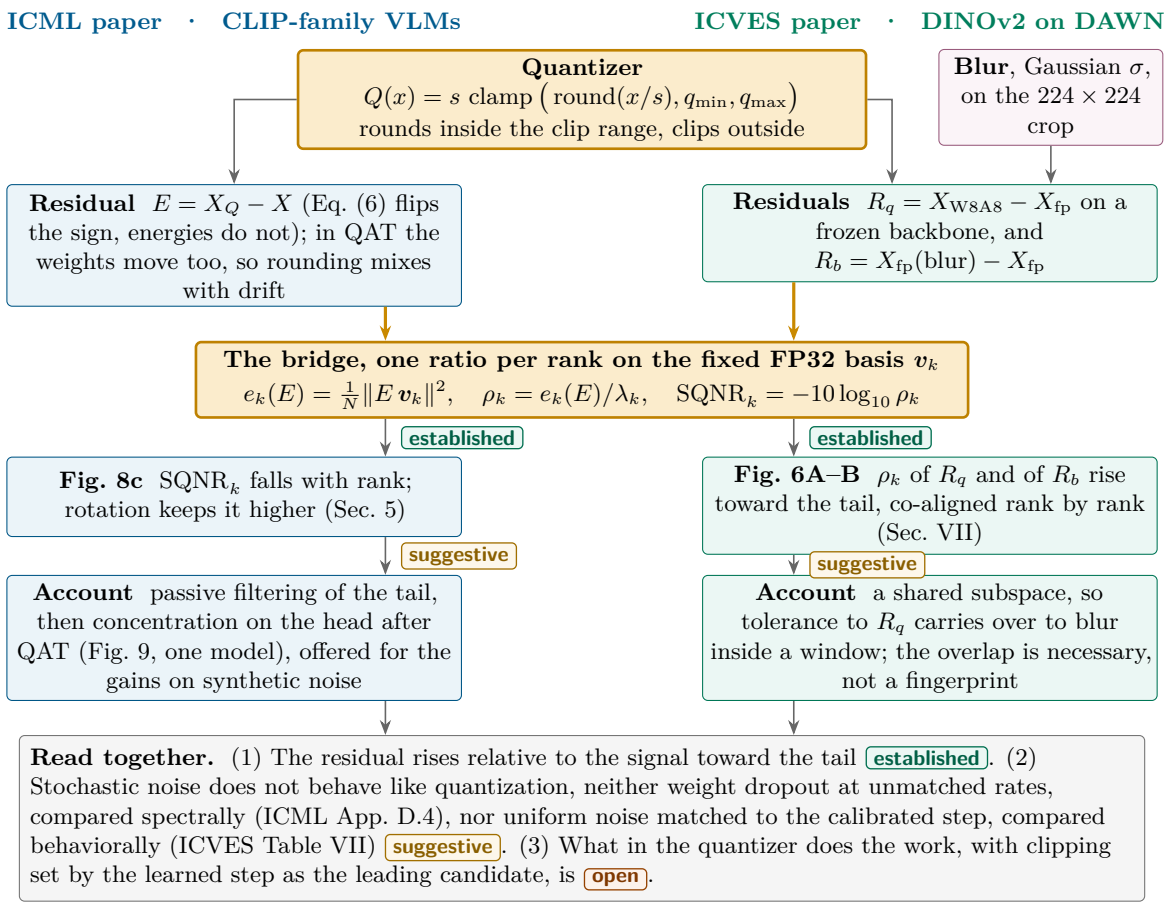


Figure 37: The bridge between the two papers. Both chains start from the same kind of uniform quantizer and pass through the per-rank ratio ρ_k on the fixed FP32 basis (Definition 8.1), which ICML reports as SQNR_k and ICVES as the ratio itself. The tag on each arrow says how far that step is supported. The measurements describe plotted curves established, each paper's account goes beyond them suggestive, and the joint reading holds one established, one suggestive, and one open claim. The residuals differ, since ICML's QAT moves the weights and ICVES freezes them.

and the ratios (Fig. 6B) of the quantization residual R_q and the blur residual R_b of a frozen DINOv2 encoder (Figure 37).

The joint reading at the foot of the figure rests on three observations. First, relative to the signal, the quantization residual grows toward the spectral tail in both papers (ICML Sec. 5; ICVES Sec. VII) established, as descriptions of the plotted curves. Second, the ICVES residual is not a flat floor but decays with rank, more slowly than the signal (Fig. 6A), so the flat floor of Proposition 8.1 is a model that neither paper measures; whether the ICML residual is flat is open (Section 17) open. Third, stochastic noise does not behave like quantization in either paper, but the controls differ. ICML compares the spectral signature of weight dropout at 10, 20, and 30%, rates not matched to the quantization error (App. D.4, Fig. 14). ICVES trains with uniform activation noise matched to the calibrated step, which does not reproduce the blur gain (Table VII). Only the second control is behavioral and matched in size suggestive.

21.2 Where the two accounts part

Table 8 lists the differences that change what a result can mean. ICML's quantization-aware training (QAT) updates the weights (App. C.2), so its residual mixes rounding with training

drift; ICVES freezes the backbone and trains only the step sizes of learned step size quantization (LSQ) and the classifier head (Sec. III; L3). ICML scores against the pretrained FP32 model, ICVES against an FP32 control given the same continuation steps. The two report probability calibration moving in opposite directions, in different regimes (Section 16), and reconciling the signs is open. Neither varies the activation width.

One apparent conflict dissolves once you see that the two papers measure different things. On a basis recomputed from the quantized features of one CLIP model, ICML finds discriminative information concentrating in the spectral head (Fig. 9), and PTQ, which retrains no weights, gains in the head band as well (Fig. 9A). ICVES summarizes the spectrum of a frozen DINOv2 encoder in one number, the effective rank, 90.4 against 90.7 (Fig. 6A), which a modest shift of energy toward the head need not move much. The ICML abstract's "rely more heavily on robust, low-rank features" and the ICVES invariance agree if the first is read as reduced sensitivity to the tail, not as its removal.

Resolving an ambiguity about "arithmetic"

ICML uses the word for numerical artifacts of low precision (App. D.3, p. 18). In the ICVES slogan "training does the work, not the arithmetic" (Abstract; Sec. VII) it means the untrained quantized forward pass, the post-training quantization (PTQ) variant. These notes name the control instead.

Worked example, what training the quantizer adds

The ICVES PTQ variant uses the same quantizers and range calibration as QAT with no training (Sec. III, VII). Both are scored against the same FP32 control, so $\Delta\mathrm{Acc}_{\mathrm{QAT}} - \Delta\mathrm{Acc}_{\mathrm{PTQ}} = \mathrm{Acc}_{\mathrm{QAT}} - \mathrm{Acc}_{\mathrm{PTQ}}$ is what 400 steps of training the step sizes and the head add, whatever the control. In balanced-accuracy points (Bouguerra et al., 2026b, Table VIII, Sec. VII) it is +3.1 clean, +1.2, +4.4, and +3.1 at Gaussian $\sigma = 4, 6, 8$, and +1.0 and +0.8 at defocus s4, s5. At $\sigma = 6$ the untrained quantizer carries none of the gain (-0.3 against $+4.1$); under defocus it carries $2.6/3.6 = 72\%$ and $5.1/5.9 = 86\%$ of it, so the arithmetic "is not wholly inert" (Sec. VII). "Training does the work" fits Gaussian blur, not defocus suggestive, a reading of rounded values with no printed interval for any PTQ cell.

21.3 What is established, suggestive, and open

Table 20 sorts the claims that matter for the tutorial's question with the tags of Section 14, following each paper's own hedges. Established means measured under the paper's protocol, and says nothing about other models or benchmarks.

Every established row is a measurement under one protocol, and every mechanistic row is suggestive or open. The controls closest to isolating a cause are negative, showing what quantization is *not*, namely stochastic noise, unmatched in ICML and step-matched in ICVES. The noise floor of Section 8 explains why the two residuals can co-align; it does not explain why training through one buys tolerance to the other.

21.4 Where the effect holds, and what to do about it

The gain has an address. In ICML it appears for synthetic corruptions among successful W8A8 runs (Fig. 7) and, in one TensorRT engine, under blur, where INT8 loses 2.21 to 4.92 points less accuracy than FP16 (Table 3, "Added rob."). In ICVES it appears only in balanced accuracy, only for Gaussian and defocus blur, and only inside a window (Figure 34), while plain accuracy slips by up to 2.9 points and confidence-based reliability gets slightly worse (Sec. V). The papers' σ values cannot be compared, since σ counts pixels of the image blurred (Section 9). Five practices follow, most from ICVES Sec. VIII and ICML App. B.1.

Table 20: The claims of the two papers by evidential tier, with locators in Bouguerra et al. (2026a) (ICML) and Bouguerra et al. (2026b) (ICVES). Values as printed; the tiers are the notes' classification, following the papers' hedges; "notes" marks the notes' own.

Claim	Evidence (locator)	Tier
Of 6720 significant W8A8 runs, 54.8% ($N = 3682$) degrade both accuracy and calibration and 7.8% ($N = 527$) improve both	ICML Fig. 1	established
Among successful W8A8 runs, synthetic-corruption robustness rises (+8.9% relative gap change, $n = 2182$), covariate-shift robustness falls slightly (−0.3%, $n = 1632$)	ICML Fig. 7	established
Simple PTQ amplifies spurious reliance, sharply at 4 bits (ΔRSG 2.6 at W8A8, 12.5 at W4A8); Rot+LSQ does not at W8A8 (−0.1)	ICML Table 1	established
On the FP32 basis SQNR_k falls with rank, higher for rotation; on each variant's own basis the head band 0–8 beats FP32	ICML Sec. 5, Fig. 8c (aggregated); Fig. 9A (one model)	established
The quantized detector stays within 1.7 mAP of FP32 everywhere, a bound, not an equivalence	ICVES Table I; Sec. IV.A	established
The W8A8 QAT recognizer beats its FP32 control by +2.5 to +5.9 balanced-accuracy points at Gaussian $\sigma = 4, 6$ and defocus s4, s5 (clean −0.2); the Gaussian gain peaks at $\sigma = 6$ and reverses (−2.6, −6.0 at $\sigma = 10, 14$)	ICVES Table V (bootstrap, conditional on three runs); Table VIII	established
Relative to the signal, R_q and R_b rise toward the tail and co-align; effective rank 90.4 against 90.7	ICVES Fig. 6A–B (rank formula unstated)	established
Stochastic noise does not stand in for quantization. Step-matched noise misses the gain (−1.2, −0.9, −1.3, +0.6 where QAT gains) but helps clean accuracy (+1.8); unmatched dropout shifts the spectrum differently	ICVES Table VII (three-seed point estimates); ICML App. D.4, Fig. 14 (one model)	suggestive
The spectral overlap is necessary, not sufficient; sensor noise and contrast co-align too, and the cosine falls from 0.98 to 0.58 as blur grows	ICVES Sec. VII; Fig. 6C; Table VIII (cosine definition unstated)	suggestive
Training the quantizer carries the Gaussian gain; PTQ inherits most of the defocus gain	ICVES Table VIII; Sec. VII	suggestive
Passive filtering of the tail, with QAT compensating in the head, explains the synthetic-noise gains; tail truncation of an FP32 model reproduces the signature	ICML Sec. 5; Fig. 9 (one model); App. D.3	suggestive
Activation clipping set by the learned step does the work	ICVES Sec. VII; L5	open
Determinism, signal-dependent error, or trained capacity separates quantizer from noise	notes; research note	open
Coarser activation quantization widens the severity window	ICVES Sec. VII; L2	open
Why the gain reverses past $\sigma = 8$, while FP32 still scores 56.1 at $\sigma = 10$	ICVES Sec. VII; Table VIII	open

- *Evaluate per severity, per corruption family, and per class*, since a suite average dilutes a window and plain accuracy hides a minority-class gain (Section 12).
- *Benchmark stack-matched and state the baseline*, since the ICVES vendor INT8 path ran slower than the vendor's FP16 engine (Table IV) and ICML's headline speed-up is against PyTorch FP32 (Section 16).
- *Recalibrate confidence after quantizing*, as ICVES recommends for score thresholds (Sec. V) and as one refitted logit scale does for ICML's vision-only LAION models (Fig. 5).
- *Avoid naive PTQ; prefer a trained quantizer.* Pooled PTQ fails ICML's 5% tolerance in 44.1% of evaluations against 25.0% for QAT (App. D.4; rotation PTQ, 5.6%, App. F), and the ICVES recipe, LSQ on clean data, "needs no corrupted data, no extra loss term, and no architectural change" (Sec. VIII).
- *Do not extrapolate below 8 bits*, where even Rot+LSQ amplifies spurious reliance (ΔRSG 4.0 at W4A8, ICML Table 1) and the ICVES window is unmeasured.

Each ICVES item rests on one benchmark, detector, encoder, and GPU (L1), each ICML item on discriminative CLIP-family models under simulated quantization with one deployed engine (App. A), so treat a robustness gain as a hypothesis to verify on your own data.

21.5 Open problems for a student project

Each open row of Table 20 can be attacked with the ICVES recognizer protocol (Section 18; notebook Part 5 runs it at one seed), changing one thing at a time over at least three seeds and comparing with the same FP32 control by the paired bootstrap of Section 13. Projects 1 and 5 test the ICVES paper's candidate and prediction; projects 2 to 4 are the notes' designs, built on a research note written for this tutorial.

1. Clipping without rounding. Use $\mathrm{clamp}(x, sq_{\min}, sq_{\max})$ in place of each activation quantizer, with s the step that the same seed's W8A8 QAT run learned, frozen (without rounding, the LSQ gradient of Equation (28) would move it). If the variant recovers most of the window gain, clipping carries it; if none, the ICVES clipping candidate (L5) fails.

2. Rounding without clipping. The complement, $s\,\mathrm{round}(x/s)$ with the learned step and no clamp. If neither this nor project 1 reproduces the gain, the interaction of the two regimes of Equation (14) carries it.

3. A step-matched, capacity-matched noise control. The Table VII noise follows the calibrated step, not the learned one, and appears to train no scale (Table 19). Draw $n \sim \mathcal{U}(-s_t/2, s_t/2)$, with s_t the QAT run's step at training step t, and match the trainable parameters with one scale per insertion point after the noise, $x \mapsto a\,(x + n)$, which cannot change the noise-to-signal ratio. The research note's trainable noise scale would instead be driven toward zero by a task loss that the noise hurts.

4. Deterministic or stochastic, signal-dependent or not. Rounding error is a fixed function of the input; the Table VII noise is independent of it and redrawn at every step. Four arms separate the two properties: the quantizer; stochastic rounding, up or down with probability set by the distance (mean error variance $s^2/6$, so use step $s/\sqrt{2}$); one noise tensor frozen per crop and insertion point; and fresh noise. The frozen-against-fresh pair is the research note's, and the stochastic-rounding arm is the notes' extension.

5. The bit-width sweep on the real encoder. Hold the weights at 8 bits and set the activations to 8, 6, 5, and 4 bits over the full σ grid, as designed in Exercise II.14. The ICVES account predicts that the window widens (Sec. VII; L2) and, by the paper's statement, is falsified if it does not move; fix the decision rule before running.

21.6 The sentence to leave with

Quantized to 8 bits, and most dependably when the quantizer is trained, a model can become more reliable in specific, measurable places, CLIP-family models under synthetic noise and a road-scene recognizer under moderate-to-heavy blur, and less reliable in others. Both papers see its residual rise relative to the signal toward the tail of the feature spectrum, both find that stochastic noise does not stand in for it (only ICVES matches the noise's size), and neither has isolated what in the quantizer does the work.

Running example: the bus in the rain

Back to the bus in the rain at $\sigma = 6$, which ICVES Fig. 1A shows the FP32 control calling a truck and the W8A8 model a bus in all three seed pairs. There the quantized recognizer scores 70.8 ± 1.2 balanced accuracy against 66.8, a gain of $+4.1$ with interval $[+1.0, +7.4]$ (Bouguerra et al., 2026b, Table V) established, and under heavy Gaussian blur its corrections outnumber its reverse errors. Under defocus they do not (Sec. V), so plain accuracy can fall while balanced accuracy rises, as in Section 12. At $\sigma = 14$ the difference turns to -6.0 (Table VIII), at 4-bit activations it is untested open, and the "bus" threshold needs refitting.

In the hands-on session

Notebook Part 1 (LSQ shrinking a 3-bit step from 2.02 to 1.48) is project 1 in miniature. In Part 3, Exercise 3.2, matched noise does as well as the quantizer in a linear classifier (Section 8), unlike ICVES Table VII. What does the toy lack? The candidates of Section 20 (determinism, signal dependence, clipping, trained capacity) are the places to look. Part 5 (GPU) runs the recognizer at one seed, where cells move by about 2 points; its exercise (c) pilots project 5.

Exercises

Exercise II.15: Assign the tier

Give each claim a tier, and name what would move it up. (a) The ICVES W8A8 recognizer is more accurate than its FP32 control at Gaussian $\sigma = 6$. (b) The recognizer tolerates blur because its quantization and blur residuals share the high-rank tail. (c) Clipping by the learned step causes the gain. (d) Quantization dampens the energy of the high-rank tail.

Solution sketch. (a) established for DAWN in balanced accuracy (Table V), conditional on three runs; a second dataset and encoder would widen it (L1). (b) suggestive; noise and contrast co-align too and the cosine falls as the gain rises, so an intervention on the tail is needed. (c) open; projects 1 and 2. (d) A misreading. ICML Fig. 8b shows the tail's projected magnitude rising; what is established is a low SQNR_k there (Section 8).

Exercise II.16: A tension inside the ICML paper

Rotation keeps the highest SQNR_k in ICML Fig. 8c, so by the paper's own account it filters the tail least. Yet App. B Table 4 lists synthetic-noise robustness as improved for PTQ, QAT, and Rotation+LSQ alike. (a) If filtering strength set synthetic robustness, which family should gain least? (b) What numbers would test that, and does the paper

print them? (c) Which ICVES prediction runs on the same logic?

Solution sketch. (a) Rotation+LSQ. (b) Synthetic-robustness changes per method family at matched accuracy; Fig. 7 pools the methods, and Table 4 is qualitative, stated to apply only to "easy use cases where we observed gains". (c) The bit-width prediction, a coarser step filtering more and so widening the window (ICVES L2). Neither test has been run open.

Check yourself before moving on

1. Which one quantity lets you read ICML Fig. 8c and ICVES Fig. 6B with the same eyes?[a]
2. Why do the ICML head concentration and the unchanged ICVES effective rank not conflict?[b]
3. Name one place where quantization made reliability worse in each paper.[c]

[a] The ratio $\rho_k = e_k/\lambda_k$ of a residual on the fixed FP32 basis; ICML plots $\mathrm{SQNR}_k = -10\log_{10}\rho_k$, ICVES plots ρ_k.

[b] They measure different things in different interventions. ICML reads subspace accuracy on a basis recomputed for one CLIP model, where even PTQ gains in the head band (Fig. 9A); ICVES summarizes a frozen encoder, whose features are not re-learned (L3), in one effective rank.

[c] ICML, the 54.8% of significant runs degrading accuracy and calibration (Fig. 1) or spurious reliance under PTQ (Table 1); ICVES, the reversal at $\sigma = 10, 14$ (Table VIII) or plain accuracy slipping by up to 2.9 points (Sec. V).

A Notation

The symbols below are used the same way in every section. Where the two papers write something else for the same object, Section B gives the translation.

Symbol	Meaning
Numbers and the quantizer	
b	bit width; W8A8 means 8-bit weights and 8-bit activations
s	quantization step, the scale that LSQ learns; never written Δ
$q_{\min}$, $q_{\max}$	smallest and largest integer level, -2^{b-1} and $2^{b-1}-1$
round	$\mathrm{round}(u)$ is the integer nearest to u, ties to even
clamp	$\mathrm{clamp}(u, q_{\min}, q_{\max}) = \min(\max(u, q_{\min}), q_{\max})$, which limits u to $[q_{\min}, q_{\max}]$
$Q(x)$	the symmetric quantizer $s\,\mathrm{clamp}(\mathrm{round}(x/s), q_{\min}, q_{\max})$, Equation (13)
z	zero point of the affine quantizer, Equation (15)
$[sq_{\min},\ sq_{\max}]$	the clip range, Equation (14)
$e = Q(x) - x$	quantization error, quantized minus original: rounding error inside the clip range, clipping error outside; energies do not depend on the sign
η^2	rounding-noise variance, $s^2/12$ in the high-resolution model, Equation (16)
SQNR	signal-to-quantization-noise ratio in dB, Equation (17)
Features and their spectrum	
$\boldsymbol{x} \in \mathbb{R}^d$	one feature vector; $d = 384$ for DINOv2 ViT-S/14
$X \in \mathbb{R}^{N\times d}$	N feature vectors as rows; X_c after subtracting the mean $\boldsymbol{\mu}$
C	feature covariance $\frac{1}{N}X_c^\top X_c = V\Lambda V^\top$, Equations (30) and (31)

Symbol	Meaning
λ_k, $\boldsymbol{v}_k$	k-th eigenvalue (the variance along $\boldsymbol{v}_k$) and eigenvector, sorted so that $\lambda_1 \geq \lambda_2 \geq \dots$
k	the rank index, the position of a direction in the sorted spectrum, $k = 1, \dots, d$ (the papers' plots start at 0)
erank	effective rank, Equation (34); the participation ratio is Equation (35)
E	a residual matrix, for example $X_Q - X$ or $X(\text{blur}) - X$
$e_k(E)$	per-rank energy $\frac{1}{N}\|E\boldsymbol{v}_k\|^2$ on the fixed full-precision basis, Definition 8.1
ρ_k	residual-to-signal ratio e_k/λ_k
SQNR_k	per-rank SQNR $-10\log_{10}\rho_k$ in dB, Equation (38)
g_k	gain of the quantized component along $\boldsymbol{v}_k$, Equation (39)
Images, models, and metrics	
σ	Gaussian blur standard deviation in pixels of the image it is applied to
κ	CLIP logit scale (inverse temperature), Equation (49)
T	temperature of a softmax, Equation (62)
ΔAcc	accuracy of the quantized model minus accuracy of the full-precision model, in percentage points (pp), Equation (59)
δ	relative drop, positive when the quantized model is worse, Equation (58)
BAcc	balanced accuracy, the mean per-class recall, Equation (57)
ECE	expected calibration error, Equation (61)
AUROC, FPR	area under the ROC curve, false positive rate, Equation (63)

B Glossary of terms the two papers use differently

The two papers address different communities, one studying vision-language models at scale and one studying a deployed road-scene pipeline, and several words carry different meanings in them. Each row gives the word, how the ICML paper (Bouguerra et al., 2026a) uses it, how the ICVES paper (Bouguerra et al., 2026b) uses it, and what these notes write instead.

Word	ICML paper	ICVES paper	These notes
rank	the index of a direction in the feature spectrum ("high-rank tail"), the rank of a LoRA update, and "full-rank" noise that touches every direction	the index of a direction ("high-rank directions") and the scalar "effective rank"	rank index k, matrix rank, LoRA rank, and effective rank, always qualified
spectrum	the feature covariance spectrum, the Fourier "spectral bias", and the singular values of weight matrices	the feature spectrum on the full-precision basis, and "frequency sensitivity" of the input	always say which one: image Fourier spectrum, feature covariance spectrum, or weight singular values
low-pass	a metaphor for filtering in the rank domain, and literal image frequency	blur as a literal low-pass filter of the image	"low-pass" only for images; for features, "suppresses the high-rank tail relative to the signal"
noise floor	low-variance, ambiguous content of the representation (one use)	not used	rounding-noise level η^2 for one tensor; residual profile e_k for the features; the flat floor is a model

Word	ICML paper	ICVES paper	These notes
noise	rounding error, a Gaussian-noise corruption, label noise in web data, and a figure panel that also covers blur and contrast	rounding residual, sensor noise, and the matched uniform noise of the control	always qualified: rounding error, corruption noise, label noise
energy	the energy OOD score, the magnitude spectrum, cumulative explained variance	per-rank energy, residual energy, "matched energy"	"energy score" only for OOD detection; "per-rank energy e_k" for projections, with mean or sum stated
W8A8	all linear and convolution layers except the last projections; granularity not stated; also W6A8 and W4A8	per-channel weights, per-tensor activations; 48 block linear layers of the encoder, or 37 linear and 69 convolution layers of the detector	the table in Section 14
PTQ	eight methods, all calibrated on 1000 proxy pairs	same quantizers and calibration as QAT, with no training	post-training quantization, meaning no gradient training of the model
QAT	weights updated with distillation on proxy data	backbone frozen; only the LSQ scales and the classifier head train	"QAT that moves weights" versus "QAT that moves only scales and head"
calibration	probability calibration (ECE) and quantizer calibration on a proxy set	quantizer calibration on clean images and confidence calibration	range calibration versus probability calibration, qualified wherever both senses could meet
detection	out-of-distribution detection	object detection	OOD detection versus object detection
head, tail	spectral head and tail; a text-projection head	spectral head and tail; the linear classifier head; the detector heads	spectral head and tail versus classifier head
semantic	new classes (semantic shift) and, in one figure, covariate shift	not used	covariate shift and semantic shift only
robustness	accuracy kept after quantization; relative drop under shift; "added robustness" in pp	accuracy on corrupted inputs; $\Delta\mathrm{Acc}(\sigma)$ in pp	the conversion of Equation (60)
severity	severity 3 of 5 on CIFAR-10-C, and mild, moderate, severe blur	severities 1 to 5 mapped to $\sigma \in \{1, 2, 3, 4, 6\}$, plus $\sigma =$ 8, 10, 14	σ in pixels at a stated resolution
residual	$X - X_Q$	$X_{\mathrm{W8A8}} - X_{\mathrm{fp}}$ and $X_{\mathrm{fp}}(\mathrm{blur}) - X_{\mathrm{fp}}$	$E = X_Q - X$; energies ignore the sign
cosine	cosine between reconstructed and full-precision embeddings	cosine between two per-rank profiles	embedding cosine versus profile cosine
Δ	spectral distortion, relative accuracy change, and the step size	accuracy differences in pp	reserved for differences in results; the step is s
arithmetic	"finite-precision arithmetic", rejected as the cause	"training does the work, not the arithmetic", meaning training-free quantization	describe the control instead of using the slogan
speed-up	against PyTorch FP32	against a stack-matched FP16 pipeline	always state the baseline
vendor engine	TensorRT, named	"vendor INT8 path", not named	TensorRT only for the ICML paper

Word	ICML paper	ICVES paper	These notes
outlier	outlier features (“massive activations”) that rotation redistributes (App. F), and, in the name of the OOD score EOE (Sec. 3.2, Fig. 6), an unusual input	activation outliers that rebalancing or foldable rotations handle (Sec. II); no OOD sense	outlier channel, outlier token, or rare value for activations (Section 6.2); OOD input or anomalous region for inputs (Section 12.4)

C Key equations

The equations the notes reuse most, grouped by topic, each copied from the section that derives it. The number on the right of each entry points back to the derivation.

Numbers and the quantizer

Weight memory, Equation (7), and Amdahl's law, Equation (12):

$$M_{\mathrm{w}} = \frac{1}{8}\sum_{\ell} P_\ell\, b_\ell \text{ bytes}, \qquad G = \frac{1}{(1-f) + f/r} \;\le\; \frac{1}{1-f}.$$

The symmetric and the affine quantizer, Equations (13) and (15), and the clip range, Equation (14):

$$Q(x) = s \cdot \mathrm{clamp}\Big(\mathrm{round}(x/s),\, q_{\min},\, q_{\max}\Big),$$

$$Q_{s,z}(x) = s\Big(\mathrm{clamp}(\mathrm{round}(x/s) + z,\, q_{\min},\, q_{\max}) - z\Big),$$

$$\text{clip range} = \big[\, sq_{\min},\; sq_{\max} \,\big] = \big[\, -2^{b-1}s,\; (2^{b-1}-1)s \,\big].$$

Rounding error as noise

The high-resolution model, Equation (16), the SQNR, Equation (17), and the 6.02 dB per bit rule, Equation (18), for a signal of power P_S and range R:

$$\eta^2 = \mathrm{Var}(e) = \frac{s^2}{12}, \qquad \mathrm{SQNR} = 10\log_{10}\frac{P_S}{P_N}\ \mathrm{dB}, \qquad \mathrm{SQNR}(b) \approx 10\log_{10}\frac{12\,P_S}{R^2} + 6.02\,b\ \mathrm{dB}.$$

The error decomposition, Equation (19); a negative $\mathbb{E}[x\,e]$ means attenuation:

$$\mathbb{E}\Big[Q(x)^2\Big] = \mathbb{E}\Big[x^2\Big] + 2\,\mathbb{E}[x\,e] + \mathbb{E}\Big[e^2\Big].$$

Networks and training

Softmax and cross-entropy, Equation (20):

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad L(\boldsymbol{z}, y) = -\log p_y, \qquad \frac{\partial L}{\partial z_k} = p_k - \mathbf{1}[k = y].$$

The straight-through estimator, Equation (27), and the LSQ step gradient with $u = x/s$, Equation (28):

$$\frac{\partial Q}{\partial x} \approx \begin{cases} 1, & sq_{\min} < x < sq_{\max}, \\ 0, & \text{otherwise}, \end{cases}$$

$$\frac{\partial Q}{\partial s} = \begin{cases} \mathrm{round}(u) - u, & q_{\min} < u < q_{\max}, \\ q_{\min}, & u \le q_{\min}, \\ q_{\max}, & u \ge q_{\max}, \end{cases} \qquad g = \frac{1}{\sqrt{N\,q_{\max}}}.$$

Spectra

Covariance and its eigendecomposition, Equations (30) and (31), and the link to the singular values of $X_c = USV^\top$, Equation (32):

$$C = \frac{1}{N} X_c^\top X_c = V \Lambda V^\top = \sum_{k=1}^{d} \lambda_k \, \boldsymbol{v}_k \boldsymbol{v}_k^\top, \qquad \lambda_k = \frac{S_{kk}^2}{N}.$$

Effective rank and participation ratio with $p_k = \lambda_k / \sum_j \lambda_j$, Equations (34) and (35):

$$\operatorname{erank}(C) = \exp\Big(- \sum_{k=1}^{d} p_k \ln p_k \Big), \qquad \operatorname{PR}(C) = \frac{\big(\sum_k \lambda_k\big)^2}{\sum_k \lambda_k^2} = \frac{1}{\sum_k p_k^2}.$$

Noise and the spectrum

Independent noise adds its covariance, Equation (37):

$$\operatorname{Cov}(\boldsymbol{x} + \boldsymbol{e}) = C + \Sigma_E.$$

The per-rank SQNR of a residual E, Equation (38), and the gain-plus-noise decomposition along $\boldsymbol{v}_k$ with c_k the signal-error covariance, Equation (39):

$$\operatorname{SQNR}_k(E) = -10 \log_{10} \rho_k(E) = 10 \log_{10} \frac{\lambda_k}{e_k(E)} \ \text{dB},$$

$$z_Q = g_k \, z + n_k, \quad g_k = 1 + \frac{c_k}{\lambda_k}, \quad \mathbb{E}\big[n_k^2\big] = e_k - \frac{c_k^2}{\lambda_k}.$$

Images

Convolution, Equation (42), and the frequency response of a Gaussian blur, Equation (44):

$$(h * I)(x, y) = \sum_m \sum_n h(m, n) \, I(x - m, \, y - n),$$

$$H_\sigma(f) = \exp\!\big(-2\pi^2 \sigma^2 f^2\big) \quad (f \text{ in cycles per pixel}).$$

Foundation models and detection

CLIP zero-shot classification with unit embeddings $\boldsymbol{u}$ and $\boldsymbol{t}_y$, Equation (49); the contrastive loss is Equation (48):

$$p(y \mid \text{image}) = \frac{\exp\left(\kappa \, \boldsymbol{u}^\top \boldsymbol{t}_y\right)}{\sum_{y'} \exp\left(\kappa \, \boldsymbol{u}^\top \boldsymbol{t}_{y'}\right)}.$$

Intersection over union and average precision, Equations (50) and (53):

$$\operatorname{IoU}(A, B) = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}, \qquad \operatorname{AP} = \int_0^1 p_{\text{interp}}(r) \, dr.$$

Reliability

Accuracy and balanced accuracy from per-class recalls r_i, Equation (57):

$$\operatorname{Acc} = \sum_{i=1}^{K} \frac{N_i}{N} \, r_i, \qquad \operatorname{BAcc} = \frac{1}{K} \sum_{i=1}^{K} r_i.$$

Relative drop, absolute difference, and the conversion between retention and absolute gain, Equations (58) to (60):

$$\delta = \frac{A(f) - A(q)}{A(f)}, \qquad \Delta\mathrm{Acc}(\sigma) = \mathrm{Acc}_q(\sigma) - \mathrm{Acc}_f(\sigma), \qquad D_f(\sigma) - D_q(\sigma) = \Delta\mathrm{Acc}(\sigma) - \Delta\mathrm{Acc}(0).$$

Expected calibration error and temperature scaling, Equations (61) and (62):

$$\mathrm{ECE} = \sum_{b=1}^{M} \frac{n_b}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|, \qquad p_T(y \mid x) = \frac{\exp(z_y/T)}{\sum_{y'} \exp(z_{y'}/T)}.$$

The AUROC as a probability, Equation (63), and the relative spurious gap, Equation (64):

$$\mathrm{AUROC} = \Pr(S_{\mathrm{ID}} > S_{\mathrm{OOD}}) + \tfrac{1}{2}\Pr(S_{\mathrm{ID}} = S_{\mathrm{OOD}}), \qquad \mathrm{RSG}(m) = \frac{A(m, \mathcal{D}_N) - A(m, \mathcal{D}_C)}{A(m, \mathcal{D}_N)}.$$

Statistics

The percentile bootstrap interval, Equation (66), and the Benjamini and Hochberg and Holm rules for m tests with sorted p-values $p_{(1)} \le \cdots \le p_{(m)}$, Equations (67) and (68):

$$\mathrm{CI}_{1-\alpha} = \Big[\, \hat{\theta}^*_{(\lceil B\alpha/2 \rceil)},\ \hat{\theta}^*_{(\lceil B(1-\alpha/2) \rceil)} \,\Big],$$

$$k^\star = \max\Big\{k :\ p_{(k)} \le \frac{k}{m}\, q\Big\}, \qquad k^\dagger = \min\Big\{k :\ p_{(k)} > \frac{\alpha}{m - k + 1}\Big\}.$$

References

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 Spring)*, 483–485. https://doi.org/10.1145/1465482.1465560

Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., & Hensman, J. (2024). QuaRot: Outlier-free 4-bit inference in rotated LLMs. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in neural information processing systems* (pp. 100213–100240, Vol. 37). Curran Associates, Inc. https://doi.org/10.52202/079017-3180

AskariHemmat, M., Askari Hemmat, R., Hoffman, A., Lazarevich, I., Saboori, E., Mastropietro, O., Sah, S., Savaria, Y., & David, J.-P. (2022). *QReg: On regularization effects of quantization*. arXiv: 2206.12372. https://arxiv.org/abs/2206.12372

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization*. arXiv: 1607.06450. https://arxiv.org/abs/1607.06450

Banner, R., Nahshan, Y., & Soudry, D. (2019). Post training 4-bit quantization of convolutional networks for rapid-deployment. *Advances in Neural Information Processing Systems*, *32*. https://neurips.cc/virtual/2019/poster/13871

Bengio, Y., Léonard, N., & Courville, A. (2013). *Estimating or propagating gradients through stochastic neurons for conditional computation*. arXiv: 1308.3432. https://arxiv.org/abs/1308.3432

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, *57*(1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

Benjamini, Y., & Yekutieli, D. (2001). The control of the false discovery rate in multiple testing under dependency. *The Annals of Statistics*, *29*(4). https://doi.org/10.1214/aos/1013699998

Bennett, W. R. (1948). Spectra of quantized signals. *Bell System Technical Journal*, *27*(3), 446–472. https://doi.org/10.1002/j.1538-7305.1948.tb01340.x

Birhane, A., Prabhu, V. U., & Kahembwe, E. (2021). *Multimodal datasets: Misogyny, pornography, and malignant stereotypes*. arXiv: 2110.01963. https://arxiv.org/abs/2110.01963

Bishop, C. M. (1995). Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, *7*(1), 108–116. https://doi.org/10.1162/neco.1995.7.1.108

Blum, H., Sarlin, P.-E., Nieto, J., Siegwart, R., & Cadena, C. (2021). The Fishyscapes benchmark: Measuring blind spots in semantic segmentation. *International Journal of Computer Vision*, *129*(11), 3119–3135. https://doi.org/10.1007/s11263-021-01511-6

Bouguerra, A., Montoya, D., Gomez-Villa, A., Mraidha, C., & Arnez, F. (2026a). Less precise can be more reliable: A systematic evaluation of quantization's impact on VLMs beyond accuracy. *Proceedings of the 43rd International Conference on Machine Learning*, *306*.

Bouguerra, A., Radermacher, A., Arnez, F., & Mraidha, C. (2026b). Quantization and corruption robustness in deployed road-scene perception. *Proceedings of the 2026 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*. To appear.

Cao, C., Zhong, Z., Zhou, Z., Liu, Y., Liu, T., & Han, B. (2024). Envisioning outlier exposure by large language models for out-of-distribution detection. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 5629–5659, Vol. 235). PMLR. https://proceedings.mlr.press/v235/cao24d.html

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In A. Vedaldi, H. Bischof, T. Brox, & J.-M.

Frahm (Eds.), *Computer vision – ECCV 2020* (pp. 213–229). Springer International Publishing. https://doi.org/10.1007/978-3-030-58452-8_13

Ceccarelli, A., & Secci, F. (2023). RGB cameras failures and their effects in autonomous driving applications. *IEEE Transactions on Dependable and Secure Computing*, *20*(4), 2731–2745. https://doi.org/10.1109/TDSC.2022.3156941

Chee, J., Cai, Y., Kuleshov, V., & De Sa, C. (2023). QuIP: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, *36*, 4396–4429. https://doi.org/10.52202/075280-0196

Courbariaux, M., Bengio, Y., & David, J.-P. (2015). BinaryConnect: Training deep neural networks with binary weights during propagations. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 28). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2015/hash/3e15cc11f979ed25912dff5b0669f2cd-Abstract.html

Darcet, T., Oquab, M., Mairal, J., & Bojanowski, P. (2024). Vision transformers need registers. *International Conference on Learning Representations*. https://openreview.net/forum?id=2dnO3LLiJ1

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255. https://doi.org/10.1109/CVPR.2009.5206848

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8() / GPT3.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, *35*, 30318–30332. https://doi.org/10.52202/068431-2198

Ding, Y., Qin, H., Yan, Q., Chai, Z., Liu, J., Wei, X., & Liu, X. (2022). Towards accurate post-training quantization for vision transformer. *Proceedings of the 30th ACM International Conference on Multimedia*, 5380–5388. https://doi.org/10.1145/3503161.3547826

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations*. https://openreview.net/forum?id=YicbFdNTTy

Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, *1*(3), 211–218. https://doi.org/10.1007/BF02288367

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1), 1–26. https://doi.org/10.1214/aos/1176344552

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall. https://doi.org/10.1007/978-1-4899-4541-9

Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., & Modha, D. S. (2020). Learned step size quantization. *International Conference on Learning Representations*. https://openreview.net/forum?id=rkgO66VKDS

Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010). The Pascal visual object classes (VOC) challenge. *International Journal of Computer Vision*, *88*(2), 303–338. https://doi.org/10.1007/s11263-009-0275-4

Fan, K. (1949). On a theorem of Weyl concerning eigenvalues of linear transformations I. *Proceedings of the National Academy of Sciences*, *35*(11), 652–655. https://doi.org/10.1073/pnas.35.11.652

Field, D. J. (1987). Relations between the statistics of natural images and the response properties of cortical cells. *Journal of the Optical Society of America A*, *4*(12), 2379–2394. https://doi.org/10.1364/JOSAA.4.002379

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). OPTQ: Accurate quantization for generative pre-trained transformers [The arXiv version is titled "GPTQ"]. *International Conference on Learning Representations*. https://iclr.cc/virtual/2023/poster/10855

Fu, M., Yu, H., Shao, J., Zhou, J., Zhu, K., & Wu, J. (2025). Quantization without tears. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4462–4472. https://doi.org/10.1109/CVPR52734.2025.00421

Gao, P., Trautmann, E., Yu, B., Santhanam, G., Ryu, S., Shenoy, K., & Ganguli, S. (2017). *A theory of multineuronal dimensionality, dynamics and measurement*. bioRxiv. https://doi.org/10.1101/214262

Geirhos, R., Rubisch, P., Michaelis, C., Bethge, M., Wichmann, F. A., & Brendel, W. (2019). ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. *International Conference on Learning Representations*. https://openreview.net/forum?id=Bygh9j09KX

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2022). A survey of quantization methods for efficient neural network inference. In G. K. Thiruvathukal, Y.-H. Lu, J. Kim, Y. Chen, & B. Chen (Eds.), *Low-power computer vision: Improve the efficiency of artificial intelligence* (pp. 291–326). Chapman; Hall/CRC. https://doi.org/10.1201/9781003162810-13

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, *23*(1), 5–48. https://doi.org/10.1145/103162.103163

Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.

Gray, R. M., & Neuhoff, D. L. (1998). Quantization. *IEEE Transactions on Information Theory*, *44*(6), 2325–2383. https://doi.org/10.1109/18.720541

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In D. Precup & Y. W. Teh (Eds.), *Proceedings of the 34th international conference on machine learning* (pp. 1321–1330, Vol. 70). PMLR. https://proceedings.mlr.press/v70/guo17a.html

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Hendrycks, D., Basart, S., Mu, N., Kadavath, S., Wang, F., Dorundo, E., Desai, R., Zhu, T., Parajuli, S., Guo, M., Song, D., Steinhardt, J., & Gilmer, J. (2021). The many faces of robustness: A critical analysis of out-of-distribution generalization. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 8320–8329. https://doi.org/10.1109/ICCV48922.2021.00823

Hendrycks, D., & Dietterich, T. (2019). Benchmarking neural network robustness to common corruptions and perturbations. *International Conference on Learning Representations*. https://openreview.net/forum?id=HJz6tiCqYm

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations*. https://openreview.net/forum?id=Hkg4TI9xl

Hendrycks, D., Mu, N., Cubuk, E. D., Zoph, B., Gilmer, J., & Lakshminarayanan, B. (2020). AugMix: A simple data processing method to improve robustness and uncertainty. *International Conference on Learning Representations*. https://openreview.net/forum?id=S1gmrxHFvB

Hendrycks, D., Zhao, K., Basart, S., Steinhardt, J., & Song, D. (2021). Natural adversarial examples. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 15257–15266. https://doi.org/10.1109/CVPR46437.2021.01501

Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the knowledge in a neural network* [Presented at the NIPS 2014 Deep Learning Workshop]. arXiv: 1503.02531. https://doi.org/10.48550/arXiv.1503.02531

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70. https://www.jstor.org/stable/4615733

Hooker, S., Courville, A., Clark, G., Dauphin, Y., & Frome, A. (2019). *What do compressed deep neural networks forget?* arXiv: 1911.05248. https://doi.org/10.48550/arXiv.1911.05248

Hooker, S., Moorosi, N., Clark, G., Bengio, S., & Denton, E. (2020). *Characterising bias in compressed models*. arXiv: 2010.03058. https://arxiv.org/abs/2010.03058

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9

Huang, R., & Li, Y. (2021). MOS: Towards scaling out-of-distribution detection for large semantic space. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8710–8719. https://openaccess.thecvf.com/content/CVPR2021/html/Huang_MOS_Towards_Scaling_Out-of-Distribution_Detection_for_Large_Semantic_Space_CVPR_2021_paper.html

Huh, M., Mobahi, H., Zhang, R., Cheung, B., Agrawal, P., & Isola, P. (2023). The low-rank simplicity bias in deep networks. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=bCiNWDmlY2

IEEE. (2019). *IEEE standard for floating-point arithmetic*. IEEE Std 754-2019. https://doi.org/10.1109/IEEESTD.2019.8766229

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2704–2713. https://doi.org/10.1109/CVPR.2018.00286

Javed, S., Le, H., & Salzmann, M. (2025). QT-DoG: Quantization-aware training for domain generalization. *Proceedings of the 42nd International Conference on Machine Learning*, *267*, 26981–27004. https://proceedings.mlr.press/v267/javed25a.html

Jia, C., Yang, Y., Xia, Y., Chen, Y.-T., Parekh, Z., Pham, H., Le, Q., Sung, Y.-H., Li, Z., & Duerig, T. (2021). Scaling up visual and vision-language representation learning with noisy text supervision. *Proceedings of the 38th International Conference on Machine Learning*, *139*, 4904–4916. https://proceedings.mlr.press/v139/jia21b.html

Jiang, X., Liu, F., Fang, Z., Chen, H., Liu, T., Zheng, F., & Han, B. (2024). Negative label guided OOD detection with pretrained vision-language models. *International Conference on Learning Representations*. https://openreview.net/forum?id=xUO1HXz4an

Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *374*(2065), Article 20150202. https://doi.org/10.1098/rsta.2015.0202

Kenk, M. A., & Hassaballah, M. (2020). *DAWN: Vehicle detection in adverse weather nature dataset*. arXiv: 2008.05402. https://arxiv.org/abs/2008.05402

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Krishnamoorthi, R. (2018). *Quantizing deep convolutional networks for efficient inference: A whitepaper*. arXiv: 1806.08342. https://arxiv.org/abs/1806.08342

Krizhevsky, A. (2009, April 8). *Learning multiple layers of features from tiny images* (Technical report). University of Toronto. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

Kumar, A., Raghunathan, A., Jones, R., Ma, T., & Liang, P. (2022). Fine-tuning can distort pretrained features and underperform out-of-distribution. *International Conference on Learning Representations*. https://openreview.net/forum?id=UYneFzXSJWh

Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, *8*(4), 355–362. https://doi.org/10.1177/1948550617697177

Li, Y., Xu, S., Zhang, B., Cao, X., Gao, P., & Guo, G. (2022). Q-ViT: Accurate and fully quantized low-bit vision transformer. *Advances in Neural Information Processing Systems*, *35*, 34451–34463. https://doi.org/10.52202/068431-2496

Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., & Gu, S. (2021). BRECQ: Pushing the limit of post-training quantization by block reconstruction. *International Conference on Learning Representations*. https://iclr.cc/virtual/2021/poster/2596

Li, Z., Xiao, J., Yang, L., & Gu, Q. (2023). RepQ-ViT: Scale reparameterization for post-training quantization of vision transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 17181–17190. https://doi.org/10.1109/ICCV51070.2023.01580

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of Machine Learning and Systems*, *6*. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), *Computer vision – ECCV 2014* (pp. 740–755). Springer International Publishing. https://doi.org/10.1007/978-3-319-10602-1_48

Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Jiang, Q., Li, C., Yang, J., Su, H., Zhu, J., & Zhang, L. (2024). Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, & G. Varol (Eds.), *Computer vision – ECCV 2024* (pp. 38–55). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-72970-6_3

Liu, W., Wang, X., Owens, J., & Li, Y. (2020). Energy-based out-of-distribution detection. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems* (pp. 21464–21475, Vol. 33). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2020/hash/f5496252609c43eb8a3d147ab9b9c006-Abstract.html

Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., & Blankevoort, T. (2025). SpinQuant: LLM quantization with learned rotations. *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/e5b1c0d4866f72393c522c8a00eed4eb-Abstract-Conference.html

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 11966–11976. https://doi.org/10.1109/CVPR52688.2022.01167

Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, *28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*. https://openreview.net/forum?id=Bkg6RiCqY7

Max, J. (1960). Quantizing for minimum distortion. *IEEE Transactions on Information Theory*, *6*(1), 7–12. https://doi.org/10.1109/TIT.1960.1057548

Michaelis, C., Mitzkus, B., Geirhos, R., Rusak, E., Bringmann, O., Ecker, A. S., Bethge, M., & Brendel, W. (2019). *Benchmarking robustness in object detection: Autonomous driving when winter is coming*. arXiv: 1907.07484. https://arxiv.org/abs/1907.07484

Ming, Y., Cai, Z., Gu, J., Sun, Y., Li, W., & Li, Y. (2022). Delving into out-of-distribution detection with vision-language representations. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 35087–35102, Vol. 35). Curran Associates, Inc. https://doi.org/10.52202/068431-2543

Miyai, A., Yang, J., Zhang, J., Ming, Y., Lin, Y., Yu, Q., Irie, G., Joty, S., Li, Y., Li, H. H., Liu, Z., Yamasaki, T., & Aizawa, K. (2025). Generalized out-of-distribution detection and beyond in vision language model era: A survey. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=FO3IA4lUEY

Moon, J., Kim, D., Cheon, J., & Ham, B. (2024). Instance-aware group quantization for vision transformers. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16132–16141. https://doi.org/10.1109/CVPR52733.2024.01527

Naeini, M. P., Cooper, G. F., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *29*(1), 2901–2907. https://doi.org/10.1609/aaai.v29i1.9602

Nagel, M., Amjad, R. A., van Baalen, M., Louizos, C., & Blankevoort, T. (2020). Up or down? Adaptive rounding for post-training quantization. In H. Daumé III & A. Singh (Eds.), *Proceedings of the 37th international conference on machine learning* (pp. 7197–7206, Vol. 119). PMLR. https://proceedings.mlr.press/v119/nagel20a.html

Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., van Baalen, M., & Blankevoort, T. (2021). *A white paper on neural network quantization*. arXiv: 2106.08295 [cs.LG]. https://arxiv.org/abs/2106.08295

Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., … Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=a68SUt6zFt

Ordonez, V., Kulkarni, G., & Berg, T. L. (2011). Im2Text: Describing images using 1 million captioned photographs. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, & K. Weinberger (Eds.), *Advances in neural information processing systems* (Vol. 24). Curran Associates. https://proceedings.neurips.cc/paper_files/paper/2011/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

Panter, P. F., & Dite, W. (1951). Quantization distortion in pulse-count modulation with nonuniform spacing of levels. *Proceedings of the IRE*, *39*(1), 44–48. https://doi.org/10.1109/JRPROC.1951.230419

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In M. Meila & T. Zhang (Eds.), *Proceedings of the 38th international conference on machine learning* (pp. 8748–8763, Vol. 139). PMLR. https://proceedings.mlr.press/v139/radford21a.html

Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., & Courville, A. (2019). On the spectral bias of neural networks. In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th international conference on machine learning* (pp. 5301–5310, Vol. 97). PMLR. https://proceedings.mlr.press/v97/rahaman19a.html

Recht, B., Roelofs, R., Schmidt, L., & Shankar, V. (2019). Do ImageNet classifiers generalize to ImageNet? In K. Chaudhuri & R. Salakhutdinov (Eds.), *Proceedings of the 36th international conference on machine learning* (pp. 5389–5400, Vol. 97). PMLR. https://proceedings.mlr.press/v97/recht19a.html

Robinson, W. S. (1950). Ecological correlations and the behavior of individuals. *American Sociological Review*, *15*(3), 351–357. https://doi.org/10.2307/2087176

Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). Towards total recall in industrial anomaly detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 14298–14308. https://doi.org/10.1109/CVPR52688.2022.01392

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *15th European Signal Processing Conference (EUSIPCO 2007)*, 606–610. https://www.eurasip.org/Proceedings/Eusipco/Eusipco2007/Papers/a5p-h05.pdf

Sagawa, S., Koh, P. W., Hashimoto, T. B., & Liang, P. (2020). Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *International Conference on Learning Representations*. https://openreview.net/forum?id=ryxGuJrFvS

Schuhmann, C., Beaumont, R., Vencu, R., Gordon, C., Wightman, R., Cherti, M., Coombes, T., Katta, A., Mullis, C., Wortsman, M., Schramowski, P., Kundurthy, S., Crowson, K., Schmidt, L., Kaczmarczyk, R., & Jitsev, J. (2022). LAION-5B: An open large-scale dataset for training next generation image-text models [Datasets and Benchmarks Track]. *Advances in Neural Information Processing Systems*, *35*, 25278–25294. https://doi.org/10.52202/068431-1833

Sharma, P., Ding, N., Goodman, S., & Soricut, R. (2018). Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2556–2565. https://doi.org/10.18653/v1/P18-1238

Sun, Y., Ming, Y., Zhu, X., & Li, Y. (2022). Out-of-distribution detection with deep nearest neighbors. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, & S. Sabato (Eds.), *Proceedings of the 39th international conference on machine learning* (pp. 20827–20840, Vol. 162). PMLR. https://proceedings.mlr.press/v162/sun22d.html

Thakkar, V., Ramani, P., Cecka, C., Shivam, A., Lu, H., Yan, E., Kosaian, J., Hoemmen, M., Wu, H., Kerr, A., Nicely, M., Merrill, D., Blasig, D., Qiao, F., Majcher, P., Springer, P., Hohnerbach, M., Wang, J., & Gupta, M. (2023, January 23). *CUTLASS* (Version 3.0.0). https://github.com/NVIDIA/cutlass

Thomee, B., Shamma, D. A., Friedland, G., Elizalde, B., Ni, K., Poland, D., Borth, D., & Li, L.-J. (2016). YFCC100M: The new data in multimedia research. *Communications of the ACM*, *59*(2), 64–73. https://doi.org/10.1145/2812802

Tseng, A., Chee, J., Sun, Q., Kuleshov, V., & De Sa, C. (2024). QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 48630–48656, Vol. 235). PMLR. https://proceedings.mlr.press/v235/tseng24a.html

van den Oord, A., Li, Y., & Vinyals, O. (2018). *Representation learning with contrastive predictive coding*. arXiv: 1807.03748. https://arxiv.org/abs/1807.03748

van der Schaaf, A., & van Hateren, J. H. (1996). Modelling the power spectra of natural images: Statistics and information. *Vision Research*, *36*(17), 2759–2770. https://doi.org/10.1016/0042-6989(96)00002-8

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Wang, C., Wang, Z., Xu, X., Tang, Y., Zhou, J., & Lu, J. (2024). Q-VLM: Post-training quantization for large vision-language models. *Advances in Neural Information Processing Systems*, *37*, 114553–114573. https://doi.org/10.52202/079017-3638

Wang, H., Ge, S., Lipton, Z., & Xing, E. P. (2019). Learning robust global representations by penalizing local predictive power. *Advances in Neural Information Processing Systems*,

32. https://proceedings.neurips.cc/paper_files/paper/2019/hash/3eefceb8087e964f89c2d59e8a249915-Abstract.html

Wang, H., Wu, X., Huang, Z., & Xing, E. P. (2020). High-frequency component helps explain the generalization of convolutional neural networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8681–8691. https://doi.org/10.1109/CVPR42600.2020.00871

Wang, Q., Lin, Y., Chen, Y., Schmidt, L., Han, B., & Zhang, T. (2024). A sober look at the robustness of CLIPs to spurious features. *Advances in Neural Information Processing Systems*, *37*, 122484–122523. https://doi.org/10.52202/079017-3893

Wei, X., Gong, R., Li, Y., Liu, X., & Yu, F. (2022). QDrop: Randomly dropping quantization for extremely low-bit post-training quantization. *International Conference on Learning Representations*. https://iclr.cc/virtual/2022/poster/5893

Wei, X., Zhang, Y., Zhang, X., Gong, R., Zhang, S., Zhang, Q., Yu, F., & Liu, X. (2022). Outlier suppression: Pushing the limit of low-bit transformer language models. *Advances in Neural Information Processing Systems*, *35*, 17402–17414. https://doi.org/10.52202/068431-1265

Widrow, B., & Kollár, I. (2008). *Quantization noise: Roundoff error in digital computation, signal processing, control, and communications*. Cambridge University Press. https://doi.org/10.1017/CBO9780511754661

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, *52*(4), 65–76. https://doi.org/10.1145/1498765.1498785

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of the 40th international conference on machine learning* (pp. 38087–38099, Vol. 202). PMLR. https://proceedings.mlr.press/v202/xiao23c.html

Xiao, J., Hays, J., Ehinger, K. A., Oliva, A., & Torralba, A. (2010). SUN database: Large-scale scene recognition from abbey to zoo. *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 3485–3492. https://doi.org/10.1109/CVPR.2010.5539970

Xiao, Y., Liu, A., Zhang, T., Qin, H., Guo, J., & Liu, X. (2023). RobustMQ: Benchmarking robustness of quantized models. *Visual Intelligence*, *1*(1), Article 30. https://doi.org/10.1007/s44267-023-00031-w

Xu, Y., Xie, L., Gu, X., Chen, X., Chang, H., Zhang, H., Chen, Z., Zhang, X., & Tian, Q. (2024). QA-LoRA: Quantization-aware low-rank adaptation of large language models. *International Conference on Learning Representations*, 52401–52418. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e6c2e85db1f1039177c4495ccd399ac4-Abstract-Conference.html

Xu, Z.-Q. J., Zhang, Y., Luo, T., Xiao, Y., & Ma, Z. (2020). Frequency principle: Fourier analysis sheds light on deep neural networks. *Communications in Computational Physics*, *28*(5), 1746–1767. https://doi.org/10.4208/cicp.OA-2020-0085

Yang, J., Wang, P., Zou, D., Zhou, Z., Ding, K., Peng, W., Wang, H., Chen, G., Li, B., Sun, Y., Du, X., Zhou, K., Zhang, W., Hendrycks, D., Li, Y., & Liu, Z. (2022). OpenOOD: Benchmarking generalized out-of-distribution detection. *Advances in Neural Information Processing Systems*, *35*, 32598–32611. https://doi.org/10.52202/068431-2362

Yang, J., Zhou, K., Li, Y., & Liu, Z. (2024). Generalized out-of-distribution detection: A survey. *International Journal of Computer Vision*, *132*(12), 5635–5662. https://doi.org/10.1007/s11263-024-02117-4

Yin, D., Gontijo Lopes, R., Shlens, J., Cubuk, E. D., & Gilmer, J. (2019). A Fourier perspective on model robustness in computer vision. *Advances in Neural Information Processing*

Systems, *32*. https://proceedings.neurips.cc/paper_files/paper/2019/hash/b05b57f6add810d3b7490866d74c0053-Abstract.html

Yu, J., Wang, Z., Vasudevan, V., Yeung, L., Seyedhosseini, M., & Wu, Y. (2022). CoCa: Contrastive captioners are image-text foundation models. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=Ee277P3AYC

Yuan, Z., Xue, C., Chen, Y., Wu, Q., & Sun, G. (2022). PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization. In S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, & T. Hassner (Eds.), *Computer vision – ECCV 2022* (pp. 191–207). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-19775-8_12

Zandieh, A., Daliri, M., Hadian, M., & Mirrokni, V. (2026). TurboQuant: Online vector quantization with near-optimal distortion rate. *International Conference on Learning Representations*. https://iclr.cc/virtual/2026/poster/10006985

Zhai, X., Mustafa, B., Kolesnikov, A., & Beyer, L. (2023). Sigmoid loss for language image pre-training. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 11941–11952. https://doi.org/10.1109/ICCV51070.2023.01100

Zhang, J., Yang, J., Wang, P., Wang, H., Lin, Y., Zhang, H., Sun, Y., Du, X., Li, Y., Liu, Z., Chen, Y., & Li, H. (2024). OpenOOD v1.5: Enhanced benchmark for out-of-distribution detection. *Journal of Data-centric Machine Learning Research*, *2*(3), 1–32. https://openreview.net/forum?id=cnnTnJQigs

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16965–16974. https://doi.org/10.1109/CVPR52733.2024.01605